

# Quantum error correction and fault tolerance: A comprehensive tutorial

Daniel J. Spencer<sup>\*1,2,3</sup>, Shubham P. Jain<sup>\*1,2</sup>, Andrew Tanggara<sup>\*4,5</sup>, Zeen Sun<sup>\*4</sup>, Tobias Haug<sup>6</sup>, Derek Khu<sup>7</sup>, and Kishor Bharti<sup>†1,8</sup>

<sup>1</sup>*Joint Center for Quantum Information and Computer Science, NIST/University of Maryland, College Park, MD 20742, USA*

<sup>2</sup>*Department of Physics, University of Maryland, College Park, MD 20742, USA*

<sup>3</sup>*Joint Quantum Institute, NIST/University of Maryland, College Park, MD 20742, USA*

<sup>4</sup>*Centre for Quantum Technologies, National University of Singapore, 3 Science Drive 2, Singapore 117543*

<sup>5</sup>*Nanyang Quantum Hub, School of Physical and Mathematical Sciences, Nanyang Technological University, Singapore 639673*

<sup>6</sup>*Quantum Research Center, Technology Innovation Institute, Abu Dhabi, UAE*

<sup>7</sup>*Institute for Infocomm Research (I<sup>2</sup>R), Agency for Science, Technology and Research (A\*STAR), 1 Fusionopolis Way, #21-01, Connexis South Tower, Singapore 138632, Republic of Singapore*

<sup>8</sup>*Department of Computer Science and Institute for Advanced Computer Studies, University of Maryland, College Park, Maryland 20742, USA*

## Abstract

Noise is one of the central obstacles to building useful quantum computers, and quantum error correction (QEC) provides the framework for protecting quantum information against it. Unlike classical error correction, QEC must preserve fragile quantum states without copying them, measuring them directly, or destroying the information they encode. Driven by rapid progress in both theory and experiment, this challenge has grown into one of the most active areas of quantum information science. This tutorial gives a guided introduction to modern QEC, developing the core concepts of codes, syndromes, stabilizers, decoding, and fault tolerance before connecting them to major code families and current research directions. We cover both established constructions and newer developments, including topological and subsystem codes, bosonic and qudit codes, dynamical codes, and quantum low-density parity-check (qLDPC) codes. The emphasis is on building operational understanding: explaining not only what the main objects are, but how they are used in code design, error diagnosis, decoding, and fault-tolerant computation. The tutorial is intended for newcomers seeking a first path through QEC, as well as researchers looking for a coherent reference for the concepts, code families, and tools that arise in current work.

---

<sup>\*</sup>These authors contributed equally

<sup>†</sup>Corresponding author: [kishor.bharti1@gmail.com](mailto:kishor.bharti1@gmail.com)

*The following tutorial is derived in part from an online lecture series held between August 20, 2025 and December 17, 2025. These lectures are now archived on YouTube and can be found [here](#).*

# Contents

|          |                                                                  |           |
|----------|------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Motivation</b>                                                | <b>6</b>  |
| <b>2</b> | <b>Brief introduction to quantum information</b>                 | <b>8</b>  |
| 2.1      | Quantum states, operators, and measurements . . . . .            | 8         |
| 2.2      | Postulates of quantum mechanics . . . . .                        | 11        |
| 2.3      | Quantum circuit model . . . . .                                  | 13        |
| <b>3</b> | <b>Fundamentals of classical and quantum error correction</b>    | <b>16</b> |
| 3.1      | Classical error correction . . . . .                             | 16        |
| 3.2      | Quantum repetition code . . . . .                                | 19        |
| 3.3      | Nine-qubit Shor code . . . . .                                   | 22        |
| 3.4      | Noise channels . . . . .                                         | 24        |
| 3.5      | Brief history of QEC . . . . .                                   | 28        |
| <b>4</b> | <b>Stabilizer codes</b>                                          | <b>30</b> |
| 4.1      | Primer on group theory . . . . .                                 | 30        |
| 4.2      | Stabilizer codes . . . . .                                       | 34        |
| 4.3      | Five-qubit perfect code . . . . .                                | 36        |
| 4.4      | CSS codes . . . . .                                              | 37        |
| <b>5</b> | <b>Topological codes</b>                                         | <b>40</b> |
| 5.1      | Primer on topology . . . . .                                     | 40        |
| 5.2      | Toric code . . . . .                                             | 49        |
| 5.3      | Surface code . . . . .                                           | 56        |
| <b>6</b> | <b>Subsystem codes</b>                                           | <b>63</b> |
| 6.1      | Motivation: the 9-qubit Shor code . . . . .                      | 63        |
| 6.2      | The mathematical framework of subsystem codes . . . . .          | 64        |
| 6.3      | The Bacon–Shor code . . . . .                                    | 65        |
| 6.4      | Knill–Laflamme conditions for subsystem codes . . . . .          | 66        |
| 6.5      | Topological subsystem codes . . . . .                            | 67        |
| 6.6      | Why subsystem codes? . . . . .                                   | 70        |
| <b>7</b> | <b>Dynamical codes</b>                                           | <b>73</b> |
| 7.1      | “Hidden” dynamical logical qubit . . . . .                       | 73        |
| 7.2      | 4-qubit dynamical code . . . . .                                 | 75        |
| 7.3      | Hastings–Haah honeycomb Floquet code . . . . .                   | 82        |
| 7.4      | Conservation of logical information in dynamical codes . . . . . | 85        |
| <b>8</b> | <b>Bosonic codes</b>                                             | <b>86</b> |
| 8.1      | Bases of a simple harmonic oscillator . . . . .                  | 86        |
| 8.2      | Error channels . . . . .                                         | 88        |
| 8.3      | Code examples . . . . .                                          | 91        |
| 8.4      | Further reading . . . . .                                        | 95        |

|           |                                                                     |            |
|-----------|---------------------------------------------------------------------|------------|
| <b>9</b>  | <b>Decoders</b>                                                     | <b>96</b>  |
| 9.1       | The decoding problem . . . . .                                      | 96         |
| 9.2       | Optimization formulation of decoding . . . . .                      | 98         |
| 9.3       | Degeneracy . . . . .                                                | 99         |
| 9.4       | How to measure syndromes . . . . .                                  | 101        |
| 9.5       | Noise models for QEC simulation . . . . .                           | 102        |
| 9.6       | Detector model . . . . .                                            | 102        |
| 9.7       | Decoder landscape . . . . .                                         | 103        |
| 9.8       | Mixed-integer programming decoders and relaxations . . . . .        | 103        |
| 9.9       | Minimum weight perfect matching . . . . .                           | 104        |
| 9.10      | Union-Find . . . . .                                                | 105        |
| 9.11      | Belief propagation . . . . .                                        | 106        |
| 9.12      | Tensor network-based decoders . . . . .                             | 107        |
| 9.13      | Machine learning-based decoders . . . . .                           | 108        |
| 9.14      | Tesseract and decision-tree decoders . . . . .                      | 108        |
| 9.15      | Combinatorial decoders . . . . .                                    | 109        |
| 9.16      | Renormalization group decoders . . . . .                            | 109        |
| <b>10</b> | <b>Fundamental bounds in quantum error correction</b>               | <b>111</b> |
| 10.1      | The quantum Hamming bound (sphere-packing limit) . . . . .          | 111        |
| 10.2      | The quantum Singleton bound (information-theoretic limit) . . . . . | 113        |
| 10.3      | The quantum Gilbert-Varshamov bound (existence guarantee) . . . . . | 114        |
| 10.4      | The constraint of geometric locality . . . . .                      | 115        |
| 10.5      | Bounds from optimization theory . . . . .                           | 117        |
| <b>11</b> | <b>Quantum low-density parity-check codes</b>                       | <b>119</b> |
| 11.1      | The LDPC code construction . . . . .                                | 119        |
| 11.2      | The hypergraph product construction . . . . .                       | 120        |
| 11.3      | LDPC codes from algebraic constructions . . . . .                   | 123        |
| 11.4      | Breaking the $\sqrt{n}$ barrier: advanced constructions . . . . .   | 124        |
| <b>12</b> | <b>Qudit codes</b>                                                  | <b>132</b> |
| 12.1      | Motivation for and basic background on qudits . . . . .             | 132        |
| 12.2      | Finite field theory . . . . .                                       | 134        |
| 12.3      | Major families of qudit stabilizer codes . . . . .                  | 136        |
| 12.4      | Advances in code bounds and asymptotic performance . . . . .        | 141        |
| 12.5      | Experimental realizations of qudit codes . . . . .                  | 141        |
| <b>13</b> | <b>Fault tolerance I</b>                                            | <b>143</b> |
| 13.1      | Error propagation . . . . .                                         | 143        |
| 13.2      | Syndrome extraction schemes . . . . .                               | 145        |
| 13.3      | Formalism for fault tolerance . . . . .                             | 147        |
| 13.4      | Noise models . . . . .                                              | 149        |
| 13.5      | Threshold theorem . . . . .                                         | 150        |
| 13.6      | Recursive encoding . . . . .                                        | 152        |
| 13.7      | Recursive gadgets . . . . .                                         | 152        |
| 13.8      | Why concatenation works . . . . .                                   | 152        |
| 13.9      | Why transversality is powerful . . . . .                            | 153        |

|                                                                    |            |
|--------------------------------------------------------------------|------------|
| <b>14 Fault tolerance II</b>                                       | <b>154</b> |
| 14.1 Constant-overhead FTQC scheme using qLDPC codes . . . . .     | 154        |
| 14.2 Decoding threshold for ideal syndrome extraction . . . . .    | 154        |
| 14.3 Decoding threshold for noisy syndrome extraction . . . . .    | 157        |
| 14.4 Threshold theorem for constant-overhead FTQC scheme . . . . . | 174        |
| 14.5 Magic state distillation . . . . .                            | 187        |
| 14.6 Code switching . . . . .                                      | 192        |
| 14.7 Outlook . . . . .                                             | 193        |
| <b>Acknowledgments</b>                                             | <b>194</b> |

# 1 Motivation

Quantum computing is one of the boldest ideas in modern science. Quantum computers can perform calculations beyond what is possible with classical computers by harnessing hitherto abstract concepts of quantum mechanics, such as interference, entanglement, and superposition. This promises tremendous breakthroughs in a wide range of fields, including chemistry [1–5], many-body physics [6–8], cryptography [9–12], sensing [13], optimization [14, 15], and simulation [16–20]. Initially introduced as a curious theoretical idea, quantum computing has since exploded into a fast-moving industry, with real quantum devices becoming widely available across various physical platforms. Progress appears almost daily: larger processors, higher gate fidelities, longer qubit coherence times, and more sophisticated algorithmic demonstrations.

However, it must contend with a fundamental issue. The very same quantum effects that give quantum computing its power also make it fragile and susceptible to noise. This noise is unavoidable, so the quantum computer has to constantly deal with errors trying to corrupt its internal state. Examples of such errors are bit flips, phase flips, decoherence, crosstalk, and even external events like cosmic-ray impacts. These errors accumulate over the course of a computation, at every gate, while a qubit idles waiting for the next gate, and at the point of measurement to extract the relevant information from the computation. Without some way to fix this problem, the accumulation of errors destroys the information in the computation, rendering the output useless.

Initially, this appeared to be a devastating obstacle to overcome. While classical computation allows for error correction via the redundant encoding of information by copying it and checking it, the very rules of quantum mechanics forbid this [21]. In particular, unknown quantum states cannot be cloned and the act of measuring a quantum state disturbs the state being protected. Despite this, *quantum error correction* (QEC) resolves this seemingly paradoxical problem. Quantum error-correcting codes encode quantum information distributed across a larger system, which remarkably allows one to detect and correct errors afflicting the state, all without learning or destroying the encoded state itself. Thus, while advancements in materials, control, qubit architecture, cryogenics, and algorithms help in accelerating the field forward, progress towards large-scale, useful quantum computation ultimately depends on quantum error correction.

Today’s devices, despite exciting progress over the past few decades, still exhibit physical qubit error rates on the order of  $10^{-4}$  to  $10^{-3}$  [22]. While a “one-in-a-thousand” or even “one-in-ten-thousand” error rate may sound acceptable, useful large-scale quantum computations may require logical failure rates of  $10^{-9}$  or even less than that, depending on the architecture and algorithm being deployed. This is *five to six orders of magnitude* lower than what has been achieved so far. Barring a major alternative way of achieving large-scale quantum computation, the most promising way forward is through quantum error correction.

We can quantify the physical error rate needed to achieve *fault-tolerant quantum computation* (FTQC) through the *threshold theorem* [23–25]. Informally, this cornerstone theorem guarantees the existence of a physical error rate below which any arbitrarily long quantum computation can be performed reliably by scaling up the number of qubits of the quantum computer. Thus, noise does not need to be eliminated entirely, it only needs to be suppressed to a sufficiently low level. The specific value depends on the quantum error-correcting code being used. For example, the surface code has a threshold value on the order of 0.1% to 1% and can require on the order of 1,000–10,000 *physical qubits per logical qubit* to implement [26]. While this makes the surface code highly attractive for experimental demonstrations of early fault tolerance, quantum computations requiring hundreds or thousands of *logical* qubits would require hundreds of thousands or *millions* of physical qubits, which is out of reach for current devices. Recent progress in promising code constructions like *quantum low-density parity-check* (qLDPC) codes has lowered these numbers significantly [27,

28], but there remains a lot of work to be done.

The tension between the theoretical scalability and the practical, experimental overhead required to realize large-scale quantum computation is one of the primary reasons that quantum error correction remains a popular and active field of research. Over the past several years, there has been vast progress made in both theory, with the introduction of novel code constructions, improvements in code parameters, and decoding algorithms, and experiment, with advancements in the quality of qubits, coherence times, and gate fidelities, and even some early demonstrations of logical error suppression below threshold and small fault-tolerant primitives [29–38]. However, the pace of development can make entry into the field difficult for newcomers and even for seasoned experts to keep track of.

This tutorial is written with this challenge in mind. It aims to bring a fresh, modern perspective on quantum error correction, focusing not only on the basics, but also diving deeper into recent breakthroughs and modern techniques. We take the reader on a journey through the basic formalism of quantum error correction, several promising code constructions, decoding techniques, the limitations of quantum error-correcting codes, and fault tolerance, the pinnacle of quantum error correction. This tutorial is designed to be a comprehensive entry point for those new to quantum error correction and a pedagogical reference for researchers actively working in quantum information more broadly.

## 2 Brief introduction to quantum information

While we assume the reader is familiar with the basics of quantum information and quantum computing, we nonetheless provide a brief overview as a helpful reference if needed and to declare the notation that we use throughout the rest of the tutorial. For a more comprehensive introduction to quantum information and quantum computing, we refer the reader to the many excellent textbooks [39, 40] and lecture notes [41–46].

### 2.1 Quantum states, operators, and measurements

A quantum system is described mathematically by a state vector, which is an element of a complex-valued Hilbert space. Informally, a Hilbert space is a vector space with an inner product, together with a completeness property.<sup>1</sup> For a single qubit, the relevant Hilbert space is two-dimensional. We will use *Dirac notation* for state vectors: a state is written as a *ket*,  $|\psi\rangle$ , and its dual vector is written as a *bra*,  $\langle\psi|$ . The bra is the Hermitian conjugate of the ket, meaning its complex conjugate transpose, that is,  $\langle\psi| = |\psi\rangle^\dagger$ .

Any single-qubit state can be written as a *superposition* of *computational basis* vectors, which are usually labeled as  $|0\rangle$  and  $|1\rangle$ . Formally, a superposition of states is just a linear combination of the state vectors. A general state  $|\psi\rangle$  can thus be written as

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle, \quad (2.1)$$

where  $\alpha, \beta \in \mathbb{C}$ . The state is *physical* if it is normalized, meaning its  $\ell_2$ -norm is equal to one. In that case, the coefficients  $\alpha$  and  $\beta$  are *probability amplitudes*, and their squared magnitudes give the probabilities of obtaining the corresponding outcomes when measuring in the computational basis. Thus, the probability of obtaining outcome 0 is  $|\alpha|^2$  and the probability of obtaining outcome 1 is  $|\beta|^2$ . Since these two outcomes are exhaustive and probabilities must sum to 1, any physical quantum state satisfies

$$|\alpha|^2 + |\beta|^2 = 1. \quad (2.2)$$

The *dual* of the state  $|\psi\rangle$  is given by

$$\langle\psi| = \alpha^* \langle 0| + \beta^* \langle 1|, \quad (2.3)$$

where, recall,  $\alpha$  and  $\beta$  are complex-valued, so we must take their complex conjugate, which we denote with the asterisk. Dirac notation is remarkably convenient for the *inner product* between two vectors  $|\psi\rangle$  and  $|\phi\rangle$ , written as the complex-valued scalar  $\langle\phi|\psi\rangle = \langle\psi|\phi\rangle^*$ . The norm of a vector is obtained from its inner product with itself:

$$\| |\psi\rangle \|_2 = \sqrt{\langle\psi|\psi\rangle}. \quad (2.4)$$

If two vectors  $|\psi\rangle$  and  $|\phi\rangle$  are normalized, that is,

$$\langle\psi|\psi\rangle = \langle\phi|\phi\rangle = 1, \quad (2.5)$$

then the magnitude of their inner product  $|\langle\phi|\psi\rangle|$  quantifies the degree to which these two vectors *overlap*. The normalization enforces that the squared magnitude of the probability amplitudes adds up to 1. If the inner product between two vectors is 0, then we say that the vectors are *orthogonal*.

---

<sup>1</sup>A vector space is complete if every Cauchy sequence in the space converges to a limit that also lies in the space. In particular, every finite-dimensional normed vector space is automatically complete.



The computational basis vectors are both orthogonal and normalized, so they form an *orthonormal set*:

$$\langle 0|1\rangle = \langle 1|0\rangle = 0, \quad (2.6)$$

$$\langle 0|0\rangle = \langle 1|1\rangle = 1. \quad (2.7)$$

If we have more than one qubit, then we naturally have a larger Hilbert space of dimension greater than 2. In particular, if we have  $n$  qubits, each with its own two-dimensional Hilbert space  $\mathcal{H}_i$ , then their joint Hilbert space is the *tensor product* of the  $\mathcal{H}_i$ :

$$\mathcal{H} = \mathcal{H}_1 \otimes \cdots \otimes \mathcal{H}_n \quad (2.8)$$

of dimension  $2^n$ . As  $\mathcal{H}$  is a vector space, any vector  $|\psi\rangle \in \mathcal{H}$  can be written as a linear combination of basis elements of  $\mathcal{H}$ . The *joint computational basis* for  $\mathcal{H}$  is given by the tensor products of the single-qubit computational basis vectors of the form  $|i_1\rangle \otimes \cdots \otimes |i_n\rangle$ , where  $i_j \in \{0, 1\}$  for all  $j$ . In particular, any  $n$ -qubit state can be expanded in the joint computational basis as

$$|\psi\rangle = \sum_{i_1, \dots, i_n \in \{0, 1\}} c_{i_1 \dots i_n} |i_1\rangle \otimes \cdots \otimes |i_n\rangle. \quad (2.9)$$

#### Example 2.1: Two-qubit joint state

A general two-qubit state can be written as a linear combination of the four joint computational-basis states  $|00\rangle, |01\rangle, |10\rangle, |11\rangle$ . As a special case, suppose the state is a *product state*  $|\psi\rangle = |\psi_1\rangle \otimes |\psi_2\rangle$ , where  $|\psi_1\rangle = \alpha'|0\rangle + \beta'|1\rangle$  and  $|\psi_2\rangle = \gamma'|0\rangle + \delta'|1\rangle$ . Then,

$$|\psi\rangle = (\alpha'|0\rangle + \beta'|1\rangle) \otimes (\gamma'|0\rangle + \delta'|1\rangle) \quad (2.10)$$

$$= \alpha'\gamma'|00\rangle + \alpha'\delta'|01\rangle + \beta'\gamma'|10\rangle + \beta'\delta'|11\rangle \quad (2.11)$$

$$= \alpha|00\rangle + \beta|01\rangle + \gamma|10\rangle + \delta|11\rangle, \quad (2.12)$$

where we have introduced the shorthand notation  $|a\rangle \otimes |b\rangle \equiv |ab\rangle$ , which we make use of throughout this tutorial.

#### Exercise 2.1: 2-qubit measurement

Given a 2-qubit state of the form

$$|\psi\rangle = \alpha|00\rangle + \beta|01\rangle + \gamma|10\rangle + \delta|11\rangle, \quad (2.13)$$

what is the probability of measuring  $|\psi\rangle$  in *either* the state  $|00\rangle$  or  $|10\rangle$ ?

For the state vector description used in this section, if a multipartite state can be expressed as  $|\psi\rangle = |\psi_1\rangle \otimes \cdots \otimes |\psi_n\rangle$ , then it is called a *product state*. Pure states that cannot be written in this way are called *entangled states*.<sup>2</sup> Among the most familiar examples of entangled states are the Bell states, introduced in [Example 2.2](#) below. Entanglement is one of the key features that distinguishes quantum mechanics from classical mechanics and, as will become clear throughout this tutorial, it is a central ingredient in quantum error correction.

<sup>2</sup>Pure states are states that are fully described by a single state vector. The more general quantum states are known as *mixed states* and will be introduced in the next chapter.

### Example 2.2: Two-qubit entangled state

Taking the two-qubit state from [Example 2.1](#), if we set  $\beta = \gamma = 0$ , then we have the state

$$|\psi\rangle = \alpha |00\rangle + \delta |11\rangle, \quad (2.14)$$

which cannot be written as a tensor product when both  $\alpha$  and  $\delta$  are nonzero. This is an example of an entangled state. A particularly important special case is the Bell state  $|\Phi^+\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$ .

So far, we have discussed static quantum states. To describe dynamics, and in particular quantum computations, we need transformations of those states. These transformations are described by *operators*, which we denote by capital letters such as  $A$ .<sup>3</sup> Mathematically, operators are matrices acting on states by matrix multiplication. For an  $n$ -qubit state, the relevant operators are  $2^n \times 2^n$  complex-valued matrices acting on the  $2^n$ -dimensional Hilbert space. Operators act on kets from the left and on bras from the right:  $A|\psi\rangle$  and  $\langle\psi|A$ , respectively. We summarize several useful properties of operators in [Box 2.1](#).

### Box 2.1: Properties of operators

Let  $A$ ,  $B$ , and  $C$  be operators. Then, they satisfy the following properties:

- **Associativity:**  $A + (B + C) = (A + B) + C$  and  $A(BC) = (AB)C$
- **Additive commutativity:**  $A + B = B + A$
- **Non-commutativity (in general):**  $AB \neq BA$

The last property above has to do with the *commutator* of two operators, which is given as

$$[A, B] := AB - BA. \quad (2.15)$$

If  $[A, B] = 0$ , that is,  $AB = BA$ , then we say that  $A$  and  $B$  *commute*. If  $AB = -BA$ , then we say that  $A$  and  $B$  *anticommute*. The Hermitian conjugate applies to operators in the following ways:

- **Antidistributivity:**  $(AB)^\dagger = B^\dagger A^\dagger$
- **Antilinearity:**  $(cA)^\dagger = c^* A^\dagger$
- **Involution:**  $(A^\dagger)^\dagger = A$

In particular, if  $A = A^\dagger$ , then we say that  $A$  is *Hermitian*. Hermitian observables have real eigenvalues and, as such, all physical observables in quantum mechanics are Hermitian.

In quantum computation, we often care about *unitary* operators, where an operator is unitary if  $A^{-1} = A^\dagger$ . Equivalently,  $AA^\dagger = A^\dagger A = I$ , where  $I$  is the identity operator. As we mentioned above, operators are matrices and so can be written in a basis using the *outer product*. For example,

<sup>3</sup>Many references also put a “hat” on top of operators, as in  $\hat{A}$ , but because we will be writing many operators, we omit the hats for readability.

we can decompose the operator  $A$  as

$$A = \sum_{m,n} |m\rangle \langle m| A |n\rangle \langle n| = \sum_{m,n} A_{m,n} |m\rangle \langle n|, \quad (2.16)$$

where the matrix elements are  $A_{mn} = \langle m|A|n\rangle$  and the outer products  $|m\rangle \langle n|$  form a basis for the space of operators. Just as we did for state vectors, so too can we tensor together operators to act on individual Hilbert spaces. For example, we can tensor together several single-qubit gates that act individually on a set of qubits. Operators that cannot be decomposed into a tensor product of single-subsystem operators can generate entanglement. One very important example is the CNOT operator, which can be used to entangle two qubits.

Given an operator  $A$ , an important concept is that of its *eigenstates* and *eigenvalues* (e.g., for the stabilizer codes that we will meet in [Section 4](#)). An eigenstate  $|a_i\rangle$  of the operator  $A$  is a state such that  $A|a_i\rangle = a_i|a_i\rangle$ , where  $a_i \in \mathbb{C}$  is called the eigenvalue. If  $A$  is Hermitian (as mentioned in [Box 2.1](#)), then  $A$  has real eigenvalues:  $a_i \in \mathbb{R} \forall i$ . Furthermore, the set of eigenstates of any Hermitian operator constitute its *eigenbasis*, which is also a basis for the space of all states. When written in this basis, the operator becomes diagonal with the matrix elements being its eigenvalues, that is,  $A = \sum_i a_i |a_i\rangle \langle a_i|$ . The set of all eigenstates sharing the same eigenvalue forms a subspace called an *eigenspace*. If multiple orthogonal eigenstates share the same eigenvalue, then the eigenvalue is termed *degenerate*, meaning the choice of an orthonormal basis for this eigenspace is no longer unique. One useful thing to do with the eigenbasis is to take its expectation value with respect to a specific state  $|\psi\rangle$ . If we expand  $|\psi\rangle$  in the eigenbasis of  $A$  as  $|\psi\rangle = \sum_i c_i |a_i\rangle$ , then the expectation value of  $A$  with respect to  $|\psi\rangle$  is

$$\langle \psi | A | \psi \rangle = \sum_i a_i |c_i|^2, \quad (2.17)$$

so the expectation value is the average of the eigenvalues  $a_i$  with respect to the probability distribution  $\{|c_i|^2\}_i$  defined by the state in the eigenbasis of  $A$ , where  $\sum_i |c_i|^2 = 1$  is the normalization condition.

## 2.2 Postulates of quantum mechanics

We now formalize the above notions and condense several themes into a series of postulates, giving us the *postulates of quantum mechanics*. This section follows the presentation in *Quantum Computation and Quantum Information* by Nielsen and Chuang [39]. The first postulate has to do with the state space that we work with in quantum mechanics:

### Postulate 2.1: State space

An isolated quantum system can be fully described by a *state vector*, which is a unit vector in a complex-valued vector space called a *Hilbert space*. A Hilbert space is a complete vector space equipped with an inner product.

As we saw in the previous section, the state space is where the kets and bras live. Mathematically, we wrote  $|\psi\rangle \in \mathcal{H}$ , where  $|\psi\rangle$  is the state vector and  $\mathcal{H}$  is the Hilbert space. The specific state space for a given quantum system is determined by the physics of the system. For example, a qubit has a natural state space of a two-dimensional Hilbert space. Other physical systems, such as photons or atoms, come with different spaces. Of course, quantum states do not just exist statically without

changing; to have interesting dynamics like a quantum computation, the state needs to *evolve*. Evolution of quantum states is the subject of the next postulate:

**Postulate 2.2: Evolution**

Given a closed quantum system described by the state vector  $|\psi\rangle$ , its evolution from time  $t_0$  to time  $t_1$  is described by a unitary operator  $U$ , which depends only on  $t_0$  and  $t_1$ :

$$|\psi(t_1)\rangle = U(t_0; t_1) |\psi(t_0)\rangle . \quad (2.18)$$

For a single qubit, this evolution is described by a unitary operator, which in the language of quantum computing is called a *quantum gate*. We discuss quantum gates and quantum circuits in more detail in the next section. Mathematically, the evolution of the state  $|\psi\rangle$  of a *closed* quantum system, meaning one idealized as isolated from its environment, is described by the *Schrödinger equation*:

$$i\hbar \frac{d}{dt} |\psi\rangle = H |\psi\rangle , \quad (2.19)$$

where  $H$  is called the system's *Hamiltonian* and  $\hbar$  is the reduced Planck's constant. Thus, given an initial state and the Hamiltonian, we can find the state at any given later time. This description applies to *closed* systems. By contrast, an *open* quantum system can interact with external degrees of freedom, so the system of interest cannot in general be described by unitary evolution alone. A widely used model for many such situations is *Lindbladian* dynamics. In this tutorial, we will not need to work directly with these equations very often and will mostly use only unitary gates for evolution of quantum states. At the end of the evolution of our quantum system, we have some final state that encodes information we would like to extract. The only way to access this information is by *measuring* the state, which leads to the next postulate:

**Postulate 2.3: Measurement**

A quantum state  $|\psi\rangle$  can be measured, where the set of measurements are described by a set  $\{M_m\}$  of measurement operators, one for each possible measurement outcome  $m$ . The probability that the specific measurement outcome  $m$  occurs is given by  $p(m) = \langle\psi|M_m^\dagger M_m|\psi\rangle$  and the state of the system right after the measurement is given by

$$|\psi'\rangle = \frac{M_m |\psi\rangle}{\sqrt{\langle\psi|M_m^\dagger M_m|\psi\rangle}} . \quad (2.20)$$

The set of measurements  $\{M_m\}$  is *complete*:

$$\sum_m M_m^\dagger M_m = I , \quad (2.21)$$

which means that one of the possible measurement outcomes described by  $\{M_m\}$  must occur.

This post-measurement update is often described informally as the *collapse of the wavefunction*: after a measurement outcome is obtained, the state is updated to the corresponding post-measurement state. There are several types of quantum measurements. Three notable types are projective measurements, positive operator-valued measures (POVMs), and more general measurement channels. It often suffices to consider projective measurements, and we will introduce more

general formalisms only when needed. A projective measurement is described by *projection operators*  $P$ , which satisfy  $P^2 = P$ . These projectors are the outer products of the observable's eigenstates mapping a superposition onto one of the allowed outcomes. We demonstrate these ideas through the following example:

**Example 2.3:** Single-qubit measurement

Take a qubit in a superposition state

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle, \quad (2.22)$$

where  $|\alpha|^2 + |\beta|^2 = 1$ . The probability of measuring the system in the state  $|0\rangle$  is

$$p(0) = |\langle 0|\psi\rangle|^2 = |\alpha|^2 \quad (2.23)$$

and the probability of measuring the system in the state  $|1\rangle$  is

$$p(1) = |\langle 1|\psi\rangle|^2 = |\beta|^2. \quad (2.24)$$

If, for example, we measured the state in  $|1\rangle$ , then the post-measurement state is

$$|\psi'\rangle = |1\rangle. \quad (2.25)$$

The final postulate tells us how to combine multiple quantum systems into a *composite system*:

**Postulate 2.4:** Composite systems

Independent quantum systems can be combined into a *composite system*, whose state space is described by the tensor product of the state spaces of the component systems.

We saw composite systems in the previous section by way of taking the *tensor product* of individual quantum states. That is, given  $n$  *independent* quantum systems  $|\psi_i\rangle$ , the composite system is described by the following state:

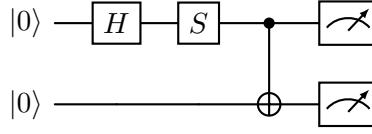
$$|\psi\rangle = |\psi_1\rangle \otimes \dots \otimes |\psi_n\rangle. \quad (2.26)$$

We often shorten the notation by omitting the tensor-product symbols, for example  $|0\rangle \otimes |1\rangle \equiv |01\rangle$ . As we saw in the previous section, some states cannot be written as tensor products of subsystem states. Such states are called *entangled*, and entanglement is one of the key features that gives quantum computation its power.

## 2.3 Quantum circuit model

Fundamentally, a quantum computation is a process that evolves an initial state to a final state from which we hope to extract useful information. We can describe this process using a tool analogous to the Boolean logic circuits of classical computing: the *quantum circuit model*. There are three basic steps in a quantum computation: initialization of the starting state, evolution of the state (i.e., the *quantum algorithm*), and measurement of the final state.

We represent a quantum circuit using a circuit diagram like the one seen in [Figure 1](#). We read the circuit from left to right, where each qubit is assigned to one wire. The quantum algorithm is a sequence of quantum *gates*, which transform the initial state of the qubits to a final state. We then



**Figure 1:** Example of a quantum circuit. The figure illustrates the basic structure of a circuit diagram: wires denote qubits, gates represent unitary operations, and meter symbols indicate measurements.

measure the qubits, represented by the meter symbols in the figure. Gates can act on one, two, or more qubits, though single-qubit and two-qubit gates are the most common and multi-qubit gates can be decomposed into a sequence of simpler gates.

Quantum gates are unitary operators, which we saw above in [Section 2.1](#), and any unitary operator qualifies as a valid quantum gate. However, there are several specific gates that we encounter repeatedly in quantum computing. A very important set of gates, especially for quantum error correction, are the *Pauli gates*. We will formally meet the Pauli group and its constituent matrices in [Section 4.1](#), but we introduce the single-qubit Pauli gates now, of which there are four: the identity matrix  $I$  and the Pauli- $X$ ,  $Y$ , and  $Z$  matrices. In matrix form, these are

$$I = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}, \quad X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \quad (2.27)$$

$$Y = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}, \quad Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}. \quad (2.28)$$

The identity matrix does nothing to a qubit, leaving it in whatever state it started in. The Pauli- $X$  gate acts as a bit flip operator in classical computation, taking the state  $|0\rangle$  to  $|1\rangle$  and *vice versa*. The Pauli- $Z$  gate is a uniquely quantum mechanical operator and is also known as a *phase flip*, where it adds a phase of  $-1$  to the state  $|1\rangle$ , bringing it to  $-|1\rangle$ , while it does nothing to the state  $|0\rangle$ . The Pauli- $Y$  gate in a sense acts as both a bit flip and phase flip, where it brings the state  $|0\rangle$  to  $-i|1\rangle$  and the state  $|1\rangle$  to  $i|0\rangle$ ; in fact, one can calculate that  $Y = iXZ = -iZX$ . Observe that  $I$  commutes with each of  $X, Y, Z$ , but any two of  $X, Y, Z$  anticommute.

Another important quantum gate is the *Hadamard gate*, which is often used to create superposition states. In the computational basis, the matrix representation is

$$H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} \quad (2.29)$$

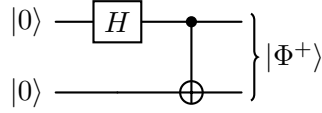
For example, applying the Hadamard gate to the state  $|0\rangle$  brings it to the superposition state  $\frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$ . This is the “plus state,” denoted  $|+\rangle$ , so the Hadamard gate can be viewed as a basis transformation between the computational ( $Z$ ) basis and the  $X$  basis.

### Exercise 2.2:

Show by direct matrix-vector calculation that the Hadamard matrix applied to the computational basis state

$$|0\rangle = \begin{pmatrix} 1 \\ 0 \end{pmatrix} \quad (2.30)$$

produces the state  $|+\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$ .



**Figure 2:** Quantum circuit that prepares the two-qubit Bell state  $|\Phi^+\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$ .

Other important gates include the phase or  $S$  gate and the  $\pi/8$  or  $T$  gate, which are given by

$$S = \begin{pmatrix} 1 & 0 \\ 0 & i \end{pmatrix}, \quad T = \begin{pmatrix} 1 & 0 \\ 0 & e^{i\pi/4} \end{pmatrix}. \quad (2.31)$$

There are also important two-qubit gates, in particular the *controlled* gates. In a controlled gate, there are two qubits, one called the *control* and the other called the *target*, where the gate is applied to the target qubit only if the control qubit is in a particular state. Common examples include the CNOT and CZ gates, which apply the  $X$  gate or  $Z$  gate, respectively, only if the control qubit is in a particular state, usually  $|1\rangle$ . In matrix form, these gates are

$$\text{CNOT} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}, \quad \text{CZ} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & -1 \end{pmatrix}. \quad (2.32)$$

Such gates can be used to create *entangled* states and are thus very important for quantum computation. For example, in [Figure 2](#), we use a CNOT gate to create a Bell state, a useful two-qubit entangled state.

### Exercise 2.3: A simple entangling circuit

Determine the state prepared by the circuit in [Figure 1](#) immediately before measurement. What probabilities would you expect if the two qubits are then measured in the computational basis?

The concepts in this chapter will appear repeatedly throughout the rest of the tutorial. Quantum codes are subspaces of larger Hilbert spaces, encoding and recovery are implemented by circuits and operators, and error diagnosis relies on carefully chosen measurements. In particular, the idea of extracting information about an error without learning the encoded quantum state is one of the central themes of quantum error correction.

### 3 Fundamentals of classical and quantum error correction

We now shift our attention to error correction. As we saw in the previous chapter, quantum information is powerful because it can be stored in superpositions, take advantage of entanglement, and transformed by coherent quantum dynamics, but this is also precisely what makes it fragile. While we would like our quantum states to remain isolated from the external environment, this is impossible in practice. Realistic quantum systems experience undesired interactions with the environment, imperfect gates and measurements, and errors in state preparation such that long quantum computations become challenging to perform reliably.

As such, we need a way to protect quantum states and information. This is exactly where error correction comes in. The basic idea of QEC is to encode quantum information into a larger quantum system such that physical errors can be detected and appropriately corrected. By addressing these physical errors, we can thus avoid incurring *logical* errors, which would corrupt our computation and final results. Before getting to the rich field of QEC, though, we turn to classical error correction, from which many ideas like redundancy, codespaces, distances, and logical operations, can be extended to the quantum setting. In particular, we will see how some of these ideas can be directly extended in the quantum regime, but also how many cannot, where unique quantum mechanical effects like the impossibility of copying quantum information, the disturbing of quantum states via measurement, and other errors like phase-flip errors greatly enrich the QEC program.

#### 3.1 Classical error correction

In classical computation [47], we encode information using *bits*. A bit is an abstract concept that takes one of two values (or *states*), where two is the minimal number of possible states needed for information processing. Bits can be represented by the two sides of a coin, the on/off settings of a light switch, the low/high voltages of a transistor, or the elements of the binary field  $\mathbb{F}_2$  (i.e., the numbers 0 and 1), to name a few. If we (the *sender*) wish to transmit a message to someone (the *receiver*) over a communication channel, we can write that message in *binary*, that is, as a series of 0's and 1's according to some mapping in which letters, numbers, punctuation, and other symbols each have a unique binary representation. Once the receiver gets the binary message, they can use the mapping to recover the original symbols and read the message. However, no communication channel is perfect, and there is some (usually small) probability that one or more transmitted bits will flip. While this may be acceptable for a few bits representing letters, if we are trying to send numerical information, even one bit-flip can dramatically change the value we are trying to send. We will assume that we have already reduced the bit-flip rate in the communication channel as much as possible and now seek mechanisms to *detect* and *correct* the remaining errors. Those mechanisms are, of course, what error correction provides.

The field of error correction [48–50] was initiated by Richard Hamming, who introduced [51] the 7-bit [Hamming code](#) <sup>Ec</sup><sub>200</sub> in 1947.<sup>4</sup> Shortly after, Claude Shannon incorporated error-correcting codes into his mathematical theory of communication in 1948 [52]. The Hamming code, the [repetition code](#) <sup>Ec</sup><sub>200</sub>, and many other classical error-correcting codes belong to the class of [linear codes](#) <sup>Ec</sup><sub>200</sub>. Other important examples include [Reed-Solomon codes](#) <sup>Ec</sup><sub>200</sub>, [low-density parity-check](#) <sup>Ec</sup><sub>200</sub> (LDPC) codes, and [turbo codes](#) <sup>Ec</sup><sub>200</sub>. Linear codes have many useful properties, and we will see their quantum analogue in the form of *stabilizer codes* below. [Stabilizer codes](#) <sup>Ec</sup><sub>200</sub> are one of the most important classes of quantum error-correcting codes.

---

<sup>4</sup>Throughout this tutorial, when referring to specific codes, we link to the corresponding entry in the Error Correction Zoo with an “EC Zoo” symbol <sup>Ec</sup><sub>200</sub>.



Today, classical error-correcting codes are used across many important technologies, several of which we encounter in our everyday lives. These include cellular networks, fiber-optic communication, QR codes, Wi-Fi, flash memory, hard drives, random-access memory (RAM), and even deep space telecommunication systems like those on the Voyager 2 spacecraft. Without error-correcting codes, many of these technologies would be very difficult or even impossible to realize at a practical scale.

### 3.1.1 Classical repetition code

The central idea behind the Hamming code and many others is *redundancy*, which is especially easy to see in *repetition codes*. This is perhaps the most basic family of classical error-correcting codes. At this stage, we only need to consider a single kind of error: the flipping of a bit from “0” to “1” or *vice versa*. This is called a *bit-flip error*. The repetition code works exactly as one would expect, where we just repeat the message that we want to send over a noisy channel. We do this already, for example, when we are talking to someone in a noisy room, where if the person does not hear us, we simply repeat whatever we are trying to tell them. With enough repetitions (hopefully not more than 2!), the person should have heard enough of the noisy messages to piece together the true message we are trying to send them.

To formalize these ideas a bit, consider a message  $m$  consisting of several bits. In the three-bit repetition code, each logical bit is encoded using three physical bits: 0 is encoded as  $\bar{0} = 000$  and 1 as  $\bar{1} = 111$ . Here, the overline denotes the *logical* bit, that is, the underlying “true” information we wish to protect.

#### Example 3.1: Three-bit repetition code

Imagine we wish to send the number 24 as the message. This message in binary is  $m = 11000$ . To encode it in a three-bit repetition code, we encode each bit in  $m$  using three bits, so that the actual message that we send over the noisy channel is

$$m \rightarrow \underbrace{111}_1 \underbrace{111}_1 \underbrace{000}_0 \underbrace{000}_0 \underbrace{000}_0. \quad (3.1)$$

We call the three-bit blocks encoding each logical bit a *codeword*. Thus, the three-bit repetition code has two codewords: 000 and 111. A bit-flip error results in the flip of one or more of the bits. For example, a single bit-flip error can occur in either the first, second, or third bit, resulting in:

$$\bar{0} = 000 \rightarrow \begin{cases} 100 \\ 010 \\ 001 \end{cases} \quad (3.2)$$

$$\bar{1} = 111 \rightarrow \begin{cases} 011 \\ 101 \\ 110 \end{cases}. \quad (3.3)$$

The first step in error correction is *error detection*, that is, determining whether an error has occurred. The next step is *decoding*: inferring the most likely error from the received word. For the three-bit repetition code, if any received codeword is not all 0’s or all 1’s, then we detect that an error has occurred. Decoding can be done by taking a majority vote, which tells us which logical bit was most likely sent. For example, if we receive the codeword 011, then because there are more

1's than 0's, we infer that the first bit-flipped. We then correct the error by flipping that bit back, recovering 111, which encodes the logical bit 1.

When discussing error-correcting codes, an important concept is the *code distance*, which is the smallest number of bit-flips required to transform one codeword into another. Sticking with the three-bit repetition code, this means the smallest number of bit-flips required to transform 000 into 111, or *vice versa*, which is three. Thus, we say that the three-bit repetition code has distance  $d = 3$ . In general, an  $n$ -bit repetition code has distance  $n$ . We can formalize this idea using the *Hamming distance*, which will be useful throughout this tutorial. The Hamming distance  $d_H(a, b)$  between two  $n$ -bit strings  $a, b \in \{0, 1\}^n$  is the minimum number of bit-flips required to transform  $a$  into  $b$ . The code distance of a classical code is the minimum Hamming distance between any two distinct codewords. Thus, the code distance for the three-bit repetition code is the Hamming distance between its two codewords:  $d = d_H(000, 111) = 3$ .

The code distance determines two important quantities. The first one is the maximum number of errors that can be detected, which is  $d - 1$  for a code with distance  $d$ . The second is the maximum number of errors that can be corrected, which is  $\lfloor \frac{d-1}{2} \rfloor$ , where  $\lfloor \cdot \rfloor$  is the floor operator. Naturally, we want a high code distance! We often summarize a classical code that encodes  $k$  logical bits into  $n$  physical bits and has distance  $d$  by its *code parameters*  $[n, k, d]$ . For example, the three-bit repetition code is a  $[3, 1, 3]$  code.

We need to actually do something with our encoded data in order to perform a computation, and we can do so using a set of *logical operators*, which transform the logical bits. For example, the logical NOT operator flips a logical bit by flipping each of the physical bits:

$$\text{NOT}(\bar{0}) = \text{NOT}(000) = 111 \equiv \bar{1} \quad (3.4)$$

$$\text{NOT}(\bar{1}) = \text{NOT}(111) = 000 \equiv \bar{0}. \quad (3.5)$$

Logical operations (or logical *gates*) are an important part of developing a quantum error-correcting code.

### 3.1.2 Classical linear codes

The three-bit repetition code is the simplest example of a broader and very important class of classical codes: *linear codes*. We focus on *binary* linear codes. A binary linear code is a subspace  $\mathcal{C} \subseteq \mathbb{F}_2^n$ , where  $\mathbb{F}_2^n$  is the  $n$ -dimensional vector space over the field  $\mathbb{F}_2$  (equivalently,  $\{0, 1\}^n$  with bitwise addition modulo 2). Thus, if  $u, v \in \mathcal{C}$ , then  $u + v \in \mathcal{C}$  as well. The elements of  $\mathcal{C}$  are called *codewords*. In particular, taking  $u = v$  shows that  $u + u = 0^n$  must belong to every binary linear code.

There are three common ways to describe a linear code. The most direct is simply to list all of its codewords, although this quickly becomes impractical for larger codes. The second and more compact description is to specify a set of generators. We say that  $u_1, \dots, u_m \in \mathbb{F}_2^n$  *generate* the code  $\mathcal{C}$  if

$$\mathcal{C} = \{\alpha_1 u_1 + \dots + \alpha_m u_m \mid \alpha_1, \dots, \alpha_m \in \mathbb{F}_2\}. \quad (3.6)$$

If the generators are linearly independent, then they form a minimal generating set, and it is often convenient to arrange them as the rows of a matrix  $G$ , called the *generator matrix*. The generator description is also useful for understanding the code distance. Since a linear code contains the all-zero codeword, the distance of a binary linear code is equal to the minimum *Hamming weight* of any nonzero codeword, where the Hamming weight of a bit string is the number of 1's it contains. Indeed, if  $a$  and  $b$  are codewords, then  $a + b$  is also a codeword, and its Hamming weight is exactly the Hamming distance  $d_H(a, b)$  that we introduced above. Thus, to compute the distance, it suffices to

examine the nonzero words generated by  $G$ . For the three-bit repetition code, the generator matrix is simply  $G = (111)$ , so the only nonzero codeword is 111, which has Hamming weight 3, and hence the code distance is  $d = 3$ .

The third kind of description is in terms of *parity checks*. A set of parity checks is a collection of vectors  $v_1, \dots, v_r \in \mathbb{F}_2^n$  such that

$$\mathcal{C} = \{u \in \mathbb{F}_2^n \mid u \cdot v_1 = \dots = u \cdot v_r = 0\}, \quad (3.7)$$

where the dot product is taken over  $\mathbb{F}_2$ . In other words, a string belongs to the code exactly when it satisfies all parity checks. Arranging these parity checks as the rows of a matrix  $H$  gives the *parity-check matrix*. The linear span of these rows form the *dual code*, denoted  $\mathcal{C}^\perp$ , which consists of all vectors orthogonal to  $\mathcal{C}$ . Thus,  $\mathcal{C}^\perp$  is exactly the dual space of the codespace, and  $GH^\top = 0$ . If a code over  $\mathbb{F}_2^n$  has  $m$  independent generators and  $r$  independent parity checks, then it follows from the rank-nullity theorem [53] that  $n = m + r$ . Thus, the code encodes  $m$  bits and has  $2^m = 2^{n-r}$  codewords.

The parity-check matrix also gives a compact way to detect and diagnose errors. Given a received word  $w \in \mathbb{F}_2^n$ , we define its *syndrome* by

$$s = Hw^\top \in \mathbb{F}_2^r. \quad (3.8)$$

If  $w$  is a valid codeword, then  $s = 0$  because it satisfies all parity checks. A nonzero syndrome indicates that an error has occurred, and different syndromes can be used to infer different likely error patterns. For the three-bit repetition code, a convenient parity-check matrix is

$$H = \begin{pmatrix} 1 & 1 & 0 \\ 1 & 0 & 1 \end{pmatrix}. \quad (3.9)$$

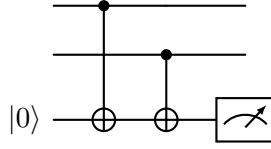
Its two rows simply check whether the first bit agrees with the second and third bits, respectively. Accordingly, both codewords satisfy  $H(000)^\top = 0$  and  $H(111)^\top = 0$ . If instead we receive, for example, the corrupted word 011, then

$$H(011)^\top = \begin{pmatrix} 1 \\ 1 \end{pmatrix}, \quad (3.10)$$

which tells us that the first bit disagrees with both of the others. The most likely error is therefore a flip of the first bit, and we decode by flipping it back to recover 111. In this way, syndrome decoding gives a compact algebraic version of the majority-vote rule. We will return to generator and parity-check descriptions in [Section 4.4](#), where they provide the classical input data for quantum CSS codes.

### 3.2 Quantum repetition code

We can naturally extend the classical repetition code to qubits and get the *quantum repetition code* [Eq. 2.60](#). However, there is an immediate obstacle in the quantum setting. Classically, repetition works by copying the information several times, but for an unknown quantum state, this is impossible in general because of the *no-cloning theorem*: there is no physical operation that takes an arbitrary state  $|\psi\rangle$  to  $|\psi\rangle|\psi\rangle$ . Instead, a quantum repetition code stores redundancy in correlations among several qubits rather than in independent copies of the original state. With that distinction in mind, we now introduce the three-qubit repetition code, where the extension to more than three qubits



**Figure 3:** Quantum circuit for measuring the joint parity of two qubits. Ancilla measurement of 1 implies odd parity, while 0 implies even parity.

is straightforward. We start by defining the codewords, which are the same codewords as in the classical repetition code but written as kets:

$$|\bar{0}\rangle = |000\rangle \text{ and } |\bar{1}\rangle = |111\rangle. \quad (3.11)$$

Here,  $|\bar{0}\rangle$  and  $|\bar{1}\rangle$  are the *logical computational basis states* of the encoded *logical qubit*, and the error-free encoded state lives in the *codespace* that they span. That is, we can write a general single-qubit quantum state  $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$  encoded into the three-qubit repetition code as

$$|\bar{\psi}\rangle = \alpha|\bar{0}\rangle + \beta|\bar{1}\rangle \quad (3.12)$$

$$= \alpha|000\rangle + \beta|111\rangle. \quad (3.13)$$

In the classical repetition code, the only error we needed to consider was a bit-flip. For quantum states, however, we must also contend with *phase-flip errors*, which have no classical analogue. We represent bit-flip and phase-flip errors using the Pauli operators  $X$  and  $Z$ , respectively, introduced in [Section 2.3](#). Their actions on the computational basis are

$$X|0\rangle = |1\rangle, \quad X|1\rangle = |0\rangle, \quad (3.14)$$

$$Z|0\rangle = |0\rangle, \quad Z|1\rangle = -|1\rangle. \quad (3.15)$$

Correcting these two errors turns out to be sufficient for correcting more general errors. We will formalize this statement later in this tutorial.

Just as in the classical repetition code, the first step in QEC is *error detection*. We again consider bit-flip errors in the three-qubit repetition code. However, for encoded quantum states we cannot simply inspect the individual qubits to determine which one was flipped. Doing so would reveal information about the encoded state and collapse the superposition (remember [Postulate 2.3](#)), destroying the quantum information we are trying to protect.

To get around this, we perform carefully chosen measurements that reveal information about the error while leaving the encoded information intact. A basic example is a *parity measurement* on a pair of qubits. There are two possible outcomes: even parity (i.e., 00 or 11) and odd parity (i.e., 01 or 10). This measurement reveals only whether two qubits are the same or different, rather than the value of either qubit individually. The state is therefore projected onto either the even-parity subspace spanned by  $|00\rangle$  and  $|11\rangle$  or the odd-parity subspace spanned by  $|01\rangle$  and  $|10\rangle$ . If the encoded state already lies entirely within one of these parity sectors, then the measurement does not disturb the logical information.

In practice, such measurements are often implemented using *ancilla qubits*, which is one reason QEC requires more overhead than its classical counterpart. A quantum circuit to measure the parity of the first two qubits of the encoded state using the three-qubit repetition code is given in [Figure 3](#). Note that the parity measurement circuit is equivalent to measuring the operator  $Z \otimes Z \otimes I$  on the

| Error                   | Syndrome                |                         |                         |
|-------------------------|-------------------------|-------------------------|-------------------------|
|                         | $Z \otimes Z \otimes I$ | $Z \otimes I \otimes Z$ | $I \otimes Z \otimes Z$ |
| $X \otimes I \otimes I$ | -1                      | -1                      | +1                      |
| $I \otimes X \otimes I$ | -1                      | +1                      | -1                      |
| $I \otimes I \otimes X$ | +1                      | -1                      | -1                      |

Table 1: Syndrome measurements for single-qubit bit-flip error.

first two qubits:

$$Z \otimes Z \otimes I |000\rangle = |000\rangle \quad (3.16)$$

$$Z \otimes Z \otimes I |010\rangle = -|010\rangle \quad (3.17)$$

$$Z \otimes Z \otimes I |100\rangle = -|100\rangle \quad (3.18)$$

$$Z \otimes Z \otimes I |110\rangle = |110\rangle . \quad (3.19)$$

Measurement of a +1 eigenvalue implies even parity, while a -1 eigenvalue implies odd parity. We can similarly probe the other pairs of qubits by measuring  $Z \otimes I \otimes Z$  and  $I \otimes Z \otimes Z$ . These are examples of *syndrome measurements*: their outcomes help us decode which error occurred. We will often use the shorthand  $A \otimes B \otimes C \equiv ABC$ , which is *not* matrix multiplication of  $A$ ,  $B$ , and  $C$ . Suppose an error  $E$  occurs on the encoded state; for bit-flip errors, the possibilities are  $XII$ ,  $IXI$ , and  $IIX$ . If  $E$  commutes with a syndrome operator, the corresponding measurement yields +1, whereas if  $E$  anticommutes with it, the outcome is -1; we characterize all of these in [Table 1](#). Once the error has been identified, we correct it by applying its inverse, which for Pauli operators is the same operator again.

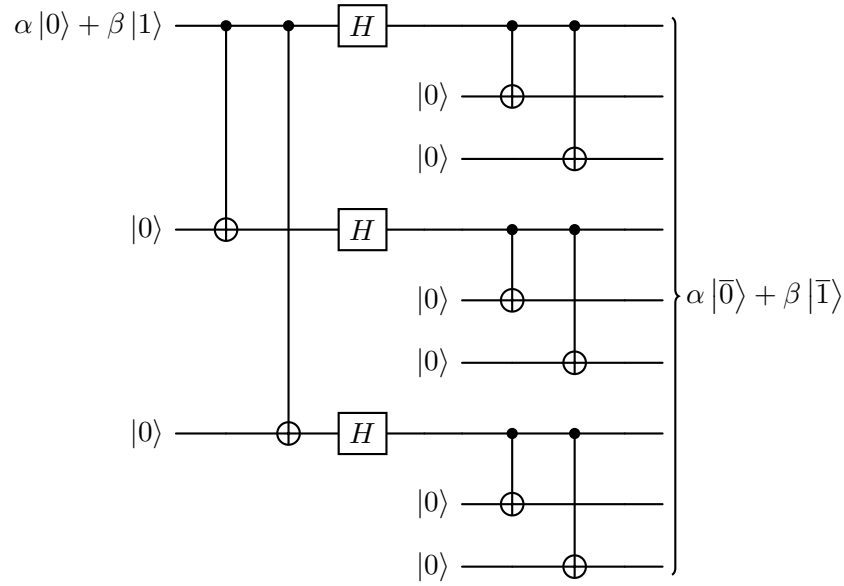
Encoded quantum information is useful only if we can also manipulate it. This is done using *logical operators*, which act on encoded states in the same way that ordinary gates act on unencoded qubits. For example, for the three-qubit repetition code, we need to be able to implement logical  $X$  and logical  $Z$  operations, meaning we need  $\bar{X}$  and  $\bar{Z}$  with the action

$$\bar{X} : |\bar{0}\rangle \rightarrow |\bar{1}\rangle \text{ and } |\bar{1}\rangle \rightarrow |\bar{0}\rangle , \quad (3.20)$$

$$\bar{Z} : |\bar{0}\rangle \rightarrow |\bar{0}\rangle \text{ and } |\bar{1}\rangle \rightarrow -|\bar{1}\rangle . \quad (3.21)$$

Let us start with logical  $X$  operations, written as  $\bar{X}$ . Recall that the logical states are encoded as  $|\bar{0}\rangle = |000\rangle$  and  $|\bar{1}\rangle = |111\rangle$ . It is easy to see that  $\bar{X} = X \otimes X \otimes X \equiv XXX$  takes  $|\bar{0}\rangle$  to  $|\bar{1}\rangle$  and *vice versa*. Thus, it has the desired logical action. For the logical  $Z$  operator, written  $\bar{Z}$ , one can choose  $\bar{Z} = Z \otimes I \otimes I \equiv ZII$ . In the next chapter we will see that logical operators are not unique: multiplying a logical operator by a stabilizer produces another operator with the same logical action. For example,  $ZZZ$ ,  $IIZ$ , and  $IZI$  all have the same logical action as  $ZII$  on this code.

The final notion we consider in this section is the code distance, which we already encountered for the classical repetition code. For a quantum code, we first define the *Pauli weight* of an operator as the number of qubits on which it acts nontrivially, that is, with a non-identity operator. For example, the weight of  $XII$  is 1, while the weight of  $ZIZ$  is 2. The distance of a quantum code is the minimum Pauli weight of any nontrivial logical operator. For the three-qubit repetition code, it is useful to distinguish the  $X$ -type and  $Z$ -type distances, denoted  $d_X$  and  $d_Z$ , respectively. The logical  $X$  operator is  $XXX$ , so  $d_X = 3$ , which means the code can detect up to  $d_X - 1 = 2$  bit-flip



**Figure 4:** Quantum circuit for encoding a single qubit into the nine-qubit Shor code.

errors and correct up to  $\lfloor (d_X - 1)/2 \rfloor = 1$  bit-flip error. By contrast, the logical  $Z$  operator can be implemented by any of  $ZII$ ,  $IZI$ ,  $IIZ$ , or  $ZZZ$ , where the minimum weight is 1 and hence  $d_Z = 1$ , meaning that the code cannot detect or correct any phase-flip errors. The overall code distance is therefore

$$d = \min\{d_X, d_Z\} = 1. \quad (3.22)$$

Thus, although the three-qubit repetition code protects against bit-flips, it does not protect against phase-flips. To handle both error types, we need more sophisticated QEC codes.

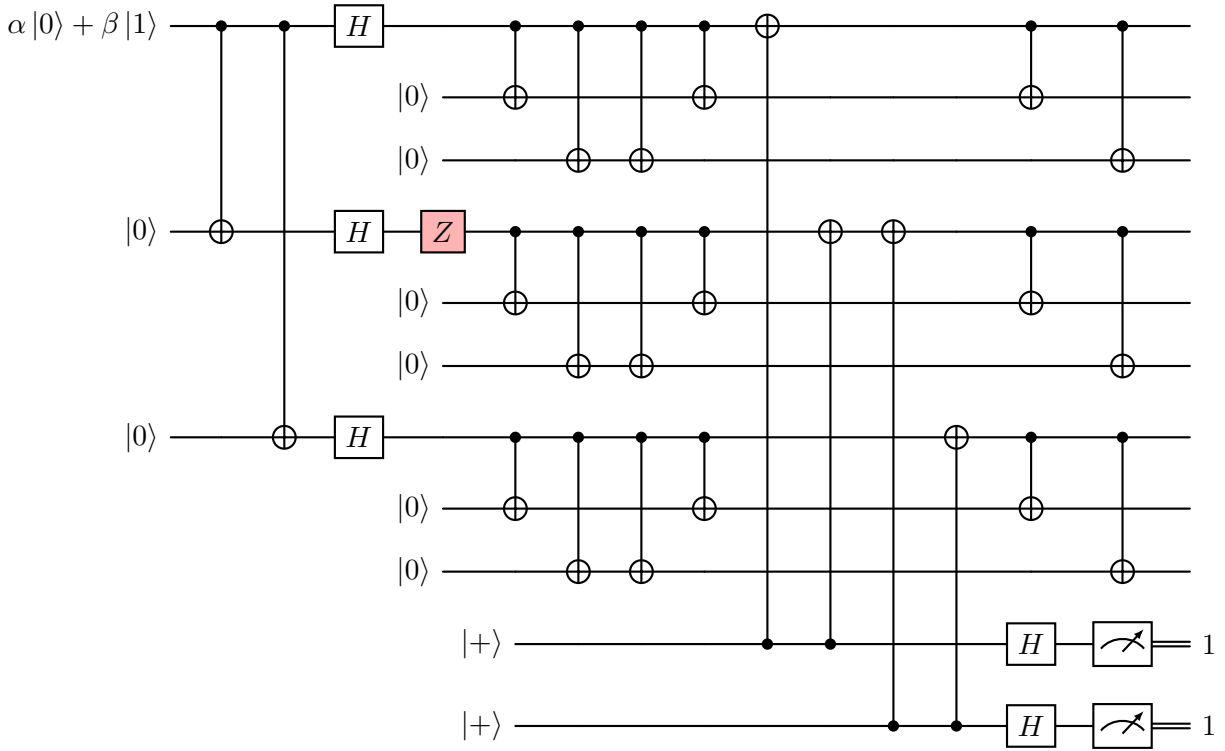
Before presenting such a code, we note that we can use a modified repetition code to detect phase-flip errors at the cost of not being able to detect bit-flip errors. The change is quite simple: we take the encoding circuit for the three-qubit repetition code for detecting bit-flip errors and we simply apply a Hadamard gate to all three qubits after the CNOT gates. This is just a basis change, and so the logical qubit that we are encoding into our three-qubit repetition code is now

$$|\bar{\psi}\rangle = \alpha |+++\rangle + \beta |--\rangle, \quad (3.23)$$

with the basis code states  $HHH|000\rangle = |+++\rangle$  and  $HHH|111\rangle = |--\rangle$ . Then, a phase-flip error on a single qubit takes the state from  $|+\rangle$  to  $|-\rangle$ , or *vice versa*. The error detection and correction is then the same as it is for the bit-flip error three-qubit repetition code, just with the syndrome operators conjugated by Hadamard gates on each qubit.

### 3.3 Nine-qubit Shor code

We saw in the previous subsection that the three-qubit repetition code can only correct one type of error at a time, so we need a quantum error-correcting code that can correct both bit-flip and phase-flip errors. The first code proposed to do just this was the nine-qubit [Shor code](#) <sup>Ec 200</sup>, which was introduced by Peter Shor in 1995 [54]. The Shor code is a  $[[9, 1, 3]]$  code, meaning it encodes 1 logical qubit into 9 physical qubits and has a distance of 3, such that it can detect any two-qubit



**Figure 5:** Quantum circuit illustrating a single  $Z$  error on the middle block of the nine-qubit Shor code, followed by a syndrome measurement that identifies which block the error occurred in. The syndrome outcome 11 indicates that the  $Z$  error occurred on the middle block of three qubits. The full syndrome-extraction circuit is larger and measures all relevant  $X$ - and  $Z$ -type syndromes; here we show only the part relevant to the illustrated  $Z$  error.

error and correct any single-qubit error. The logical codewords for the Shor code are

$$|\bar{0}\rangle = \frac{1}{2\sqrt{2}} (|000\rangle + |111\rangle)^{\otimes 3} \text{ and } |\bar{1}\rangle = \frac{1}{2\sqrt{2}} (|000\rangle - |111\rangle)^{\otimes 3}, \quad (3.24)$$

which is obtained by combining two simpler codes: the three-qubit repetition code for bit-flip errors and the modified version of that code for phase-flip errors. The process of combining existing codes in this way is called *concatenation*, and we will see several examples of it throughout this tutorial. At the first level of the code, which we call the *outer code*, we encode one qubit into three qubits. At the second level, which we call the *inner code*, we encode each of those three qubits into three more qubits, giving a total of nine qubits. Each layer corrects a different type of error, either bit-flips or phase-flips, and this can be done in either order. To be concrete, we take the outer code to be the modified three-qubit repetition code for phase-flip errors and the inner code to be the original three-qubit repetition code for bit-flip errors. The circuit diagram for this encoding is given in Figure 4.

Using this encoding, we can write an encoded logical qubit as

$$|\bar{\psi}\rangle = \alpha |\bar{0}\rangle + \beta |\bar{1}\rangle, \quad (3.25)$$

where  $|\bar{0}\rangle$  and  $|\bar{1}\rangle$  are given above. The encoding produces three blocks of qubits, as indicated in Figure 4, where each block is a three-qubit repetition encoding used to protect against bit-flips. Since the single-qubit Pauli operators  $X$ ,  $Y$ , and  $Z$  form a basis for single-qubit errors, and since

$Y$  simply combines both a bit-flip and a phase-flip, it suffices to understand how the code handles bit-flip and phase-flip errors. Here, a bit-flip corresponds to  $X$ , a phase-flip to  $Z$ , and a combined bit-and-phase-flip to  $Y$ .

Let's now see how we can detect and correct a single  $X$  error. First, note that each block encodes a single qubit into three qubits using the usual three-qubit repetition code. Thus, we can treat each of these blocks independently and, within each block, perform the syndrome measurement and  $X$  error correction as described above in [Section 3.2](#). This allows us to detect and correct at most one bit-flip error.

For  $Z$  errors, things are slightly more complicated because of the inner encoding used to handle  $X$  errors. In particular, a single  $Z$  error acting on any one qubit within a given block has the same logical effect as a  $Z$  error on either of the other two qubits in that block. One way to detect and correct such an error is to first decode the inner code, leaving the three qubits of the outer modified repetition code for  $Z$  errors. We then perform the appropriate syndrome measurement and correction, and finally re-encode using the inner code to recover the original nine-qubit encoding. We illustrate a single  $Z$  error and the corresponding syndrome measurement in [Figure 5](#).

### 3.4 Noise channels

So far we have been somewhat vague about what exactly we mean by an “error,” so we now make that notion precise. In particular, we describe how errors act on quantum states and how to model the resulting noisy states. Up to this point, we have described quantum states using state vectors  $|\psi\rangle$ , which are sufficient for fully-known, isolated states, called *pure states*. After noise acts, however, we may only know that the system is in one of several possible pure states with certain classical probabilities. This motivates the *density matrix*, which provides a unified description of both pure and mixed states.

#### Definition 3.1: Density matrix

A *density matrix* (or *density operator*), typically denoted by  $\rho$ , is a positive semidefinite matrix (i.e.,  $\rho \succeq 0$ ) acting on the state space of a quantum system, such that its trace is equal to one ( $\text{Tr}[\rho] = 1$ ). For a statistical ensemble in which the quantum system is in the pure state  $|\psi_i\rangle$  with classical probability  $p_i$ , the density matrix is

$$\rho = \sum_i p_i |\psi_i\rangle\langle\psi_i|, \quad (3.26)$$

where the probabilities satisfy  $p_i \geq 0$  and  $\sum_i p_i = 1$ . Here, the  $p_i$  quantify classical ignorance about which pure state was prepared; they are not the same as Born-rule probabilities extracted from amplitudes inside any one of the states  $|\psi_i\rangle$ .

The two defining properties of a density matrix have clear physical meanings. Positive semidefiniteness guarantees that all measurement probabilities computed from  $\rho$  are nonnegative, while the trace-one condition guarantees that these probabilities sum to one. Thus, density matrices are precisely the operators that can represent physical quantum states. Pure states are the special case  $\rho = |\psi\rangle\langle\psi|$ , while more general density matrices describe *mixed states*, which encode classical uncertainty or correlations with an environment.

Density matrices also let us describe composite systems beyond the pure-state setting. A bipar-



title mixed state  $\rho_{AB}$  is called *separable* if it can be written as

$$\rho_{AB} = \sum_i p_i \rho_A^{(i)} \otimes \rho_B^{(i)}, \quad (3.27)$$

with  $p_i \geq 0$  and  $\sum_i p_i = 1$ . If no such decomposition exists, the state is *entangled*. This distinction will matter below when we explain why quantum channels must remain physical even when the system of interest is entangled with another one.

With this mathematical tool, we can describe noisy dynamics in a unified way. Errors may transform pure states into mixed states, and the resulting evolution of the density matrix  $\rho$  is captured by the formalism of *quantum channels* [55].

### Definition 3.2: Quantum channel

Let  $\mathcal{E}$  be a quantum channel,  $\rho$  (and any  $\rho$  with a subscript) be a quantum state, and let  $c_1$  and  $c_2$  be complex numbers. A quantum channel is a map

$$\mathcal{E} : \mathcal{L}(\mathcal{H}_S) \rightarrow \mathcal{L}(\mathcal{H}_{S'}), \quad (3.28)$$

where the input Hilbert space  $\mathcal{H}_S$  and output Hilbert space  $\mathcal{H}_{S'}$  satisfy the following properties:

1. **Linearity:**  $\mathcal{E}(c_1\rho_1 + c_2\rho_2) = c_1\mathcal{E}(\rho_1) + c_2\mathcal{E}(\rho_2)$ ,
2. **Trace-preservation:**  $\text{Tr}[\mathcal{E}(\rho)] = \text{Tr}[\rho]$ ,
3. **Complete positivity:**  $(\mathcal{E} \otimes \mathcal{I}_R)(\rho_{SR}) \succeq 0$  for every auxiliary system  $R$  and every joint state  $\rho_{SR} \succeq 0$  on  $\mathcal{H}_S \otimes \mathcal{H}_R$ .

$\rho \succeq 0$  means that  $\rho$  is positive semidefinite.

Thus, a quantum channel is a completely positive trace-preserving (CPTP) map. Each part of this definition is needed for physical consistency. Linearity reflects the fact that the evolution of a statistical mixture is the corresponding mixture of the evolved states. Trace preservation guarantees that total probability remains 1. Positivity ensures that valid density matrices are mapped to valid density matrices. Finally, complete positivity is stronger than ordinary positivity: it guarantees that the map remains physical even when the system is part of a larger entangled state. Without complete positivity, a map could appear valid on isolated systems but produce an unphysical output, such as a matrix with negative eigenvalues, when applied to one half of an entangled pair.

While the above definition is mathematically clean, it is not especially convenient for calculations. For that purpose, we use the *Kraus representation*:

### Definition 3.3: Kraus Representation

Any CPTP map can be expressed in the *Kraus representation* (or operator-sum representation) as:

$$\mathcal{E}(\rho) = \sum_i E_i \rho E_i^\dagger, \quad (3.29)$$

where  $\rho$  is a quantum state in a Hilbert space  $\mathcal{H}_S$  and the operators  $\{E_i\}$  are linear operators mapping  $\mathcal{H}_S$  to  $\mathcal{H}_{S'}$ , known as *Kraus operators*. For the map to be trace-preserving, the

Kraus operators must satisfy the completeness relation:

$$\sum_i E_i^\dagger E_i = I_S, \quad (3.30)$$

where  $I_S$  is the identity operator on the input Hilbert space  $\mathcal{H}_S$ .

We practice using this definition in the following exercise:

**Exercise 3.1:** Trace preservation in the Kraus representation

Prove that the map in Eq. (3.29) is trace-preserving, meaning that  $\text{Tr}[\mathcal{E}(\rho)] = \text{Tr}[\rho]$  for all density matrices  $\rho$ , if and only if the Kraus operators satisfy the completeness relation in Eq. (3.30).

[Hint: use the cyclic property of the trace.]

Using this Kraus representation of quantum channels, we introduce our first quantum noise channel, the *dephasing channel*, in the following example.

**Example 3.2:** Dephasing channel

The dephasing channel models the decay of the coherence encoded in a density matrix. The Kraus operators for a single-qubit dephasing channel are

$$E_0 = \sqrt{1-p}I \text{ and } E_1 = \sqrt{p}Z, \quad (3.31)$$

where  $p$  is the probability of a phase-flip occurring. We can check that the completeness relation is satisfied:

$$E_0^\dagger E_0 + E_1^\dagger E_1 = (1-p)I + pZ^2 = I. \quad (3.32)$$

The action of the channel on a density matrix

$$\rho = \begin{pmatrix} \rho_{00} & \rho_{01} \\ \rho_{10} & \rho_{11} \end{pmatrix} \quad (3.33)$$

is

$$\mathcal{E}(\rho) = E_0 \rho E_0^\dagger + E_1 \rho E_1^\dagger \quad (3.34)$$

$$= (1-p)\rho + pZ\rho Z \quad (3.35)$$

$$= \begin{pmatrix} \rho_{00} & (1-2p)\rho_{01} \\ (1-2p)\rho_{10} & \rho_{11} \end{pmatrix}. \quad (3.36)$$

When  $p = \frac{1}{2}$ , this becomes a completely dephasing channel because the off-diagonal elements vanish, meaning that coherence is entirely lost. In experimental settings, dephasing is often used as a simple model for  $T_2$  relaxation, which characterizes the timescale over which phase information between the states  $|0\rangle$  and  $|1\rangle$  decays.

Another common quantum noise channel is the *depolarizing channel*, which we explore in the next example:

**Example 3.3:** Depolarizing channel

The *depolarizing channel* is a noise channel in which a qubit state is replaced by the maximally-mixed state  $\frac{1}{2}I$  with probability  $p$ . The original state is then preserved with probability  $1 - p$ . The Kraus operators are

$$E_0 = \sqrt{1 - \frac{3p}{4}}I, \quad E_i = \frac{\sqrt{p}}{2}\sigma_i \quad \text{for } i \in \{1, 2, 3\} \quad (3.37)$$

The action of the channel on an arbitrary single-qubit state is then

$$\mathcal{E}(\rho) = (1 - p)\rho + p\frac{I}{2} = \begin{pmatrix} (1 - p/2)\rho_{00} & (1 - p)\rho_{01} \\ (1 - p)\rho_{10} & (1 - p/2)\rho_{11} \end{pmatrix}. \quad (3.38)$$

The depolarizing channel models a symmetric loss of information: the state remains unchanged with probability  $1 - \frac{3p}{4}$ , while each of the errors  $X$ ,  $Y$ , and  $Z$  occurs with probability  $\frac{p}{4}$ . When  $p = 1$ , the channel is *completely depolarizing* because every input state is replaced by the maximally mixed state.

The final channel we consider is the *amplitude damping channel*, which also has some energy dissipation in addition to the loss of quantum information:

**Example 3.4:** Amplitude damping channel

Given a single qubit, the *amplitude damping channel* models the process of spontaneous emission, where the excited state  $|1\rangle$  decays to the ground state  $|0\rangle$ . The Kraus operators for this channel are

$$E_0 = \begin{pmatrix} 1 & 0 \\ 0 & \sqrt{1 - p} \end{pmatrix} \quad \text{and} \quad E_1 = \begin{pmatrix} 0 & \sqrt{p} \\ 0 & 0 \end{pmatrix}, \quad (3.39)$$

where  $p$  is the probability that the excited state  $|1\rangle$  decays to the ground state  $|0\rangle$ . We can check that the completeness relation is satisfied:

$$E_0^\dagger E_0 + E_1^\dagger E_1 = I. \quad (3.40)$$

The action on an arbitrary quantum state  $\rho$  is given by

$$\mathcal{E}(\rho) = E_0 \rho E_0^\dagger + E_1 \rho E_1^\dagger \quad (3.41)$$

$$= \begin{pmatrix} \rho_{00} + p\rho_{11} & \sqrt{1 - p}\rho_{01} \\ \sqrt{1 - p}\rho_{10} & (1 - p)\rho_{11} \end{pmatrix}. \quad (3.42)$$

The amplitude damping channel models the  $T_1$  relaxation, also called the longitudinal relaxation.  $T_1$  is thus the timescale of energy loss that occurs when the excited state  $|1\rangle$  decays to the ground state  $|0\rangle$ . In real-world quantum computing platforms, both  $T_1$  and  $T_2$  represent critical performance metrics that directly determine the feasibility and scalability of quantum information processing.

The previous examples all acted on a single qubit, but the same formalism extends naturally to many-qubit systems where one often models the multi-qubit channel by applying the relevant single-qubit channels independently to each qubit. One important family is the *Pauli channel*, defined as follows:

**Definition 3.4:** Pauli channel

An  $n$ -qubit *Pauli channel* is given by

$$\mathcal{E}(\rho) = \sum_i p_i P_i \rho P_i^\dagger, \quad (3.43)$$

where  $P_i$  are tensor products of  $I, X, Y$  and  $Z$ , also called Pauli strings:  $P_i \in \{I, X, Y, Z\}^{\otimes n}$ , and probabilities  $p_i$  with  $\sum_i p_i = 1$

When  $n = 1$ , this family includes familiar examples such as dephasing and depolarizing noise.

**Exercise 3.2:** Single-qubit Pauli channels

Consider the single-qubit version of the Pauli channel in [Definition 3.4](#), where the Pauli operators are  $\{I, X, Y, Z\}$  with probabilities  $\{p_I, p_X, p_Y, p_Z\}$ . Determine these probabilities for the channel to become:

1. The dephasing channel in [Example 3.2](#),
2. The depolarizing channel in [Example 3.3](#).

We conclude this section with the *Knill-Laflamme condition*, which formalizes what errors a quantum code can correct:

**Theorem 3.1:** Knill–Laflamme condition

Let  $P$  be the projector onto the codespace  $\mathcal{C}$ , and let  $\{E_a\}$  be a set of errors. Then,  $\mathcal{C}$  corrects these errors if and only if

$$P E_a^\dagger E_b P = C_{ab} P, \quad (3.44)$$

where  $C_{ab} \in \mathbb{C}$ .

We interpret this to mean that the combined action of any pair of correctable errors is indistinguishable from a scalar multiple of the identity. That is, the error process must not reveal any information about the encoded quantum state.

### 3.5 Brief history of QEC

We conclude this chapter with a brief history of QEC, for those interested in its historical development. QEC was introduced shortly after Peter Shor kick-started the field of quantum computing with his factoring algorithm in 1994 (though finally published in 1997) [\[56\]](#). While the three-qubit bit-flip code had been introduced in 1985 by Asher Peres [\[57\]](#), ten years later, Shor [\[54\]](#) introduced the first QEC code that could correct both bit-flip and phase-flip errors with his nine-qubit code. Shortly after, in 1996, Andrew Steane also pointed out that decoherence is a fundamental aspect of quantum mechanics and so any computation based on quantum mechanical systems must inevitably deal with this problem, which he proposed can be done with his seven-qubit code [\[58\]](#). The years 1996 and 1997 were then quite busy for QEC, where after Steane proposed his seven-qubit code, several other advancements were made. The five-qubit code—the smallest possible QEC code that can protect a single logical qubit against single-qubit errors—was independently proposed by Laflamme, Miquel, Paz, and Zurek [\[59\]](#) and Bennett, DiVincenzo, Smolin, and Wootters [\[60\]](#) in 1996.

A more general framework for these codes was then introduced by Calderbank and Shor [61] and Steane [62] in the form of Calderbank-Shor-Steane (CSS) codes. Further generalizations were then made independently by Daniel Gottesman in his seminal doctoral thesis [63] and by Calderbank, Rains, Shor, and Sloane [61].

Fault-tolerant quantum memory was then proposed by Alexei Kitaev [25], where he also proposed the toric code, still a highly relevant code construction today. Later, Shor again made a major contribution, showing how to do universal logical FTQC without using concatenation [64]. Further details of FTQC were then developed by Kitaev [65]; Cirac, Pellizzari and Zoller [66]; Zurek and Laflamme [67]; DiVincenzo and Shor; and Gottesman [68]. All of this work was then put together independently by Knill, Laflamme, and Zurek [24]; Aharonov and Ben-Or [69]; and Kitaev [25] in 1996 and 1997 in the form of the fault-tolerant threshold theorem, one of the most important results of QEC. The proof for the threshold theorem was subsequently made more rigorous and improved upon by Aliferis, Gottesman, and Preskill in 2005 [70].

The above advancements laid much of the groundwork for QEC upon which further advancements were made in the late 1990s. Beyond the stabilizer formalism, Rains, Hardin, Shor, and Sloane [71] introduced a non-additive (i.e., non-stabilizer) construction in 1997. Also in 1997, Leung, Nielsen, Chuang, and Yamamoto [72] introduced approximate error correction, expanding the somewhat narrow consideration of single-qubit Pauli error channels. All of the above advancements were made for qubits, but of course, quantum systems can have more than two states, so Gottesman [73] and Rains [74] independently introduced formalism to consider higher dimensional QEC codes. Furthermore, in the late 1990s, there was abundant work on quantum analogues of error correction bounds; see, for example, references [75–82].

Over the next few years, in the early 2000s, several other improvements and advancements were made by many researchers. In 2002, Dennis, Kitaev, Landahl, and Preskill [83] studied many details of the toric code, which was introduced by Kitaev a few years earlier. Then, in 2004, Bravyi and Kitaev [84] introduced magic state distillation, still a hot topic of research today. In 2006, Bombin and Delgado [85, 86] proposed a new type of code, the color code. While the surface code was introduced by Kitaev years before, it was popularized in 2012 by Fowler, Mariantoni, Martinis, and Cleland [26].

We stop our early history of QEC there, though that is not to say that this is where the advancements in QEC research stopped. QEC remains one of the fastest growing subfields of quantum information science and quantum computing, and is finally even seeing exciting experimental demonstrations of high quality error correction and early fault tolerance [29–38].

Finally, we highlight several introductory references specific to QEC. The books by Lidar and Brun [87] and Gaitan [88] are well-known references, though much progress has been made in QEC since their publication in 2013. Though the textbook is not yet available in print (at the time of writing), Daniel Gottesman presents a more recent view of QEC in his *Surviving as a Quantum Computer in a Classical World* [89]. There are also several freely available online references, such as lecture notes by Michael Kastoryano [90] and the introductory guide by Joschka Roffe [91]. Finally, the Error Correction Zoo [92] is an invaluable resource for researchers and students alike.

## 4 Stabilizer codes

We now turn to one of the most important and successful classes of quantum error-correcting codes: the *stabilizer codes*, first introduced by Daniel Gottesman (see [Section 3.5](#) for more history). The stabilizer formalism is one of the most useful tools in quantum error correction and several of the codes that we have already encountered can be described using it. We make use of the notions in this chapter, including group theory, the stabilizer formalism, and CSS codes, throughout this tutorial.

### 4.1 Primer on group theory

Before introducing stabilizer codes, we need to understand some basic *group theory* [93]. Group theory is a field of mathematics that studies highly-structured collections of objects equipped with a binary operation satisfying certain properties. Since we primarily focus on discrete errors in this tutorial, this primer deals exclusively with finite groups. We begin with the concept of an *equivalence relation*:

#### Definition 4.1: Equivalence relation

A relation  $\sim$  on a set  $G$  is an *equivalence relation* if it satisfies the following conditions:

1. **Reflexivity:**  $g \sim g$  for all  $g \in G$ .
2. **Symmetry:**  $g \sim g'$  if and only if  $g' \sim g$ .
3. **Transitivity:** if  $g \sim g'$  and  $g' \sim g''$ , then  $g \sim g''$ .

For any element  $g \in G$ , its equivalence class, denoted  $[g]$ , is defined as  $[g] = \{g' \in G : g' \sim g\}$ .

Two equivalence classes are either *disjoint* or *equal*. Consequently, any equivalence relation partitions a set into disjoint equivalence classes. This construction is closely connected to cosets in group theory and will play a crucial role in helping us organize errors and logical operators. With this in mind, we now formally define a group:

#### Definition 4.2: Group

For a set  $G$  and a binary operation  $*$  :  $G \times G \rightarrow G$ , the tuple  $(G, *)$  is a *group* if all of the following hold:

1. **Associativity:**  $(a * b) * c = a * (b * c)$  for any  $a, b, c \in G$ .
2. **Identity:** There exists a unique *identity* element  $e \in G$  such that  $e * a = a * e = a$  for any  $a \in G$ .
3. **Inverse:** For each  $a \in G$  there exists a unique *inverse* element  $b \in G$  such that  $a * b = b * a = e$ . This unique element  $b$  is denoted by  $a^{-1}$ .

In addition, we say that  $a, b \in G$  *commute* if  $a * b = b * a$ . A group  $(G, *)$  in which all  $a, b \in G$  commute is called an *abelian* group; otherwise, it is *non-abelian*.

When it is clear from the context, we simply refer to the group  $(G, *)$  as  $G$ . Also, where appropriate, we suppress the “ $*$ ” operation and write  $a * b$  simply as  $ab$ . For example, when  $*$  is the

multiplication operator,  $\times$ , we write  $ab$  instead of  $a \times b$  for group elements  $a, b \in G$ . Let's look at a few examples of groups, starting with the integers under addition:

**Example 4.1:** Integer addition

The set of integers  $\mathbb{Z} = \{\dots, -2, -1, 0, 1, 2, \dots\}$  under the addition operation  $+$  is an abelian group  $(\mathbb{Z}, +)$  with identity element 0. The inverse element of  $a \in \mathbb{Z}$  is  $-a$  such that  $a + (-a) = (-a) + a = 0$ .

By restricting the integers to a finite set and using modular arithmetic, we can construct an important class of finite abelian groups known as the cyclic groups.

**Example 4.2:** Integers modulo  $n$

The group  $(\mathbb{Z}_n, +)$  (also written as  $\mathbb{Z}/n\mathbb{Z}$ ) represents the integers modulo a positive integer  $n$  under modular addition. For example,  $\mathbb{Z}_2 = \{0, 1\}$  contains only two elements, where the group operation is defined by  $0 + 0 = 0$ ,  $0 + 1 = 1$ , and  $1 + 1 = 0$ .

As a more sophisticated example, we now look at matrix multiplication over real matrices:

**Example 4.3:** Invertible real matrix multiplication

The *general linear group*  $\text{GL}_n(\mathbb{R})$  is the set of  $n \times n$  invertible real matrices under matrix multiplication. Its identity element is the  $n \times n$  identity matrix  $I_n$ , and the inverse of  $A \in \text{GL}_n(\mathbb{R})$  is its matrix inverse  $A^{-1}$ . The group  $\text{GL}_n(\mathbb{R})$  is non-abelian for  $n \geq 2$ . For example, matrices

$$A = \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix} \quad \text{and} \quad B = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix} \quad (4.1)$$

are invertible, so both belong to  $\text{GL}_2(\mathbb{R})$ . However,

$$AB = \begin{bmatrix} 1 & 1 \\ 1 & 2 \end{bmatrix} \neq BA = \begin{bmatrix} 2 & 1 \\ 1 & 1 \end{bmatrix}, \quad (4.2)$$

which means that  $\text{GL}_2(\mathbb{R})$  is a non-abelian group.

Fundamental to the stabilizer formalism and the study of quantum error-correcting codes in general is the concept of a *generator*, which define as:

**Definition 4.3:** Generator

Given a group  $G$ , a subset  $S \subseteq G$  is a *generating set* if every element of  $G$  can be written as a finite product of elements of  $S$  and their inverses. Equivalently,

$$G = \langle S \rangle = \{s_1^{e_1} s_2^{e_2} \cdots s_k^{e_k} : k \geq 0, s_i \in S, e_i \in \mathbb{Z}\}. \quad (4.3)$$

As an example of how to produce a group from a generating set, consider the Pauli group, which we have already seen as an important group in our study of quantum error correction:

**Example 4.4: Pauli group**

The  $n$ -qubit Pauli group  $\mathcal{P}_n$  consists of all  $n$ -fold tensor products of  $I$ ,  $X$ ,  $Y$ , and  $Z$ , together with overall phases  $\pm 1$  and  $\pm i$ , that is,

$$\mathcal{P}_n = \{I, X, Y, Z\}^{\otimes n} \times \{\pm 1, \pm i\}. \quad (4.4)$$

A generating set for  $\mathcal{P}_n$  is

$$\langle i, X_1, Z_1, X_2, Z_2, \dots, X_n, Z_n \rangle, \quad (4.5)$$

where  $X_i$  and  $Z_i$  are the  $n$ -qubit operators that act as  $X$  or  $Z$  on the  $i^{\text{th}}$  qubit and as the identity on all other qubits. Up to an overall phase, Pauli operators are self-inverse:  $P^2 = \pm I$  for any  $P \in \mathcal{P}_n$ . Taking  $n = 2$  as an example, we can check that  $X_1 X_2$  and  $Z_1 X_2$  do not commute, that is,  $(X_1 X_2)(Z_1 X_2) = -(Z_1 X_2)(X_1 X_2)$ , so the Pauli group is overall a non-abelian group.

Another important structure is the *subgroup*. Since stabilizers of a code constitute a particular subgroup of the Pauli group, understanding subgroup properties is essential for stabilizer code theory. We define a subgroup as follows:

**Definition 4.4: Subgroup**

Given a group  $(G, *)$ , a subset  $H \subseteq G$  is a *subgroup* if  $H$  is itself a group under the binary operation induced by  $G$ .

Related is the concept of *cosets*:

**Definition 4.5: Coset**

Let  $G$  be a group and  $H$  a subgroup of  $G$ . For any element  $g \in G$ :

1. The *left coset* of  $H$  containing  $g$  is defined as  $gH = \{gh : h \in H\}$ .
2. The *right coset* of  $H$  containing  $g$  is defined as  $Hg = \{hg : h \in H\}$ .

We can now define normal subgroups and quotient groups:

**Definition 4.6: Normal subgroup and quotient group**

A subgroup  $N$  of a group  $G$  is called *normal* if  $gN = Ng$  for all  $g \in G$ . Equivalently,  $gNg^{-1} = N$  for all  $g \in G$ . If  $H$  is a normal subgroup of  $G$ , then the set of all left cosets

$$G/H = \{gH : g \in G\} \quad (4.6)$$

forms a group under the operation  $(g_1 H) \cdot (g_2 H) = (g_1 g_2) H$ , called the *quotient group* of  $G$  by  $H$ .

If we define an equivalence relation on  $G$  by declaring that  $g \sim g'$  if and only if  $g^{-1}g' \in H$ , then the corresponding equivalence class is

$$[g] = \{x \in G \mid g^{-1}x \in H\}. \quad (4.7)$$



Writing  $g^{-1}x = h$  for some  $h \in H$  gives  $x = gh$ , so

$$[g] = \{gh \mid h \in H\} = gH. \quad (4.8)$$

Thus, the cosets of a subgroup are exactly the equivalence classes induced by that subgroup, and they partition the group into disjoint classes of equal size. In the next section, this viewpoint will help us classify logical operators. Before that, we introduce two related concepts: the normalizer and the centralizer.

**Definition 4.7:** Normalizer and centralizer

Let  $G$  be a group and  $H$  a subgroup of  $G$ . The *centralizer* of  $H$  in  $G$  is

$$C_G(H) = \{g \in G : gh = hg \ \forall h \in H\}. \quad (4.9)$$

It contains all elements of  $G$  that commute with every element of  $H$ . The *normalizer* of  $H$  in  $G$  is

$$N_G(H) = \{g \in G : gHg^{-1} = H\}. \quad (4.10)$$

In general, the centralizer is contained in the normalizer: commuting with every element of  $H$  is a stronger condition than merely preserving  $H$  under conjugation. Besides studying the internal structure of a group, we will also need maps between groups that preserve the group operation. These are called *homomorphisms*:

**Definition 4.8:** Group homomorphism and isomorphism

Let  $(G, *)$  and  $(H, \cdot)$  be groups. A map  $\phi : G \rightarrow H$  is a *group homomorphism* if for all  $g_1, g_2 \in G$ , it satisfies the property:

$$\phi(g_1 * g_2) = \phi(g_1) \cdot \phi(g_2) \quad (4.11)$$

A homomorphism  $\phi$  that is also bijective (both one-to-one and onto) is called a *group isomorphism*. If such a map exists between two groups  $G$  and  $H$ , they are said to be isomorphic, denoted  $G \cong H$ .

We will also find the concept of a *kernel* useful:

**Definition 4.9:** Kernel

Let  $\phi : G \rightarrow H$  be a group homomorphism. The *kernel* of  $\phi$ , defined as  $\ker(\phi) = \{g \in G \mid \phi(g) = e_H\}$ , where  $e_H$  is the identity element in  $H$ , is a normal subgroup of  $G$ .

In words, the kernel consists of those elements of  $G$  that vanish under the map  $\phi$ , in the sense that they are sent to the identity in  $H$ . Since homomorphisms respect the group operation, conjugating such an element yields a pair of inverse elements multiplied together, which still yields the identity. So the kernel is closed under conjugation and is therefore normal.

**Theorem 4.1:** First isomorphism theorem

Let  $\phi : G \rightarrow H$  be a group homomorphism. Then the quotient group  $G/\ker(\phi)$  is isomorphic

to the image of  $\phi$ , which is a subgroup of  $H$ . This is expressed as:

$$G/\ker(\phi) \cong \text{im}(\phi) \quad (4.12)$$

For complete proofs of [Definition 4.9](#) and [Theorem 4.1](#), see [94]. Here we give a simple example motivated by QEC. Suppose we measure the two-qubit parity operator  $ZZ$ . On computational-basis states, this measurement does not tell us the state of the qubits; it only tells us whether the parity is even or odd. In particular, any superposition of  $|00\rangle$  and  $|11\rangle$  produce the same measurement outcome, while that of  $|01\rangle$  and  $|10\rangle$  produce the other outcome. We can model this using a group homomorphism. Let

$$G = \{00, 01, 10, 11\} \quad (4.13)$$

with group operation given by bitwise XOR, and let

$$H = \{0, 1\} \quad (4.14)$$

with addition modulo 2. Define the map  $\phi : G \rightarrow H$  by

$$\phi(00) = \phi(11) = 0, \quad \phi(01) = \phi(10) = 1. \quad (4.15)$$

This map records exactly the outcome of the  $ZZ$  measurement: 0 for even parity and 1 for odd parity. The kernel is

$$\ker(\phi) = \{00, 11\}, \quad (4.16)$$

which is the set of strings with even parity. The other coset is

$$01 + \ker(\phi) = \{01, 10\}, \quad (4.17)$$

which is the set of strings with odd parity. Therefore,

$$G/\ker(\phi) = \{\{00, 11\}, \{01, 10\}\} = \{[00], [01]\}, \quad (4.18)$$

so the quotient group is exactly the set of parity classes.

The induced map  $\bar{\phi} : G/\ker(\phi) \rightarrow \text{im}(\phi)$  defined by  $\bar{\phi}([g]) = \phi(g)$  is bijective, illustrating the first isomorphism theorem in this setting. In the language of quantum error correction, the theorem formalizes the idea that a syndrome measurement groups physical configurations into equivalence classes that produce the same measurement outcome. The syndrome does not identify the exact error, only its class, which is why decoding generally involves choosing a representative correction. This is closely related to the degeneracy of quantum codes. To save resources, we sometimes choose not to decode exactly but choose the most likely logical class and apply a representative correction. More details on decoding will be covered in [Section 9](#).

## 4.2 Stabilizer codes

We are now ready to introduce stabilizer codes and the stabilizer formalism [63, 95]. Many of the most important QEC codes can be described this way, including the Shor code, the Steane code, the five-qubit code, CSS codes, and surface code-type constructions. At its heart, the stabilizer formalism describes a code by the Pauli operators that leave every code state unchanged. We formalize this as follows:

**Definition 4.10:** Stabilizer code

An  $[n, k]$  *stabilizer code* is a quantum error-correcting code  $C \subseteq \mathcal{H}_{2^n}$  that encodes  $k$  logical qubits into  $n$  physical qubits, where the codespace is of dimension  $2^k$  and is defined by the  $+1$  eigenspace of the *stabilizer group*  $\mathcal{S}$ , which is an abelian subgroup of the  $n$ -qubit Pauli group  $\mathcal{P}_n$  (excluding  $-I^{\otimes n}$ ). That is,

$$\mathcal{S}(C) = \{P \in \mathcal{P}_n : P|\psi\rangle = |\psi\rangle, \forall |\psi\rangle \in C\}. \quad (4.19)$$

Thus, the stabilizer group is the set of Pauli operators that leave every code state invariant. We refer to these Pauli operators as *stabilizer operators*, *stabilizer elements*, or simply *stabilizers*. Since the codespace is a simultaneous eigenspace of all stabilizers, the stabilizer group must be abelian. As with any group, we can describe it in terms of generators. The following fact is especially useful:

**Fact 4.1:** Abelian group generators

Let  $G$  be an abelian group of self-inverse operators generated by  $m$  generators  $g_1, \dots, g_m$ . Then any element  $g \in G$  can be written as

$$g = \prod_{i=1}^m g_i^{\alpha_i}, \quad (4.20)$$

where  $\alpha_i \in \{0, 1\}$ .

This fact applies directly to stabilizer codes because stabilizer generators are Pauli operators, which are self-inverse up to a phase. Since stabilizer groups exclude  $-I^{\otimes n}$ , every stabilizer can be written as a product of generators with exponents in  $\{0, 1\}$ , so  $|\mathcal{S}| = 2^m$ . Moreover, if a stabilizer code on  $n$  physical qubits has  $m$  independent generators, then each generator halves the Hilbert-space dimension, so

$$\frac{2^n}{2^m} = 2^k \implies k = n - m, \quad (4.21)$$

directly giving us the number of logical qubits from the knowledge of the size of the stabilizer group. Let us now apply the stabilizer formalism to the three-qubit repetition code. Recall, the codewords are

$$|\bar{0}\rangle = |000\rangle \text{ and } |\bar{1}\rangle = |111\rangle. \quad (4.22)$$

This is a  $[[3, 1, 1]]$  code, so  $n = 3$ ,  $k = 1$ , and hence  $m = n - k = 2$ . Therefore, the stabilizer group has  $|\mathcal{S}| = 2^2 = 4$  elements, namely

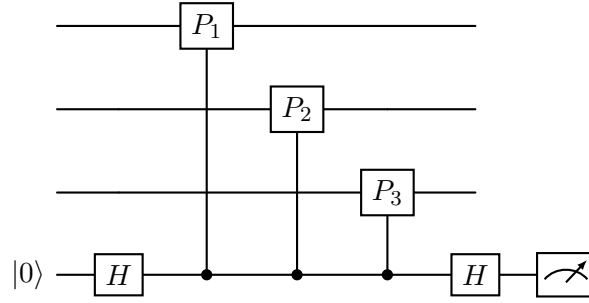
$$\mathcal{S} = \{III, ZIZ, IZZ, ZZI\}. \quad (4.23)$$

Once we know the stabilizers, we can measure them to detect errors and extract a syndrome. An example circuit for measuring the stabilizer  $P_1 \otimes P_2 \otimes P_3$  is shown in [Figure 6](#).

The stabilizer formalism also organizes the logical operators. These are the operators that act on the encoded states in the same way that ordinary Pauli operators act on an unencoded qubit. For the three-qubit repetition code, a logical bit flip and a logical phase flip act as

$$\bar{X}|\bar{0}\rangle = |\bar{1}\rangle, \quad \bar{X}|\bar{1}\rangle = |\bar{0}\rangle \quad (4.24)$$

$$\bar{Z}|\bar{0}\rangle = |\bar{0}\rangle, \quad \bar{Z}|\bar{1}\rangle = -|\bar{1}\rangle. \quad (4.25)$$



**Figure 6:** Example circuit for measuring the stabilizer  $P_1 \otimes P_2 \otimes P_3$ .

One choice of logical operators is  $\{XXX, ZII\}$ . However, logical operators are not unique: multiplying a logical operator by any stabilizer gives another operator with the same action on the codespace. For example, multiplying  $ZII$  by the stabilizer  $ZIZ$  gives  $IIZ$ , which is also a valid logical  $Z$ . More generally, if  $|\psi\rangle$  lies in the codespace and  $S_j \in \mathcal{S}$ , then  $S_j |\psi\rangle = |\psi\rangle$ , so

$$LS_j |\psi\rangle = L |\psi\rangle. \quad (4.26)$$

Thus, every logical operator  $L$  comes with an entire equivalence class  $\{LS_j : S_j \in \mathcal{S}\}$  of physically equivalent representatives.

Finally, we can describe logical operators and code distance in group-theoretic terms. The logical Pauli operators are the Pauli operators that preserve the codespace but are not themselves stabilizers, so they form the set  $N(\mathcal{S}) \setminus \mathcal{S}$ , where  $A \setminus B$  denotes the set of elements of  $A$  that are not in  $B$ . For Pauli stabilizer groups, this is the same as  $C(\mathcal{S}) \setminus \mathcal{S}$ . The reason is that Pauli conjugation can only introduce a sign, and a stabilizer group never contains  $-I$ ; thus a Pauli operator cannot normalize  $\mathcal{S}$  without actually commuting with every stabilizer. As a result, the centralizer and the normalizer of any stabilizer group coincide. The distance of a stabilizer code is therefore the minimum weight of any element of  $N(\mathcal{S}) \setminus \mathcal{S}$ . For the three-qubit repetition code, this minimum is 1, since  $ZII$ ,  $IZI$ , and  $IIZ$  are all weight-one logical operators.

### 4.3 Five-qubit perfect code

To see another example of the stabilizer formalism, we now consider the [five-qubit code](#) <sup>5c</sup><sub>300</sub>, the smallest code that can correct an arbitrary single-qubit error. It does so using four fewer qubits than the nine-qubit Shor code from [Section 3.3](#). This code, denoted as a  $[[5, 1, 3]]$  code [\[59\]](#), is especially important because it is a *perfect code*, a notion we discuss at the end of this section.

The five-qubit code is elegantly described in the stabilizer language. Since it encodes one logical qubit into five physical qubits, the stabilizers reduce the physical Hilbert-space dimension from  $2^5 = 32$  to a two-dimensional codespace. Using the relation from the previous section, we need  $5 - 1 = 4$  independent stabilizer generators, which can be written as

$$g_1 = XZZXI, \quad (4.27)$$

$$g_2 = IXZZX, \quad (4.28)$$

$$g_3 = XIXZZ, \quad (4.29)$$

$$g_4 = ZXIXZ. \quad (4.30)$$

Any valid logical state  $|\bar{\psi}\rangle$  in the codespace must satisfy  $g_i |\bar{\psi}\rangle = |\bar{\psi}\rangle \forall i \in \{1, 2, 3, 4\}$ . It can be easily verified that all these generators commute with each other, which is also a necessary condition.

**Exercise 4.1:** Mutual commutation of five-qubit code stabilizers

Verify that  $g_1, g_2, g_3$ , and  $g_4$  all mutually commute.

To manipulate the encoded qubit, we need logical Pauli operators  $\bar{X}$  and  $\bar{Z}$ . These operators must commute with all stabilizer generators but must not themselves be stabilizers. Additionally,  $\bar{X}$  and  $\bar{Z}$  must anticommute with each other, just like normal Pauli operators. In group-theoretic language, logical operators are represented by cosets of  $\mathcal{S}$  inside its normalizer. For the five-qubit code, one convenient choice is

$$\bar{Z} = ZZZZZ, \quad (4.31)$$

$$\bar{X} = XXXXX. \quad (4.32)$$

**Exercise 4.2:**  $[[5, 1, 3]]$  logical Pauli operators

Verify that  $\bar{X}$  and  $\bar{Z}$  of the five-qubit code commute with all stabilizer generators  $\{g_1, g_2, g_3, g_4\}$  and anticommute with each other.

The power of the stabilizer formalism lies in the detection of errors. Suppose a single-qubit error  $E$  occurs on the state  $|\bar{\psi}\rangle$ , transforming it to  $E|\bar{\psi}\rangle$ . To detect this error, we measure the eigenvalues of the four stabilizer generators. If a generator  $g_i$  commutes with the error  $E$ , then

$$g_i E |\bar{\psi}\rangle = E g_i |\bar{\psi}\rangle = E |\bar{\psi}\rangle. \quad (4.33)$$

If  $g_i$  anticommutes with  $E$ , then

$$g_i E |\bar{\psi}\rangle = -E g_i |\bar{\psi}\rangle = -E |\bar{\psi}\rangle. \quad (4.34)$$

Thus, by measuring the eigenvalues of all four generators, we can determine the error syndrome. Each unique single-qubit error ( $X_i$ ,  $Y_i$ , and  $Z_i$  for  $i = 1, \dots, 5$ ) produces a unique, non-trivial syndrome. [Table 2](#) lists the syndromes for all 15 possible single-qubit Pauli errors, where +1 indicates commutation and -1 indicates anticommutation. Once a syndrome is measured, we simply look it up in the table to identify the error that occurred. For example, if we measure the syndrome  $(+1, -1, +1, -1)$ , we know that a  $Z_2$  error has occurred. To correct it, we simply apply the same operator  $Z_2$  again to correct the error.

Finally, we return to the statement that the five-qubit code is *perfect*. As [Table 2](#) shows, each of the 15 nontrivial single-qubit Pauli errors gives a unique syndrome. There is therefore a one-to-one correspondence between correctable errors and nontrivial syndromes: no syndrome is wasted, and every correctable error is uniquely identified. By contrast, the nine-qubit Shor code has eight stabilizer generators and therefore  $2^8 - 1 = 255$  nontrivial syndromes available, far more than the 27 nontrivial single-qubit Pauli errors it is designed to correct. Thus the five-qubit code uses its syndrome space optimally. This efficient use of syndrome space is rare and occurs only for certain codes. Equivalently, the five-qubit code saturates the *quantum Hamming bound*, which we discuss later in the tutorial (see [Section 10.1](#)).

## 4.4 CSS codes

We conclude this chapter with an introduction to an important family of stabilizer codes known as *CSS codes*, named after their discoverers, Robert Calderbank, Peter Shor, and Andrew Steane.

| Error | $g_1$ | $g_2$ | $g_3$ | $g_4$ |
|-------|-------|-------|-------|-------|
| $X_1$ | +1    | +1    | +1    | -1    |
| $Z_1$ | -1    | +1    | -1    | +1    |
| $Y_1$ | -1    | +1    | -1    | -1    |
| $X_2$ | -1    | +1    | +1    | +1    |
| $Z_2$ | +1    | -1    | +1    | -1    |
| $Y_2$ | -1    | -1    | +1    | -1    |
| $X_3$ | -1    | -1    | +1    | +1    |
| $Z_3$ | +1    | +1    | -1    | +1    |
| $Y_3$ | -1    | -1    | -1    | +1    |
| $X_4$ | +1    | -1    | -1    | +1    |
| $Z_4$ | -1    | +1    | +1    | -1    |
| $Y_4$ | -1    | -1    | -1    | -1    |
| $X_5$ | +1    | +1    | -1    | -1    |
| $Z_5$ | +1    | -1    | -1    | +1    |
| $Y_5$ | +1    | -1    | -1    | -1    |

Table 2: The 15 error syndromes for the  $[[5, 1, 3]]$  code. Each row shows the resulting eigenvalues from measuring the four stabilizer generators against a specific single-qubit Pauli error.

Common examples include the Steane code and the Shor code, and the toric code, which we will introduce in the next chapter.

A CSS code is a quantum stabilizer code built from a pair of classical linear codes. One classical code supplies the  $Z$ -type stabilizer generators, which are used to detect bit-flip errors, while the other supplies the  $X$ -type stabilizer generators, which are used to detect phase-flip errors. This separation is what makes CSS codes especially transparent and useful. More concretely, suppose we start with two binary linear codes of length  $n$ , with parity-check matrices  $H_Z$  and  $H_X$ . Each row of  $H_Z$  is interpreted as the support of a  $Z$ -type stabilizer generator: wherever the row has a 1, we place a  $Z$ , and wherever it has a 0, we place an  $I$ . Similarly, each row of  $H_X$  defines an  $X$ -type stabilizer generator. For example, the row  $(1, 0, 1, 1)$  of  $H_Z$  would define the stabilizer  $ZIZZ$ , while the same row in  $H_X$  would define  $XIXX$ .

These generators define a valid stabilizer code only if every  $X$ -type generator commutes with every  $Z$ -type generator. Since single-qubit Pauli operators  $X$  and  $Z$  anticommute on the same qubit and commute on different qubits, this commutation condition is equivalent to requiring that each row of  $H_X$  have even overlap with each row of  $H_Z$ . In matrix form, this is the condition

$$H_X H_Z^\top = 0 \quad (4.35)$$

over  $\mathbb{F}_2$ . When this condition holds, the  $X$ -type and  $Z$ -type parity checks together define a CSS stabilizer group.

The parameters of the resulting CSS code can be read off from the underlying classical codes. If the two classical codes have parameters  $[n, k_X, d_X]$  and  $[n, k_Z, d_Z]$ , then the number of encoded qubits is

$$k = k_X + k_Z - n. \quad (4.36)$$

**Exercise 4.3:** Parameters of a CSS code

Prove the relation in Eq. (4.36).

[Hint: think of the number of  $X$ - and  $Z$ -type stabilizer generators.]

The distances of the underlying classical codes also influence the error-correcting properties of the CSS code, but the relation is subtler than the formula for  $k$ . Roughly speaking, one classical code governs protection against bit-flip ( $X$ -type) errors, while the other governs protection against phase-flip ( $Z$ -type) errors, so a CSS code can have different  $X$ - and  $Z$ -type logical distances. However, the true quantum distances are not obtained simply by taking the classical distances of the seed codes. Classical codewords of any one kind correspond only to stabilizers and therefore do not provide information about nontrivial logical operators.

More precisely, if the two classical codes are  $C_X$  and  $C_Z$ , then the relevant logical distances are

$$d_X^{(\text{CSS})} = \min\{\text{wt}(c) : c \in C_Z \setminus C_X^\perp\}, \quad (4.37)$$

$$d_Z^{(\text{CSS})} = \min\{\text{wt}(c) : c \in C_X \setminus C_Z^\perp\}, \quad (4.38)$$

where  $\text{wt}(c)$  denotes the Hamming weight of  $c$ . The overall CSS code distance is then

$$d = \min\{d_X^{(\text{CSS})}, d_Z^{(\text{CSS})}\} \geq \min\{d_X, d_Z\}. \quad (4.39)$$

Thus, the classical distances  $d_X$  and  $d_Z$  provide useful intuition and a lower bound, but the actual quantum distance is determined by the minimum weight of a nontrivial logical operator rather than by the minimum weight of a nonzero classical codeword.

**Example 4.5:** Seven-qubit Steane code

An example of a CSS code is the seven-qubit [Steane code](#) [Ec<sub>7,3</sub>](#), which is constructed by taking two copies of the  $[[7, 4, 3]]$  Hamming code, whose parity-check matrix is

$$H = \begin{pmatrix} 0 & 0 & 0 & 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & 0 & 0 & 1 & 1 \\ 1 & 0 & 1 & 0 & 1 & 0 & 1 \end{pmatrix}. \quad (4.40)$$

Thus, we take  $H_X = H_Z = H$ . Because the Hamming code is self-orthogonal in the required sense, the condition  $H_X H_Z^\top = 0$  is satisfied, so the corresponding  $X$ - and  $Z$ -type stabilizers commute. The resulting CSS code has parameters  $[[7, 1, 3]]$ , where the number of physical qubits is inherited from the block length of the Hamming code. The number of encoded logical qubits is

$$k = k_X + k_Z - n = 4 + 4 - 7 = 1. \quad (4.41)$$

In this example, the smallest nontrivial  $X$ - and  $Z$ -type logical operators both have weight 3, so

$$d = \min\{d_X^{(\text{CSS})}, d_Z^{(\text{CSS})}\} = 3. \quad (4.42)$$

Thus, the Steane code is a  $[[7, 1, 3]]$  code.

One of the main advantages of CSS codes is therefore both conceptual and practical: they reduce much of the quantum construction to familiar classical coding data while still producing genuinely quantum codes. This makes them one of the most useful bridges between classical coding theory and quantum error correction, and it is one reason they appear so often in both theory and practice.

## 5 Topological codes

In the first four chapters of this tutorial, we gave a general overview of quantum error correction. We now dive deeper into specific code constructions, starting with *topological codes*. Important examples include the toric and planar/surface codes [26, 83, 84], color codes [85, 96], topological subsystem codes [97–99], and hyperbolic surface codes [100, 101].

### 5.1 Primer on topology

We begin with an introduction to some concepts from topology and homology [102] that we will need in this chapter and several other chapters throughout the tutorial.

#### 5.1.1 Introduction to topology

Topology is a branch of mathematics dealing with the preserved properties of an object undergoing continuous transformations, such as stretching, bending, or twisting the material itself. For example, a solid square can be deformed into a solid rectangle by using such transformations. Thus, we say that these shapes are *topologically equivalent*. Other examples of topologically equivalent objects include a circle and an ellipse, as well as a sphere and an ellipsoid. However, a sphere cannot be continuously deformed into a torus; such a transition would require puncturing and gluing the surface to create a new hole, which is not allowed. As such, we say that a sphere and a torus are *topologically inequivalent*. Some other questions that arise in topology include asking how many holes an object has and what the boundary of the object is, which are answered by *homology*, which we explore in Section 5.1.2.

A famous early problem in topology, which also led to the branch of mathematics called *graph theory*, is the “Seven Bridges of Königsberg,” which was proposed by Euler. Given a map of the then-Prussian city of Königsberg (now Kaliningrad in Russia) in Figure 7, we can see that there are seven bridges connecting the various land masses comprising the city. Euler asked an interesting question: is it possible to traverse a path that crosses each bridge once and only once? Try it for yourself and you should find that there is no such path! Euler abstracted the problem and modeled it as a graph and determined that a path that crosses each bridge exactly once is only possible if there is a maximum of two vertices (the land masses) of the graph that are of odd degree. However, for the Seven Bridges of Königsberg problem, all four of the vertices are of odd degree, and there are four vertices, so such a path is not possible.

From this example, we can see that abstracting the problem and dealing only with the vertices and edges of a graph was enough to arrive at a useful result. Other details like the lengths of the bridges, the total length of the path, the distance between each bridge, etc. were not needed to answer the original question posed of whether it is possible to take a path that crosses each bridge exactly once. All we needed was the set of four vertices and seven edges, demonstrating an application of topology.

Another early result (also from Euler) in topology has to do with the relation of components of different topological dimension in a geometric object. Given the two-dimensional surface of a geometric shape like a polyhedron (treating it as a hollow shell with flat surfaces), we get several components of varying *dimension*. Thus, a 0-dimensional object is a vertex, a 1-dimensional object is an edge, a 2-dimensional object is a face or plaquette. Euler found that the quantity

$$\chi = V - E + F, \tag{5.1}$$

which is now called the *Euler characteristic*, is invariant for a given topological surface. Here,  $V$  is the number of vertices in the object,  $E$  is the number of edges, and  $F$  is the number of faces. For



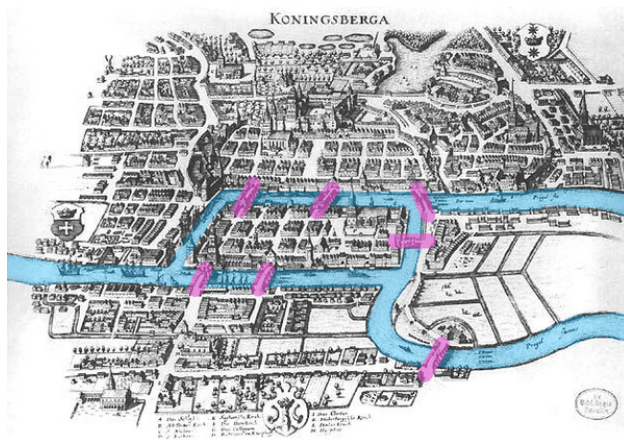
example, for convex polyhedra (tetrahedra, cubes, icosahedra, dodecahedra, etc.),  $\chi = 2$ . Since this quantity does not change (up to a topological equivalence which we will introduce shortly), we call it a *topological invariant*.

The canonical example of topology is, of course, the idea that a coffee mug and a donut are topologically equivalent. That is, one can be deformed into the other without making any cuts or tears. We mention this example to introduce the concept of a *homeomorphism*, which is exactly what the transformation from donut to coffee mug or *vice versa* is. Informally, we can think of a homeomorphism as an intrinsic deformation of one object or space into another without “breaking” or “gluing” anything. A homeomorphism is the standard function used to define this *topological equivalence*. More formally, a homeomorphism is a function between two objects or topological spaces that

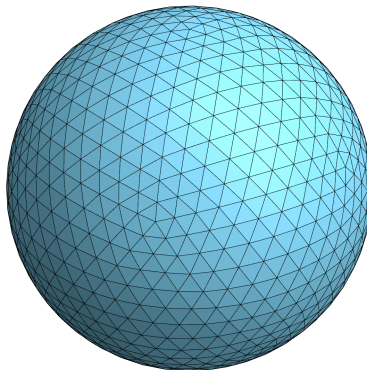
1. is continuous,
2. is a bijection (i.e., it is one-to-one and onto and therefore invertible), and
3. has a continuous inverse function.

Thus, when we say that two objects are topologically equivalent, we mean that they are equivalent *up to a homeomorphism*. Related are the topological invariants that we mentioned above, where we saw that the Euler characteristic was one such example. Now that we understand what a homeomorphism is, we can more precisely define a topological invariant as a property or numerical quantity that remains unchanged under homeomorphism. For convex polyhedra, we can stretch and deform them into other convex polyhedra (e.g., a tetrahedron to a cube), but the Euler characteristic remains 2 because it is invariant under the underlying homeomorphism.

We can generalize the Euler characteristic to distinguish general spaces via a process called *cellulation*, which partitions a space into a collection of constituent cells. For a two-dimensional surface, this amounts to decomposing the space into polygonal cells. One example is the triangulation of the surface of a sphere, as in Figure 8. If we count the number of vertices, edges, and faces of our triangulated sphere, we should find that the Euler characteristic is  $\chi_{\text{sphere}} = 2$ . On the other hand, if we consider a torus using a square  $L \times L$  cellulation and periodic boundary conditions, as we will see when we study the toric code in Section 5.2, we will find that the number of vertices is  $L^2$ , the number of edges is  $2L^2$ , and the number of faces is  $L^2$ , where  $L$  is a parameter we can choose. This gives us an Euler characteristic of  $\chi_{\text{torus}} = L^2 - 2L^2 + L^2 = 0$ , which is not equal to  $\chi_{\text{sphere}}$ . Thus, we can conclude that a sphere and a torus are *topologically inequivalent*.



**Figure 7:** Map of the seven bridges of Königsberg. Photo credit: [Wikipedia](#).



**Figure 8:** Triangulated sphere.

Finally, we highlight an additional property that differentiates topological objects: the number of *holes* an object has, which we characterize by its *genus*. In the sphere and torus example from above, we can see that a sphere has 0 holes and a torus has 1 hole. Thus, we say that a sphere is a genus-0 object while a torus is a genus-1 object. One important result that we will find helpful when studying topological quantum error-correcting codes is that the Euler characteristic  $\chi$  of any *closed, orientable* surface of genus  $g$  is  $\chi = 2 - 2g$ . We will explore this idea further when we use it in our development of topological codes.

### 5.1.2 Introduction to homology

We now move onto a branch of topology called *homology*, which abstracts and generalizes the notion of a *boundary*. In this section, we will focus on concrete aspects of homology that are relevant to qubit topological codes, which means we will restrict ourselves to  $\mathbb{Z}_2$  homology and cohomology over square lattice cellulations. Homology is a rich field with extensions to other groups, lattice layouts, etc., but for our purposes we will only need to consider these regimes.

The most basic object in homology is an  $n$ -cell, which is an  $n$ -dimensional object in a cellulation. For example, a 0-cell is a vertex, a 1-cell is an edge, and a 2-cell is a face (or, more frequently in QEC, a *plaquette*). Given a lattice or graph with a set  $K_n$  of  $n$ -cells, we define an  $n$ -chain to be a formal  $\mathbb{Z}_2$ -linear combination of these  $n$ -cells:

$$c = \sum_{\sigma \in K_n} c_\sigma \sigma. \quad (5.2)$$

Since we work throughout with  $\mathbb{Z}_2$  homology, each coefficient  $c_\sigma$  is either 0 or 1. Equivalently, an  $n$ -chain  $c$  can be viewed as an assignment of 0 or 1 to each  $n$ -cell: a 0-chain assigns 0 or 1 to each vertex, a 1-chain assigns 0 or 1 to each edge, and so on. Intuitively, an  $n$ -chain  $c$  simply specifies a subset of  $n$ -cells:  $\{\sigma : c_\sigma \neq 0\}$ .

Choosing  $\mathbb{Z}_2$  coefficients introduces a massive geometric simplification: cell orientations can be completely ignored. In general homology theories (such as over  $\mathbb{Z}$  or  $\mathbb{Z}_3$ ), each cell must be assigned a choice of direction or orientation, and reversing this orientation corresponds to an algebraic sign change of  $-1$ . However, because  $-1 \equiv 1 \pmod{2}$ , the distinction between opposite orientations vanishes entirely, allowing us to compute boundaries and sum chains without tracking directed arrows on the lattice cells.

Given two  $n$ -chains, we add them by adding their coefficients cell by cell modulo 2. For example, if a square plaquette has boundary edges  $e_1, e_2, e_3, e_4$  (1-cells), then the 1-chain  $c''$  consisting of those

four edges is

$$c'' = \begin{array}{c} \bullet \quad \bullet \\ \text{---} \text{---} \\ \bullet \quad \bullet \end{array} = \begin{array}{c} \bullet \quad \bullet \\ \text{---} \text{---} \\ \bullet \quad \bullet \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \\ \bullet \quad \bullet \end{array} + \begin{array}{c} \bullet \quad \bullet \\ \text{---} \text{---} \\ \bullet \quad \bullet \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \\ \bullet \quad \bullet \end{array} + \begin{array}{c} \bullet \quad \bullet \\ \text{---} \text{---} \\ \bullet \quad \bullet \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \\ \bullet \quad \bullet \end{array} + \begin{array}{c} \bullet \quad \bullet \\ \text{---} \text{---} \\ \bullet \quad \bullet \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \\ \bullet \quad \bullet \end{array} = e_1 + e_2 + e_3 + e_4. \quad (5.3)$$

As another example, for two 1-chains  $c$  and  $c'$  given by

$$c = \begin{array}{c} \bullet \quad \bullet \\ \text{---} \text{---} \\ \bullet \quad \bullet \end{array} = \begin{array}{c} \bullet \quad \bullet \\ \text{---} \text{---} \\ \bullet \quad \bullet \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \\ \bullet \quad \bullet \end{array} + \begin{array}{c} \bullet \quad \bullet \\ \text{---} \text{---} \\ \bullet \quad \bullet \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \\ \bullet \quad \bullet \end{array} = e_1 + e_2, \quad c' = \begin{array}{c} \bullet \quad \bullet \\ \text{---} \text{---} \\ \bullet \quad \bullet \end{array} = \begin{array}{c} \bullet \quad \bullet \\ \text{---} \text{---} \\ \bullet \quad \bullet \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \\ \bullet \quad \bullet \end{array} + \begin{array}{c} \bullet \quad \bullet \\ \text{---} \text{---} \\ \bullet \quad \bullet \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \\ \bullet \quad \bullet \end{array} = e_2 + e_3, \quad (5.4)$$

respectively, their sum is a 1-chain

$$c + c' = \begin{array}{c} \bullet \quad \bullet \\ \text{---} \text{---} \\ \bullet \quad \bullet \end{array} + \begin{array}{c} \bullet \quad \bullet \\ \text{---} \text{---} \\ \bullet \quad \bullet \end{array} = \begin{array}{c} \bullet \quad \bullet \\ \text{---} \text{---} \\ \bullet \quad \bullet \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \\ \bullet \quad \bullet \end{array} + \begin{array}{c} \bullet \quad \bullet \\ \text{---} \text{---} \\ \bullet \quad \bullet \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \\ \bullet \quad \bullet \end{array} = e_1 + e_3, \quad (5.5)$$

where  $e_2 + e_2 = 0$  because we are doing arithmetic over  $\mathbb{Z}_2$ . In fact, the set of  $n$ -chains forms an abelian group, with the following properties:

1. **Group composition:** cell-wise addition modulo 2
2. **Associativity:** follows from associativity of addition
3. **Identity element:** the all-zero  $n$ -chain, denoted by  $0_n$
4. **Inverse:** every  $n$ -chain is self-inverse (for  $\mathbb{Z}_2$ )

We denote the group of  $n$ -chains as  $C_n$  and we use lowercase Latin letters to label specific chains in the group:  $c \in C_n$ . With  $n$ -cells and  $n$ -chains defined, we can now consider *boundary maps*, which are the fundamental operators that we consider in homology. The  $n$ -boundary map  $\partial_n$  is a group-structure-preserving map (i.e., a group homomorphism) from the set of  $n$ -chains to the set of  $(n-1)$ -chains. That is,  $\partial_n : C_n \rightarrow C_{n-1}$ . We can think of a boundary map as breaking up a higher dimensional object. For example, for a triangular face (i.e., a 2-chain consisting of a single 2-cell)  $f \in C_2$ , the boundary map  $\partial_2$  maps  $f$  to its constituent edges (1-chains)  $e_1$ ,  $e_2$ , and  $e_3$ . That is,

$$f = \begin{array}{c} \bullet \\ \text{---} \text{---} \\ \bullet \quad \bullet \end{array} \mapsto \partial_2 f = \begin{array}{c} \bullet \\ \text{---} \text{---} \\ \bullet \quad \bullet \end{array} = \begin{array}{c} \bullet \\ \text{---} \text{---} \\ \bullet \quad \bullet \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \\ \bullet \quad \bullet \end{array} + \begin{array}{c} \bullet \\ \text{---} \text{---} \\ \bullet \quad \bullet \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \\ \bullet \quad \bullet \end{array} + \begin{array}{c} \bullet \\ \text{---} \text{---} \\ \bullet \quad \bullet \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \\ \bullet \quad \bullet \end{array} = e_1 + e_2 + e_3, \quad (5.6)$$

where  $e_i \in C_1$  for  $i = 1, 2, 3$ .

As the reader may note, a boundary map  $\partial_n$  quite literally maps an  $n$ -chain  $c \in C_n$  to its boundaries in  $C_{n-1}$ . For an  $n$ -chain  $c$ , we call  $\partial_n c$  the *boundary* of  $c$ . The set of all  $(n-1)$ -chains that arise as boundaries of  $n$ -chains is  $B_{n-1} = \text{im } \partial_n \subseteq C_{n-1}$  which we call the  $(n-1)$ -boundaries. Some chains have a *null boundary* (i.e., they have no boundary), that is,  $\partial_n c = 0_{n-1}$ . For example, the 1-chain  $e_1 + e_2 + e_3$  in Eq. (5.6) is a null boundary since

$$\begin{aligned} \partial_1(e_1 + e_2 + e_3) &= \begin{array}{c} \bullet \\ \text{---} \text{---} \\ \bullet \quad \bullet \end{array} + \begin{array}{c} \bullet \\ \text{---} \text{---} \\ \bullet \quad \bullet \end{array} + \begin{array}{c} \bullet \\ \text{---} \text{---} \\ \bullet \quad \bullet \end{array} \\ &= (v_1 + v_2) + (v_2 + v_3) + (v_3 + v_1) = 0_0 \end{aligned} \quad (5.7)$$

because  $\partial_1 e_j = v_j + v_{j+1 \pmod 3}$ . Chains with no boundaries are called *cycles*. In particular, an  $n$ -cycle is an  $n$ -chain  $c$  satisfying  $\partial_n c = 0_{n-1}$ . The set of all  $n$ -cycles is denoted as  $Z_n = \ker \partial_n$ . So

the 1-chain  $e_1 + e_2 + e_3$  above is an example of a 1-cycle. Note that sometimes we will omit the subscript  $n$  for boundary map on  $n$ -cycles when it is clear from the context, e.g.  $\partial c$  means  $\partial_n c$  for  $c \in C_n$ .

We now connect all of these groups together. Given a boundary map  $\partial_n$ , its domain is  $C_n$ , its image is  $B_{n-1}$ , and its kernel is  $Z_n$ . This leads to the *fundamental lemma of homology*: the boundary of a boundary is a null chain, or

$$\partial_n \partial_{n+1} c = 0_{n-1} \quad (5.8)$$

for all  $c_n \in C_{n+1}$ , which indicates that the image of  $\partial_{n+1}$  is contained in the kernel of  $\partial_n$ , that is,  $B_n \subseteq Z_n$ . Thus  $B_n$  is a subgroup of  $Z_n$ .  $B_n$  is then a *normal* subgroup of  $Z_n$  since  $Z_n$  is an abelian group, so we can take the quotient group  $H_n = Z_n/B_n$  which is called the  $n^{\text{th}}$  homology group.

Elements of  $H_n$  are equivalence classes of elements of  $Z_n$  where any pair of  $n$ -cycles  $c, c' \in Z_n$  is equivalent, denoted as  $c \sim c'$  if they belong to the same coset of  $B_n$  in  $Z_n$ , that is,  $c, c' \in c'' + B_n = \{c'' + c^* : c^* \in B_n\}$  for some  $c'' \in Z_n$ . Note that if  $c \in c'' + B_n$ , then there is some  $c^* \in B_n$  such that  $c = c'' + c^*$  which implies that  $c'' = c + c^*$  (since we are working over  $\mathbb{Z}_2$ ), so  $c'' \in c + B_n$ . This holds in general for any pair of elements in a coset of  $B_n$ , so we have  $c' + B_n = c + B_n$  if  $c \sim c'$  and thus all  $n$ -cycles in a coset is equivalent. Therefore we have

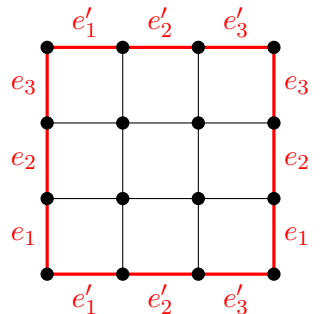
$$c \sim c' \Leftrightarrow c + B_n = c' + B_n. \quad (5.9)$$

Writing the equivalence class of an  $n$ -cycle  $c$  as  $[c] = c + B_n$ , addition between any two elements of  $H_n$  is therefore given by  $[c] + [c'] = [c + c'] = (c + c') + B_n$ . For any given coset, one can take any of its element as a representative for this group addition. Thus, any coset containing the identity element  $0_n$  is the identity element of  $H_n$ , denoted as  $[0_n] = 0_n + B_n = B_n$ . The homology group  $H_n$  therefore consists of all equivalence classes  $\{[c] = c + B_n : c \in Z_n\}$  that are closed under addition, where all distinct cosets  $c + B_n \in H_n$  partition  $Z_n$ , that is,  $\bigcup_{g \in H_n} g = Z_n$  and  $g \cap g' = \emptyset$  if  $g \neq g'$ . Thus, the homology group  $H_n$  consists of  $|H_n| = |Z_n|/|B_n|$  elements.

Taking the simple triangle example in Eq. (5.6), the only 1-boundaries are the boundary of the triangle and the null-boundary:  $B_1 = \{c, 0_1\}$  where  $c = e_1 + e_2 + e_3$ , whereas the only 1-cycles are also the boundary of the triangle and the null-boundary:  $Z_1 = \{c, 0_1\}$ . Its first homology group therefore only consists of one (identity) element since

$$c + B_1 = \{c + c, c + 0_1\} = \{0_1, c\} = \{0_1 + c, 0_1 + 0_1\} = 0_1 + B_1 = B_1. \quad (5.10)$$

We can consider a more interesting example: a  $3 \times 3$  square cellulation of a torus, which when laid flat on a 2-D surface can be illustrated as a grid with periodic boundary condition



$$, \quad (5.11)$$

where the top and bottom edges are the same and the left and right edges are the same, similarly for the top-bottom and right-left vertices. Its 1-boundary group  $B_1$  is generated by the boundaries

of the square plaquettes:

$$\partial_2(p_1) = \begin{array}{|c|c|c|c|} \hline \bullet & \bullet & \bullet & \bullet \\ \hline \bullet & \bullet & \bullet & \bullet \\ \hline \bullet & \bullet & \bullet & \bullet \\ \hline \bullet & \bullet & \bullet & \bullet \\ \hline \end{array}, \quad \partial_2(p_2) = \begin{array}{|c|c|c|c|} \hline \bullet & \bullet & \bullet & \bullet \\ \hline \bullet & \bullet & \bullet & \bullet \\ \hline \bullet & \bullet & \bullet & \bullet \\ \hline \bullet & \bullet & \bullet & \bullet \\ \hline \end{array}, \quad \dots, \quad \partial_2(p_9) = \begin{array}{|c|c|c|c|} \hline \bullet & \bullet & \bullet & \bullet \\ \hline \bullet & \bullet & \bullet & \bullet \\ \hline \bullet & \bullet & \bullet & \bullet \\ \hline \bullet & \bullet & \bullet & \bullet \\ \hline \end{array}, \quad (5.12)$$

since the boundary of two plaquettes (a 2-chain)  $p_1 + p_2$  is the sum of the boundaries of  $p_1$  and  $p_2$ :

$$\partial_2(p_1 + p_2) = \partial_2 \begin{array}{|c|c|c|c|} \hline \bullet & \bullet & \bullet & \bullet \\ \hline \bullet & \bullet & \bullet & \bullet \\ \hline \bullet & \bullet & \bullet & \bullet \\ \hline \bullet & \bullet & \bullet & \bullet \\ \hline \end{array} = \begin{array}{|c|c|c|c|} \hline \bullet & \bullet & \bullet & \bullet \\ \hline \bullet & \bullet & \bullet & \bullet \\ \hline \bullet & \bullet & \bullet & \bullet \\ \hline \bullet & \bullet & \bullet & \bullet \\ \hline \end{array} = \begin{array}{|c|c|c|c|} \hline \bullet & \bullet & \bullet & \bullet \\ \hline \bullet & \bullet & \bullet & \bullet \\ \hline \bullet & \bullet & \bullet & \bullet \\ \hline \bullet & \bullet & \bullet & \bullet \\ \hline \end{array} + \begin{array}{|c|c|c|c|} \hline \bullet & \bullet & \bullet & \bullet \\ \hline \bullet & \bullet & \bullet & \bullet \\ \hline \bullet & \bullet & \bullet & \bullet \\ \hline \bullet & \bullet & \bullet & \bullet \\ \hline \end{array}. \quad (5.13)$$

On the other hand, the 1-cycle group  $Z_1$  is generated by the boundaries of the square plaquettes in Eq. (5.12), as well as six *non-contractible* loops, namely loops that wrap around the torus in vertical and horizontal directions:

$$\begin{array}{|c|c|c|c|} \hline \bullet & \bullet & \bullet & \bullet \\ \hline \bullet & \bullet & \bullet & \bullet \\ \hline \bullet & \bullet & \bullet & \bullet \\ \hline \bullet & \bullet & \bullet & \bullet \\ \hline \end{array} \ell_1, \dots, \begin{array}{|c|c|c|c|} \hline \bullet & \bullet & \bullet & \bullet \\ \hline \bullet & \bullet & \bullet & \bullet \\ \hline \bullet & \bullet & \bullet & \bullet \\ \hline \bullet & \bullet & \bullet & \bullet \\ \hline \end{array} \ell_3, \begin{array}{|c|c|c|c|} \hline \bullet & \bullet & \bullet & \bullet \\ \hline \bullet & \bullet & \bullet & \bullet \\ \hline \bullet & \bullet & \bullet & \bullet \\ \hline \bullet & \bullet & \bullet & \bullet \\ \hline \end{array} \ell'_1, \dots, \begin{array}{|c|c|c|c|} \hline \bullet & \bullet & \bullet & \bullet \\ \hline \bullet & \bullet & \bullet & \bullet \\ \hline \bullet & \bullet & \bullet & \bullet \\ \hline \bullet & \bullet & \bullet & \bullet \\ \hline \end{array} \ell'_3. \quad (5.14)$$

where  $\ell_1 = e_1 + e_2 + e_3$ ,  $\ell'_1 = e'_1 + e'_2 + e'_3$ , and so on.

Now let us look into the homology group  $H_1 = Z_1/B_1$ . Since  $B_1$  is generated by the plaquette boundaries  $\partial_2(p_1), \dots, \partial_2(p_9)$ , we have the identity element  $[0_1] = B_1$  being the equivalence class of all plaquette boundaries, that is,  $\partial_2(p_i) \sim \partial_2(p_j)$  for all  $i, j \in \{1, \dots, 9\}$ . However, it does not contain any non-contractible loops since they cannot be generated from the plaquette boundaries. Now note that the coset  $\ell_j + B_1 \in H_1$  containing the vertical non-contractible loop  $\ell_j$  contains all other vertical non-contractible loops, as well their sum with any 1-boundaries. For example,  $[\ell_1] = \ell_1 + B_1$  contains:

$$\begin{array}{|c|c|c|c|} \hline \bullet & \bullet & \bullet & \bullet \\ \hline \bullet & \bullet & \bullet & \bullet \\ \hline \bullet & \bullet & \bullet & \bullet \\ \hline \bullet & \bullet & \bullet & \bullet \\ \hline \end{array} \ell_1 + \begin{array}{|c|c|c|c|} \hline \bullet & \bullet & \bullet & \bullet \\ \hline \bullet & \bullet & \bullet & \bullet \\ \hline \bullet & \bullet & \bullet & \bullet \\ \hline \bullet & \bullet & \bullet & \bullet \\ \hline \end{array} = \begin{array}{|c|c|c|c|} \hline \bullet & \bullet & \bullet & \bullet \\ \hline \bullet & \bullet & \bullet & \bullet \\ \hline \bullet & \bullet & \bullet & \bullet \\ \hline \bullet & \bullet & \bullet & \bullet \\ \hline \end{array} \ell_2$$

and

$$\begin{array}{|c|c|c|c|} \hline \bullet & \bullet & \bullet & \bullet \\ \hline \bullet & \bullet & \bullet & \bullet \\ \hline \bullet & \bullet & \bullet & \bullet \\ \hline \bullet & \bullet & \bullet & \bullet \\ \hline \end{array} \ell_3 + \begin{array}{|c|c|c|c|} \hline \bullet & \bullet & \bullet & \bullet \\ \hline \bullet & \bullet & \bullet & \bullet \\ \hline \bullet & \bullet & \bullet & \bullet \\ \hline \bullet & \bullet & \bullet & \bullet \\ \hline \end{array} = \begin{array}{|c|c|c|c|} \hline \bullet & \bullet & \bullet & \bullet \\ \hline \bullet & \bullet & \bullet & \bullet \\ \hline \bullet & \bullet & \bullet & \bullet \\ \hline \bullet & \bullet & \bullet & \bullet \\ \hline \end{array}.$$

By a similar argument, we also have  $\ell_i \sim \ell_j$  which correspond to coset  $[\ell_1] = \ell_1 + B_1$ . Thus, the homology group  $H_1$  is generated by  $[\ell_1]$  and  $[v_1]$ .

For the  $n^{\text{th}}$  homology group, we define the  $n^{\text{th}}$  Betti number as  $\beta_n = \dim H_n = \dim Z_n - \dim B_n$ , where the dimension  $\dim G$  of a group  $G$  is the minimum number of elements needed to generate

the group. The Betti number is a topologically invariant quantity that can informally be thought of as characterizing the number of holes in an object and is an important quantity for studying topological codes. The calculation of Betti numbers has also been given much attention recently in the quantum algorithms, where Lloyd, Garnerone, and Zanardi [103] introduced a quantum algorithm for calculating Betti numbers, giving rise to the field of quantum topological data analysis. The Betti numbers then allow us to calculate the Euler characteristic as an alternating sum of the Betti numbers. That is,

**Theorem 5.1:** Euler characteristic and Betti numbers

Given a *finite, two-dimensional* cell complex (such as a finite lattice composed of vertices, edges, and plaquettes), the Euler characteristic is  $\chi = \beta_2 - \beta_1 + \beta_0$ .

*Proof.* This identity stems from an application of the rank-nullity theorem; see, for example, Ref. [104, Sec. 2.2]. We prove it here nonetheless. Starting with the rank-nullity theorem applied to the boundary operator, we have

$$\dim C_n = \dim \ker \partial_n + \dim \operatorname{im} \partial_n \quad (5.16)$$

Recall from above that for a boundary map  $\partial_n$ , we took the domain to be  $C_n$ , the kernel to be  $Z_n$ , and the range (or image) to be  $B_{n-1}$ . Thus, we have

$$\dim C_n = \dim Z_n + \dim B_{n-1}. \quad (5.17)$$

Relating this to the Betti number, we have

$$\beta_n = \dim H_n = \dim Z_n - \dim B_n. \quad (5.18)$$

Then, recalling the Euler characteristic from Eq. (5.1) before, which relates the number of vertices, edges, and faces, we have

$$\chi = F - E + V, \quad (5.19)$$

which we can write as

$$\chi = a_2 - a_1 + a_0, \quad (5.20)$$

where  $a_n$  is the number of  $n$ -cells. Each of these is exactly equal to the rank of the chain group:  $a_n = \dim C_n$ , which we can write as

$$a_n = \dim C_n = \dim Z_n + \dim B_{n-1}. \quad (5.21)$$

Since we have no 3-cells, we have  $\dim B_2 = 0$ , so  $\beta_2 = \dim Z_2$ . Similarly,  $Z_0 = C_0$  and so  $\dim Z_0 = a_0$ . Plugging in for  $n = 0, 1, 2$ , we have

$$a_2 = \beta_2 + \dim B_1, \quad (5.22)$$

$$a_1 = \beta_1 + \dim B_1 + \dim B_0, \quad (5.23)$$

$$a_0 = \beta_0 + \dim B_0. \quad (5.24)$$

Plugging in everything, we have

$$\chi = a_2 - a_1 + a_0 \quad (5.25)$$

$$= \beta_2 + \dim B_1 - \beta_1 - \dim B_1 - \dim B_0 + \beta_0 + \dim B_0 \quad (5.26)$$

$$= \beta_2 - \beta_1 + \beta_0, \quad (5.27)$$

as claimed.  $\square$

This result can be generalized to any dimension to give the following alternating sum

$$\chi = \sum_{n \geq 0} (-1)^n \beta_n. \quad (5.28)$$

The dual theory of homology is called *cohomology*. Since we work over  $\mathbb{Z}_2$ , the chain groups  $C_n$  can be viewed as vector spaces over the finite field  $\mathbb{F}_2$  (which is simply  $\mathbb{Z}_2$  along with the usual multiplication operation). The corresponding *n-cochain group* is given by

$$C^n := \text{Hom}(C_n, \mathbb{Z}_2), \quad (5.29)$$

where  $\text{Hom}(C_n, \mathbb{Z}_2)$  is the set of all linear functionals that map an  $n$ -chain to an element of  $\mathbb{Z}_2$  with identity element  $0^n$ . Thus if  $\alpha \in C^n$  and  $c \in C_n$ , the action of  $\alpha$  on  $c$  is given by  $\langle \alpha, c \rangle \in \mathbb{Z}_2$  so that an  $n$ -cochain assigns a binary value to each  $n$ -chain. It is useful to introduce a “dual basis” as follows. For each  $n$ -chain  $c_\sigma$  consisting of a single  $n$ -cell  $\sigma$ , let  $c_\sigma^* \in C^n$  be the cochain that evaluates to 1 on  $c_\sigma$  and to 0 on all other  $n$ -chains  $c_\tau$  consisting of a single  $n$ -cell  $\tau \neq \sigma$ :

$$\langle c_\sigma^*, c_\tau \rangle = \begin{cases} 1, & \tau = \sigma, \\ 0, & \tau \neq \sigma. \end{cases} \quad (5.30)$$

Therefore, just as an  $n$ -chain can be viewed as a subset of  $n$ -cells, an  $n$ -cochain can also be viewed as a subset of  $n$ -cells, defined by its functional relationship with the  $n$ -chains.

The dual of the boundary map is called the *coboundary map*  $\delta^n : C^n \rightarrow C^{n+1}$  which is defined by the adjoint relation

$$\langle \delta^n \alpha, c \rangle = \langle \alpha, \partial_{n+1} c \rangle, \quad \alpha \in C^n, \quad c \in C_{n+1}. \quad (5.31)$$

In words,  $\delta^n \alpha$  evaluates an  $(n+1)$ -chain by first taking its boundary and then applying  $\alpha$ . For example, let  $v$  be a vertex and let  $v^* \in C^0$  be the corresponding dual 0-cochain. If an edge  $e$  has endpoint vertices  $u$  and  $w$ , then over  $\mathbb{Z}_2$ ,  $\partial_1 e = u + w$ . Hence

$$\langle \delta^0 v^*, e \rangle = \langle v^*, \partial_1 e \rangle = \langle v^*, u + w \rangle. \quad (5.32)$$

This is equal to 1 exactly when  $v$  is one of the endpoints of  $e$ . Therefore,  $\delta^0 v^*$  is the 1-cochain supported on all edges incident on  $v$ . The coboundary maps satisfy the relation  $\delta^{n+1} \delta^n = 0^{n+2}$ , which follows directly from the boundary map identity  $\partial_n \partial_{n+1} = 0_{n-1}$  for any  $n \in \mathbb{N}$  in [Eq. \(5.8\)](#). Indeed, for any  $\alpha \in C^n$  and any  $c \in C_{n+2}$ ,

$$\langle \delta^{n+1} \delta^n \alpha, c \rangle = \langle \alpha, \partial_{n+1} \partial_{n+2} c \rangle = 0. \quad (5.33)$$

We can therefore define *cocycles*, *coboundaries*, and *cohomology groups* by

$$Z^n := \ker \delta^n, \quad B^n := \text{im } \delta^{n-1}, \quad H^n := Z^n / B^n, \quad (5.34)$$

respectively.

For the square-cellulated torus example, we can identify the 0-cochains, 1-cochains, and 2-cochains with the vertices, edges, and squares, respectively, similar to the chains. However, the coboundary map now maps vertices to the edges around it and maps edges to the squares adjacent to it. For example, a coboundary map on a vertex  $v$  surrounded by edges  $e_1, e_2, e_3, e_4$  is given by

$$\begin{aligned} v = \text{---} \bullet \text{---} \overset{\text{red}}{\bullet} v \text{---} \bullet \text{---} &\mapsto \delta^0 v = \text{---} \bullet \text{---} \overset{\text{red}}{e_1} \text{---} \bullet \text{---} + \text{---} \bullet \text{---} \overset{\text{red}}{e_2} \text{---} \bullet \text{---} + \text{---} \bullet \text{---} \overset{\text{red}}{e_3} \text{---} \bullet \text{---} + \text{---} \bullet \text{---} \overset{\text{red}}{e_4} \text{---} \bullet \text{---} \\ &= e_1 + e_2 + e_3 + e_4. \end{aligned} \quad (5.35)$$



Taking the example of a  $3 \times 3$  square cellulation of a torus that we discussed for homology (c.f. Eq. (5.11)), the coboundaries of the nine vertices (recall that the top vertices are the same vertices as the ones at the bottom, so are the right and left vertices) generate the 1-coboundary group  $B^1$ :

$$\delta^0(v_1) = \begin{array}{|c|c|c|} \hline \bullet & \bullet & \bullet \\ \hline \bullet & \bullet & \bullet \\ \hline \bullet & \bullet & \bullet \\ \hline \end{array}, \quad \delta^0(v_2) = \begin{array}{|c|c|c|} \hline \bullet & \bullet & \bullet \\ \hline \bullet & \bullet & \bullet \\ \hline \bullet & \bullet & \bullet \\ \hline \end{array}, \quad \dots, \quad \delta^0(v_9) = \begin{array}{|c|c|c|} \hline \bullet & \bullet & \bullet \\ \hline \bullet & \bullet & \bullet \\ \hline \bullet & \bullet & \bullet \\ \hline \end{array}. \quad (5.36)$$

On the other hand, the 1-cocycle group  $Z^1$  is generated by the coboundaries above, as well as six 1-cocycle loops around the torus in the vertical ( $\lambda_1, \lambda_2, \lambda_3$ ) and horizontal ( $\lambda'_1, \lambda'_2, \lambda'_3$ ) directions:

$$\begin{array}{|c|c|c|} \hline \bullet & \bullet & \bullet \\ \hline \bullet & \bullet & \bullet \\ \hline \bullet & \bullet & \bullet \\ \hline \end{array}, \dots, \begin{array}{|c|c|c|} \hline \bullet & \bullet & \bullet \\ \hline \bullet & \bullet & \bullet \\ \hline \bullet & \bullet & \bullet \\ \hline \end{array}, \dots, \begin{array}{|c|c|c|} \hline \bullet & \bullet & \bullet \\ \hline \bullet & \bullet & \bullet \\ \hline \bullet & \bullet & \bullet \\ \hline \end{array}. \quad (5.37)$$

The cohomology group  $H^1 = Z^1/B^1$  here can be obtained by a similar argument to the analysis of the homology group  $H_1$ , where now we are dealing with cosets of  $B^1$  in  $Z^1$ . Elements of  $H^1$  are equivalence classes of elements of  $Z^1$ , denoted as  $[c^*] = c^* + B^1$  for  $c^* \in Z^1$ , where  $c^*, \tilde{c}^*$  are equivalent, that is,  $c^* \sim \tilde{c}^*$ , whenever they belong to the same coset. Also, the identity element of  $H^1$  is given by  $[0^1] = 0^1 + B^1 = B^1$  where  $0^1$  is the identity element of group  $C^1$ . So, any 1-cochain that is the sum of vertex coboundaries is equivalent to  $0^1$ . Again we note that coset  $\lambda_i + B^1$  contains all 1-cochain vertical loops, as well as the sum of loops with vertex coboundaries. For example,  $[\lambda_1]$  contains

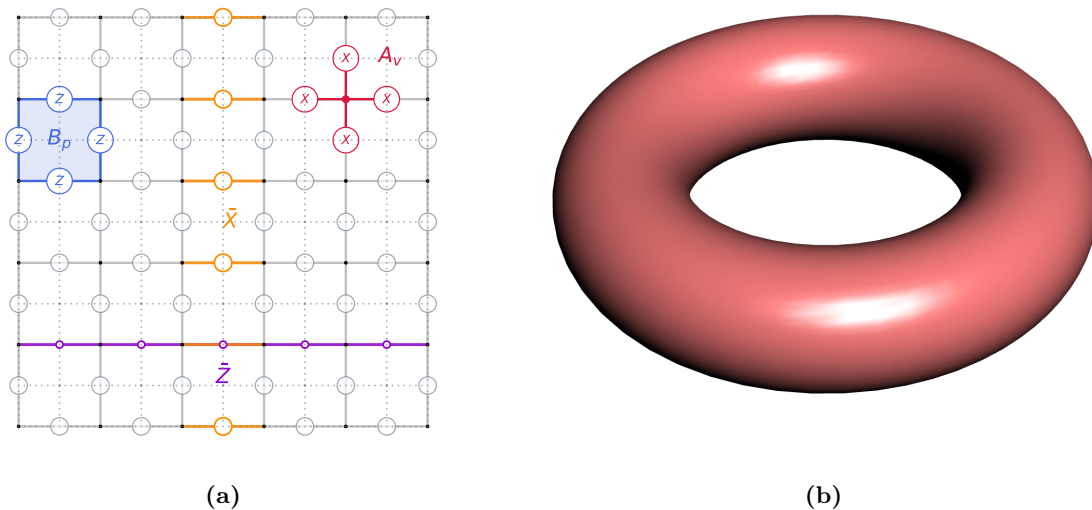
$$\begin{array}{|c|c|c|} \hline \bullet & \bullet & \bullet \\ \hline \bullet & \bullet & \bullet \\ \hline \bullet & \bullet & \bullet \\ \hline \end{array} + \begin{array}{|c|c|c|} \hline \bullet & \bullet & \bullet \\ \hline \bullet & \bullet & \bullet \\ \hline \bullet & \bullet & \bullet \\ \hline \end{array} = \begin{array}{|c|c|c|} \hline \bullet & \bullet & \bullet \\ \hline \bullet & \bullet & \bullet \\ \hline \bullet & \bullet & \bullet \\ \hline \end{array} \quad \text{and} \quad (5.38)$$

$$\begin{array}{|c|c|c|} \hline \bullet & \bullet & \bullet \\ \hline \bullet & \bullet & \bullet \\ \hline \bullet & \bullet & \bullet \\ \hline \end{array} + \begin{array}{|c|c|c|} \hline \bullet & \bullet & \bullet \\ \hline \bullet & \bullet & \bullet \\ \hline \bullet & \bullet & \bullet \\ \hline \end{array} = \begin{array}{|c|c|c|} \hline \bullet & \bullet & \bullet \\ \hline \bullet & \bullet & \bullet \\ \hline \bullet & \bullet & \bullet \\ \hline \end{array}$$

Similarly, all horizontal loops and its sum with any coboundaries is in  $[\lambda'_1]$ . Thus the first cohomology group  $H^1$  is generated by  $[\lambda_1], [\lambda'_1]$ .

Alternatively, we can define a *dual lattice* of the original square lattice, which we call the *primal lattice*, on the surface of the torus where each vertex of the dual lattice is centered on a square plaquette of the primal lattice, whereas a square plaquette of the dual lattice has a vertex of the original lattice at its center (see Figure 9a). The square lattice is a *self-dual* lattice since its dual lattice is a square lattice. However, this does not hold for other types of lattices, such as a hexagonal lattice, whose dual lattice is triangular.





**Figure 9:** (a)  $5 \times 5$  lattice with periodic boundary conditions for the toric code where qubits (hollow circles) are placed on the edges and  $Z_p$  and  $X_v$  are stabilizer generators comprised of Pauli- $Z$  (blue) and Pauli- $X$  (red) operators, respectively. These operators surround either a vertex or a plaquette. The dashed lines indicate the dual lattice. (b) A torus, shown as a geometric visualization of the periodic boundary conditions in (a). Connecting the left and right edges and the top and bottom edges of the square lattice produces the closed surface with one hole shown in the figure.

## 5.2 Toric code

We now introduce our first topological code, the *toric code* [\[50\]](#), which is one of the simplest topological codes. The toric code is defined on an  $L \times L$  square cellulation on a torus which gives a regular square lattice with periodic boundary conditions as illustrated in [Figure 9](#). As we have discussed, the lattice has 0-cells (vertices), 1-cells (edges), and 2-cells (faces or plaquettes). With an  $L \times L$  lattice, we have  $L^2$  vertices,  $L^2$  plaquettes, and  $L^2 + L^2 = 2L^2$  edges. We place physical qubits on the edges of the lattice (the circles in [Figure 9a](#)) so that the code consists of  $n = 2L^2$  physical qubits.

### 5.2.1 Stabilizers

The toric code is a CSS code, which means that there are two types of stabilizer generators: weight-4  $Z$ -type stabilizers supported on the edges around a square plaquette and weight-4  $X$ -type stabilizers supported on the edges around a vertex, which we will often refer to as *plaquette stabilizers* and *vertex stabilizers*, respectively. Since there are  $L^2$  plaquettes and  $L^2$  vertices, there are  $L^2$  stabilizer generators of each type. A plaquette stabilizer  $Z_p$  on plaquette  $p$  can be defined using the boundary map over the 2-cells (square plaquettes) and a vertex stabilizer  $X_v$  can be defined using the coboundary map over the 0-cells (vertices) as

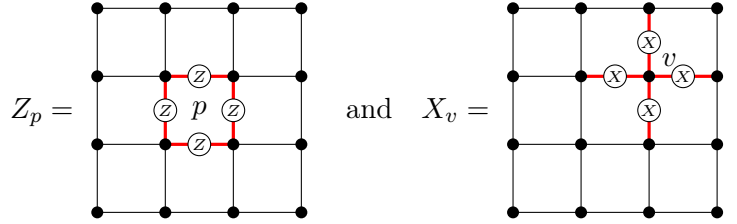
$$Z_p = \prod_{e \in \partial p} Z(e) \quad \text{and} \quad X_v = \prod_{e \in \delta v} X(e), \quad (5.39)$$

respectively<sup>5</sup>, where  $Z(e)$  (resp.,  $X(e)$ ) is an  $n$ -qubit Pauli operator with Pauli  $Z$  (resp., Pauli  $X$ ) on qubit on edge  $e$  and identity everywhere else. Note that commutation between the plaquette and vertex stabilizers is guaranteed by the action of 1-cochain  $\delta^0 v$  on 1-chain  $\partial_2 p$  for a vertex  $v$  on the corner of a plaquette  $p$ :

$$\langle \delta^0 v, \partial_2 p \rangle = \langle v, \partial_1 \partial_2 p \rangle = 0 \quad (5.40)$$

since  $\partial_1 \partial_2 = 0$ .

In our  $3 \times 3$  square lattice example, such operators on plaquette  $p$  and vertex  $v$  are



$$Z_p = \text{ (diagram of a central plaquette with red edges and Z operators) } \quad \text{and} \quad X_v = \text{ (diagram of a central vertex with red edges and X operators) }. \quad (5.41)$$

Thus, the toric code is a stabilizer code with stabilizer group  $\mathcal{S}_{\text{tc}}$  generated by the plaquette stabilizers  $Z_p$  and vertex stabilizers  $X_v$ :

$$\mathcal{S}_{\text{tc}} = \langle Z_p, X_v : \text{plaquettes } p, \text{ vertices } v \rangle. \quad (5.42)$$

Since the plaquette and vertex stabilizers are defined by the boundaries and coboundaries of  $C_2$  and  $C^0$ , respectively, we can write each  $Z$ -type and  $X$ -type stabilizer in terms of elements of the boundary group  $B_1 \subseteq C_1$  (1-chains) and coboundary group  $B^1 \subseteq C^1$  (1-cochains), respectively, as follows

$$Z_c = \prod_{e \in c} Z(e) \quad \text{and} \quad X_{c^*} = \prod_{e \in c^*} X(e) \quad (5.43)$$

for  $c \in B_1$  and  $c^* \in B^1$ . Therefore we can write the entire stabilizer group  $\mathcal{S}_{\text{tc}}$  in terms of the boundary group  $B_1$  and coboundary group  $B^1$ :

#### Lemma 5.1: Stabilizer-boundary relation

The stabilizer group  $\mathcal{S}_{\text{tc}}$  of a toric code can be expressed as

$$\mathcal{S}_{\text{tc}} = \{Z_c X_{c^*} : c \in B_1, c^* \in B^1\}. \quad (5.44)$$

### 5.2.2 Encoded logical qubits

From its stabilizer group  $\mathcal{S}_{\text{tc}}$ , we can then determine the logical operators and the codespace dimension (number of encoded logical qubits) of the toric code through its normalizer in the Pauli group:  $\mathcal{N}(\mathcal{S}_{\text{tc}})$ . We show below that we can write the normalizer in terms of cycle and cocycle groups  $Z_1, Z^1$ .

#### Lemma 5.2: Normalizer-cycle relation

For a toric code stabilizer group  $\mathcal{S}_{\text{tc}}$ , its normalizer in the Pauli group is given by

$$\mathcal{N}(\mathcal{S}_{\text{tc}}) = \langle \{Z_c, X_{c^*} : c \in Z_1, c^* \in Z^1\} \cup \{\pm i\} \rangle. \quad (5.45)$$

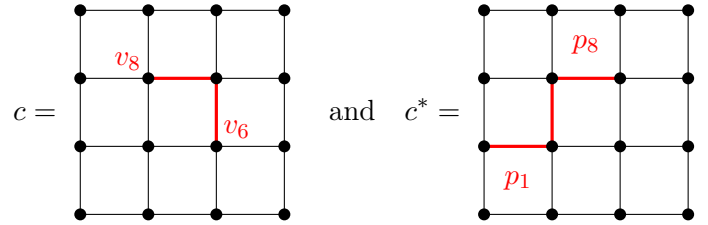
<sup>5</sup>Note that here we make a slight abuse of notation in  $e \in \partial p$  and  $e \in \delta v$  where the boundary  $\partial p$  and coboundary  $\delta v$  are considered sets of 1-cells (edges) with non-zero coefficient.

*Proof.* Note that for any  $c^* \in Z^1$  and  $c \in Z_1$  we have

$$\langle c^*, \partial_2 p \rangle = \langle \delta^1 c^*, p \rangle = 0 \quad \text{and} \quad \langle \delta^0 v, c \rangle = \langle v, \partial_1 c \rangle = 0 \quad (5.46)$$

for any plaquette  $p$  and vertex  $v$ , since  $c^* \in Z^1 = \ker \delta^1$  and  $c \in Z_1 = \ker \partial_1$  by definition. So for any  $c \in Z_1$  and  $c^* \in Z^1$ , the Pauli operators  $Z_c$  and  $X_{c^*}$  commute with any stabilizer in  $\mathcal{S}_{\text{tc}}$  since the boundary group  $B_1$  and coboundary group  $B^1$  are generated by  $\{\partial_2 p\}_p$  and  $\{\delta^0 v\}_v$ , respectively.

Moreover, the only 1-chains (resp. 1-cochains) satisfying the relations in Eq. (5.46) are those that belong in  $Z_1$  (resp. in  $Z^1$ ). This is because a 1-chain that is not in  $Z_1$ , that is,  $c \notin Z_1$  must have vertices in its boundaries, that is,  $\partial_1 c \neq 0$ , whereas a non-cocycle 1-cochain  $c^* \notin Z^1$  must have plaquettes in its coboundaries, that is,  $\delta^1 c^* \neq 0$ . For example,



$$c = \quad \text{and} \quad c^* = \quad (5.47)$$

So, for the 1-chain  $c$  we have

$$\langle \delta^0 v_6, c \rangle = \langle v_6, \partial_1 c \rangle = \langle v_6, v_6 + v_8 \rangle \neq 0 \quad \text{and} \quad \langle \delta^0 v_8, c \rangle = \langle v_8, \partial_1 c \rangle = \langle v_8, v_6 + v_8 \rangle \neq 0, \quad (5.48)$$

whereas for 1-cochain  $c^*$  we have

$$\langle c^*, \partial_2 p_1 \rangle = \langle \delta^1 c^*, p_1 \rangle = \langle p_1 + p_8, p_1 \rangle \neq 0 \quad \text{and} \quad \langle c^*, \partial_2 p_8 \rangle = \langle \delta^1 c^*, p_8 \rangle = \langle p_1 + p_8, p_8 \rangle \neq 0. \quad (5.49)$$

In a similar fashion, for any non-cycle 1-chain  $c$  and any non-cocycle 1-cochain  $c^*$ , there exists some plaquette  $p$  and some vertex  $v$  such that  $\langle c^*, \partial_2 p \rangle \neq 0$  and  $\langle \delta^0 v, c \rangle \neq 0$ .

Finally, note that we can write the cycle and cocycle groups as

$$\begin{aligned} Z_1 &= \{c \in C_1 : \langle \delta^0 v, c \rangle = 0, \text{ for all vertices } v\} \\ &\quad \text{and} \\ Z^1 &= \{c^* \in C^1 : \langle c^*, \partial_2 p \rangle = 0, \text{ for all plaquettes } p\}. \end{aligned} \quad (5.50)$$

Therefore for any  $c \in C_1$  and any  $c^* \in C^1$ , it holds that

$$\begin{aligned} Z_c X_v &= X_v Z_c, \text{ for all vertices } v \Leftrightarrow c \in Z_1 \\ &\quad \text{and} \\ X_{c^*} Z_p &= Z_p X_{c^*}, \text{ for all plaquettes } p \Leftrightarrow c^* \in Z^1. \end{aligned} \quad (5.51)$$

Since  $\mathcal{S}_{\text{tc}}$  is generated by the  $Z_p$ 's and  $X_v$ 's, and by considering Paulis with phases, the claim of the Lemma follows.  $\square$

Now let us recall that the logical operators of a stabilizer code with stabilizer group  $\mathcal{S}$  are elements of the quotient group of  $\mathcal{N}(\mathcal{S})/\mathcal{S}$ . In the toric code, the quotient group  $\mathcal{N}(\mathcal{S}_{\text{tc}})/\mathcal{S}_{\text{tc}}$  consists of cosets of  $\mathcal{S}_{\text{tc}}$  in  $\mathcal{N}(\mathcal{S}_{\text{tc}})$  of the form<sup>6</sup>

$$P\mathcal{S}_{\text{tc}} = \{Ps : s \in \mathcal{S}_{\text{tc}}\} = \{PZ_b X_{b^*} : b \in B_1, b^* \in B^1\} \quad (5.52)$$

<sup>6</sup>Note that this form of elements of the normalizer holds for CSS stabilizer codes, but not necessarily for general stabilizer codes. This is implicit in the stabilizer-boundary relation in Lemma 5.1 and normalizer-cycle relation in Lemma 5.2 that the  $X$ - and  $Z$ -type Paulis are “independently” generated. The toric code is but a special case of this more general property of the CSS codes.

for  $P \in \mathcal{N}(\mathcal{S}_{\text{tc}})$ , by [Lemma 5.1](#) linking the stabilizers to the boundaries  $B_1$  and coboundaries  $B^1$ . We can then write the elements of normalizer  $\mathcal{N}(\mathcal{S}_{\text{tc}})$  in terms of cycles  $Z_1$  and cocycles  $Z^1$ , so that these cosets can be written as

$$\{i^a Z_{c+b} X_{c^*+b^*} : b \in B_1, b^* \in B^1\} = i^a \cdot [Z_c] \cdot [X_{c^*}]$$

for

$$[Z_c] = \{Z_{c+b} : b \in B_1\} \quad \text{and} \quad [X_{c^*}] = \{X_{c^*+b^*} : b^* \in B^1\}$$

for some  $a \in \{0, \dots, 3\}$ ,  $c \in Z_1$ , and  $c^* \in Z^1$  (here we used the fact that the pair  $Z_b, X_{c^*}$  commute).

Now consider the direct sum between the homology and cohomology group:

$$H_1 \oplus H^1 = \{g \oplus g^* : g \in H_1, g^* \in H^1\} \quad (5.54)$$

with group operation “+” given by  $(g \oplus g') + (h \oplus h') = (g + h) \oplus (g' + h')$  for  $g, h \in H_1$  and  $g', h' \in H^1$ . Now we show that the group  $H_1 \oplus H^1$  is isomorphic to the group  $\mathcal{L}_{\text{tc}}$ , which is the quotient group  $\mathcal{N}(\mathcal{S}_{\text{tc}})/\mathcal{S}_{\text{tc}}$  up to phases, that consists of elements of the form  $[Z_c] \cdot [X_{c^*}]$  with group operation “\*” given by

$$([Z_c] \cdot [X_{c^*}]) * ([Z_a] \cdot [X_{a^*}]) = [Z_{c+a}] \cdot [X_{c^*+a^*}]. \quad (5.55)$$

**Lemma 5.3:**  $\mathcal{L}_{\text{tc}} - H_1 \oplus H^1$  isomorphism

The group  $\mathcal{L}_{\text{tc}}$  (the quotient group  $\mathcal{N}(\mathcal{S}_{\text{tc}})/\mathcal{S}_{\text{tc}}$ , up to phases) is isomorphic to  $H_1 \oplus H^1$ .

*Proof.* Consider a map  $\varphi$  between  $\mathcal{L}_{\text{tc}}$  and group  $H_1 \oplus H^1$  defined by

$$\varphi([Z_c] \cdot [X_{c^*}]) = [c] \oplus [c^*]. \quad (5.56)$$

$\varphi$  is well-defined because if  $\varphi([Z_c] \cdot [X_{c^*}]) = [c] \oplus [c^*]$  and  $\varphi([Z_c] \cdot [X_{c^*}]) = [a] \oplus [a^*]$ , then

$$[a] \oplus [a^*] = \varphi([Z_c] \cdot [X_{c^*}]) = [c] \oplus [c^*] \quad (5.57)$$

which means that  $[a] = [c]$  and  $[a^*] = [c^*]$ . Now we will show that  $\varphi$  is a one-to-one function (a bijection) and that it is a group homomorphism.

To show that  $\varphi$  is bijective, we show that it is both injective and surjective. First, we show that  $\varphi$  is injective, namely that, given any distinct  $g, g' \in \mathcal{L}_{\text{tc}}$ , we have  $\varphi(g) \neq \varphi(g')$ . Note that for any pair of distinct elements  $[Z_a] \cdot [X_{a^*}], [Z_c] \cdot [X_{c^*}]$  of  $\mathcal{L}_{\text{tc}}$  it holds that  $[Z_a] \neq [Z_c]$  or  $[X_{a^*}] \neq [X_{c^*}]$ . So we have

$$\varphi([Z_c] \cdot [X_{c^*}]) = [c] \oplus [c^*] \neq [a] \oplus [a^*] = \varphi([Z_a] \cdot [X_{a^*}]) \quad (5.58)$$

since  $[Z_a] \neq [Z_c]$  implies  $[a] \neq [c]$  and  $[X_{a^*}] \neq [X_{c^*}]$  implies  $[a^*] \neq [c^*]$ . Now, to show that  $\varphi$  is surjective, we need to show that for any  $[c] \oplus [c^*] \in H_1 \oplus H^1$ , there exists some  $[Z_c] \cdot [X_{c^*}] \in \mathcal{L}_{\text{tc}}$  such that  $\varphi([Z_c] \cdot [X_{c^*}]) = [c] \oplus [c^*]$ . This follows directly from the definition of  $\varphi$  that for any  $[c] \oplus [c^*] \in H_1 \oplus H^1$  we have

$$[c] \oplus [c^*] = \varphi([Z_c] \cdot [X_{c^*}]). \quad (5.59)$$

Lastly, to show that  $\varphi$  is a group homomorphism, consider again distinct  $Z_a, Z_c$  and distinct  $X_{a^*}, X_{c^*}$  so that

$$\begin{aligned} \varphi((([Z_c] \cdot [X_{c^*}]) * ([Z_a] \cdot [X_{a^*}])) &= \varphi([Z_{c+a}] \cdot [X_{c^*+a^*}]) \\ &= [c+a] \oplus [c^*+a^*] \\ &= ([c] \oplus [c^*]) + ([a] \oplus [a^*]) \end{aligned} \quad (5.60)$$

since  $[Z_a] \neq [Z_c]$  implies  $[a] \neq [c]$  and  $[X_{a^*}] \neq [X_{c^*}]$  implies  $[a^*] \neq [c^*]$  (i.e.  $a \notin [c]$  and  $a^* \notin [c^*]$ ). Hence we have shown that  $\varphi$  is an isomorphism between  $H_1 \oplus H^1$  and  $\mathcal{L}_{\text{tc}}$ , up to phases.  $\square$

Recall our discussion at the end of [Section 5.1.2](#) that  $H_1$  is generated by cosets of 1-cycle loops  $[\ell_1], [\ell'_1]$  and  $H^1$  is generated by cosets of 1-cocycle loops  $[\lambda_1], [\lambda'_1]$ . By the isomorphism in [Lemma 5.3](#), we can write  $\mathcal{L}_{\text{tc}}$  as

$$\mathcal{L}_{\text{tc}} = \langle [Z_{\ell_1}], [Z_{\ell'_1}], [X_{\lambda_1}], [X_{\lambda'_1}] \rangle, \quad (5.61)$$

that is, it is generated by  $4 = 2 \times 2$  independent elements of the form  $[Z_\ell] \cdot [X_\lambda]$ , and therefore the toric code encodes  $k = 2$  logical qubits. Alternatively, we can determine the number of logical qubits by counting the number of independent stabilizers in the toric code. Note that the number of independent vertex (resp., plaquette) stabilizers in the toric code is  $L^2 - 1$  since the product of all vertex (resp., plaquette) stabilizers is the identity. So, we can directly calculate the number of logical qubits  $k$  as

$$k = n - 2(L^2 - 1) = 2L^2 - 2L^2 + 2 = 2. \quad (5.62)$$

The number of logical qubits encoded in the toric code can also be verified using the topological invariant in [Section 5.1](#), namely the Euler characteristic  $\chi$ . Recall that the number of logical qubits is given by  $k = n - m$ , where  $n$  is the number of physical qubits and  $m$  is the number of independent stabilizers. Note that  $n = E$  by construction and  $m = (V - 1) + (F - 1)$ , where the first term is the number of independent vertex stabilizers and the second term is the number of independent plaquette stabilizers, where  $V = F = L^2$ . Then, the number of encoded logical qubits is

$$k = E - V - F + 2. \quad (5.63)$$

By rearranging and plugging in the Euler characteristic formula given in [Eq. \(5.1\)](#), we obtain

$$k = 2 - \chi. \quad (5.64)$$

If we take  $\chi = 2 - 2g$  for a genus- $g$  surface, then the number of encoded qubits is again  $k = 2g = 2$ , since the torus is a genus  $g = 1$  surface.

By identifying the independent generators of  $\mathcal{L}_{\text{tc}}$ , we can also identify the  $X$ - and  $Z$ - logical operators and find the ones with minimum weight to determine the code distance. Note that for a 1-chain  $c$ , the weight of  $Z_c$  is simply the number of edges (1-cells) in  $c$ . For example, for a 1-chain  $c = e_1 + e_2 + e_3$ , the weight of  $Z_c = Z(e_1)Z(e_2)Z(e_3)$  is three. Similar statements can be made for 1-cochain  $c^*$  and  $X_{c^*}$ . Now note that for a toric code, the coset  $[\ell_1]$  consists of a non-contractible 1-chain loop of the form  $\ell_1 + b$  for a boundary  $b \in B_1$ , whereas  $[\lambda_1]$  consists of a non-contractible 1-cochain loop of the form  $\lambda_1 + b^*$  for a coboundary  $b^* \in B^1$ ; see, for example, [Eqs. \(5.15\) and \(5.38\)](#). Note that non-contractible loops in  $\{\ell_i\}_{i=1}^3 \subseteq [\ell_1]$  and  $\{\lambda_i\}_{i=1}^3 \subseteq [\lambda_1]$  are the ones with the least number of 1-cells (edges) in it. On an  $L \times L$  torus, each of these non-contractible 1-chains 1-cochains has exactly  $L$  1-cells (edges) in it. Thus, for an  $L \times L$  toric code, its code distance must be  $d = L$ . Therefore, the toric code has parameters  $[[2L^2, 2, L]]$ .

### 5.2.3 Detecting and correcting errors

The final thing about the toric code that we discuss is how errors are detected and corrected. Since the toric code is a CSS code, we can treat bit-flip and phase-flip errors separately. The  $X$ -type errors are detected by plaquette measurements, whereas  $Z$ -type errors are detected by vertex measurements. To see this, consider first a single  $Z$  error on a qubit (edge). A  $Z$  operator commutes with every plaquette operator, but it anticommutes with the two vertex operators adjacent to that edge, since each of those contains a Pauli- $X$  acting on the same qubit. Therefore, a single  $Z$  error

flips the measurement outcome of exactly two vertex stabilizers from  $+1$  to  $-1$ . In the  $3 \times 3$  toric code example, this can be illustrated as:



where the  $Z$  error flips vertex stabilizers  $X_{v_8}$  and  $X_{v_9}$ . Now consider two adjacent  $Z$  errors, that is, the edges where the  $Z$  errors happened share a common vertex:

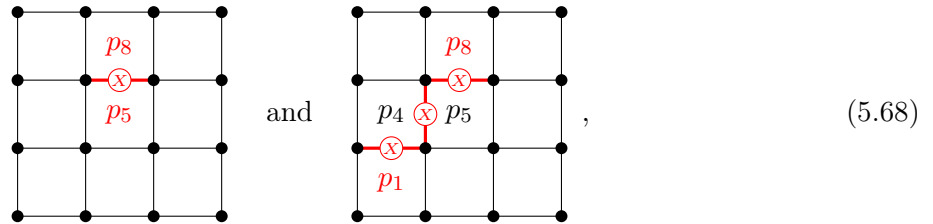


In this case, the vertex stabilizer  $X_{v_9}$  is no longer flipped since the two  $Z$  errors commute with it, however there are still two flipped vertex stabilizers:  $X_{v_8}$  and  $X_{v_6}$ . If there are  $Z$  errors on two non-adjacent edges, we have

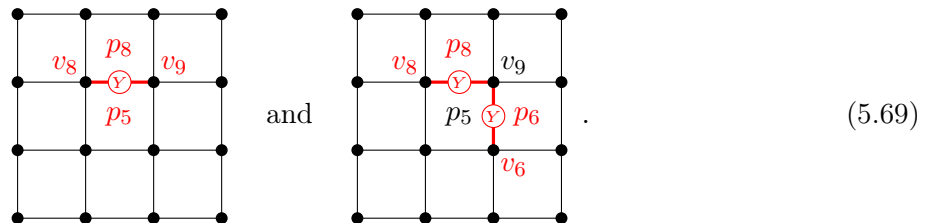


where the flipped vertex stabilizers are the pairs at the endpoints of the two strings of  $Z$  errors. In general, any combination of  $Z$  errors flips an even number of vertex stabilizers.

Similarly, a single  $X$  error anticommutes with the two plaquette operators adjacent to the corresponding edge, and so it creates two violated plaquette stabilizers. In the  $3 \times 3$  toric code, examples of  $X$  errors can be illustrated as:



where the single  $X$  error flips plaquette stabilizers  $Z_{p_5}$  and  $Z_{p_8}$ , whereas the string of three  $X$  errors flips plaquette stabilizers  $Z_{p_1}$  and  $Z_{p_8}$ . Finally, a  $Y$  error is simply the product of an  $X$  and a  $Z$  error (up to a phase), and so it flips both types of stabilizers. For example,



Naturally we can generalize this to any chain of  $X$  (resp.,  $Z$ ) errors where it flips an even number of plaquette (resp., vertex) stabilizers. In the primal-dual lattice picture,  $Z$  errors form strings on the edges of the primal lattice, while  $X$  errors form strings on the edges of the dual lattice.

Error correction proceeds by first measuring all stabilizer generators to obtain error syndromes used by the decoder to infer errors. An error (which may happen on multiple qubits/edges) therefore causes syndrome flips on vertices and plaquettes. As we have established in the discussion above, the number of syndromes from errors is even. A common decoding strategy pairs syndromes globally and chooses a likely set of recovery paths by assigning error probabilities on each edge as its weight. For example, if the probability of  $X$  and  $Z$  error on any edge is equal the inferred corrections are

$$\begin{array}{ccc}
 \begin{array}{|c|c|c|c|} \hline \bullet & \bullet & \bullet & \bullet \\ \hline \bullet & \text{v}_8 \text{---} \text{Z} & \bullet & \bullet \\ \hline \bullet & \bullet & \text{Z} \text{---} \text{v}_6 & \bullet \\ \hline \bullet & \bullet & \bullet & \bullet \\ \hline \end{array} & + & \begin{array}{|c|c|c|c|} \hline \bullet & \bullet & \bullet & \bullet \\ \hline \bullet & \text{Z} & \bullet & \text{v}_6 \\ \hline \bullet & \text{Z} & \bullet & \bullet \\ \hline \bullet & \bullet & \bullet & \bullet \\ \hline \end{array} = \begin{array}{|c|c|c|c|} \hline \bullet & \bullet & \bullet & \bullet \\ \hline \bullet & \text{v}_8 \text{---} \text{Z} & \bullet & \bullet \\ \hline \bullet & \text{Z} \text{---} \text{p}_5 & \text{Z} \text{---} \text{v}_6 & \bullet \\ \hline \bullet & \bullet & \bullet & \bullet \\ \hline \end{array} \\
 & & \text{and} & \\
 \begin{array}{|c|c|c|c|} \hline \bullet & \bullet & \bullet & \bullet \\ \hline \bullet & \text{p}_8 & \bullet & \bullet \\ \hline \bullet & \text{X} & \bullet & \bullet \\ \hline \bullet & \bullet & \bullet & \bullet \\ \hline \end{array} & + & \begin{array}{|c|c|c|c|} \hline \bullet & \bullet & \bullet & \bullet \\ \hline \bullet & \text{p}_8 & \bullet & \bullet \\ \hline \bullet & \text{X} & \bullet & \bullet \\ \hline \bullet & \bullet & \bullet & \bullet \\ \hline \end{array} = \begin{array}{|c|c|c|c|} \hline \bullet & \bullet & \bullet & \bullet \\ \hline \bullet & \text{p}_8 & \bullet & \bullet \\ \hline \bullet & \text{I} & \bullet & \bullet \\ \hline \bullet & \bullet & \bullet & \bullet \\ \hline \end{array},
 \end{array} \tag{5.70}$$

where the product between the correction on the blue edges for the 2-qubit  $Z$  error on the red edges results in a 4-qubit  $Z$  Pauli operator that is equal to the plaquette stabilizer  $Z_{p_5}$ , thus it acts trivially on any codestate. On the other hand, the correction on the single-qubit  $X$  error, directly cancels it out to an identity on the faulty qubit.

Note that distinct errors may flip the same stabilizers. For example, the following distinct two-qubit  $Z$  error flips stabilizers  $X_{v_8}$  and  $X_{v_6}$ :

$$\begin{array}{ccc}
 \begin{array}{|c|c|c|c|} \hline \bullet & \bullet & \bullet & \bullet \\ \hline \bullet & \text{v}_8 \text{---} \text{Z} & \bullet & \bullet \\ \hline \bullet & \bullet & \text{Z} \text{---} \text{v}_6 & \bullet \\ \hline \bullet & \bullet & \bullet & \bullet \\ \hline \end{array} & \text{and} & \begin{array}{|c|c|c|c|} \hline \bullet & \bullet & \bullet & \bullet \\ \hline \bullet & \text{v}_8 & \bullet & \bullet \\ \hline \bullet & \text{Z} & \text{Z} & \text{v}_6 \\ \hline \bullet & \bullet & \bullet & \bullet \\ \hline \end{array} .
 \end{array} \tag{5.71}$$

However, this is not an issue in decoding when the error probability on any edge is equal, since the two possible shortest strings between  $v_6$  and  $v_8$  are

$$\begin{array}{ccc}
 \begin{array}{|c|c|c|c|} \hline \bullet & \bullet & \bullet & \bullet \\ \hline \bullet & \text{v}_8 & \bullet & \bullet \\ \hline \bullet & \text{Z} & \text{Z} & \text{v}_6 \\ \hline \bullet & \bullet & \bullet & \bullet \\ \hline \end{array} & \text{and} & \begin{array}{|c|c|c|c|} \hline \bullet & \bullet & \bullet & \bullet \\ \hline \bullet & \text{v}_8 & \bullet & \bullet \\ \hline \bullet & \text{Z} & \text{Z} & \text{v}_6 \\ \hline \bullet & \bullet & \bullet & \bullet \\ \hline \end{array} .
 \end{array} \tag{5.72}$$

Whichever two-qubit  $Z$  error occurred, any of the two-qubit  $Z$  corrections either cancel out the

error or turn it into the plaquette stabilizer  $Z_{p_5}$ , that is, a contractible boundary:

and

(5.73)

There are other errors that flip the same stabilizers that cause a logical error when using this decoding strategy. For example,

and

(5.74)

If the probability of a  $Z$  error on any edge is equal, then the decoder will infer a correction for the 3-qubit  $Z$  error similarly to how it will infer a correction for the 2-qubit  $Z$  error (since it only sees that  $X_{v_8}$  and  $X_{v_6}$  are flipped):

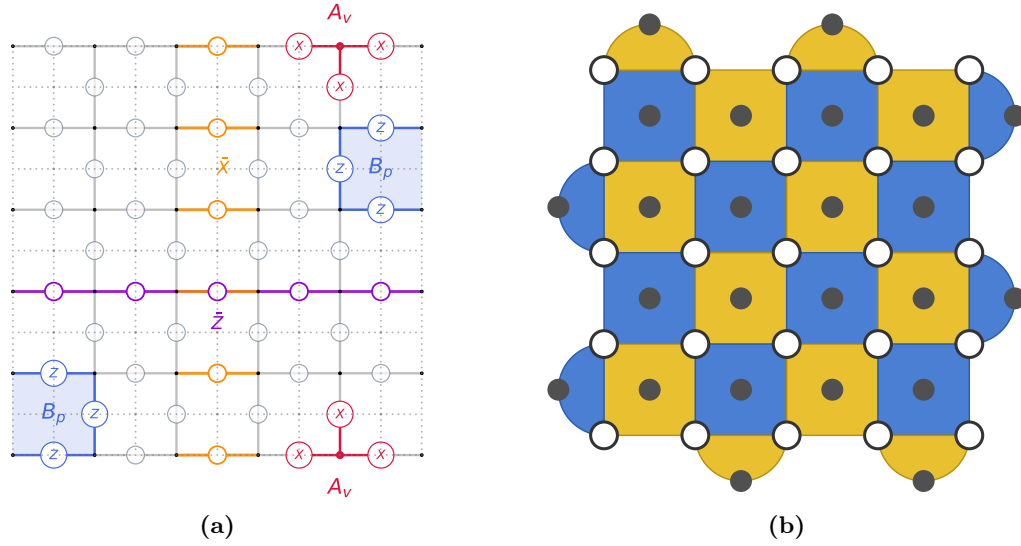
(5.75)

Note that the product between the 3-qubit error and the 2-qubit correction forms a non-contractible loop, namely a logical  $Z$  operator. This then causes a logical error, since we are effectively applying a logical  $Z$  unitary on the code through a bad correction. For a larger code and more errors, decoding gets more complicated as the number of flipped vertices and plaquettes increases, which can be addressed using a decoder such as *minimum-weight perfect matching* (MWPM). A more detailed discussion on MWPM and other decoding strategies will be covered in [Section 9](#).

### 5.3 Surface code

Practically, implementing a toric code on a physical device with connectivity or architectural constraints is challenging. One solution is to cut the torus open, laying it flat to build the code on a planar graph. This leads to a planar variant of the [surface code](#) [\[50\]](#) on a square lattice with *open*





**Figure 10:** Distance-5 unrotated and rotated surface codes. (a) A  $5 \times 5$  unrotated planar patch with open boundary conditions. Qubits are placed on edges. Interior plaquette and vertex checks have weight 4, while boundary checks have weight 3. The left and right boundaries are rough and the top and bottom boundaries are smooth. (b) A rotated surface code of distance 5. Hollow circles and gray circles denote data and syndrome qubits, respectively. Boundary syndrome measurements involve weight-2 checks. Blue and yellow plaquettes represent  $Z$ - and  $X$ -type stabilizer measurements.

boundaries, which is commonly referred to as the *unrotated surface code*. These open boundary conditions become the defining feature that results in a topology and code structure that are different from that of the toric code. As we will see, two important distinctions of the unrotated surface code from the toric code are that its stabilizers are of two different types: *interior* and *boundary* stabilizers, and that it encodes only one logical qubit instead of two.

To define the stabilizers, we first need to look into how boundaries and coboundaries are defined. Consider the following  $4 \times 4$  planar lattice with 12 vertices, 12 faces<sup>7</sup>, and  $16 + 9 = 25$  edges:



As opposed to the lattices on the surface of the torus with periodic boundary conditions, here we have a lattice with open boundaries. In fact, as shown above, there are two types of boundaries, the top and bottom boundaries of the lattice are *smooth* boundaries, whereas the right and left boundaries of the lattice are *rough* boundaries. Also, note that on each rough boundary we have three “half-plaquettes” or “half-faces,” which are plaquettes with only three edges around them. In total, there are six such plaquettes, which we label as  $\bar{p}_1, \dots, \bar{p}_6$  besides the ordinary plaquettes

<sup>7</sup>We are counting the three spaces between the edges sticking out to the left and three other spaces between the edges sticking out to the right as a special type of faces, which we will discuss in more detail shortly.

$p_1, \dots, p_6$  in the interior:

$$(5.77)$$

These are all of the 2-cells. Similarly, on each smooth boundary, we have three “half-vertices,” which are vertices with only three edges around them. In total, there are six such vertices which we label as  $\bar{v}_1, \dots, \bar{v}_6$  besides the ordinary vertices  $v_1, \dots, v_6$  in the interior:

$$(5.78)$$

These are all of the 0-cells. Since the half-plaquettes and half-vertices have only three edges around them, their boundaries are given by

$$(5.79)$$

Then, the boundary and coboundary groups are defined as

$$B_1 = \langle \partial_2 p : \text{plaquette } p \rangle \quad \text{and} \quad B^1 = \langle \delta^0 v : \text{vertex } v \rangle, \quad (5.80)$$

respectively. By again placing a qubit on the edges, the vertex and plaquette stabilizers can then be defined similarly to the toric code on the (half-) vertices and (half-) plaquettes (see also [Figure 10a](#)):

$$X_v = \prod_{e \in \delta^0 v} X(e) \quad \text{and} \quad Z_p = \prod_{e \in \partial_2 p} Z(e), \quad (5.81)$$

respectively. So, the stabilizer group of the unrotated surface code is given by

$$\mathcal{S}_{\text{sur}} = \langle X_{c^*}, Z_c : c \in B_1, c^* \in B^1 \rangle. \quad (5.82)$$

The underlying planar region is topologically a disk. If we were to compute its standard first homology group (resp., cohomology group) as we did with the torus, then the result would be trivial because every closed loop on a disk is the boundary of a patch of faces (and dually, every dual loop is a coboundary). Furthermore, standard homology and cohomology require cycles to be strictly closed loops, whereas the logical operators of the planar surface code are open strings spanning opposite sides of the primary or dual lattices.

To accommodate these open strings rigorously, we use *relative homology* and *relative cohomology*. Under this framework, any 1-chain whose boundary endpoints terminate entirely within the rough boundaries is treated as having a “null” boundary, allowing it to function mathematically as a non-trivial relative cycle. Symmetrically, 1-cochains terminating on the smooth boundaries function as relative cocycles. From this point on,  $C_n, H_1, Z_1, B_1$  (resp.,  $C^n, H^1, Z^1, B^1$ ) denote the relative  $n$ -chains, homology, cycle, and boundary groups modulo the rough boundaries (resp., relative  $n$ -cochains, cohomology, cocycle, and coboundary groups modulo the smooth boundaries) on the planar surface. These non-contractible 1-chains (connecting rough boundaries) and 1-cochains (connecting smooth boundaries) are

as well as their sum with boundaries and coboundaries, respectively, (e.g.,  $\ell_1 + \partial_2 p$  and  $\lambda_2 + \delta^0 v$ ). These non-contractible 1-chains define the cycle and cocycle groups:

$$Z_1 = \langle \{\ell_1, \dots, \ell_4\} \cup B_1 \rangle \quad \text{and} \quad Z^1 = \langle \{\lambda_1, \dots, \lambda_4\} \cup B^1 \rangle. \quad (5.84)$$

Similar to the toric code, the normalizer is given by

$$\mathcal{N}(\mathcal{S}_{\text{sur}}) = \langle \{X_{c^*}, Z_c : c \in Z_1, c^* \in Z^1\} \cup \{\pm i\} \rangle. \quad (5.85)$$

Similar to our analysis for the toric code in [Section 5.2](#), we can identify the logical operators, the number of encoded logical qubits  $k$ , and the distance  $d$  of the unrotated surface code from the homology group  $H_1 = Z_1/B_1$  and cohomology group  $H^1 = Z^1/B^1$ . By the same analysis as we have done for the torus homology we can find that the cosets in  $H_1$  are of the form

$$[\ell_1] = \{\ell_1 + b : b \in B_1\} \quad (5.86)$$

and since the non-contractible 1-chains are equivalent up to a boundary, the homology group  $H_1$  is generated by  $[\ell_1]$ . Similarly, the cosets in  $H^1$  are of the form

$$[\lambda_1] = \{\lambda_1 + b^* : b^* \in B^1\} \quad (5.87)$$

and since the non-contractible 1-cochains are equivalent up to a coboundary, the cohomology group  $H^1$  is generated by  $[\lambda_1]$ . Similar to the toric code, it can also be shown that the quotient group  $\mathcal{N}(\mathcal{S}_{\text{sur}})/\mathcal{S}_{\text{sur}}$ , up to phases, denoted by  $\mathcal{L}_{\text{sur}}$ , is isomorphic to  $H_1 \oplus H^1$ . So, representatives of the logical  $X$  and  $Z$  operators of the unrotated surface code are

$$X_{\lambda_1} = \begin{array}{|c|c|c|} \hline \bullet & \bullet & \bullet \\ \hline \bullet & \bullet & \bullet \\ \hline \bullet & \bullet & \bullet \\ \hline \bullet & \bullet & \bullet \\ \hline \end{array} \quad \text{and} \quad Z_{\ell_1} = \begin{array}{|c|c|c|} \hline \bullet & \bullet & \bullet \\ \hline \bullet & \bullet & \bullet \\ \hline \bullet & \bullet & \bullet \\ \hline \bullet & \bullet & \bullet \\ \hline \end{array} . \quad (5.88)$$

See also Figure 10a for a primal-dual lattice illustration.

Now, we can determine the number of encoded logical qubits  $k$  and the code distance  $d$  from  $H_1$  and  $H^1$ . Since there are  $2k = 2$  independent generators of  $H_1 \oplus H^1$ , there is  $k = 1$  encoded logical qubit. On the other hand, the minimum weight of non-contractible 1-chains and 1-cochains in the cosets  $[\ell_1]$  and  $[\lambda_1]$  is 4. A pair of 1-chain and 1-cochain with weight 4 are  $\ell_1$  and  $\lambda_1$ , so that the logical operators  $Z_{\ell_1}$  and  $X_{\lambda_1}$  are the minimum weight logical operators. Thus, we conclude that the code distance is  $d = 4$ . Finally, since there are 16 horizontal edges and 9 vertical edges, there are  $n = 16 + 9 = 25$  physical qubits. So, the  $4 \times 4$  unrotated surface code is a  $[[25, 1, 4]]$  code.

One can easily generalize this analysis to an  $L \times L$  unrotated surface code. We will find that for any  $L$ , the homology and cohomology groups are each generated by one coset. Thus, the code always encodes  $k = 1$  logical qubit for any  $L$ . The minimum-weight non-contractible 1-chains/1-cochains are always  $L$  since any non-contractible 1-chains/1-cochains terminating at the same type of boundaries must have a minimum weight of  $L$ . So the code distance must be  $d = L$ . Since there are  $L^2$  horizontal edges and  $(L - 1)^2$  vertical edges in such lattice, an  $L \times L$  unrotated surface code is an  $[[L^2 + (L - 1)^2, 1, L]]$  code.

We note that a variant of the unrotated surface code that has been rotated  $45^\circ$  is a popular choice in experimental demonstrations of QEC [26, 31, 37, 105]. Compared to its unrotated counterpart, the rotated surface code requires fewer physical qubits to encode the same number of logical qubits  $k$  with the same distance  $d$ . For example, in Figure 10b, the rotated surface code with distance 5 uses 25 physical qubits while the unrotated surface code with distance 5 uses 41 physical qubits.

Finally, we briefly discuss the errors on the lattice. Similar to the toric code, errors on the lattice are represented as 1-chains of Pauli operators. However, there are errors that only flip one stabilizer, namely those on the qubits on boundaries. For example,

$$\begin{array}{|c|c|c|} \hline \bullet & \bullet & \bullet \\ \hline \bullet & \bullet & \bullet \\ \hline \bullet & \bullet & \bullet \\ \hline \bullet & \bullet & \bullet \\ \hline \end{array} \quad \text{and} \quad \begin{array}{|c|c|c|} \hline \bullet & \otimes & \bullet \\ \hline \bullet & \bullet & \bullet \\ \hline \bullet & \bullet & \bullet \\ \hline \bullet & \bullet & \bullet \\ \hline \end{array} . \quad (5.89)$$

On the other hand, errors in the interior are detected similarly to the toric code as they flip two stabilizers. For example,

$$\begin{array}{|c|c|c|} \hline \bullet & \bullet & \bullet \\ \hline \bullet & \bullet & \bullet \\ \hline \bullet & \bullet & \bullet \\ \hline \bullet & \bullet & \bullet \\ \hline \end{array} \quad \text{and} \quad \begin{array}{|c|c|c|} \hline \bullet & \bullet & \bullet \\ \hline \bullet & \otimes & \bullet \\ \hline \bullet & \bullet & \bullet \\ \hline \bullet & \bullet & \bullet \\ \hline \end{array} . \quad (5.90)$$

So, a multi-qubit error that forms a 1-chain that terminates on a boundary flips a stabilizer:

$$\text{and} \quad (5.91)$$

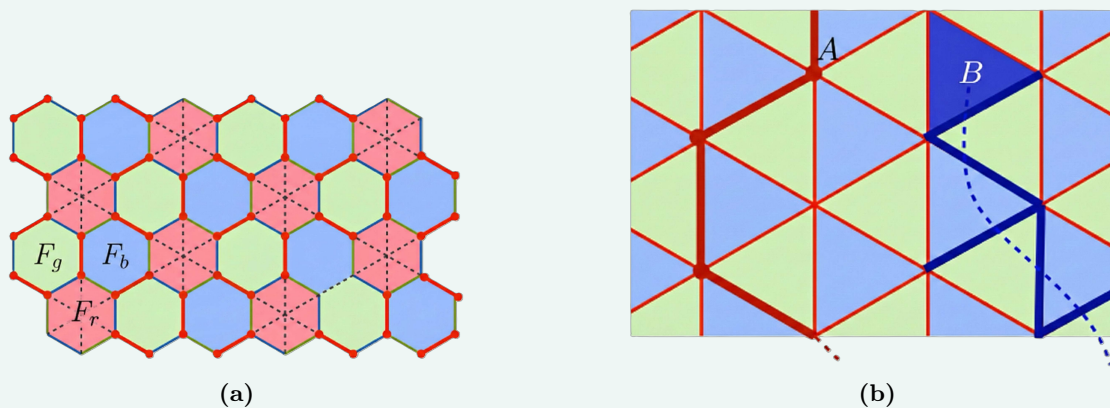
Similar to the toric code, we need to choose a 1-chain of Pauli- $Z$  (resp., 1-cochain of Pauli- $X$ ) corrections that connect each pair of flipped vertex stabilizers (resp., plaquette stabilizers) caused by  $Z$  errors (resp.,  $X$  errors) such that the product between the corrections and the errors is a stabilizer. However, a chain of  $Z$  errors (resp., a cochain of  $X$  errors) with exactly one endpoint on the rough (resp., smooth) boundary only flips one vertex stabilizer (resp., plaquette stabilizer), such as the ones in Eq. (5.91). Thus, a similar decoding strategy to the one we had for the toric code can be used. Similarly, we assign weights to edges corresponding to the probability of a certain error. However, now we need to find an error chain (resp., cochain) with an endpoint on a rough (resp., smooth) boundary, so the decoder must find a path from the flipped vertex stabilizer (resp., plaquette stabilizer) to a rough boundary (resp., smooth boundary) with the lowest weight. More details on matching decoders can be found in Section 9.

### Box 5.1: Color code

The surface code is not the only important two-dimensional topological CSS code. A closely related family is the **color code** [\[106\]](#), which is defined on a trivalent lattice whose faces can be colored with three colors, usually red, green, and blue, such that adjacent faces have different colors. Unlike the toric and surface codes discussed above, where physical qubits live on edges, a 2D color code places physical qubits on vertices. In the honeycomb example below, each hexagonal face supports two stabilizer generators with the same geometric support:

$$X_f = \bigotimes_{v \in f} X_v, \quad Z_f = \bigotimes_{v \in f} Z_v. \quad (5.92)$$

Because the lattice is trivalent and face-3-colorable, any two faces overlap on either zero or two vertices. This even-overlap property ensures that all  $X_f$  and  $Z_{f'}$  stabilizers commute, so the construction gives a CSS stabilizer code.



**Figure 11:** (a) A 3-colorable honeycomb lattice for a 2D color code, with  $F_r$ ,  $F_g$ , and  $F_b$  denoting the sets of red, green, and blue faces, respectively. (b) String operators drawn on the red shrunk lattice. Figures are adapted from Ref. [106, Figs. 1 and 3]

The logical operators can also be understood topologically. Instead of using strings on the original lattice directly, it is often convenient to use a *shrunk lattice*: choose one face color, shrink every face of that color to a vertex, and connect neighboring faces through the qubits between them. Closed strings on the shrunk lattice define Pauli string operators on the original physical qubits. As in the surface code, contractible closed strings are products of stabilizers, while noncontractible closed strings represent logical operators. The key difference is that each homology class has colored representatives, and only two of the three color choices are independent. This extra color degree of freedom is why, on a closed surface with Euler characteristic  $\chi$ , the 2D color code encodes

$$k_{\text{color}} = 4 - 2\chi \quad (5.93)$$

logical qubits, twice the number  $k_{\text{surface}} = 2 - \chi$  for the corresponding surface-code construction on the same closed surface. For example, on a torus where  $\chi = 0$ , the toric/surface-code construction encodes two logical qubits, while the 2D color code encodes four.

Thus, color codes share the same broad topological principle as surface codes: local stabilizer violations mark endpoints of string-like errors, and nontrivial homology classes support logical operators. Their distinctive feature is the colored, self-dual lattice structure, with qubits on vertices and both  $X$ - and  $Z$ -type checks attached to each face. Color codes also have distinctive logical-gate properties, but those depend on lattice and boundary details.

## 6 Subsystem codes

The central idea of a subsystem code is to encode quantum information not in a subspace of the physical Hilbert space itself, but in a tensor factor of a subspace. This generalization of the standard stabilizer formalism offers significant advantages, most notably the ability to construct codes where the check operators (measurements) have much lower weight than the stabilizers required to define the same codespace. In this chapter, we motivate subsystem codes using the standard Shor code, before rigorously defining the formalism and introducing the canonical example of a subsystem code: the Bacon–Shor code. We also discuss the Knill–Laflamme conditions in the context of subsystem codes (c.f. [Theorem 3.1](#)) and other examples of subsystem codes.

### 6.1 Motivation: the 9-qubit Shor code

To understand the utility of subsystem codes, let us revisit the standard 9-qubit Shor code, which we introduced in [Section 3.3](#). Recall, the Shor code protects against arbitrary single-qubit errors by concatenating a 3-qubit phase-flip repetition code with a 3-qubit bit-flip repetition code. The logical states are:

$$|\bar{0}\rangle = \frac{1}{\sqrt{8}}(|000\rangle + |111\rangle)^{\otimes 3}, \quad |\bar{1}\rangle = \frac{1}{\sqrt{8}}(|000\rangle - |111\rangle)^{\otimes 3}. \quad (6.1)$$

We may arrange the 9 physical qubits as three blocks of three qubits each, or more precisely, in a  $3 \times 3$  grid. The stabilizer group  $\mathcal{S}$  for the Shor code is generated by local weight-2  $Z$ -checks (detecting bit-flips within blocks) and nonlocal weight-6  $X$ -checks (detecting phase-flips between blocks). Specifically, the generators are:

- $X$ -checks:  $\begin{smallmatrix} X & X & X \\ X & X & X \\ I & I & I \end{smallmatrix}, \begin{smallmatrix} I & I & I \\ X & X & X \\ X & X & X \end{smallmatrix}$
- $Z$ -checks:  $\begin{smallmatrix} Z & Z & I \\ I & I & I \\ I & I & I \end{smallmatrix}, \begin{smallmatrix} I & Z & Z \\ I & I & I \\ I & I & I \end{smallmatrix}, \begin{smallmatrix} Z & Z & I \\ I & I & I \\ I & I & I \end{smallmatrix}, \begin{smallmatrix} I & I & I \\ I & Z & Z \\ I & I & I \end{smallmatrix}, \begin{smallmatrix} I & I & I \\ Z & Z & I \\ I & I & I \end{smallmatrix}, \begin{smallmatrix} I & I & I \\ I & I & I \\ I & Z & Z \end{smallmatrix}$

While the  $Z$ -checks are local, the  $X$ -checks are high-weight. High-weight stabilizers are experimentally challenging, since they require deeper quantum circuits for syndrome measurement, increasing the probability of errors occurring during the measurement process itself.

Can we reduce the weight of the  $X$ -stabilizers? Suppose we attempt to measure operators with lower weight that multiply to form the original stabilizers. For the 9-qubit Shor code, we could replace the weight-6  $X$ -checks with weight-2 checks acting on neighbors.

Let us define a modified Shor code (or the  $3 \times 3$  *Bacon–Shor code*). As before, we arrange the 9 qubits in a  $3 \times 3$  grid. Instead of measuring weight-6 operators, we measure the following weight-2 operators:

- Gauge  $X$ -checks:  $\begin{smallmatrix} X & I & I \\ I & I & I \\ I & I & I \end{smallmatrix}, \begin{smallmatrix} I & X & I \\ I & I & I \\ I & I & I \end{smallmatrix}, \begin{smallmatrix} I & I & X \\ I & I & I \\ I & I & I \end{smallmatrix}, \begin{smallmatrix} X & I & I \\ I & I & I \\ I & I & I \end{smallmatrix}, \begin{smallmatrix} I & I & I \\ I & X & I \\ I & I & I \end{smallmatrix}, \begin{smallmatrix} I & I & I \\ I & I & X \\ I & I & I \end{smallmatrix}$  (vertical neighbors)
- Gauge  $Z$ -checks:  $\begin{smallmatrix} Z & Z & I \\ I & I & I \\ I & I & I \end{smallmatrix}, \begin{smallmatrix} I & Z & Z \\ I & I & I \\ I & I & I \end{smallmatrix}, \begin{smallmatrix} I & I & I \\ Z & Z & I \\ I & I & I \end{smallmatrix}, \begin{smallmatrix} I & I & I \\ I & Z & Z \\ I & I & I \end{smallmatrix}, \begin{smallmatrix} I & I & I \\ Z & Z & I \\ I & I & I \end{smallmatrix}, \begin{smallmatrix} I & I & I \\ I & I & I \\ I & Z & Z \end{smallmatrix}$  (horizontal neighbors)

These lower-weight operators do *not* all commute. For example, a vertical  $XX$  pair might anticommute with a horizontal  $ZZ$  pair if they share a qubit. Because they do not commute, they cannot all be elements of a stabilizer group (which must be abelian).

However, notice that the original weight-6  $X$ -stabilizers of the Shor code can be reconstructed from these lower-weight operators. For example, the product of the three vertical  $XX$  checks along the first two rows generates the weight-6  $X$ -stabilizer on those two rows. This introduces the core

concept of subsystem codes: we measure non-commuting “gauge” operators. The actual stabilizers of the code are the specific products of these gauge operators that commute with all other gauge operators. The measurement values of the individual non-commuting gauge operators do not carry any logical information by themselves.

## 6.2 The mathematical framework of subsystem codes

We now formalize the structure of subsystem codes [97, 107, 108].

### 6.2.1 Hilbert space decomposition

Let  $\mathcal{H} = (\mathbb{C}^2)^{\otimes n}$  be the Hilbert space of  $n$  qubits. Rather than protecting a subspace  $\mathcal{C} \subseteq \mathcal{H}$ , we decompose  $\mathcal{C}$  as  $\mathcal{C} \cong \mathcal{H}_L \otimes \mathcal{H}_G$ , where  $\mathcal{H}_L$  carries the logical information we wish to protect and  $\mathcal{H}_G$  is a gauge subsystem whose state can be changed without affecting the encoded information. Formally, we define a subsystem code as follows.

#### Definition 6.1: Subsystem code

An  $[[n, k, g, d]]$  subsystem code is defined by a decomposition of the Hilbert space  $\mathcal{H} = (\mathbb{C}^2)^{\otimes n}$  of  $n$  qubits as:

$$\mathcal{H} = \mathcal{C} \oplus \mathcal{C}^\perp = (\mathcal{H}_L \otimes \mathcal{H}_G) \oplus \mathcal{C}^\perp \quad (6.2)$$

where:

- $\mathcal{C}$  is the code subspace,
- $\mathcal{H}_L$  is the logical subsystem with  $\dim \mathcal{H}_L = 2^k$ , and
- $\mathcal{H}_G$  is the gauge subsystem with  $\dim \mathcal{H}_G = 2^g$ .

The parameters  $n, k, g, d$  are defined as follows:

- $n$ : Number of physical qubits, i.e.,  $\log_2 \dim \mathcal{H}$ .
- $k$ : Number of logical qubits, i.e.,  $\log_2 \dim \mathcal{H}_L$ .
- $g$ : Number of gauge qubits, i.e.,  $\log_2 \dim \mathcal{H}_G$ .
- $d$ : Distance of the code, i.e., minimum weight of a nontrivial logical operator on the code.

### 6.2.2 Operator description

As with stabilizer codes, it is more common to define a subsystem code in terms of Pauli operators. However, beyond having to define the stabilizer group, we also need to define the *gauge group* for the code. Both of these are subgroups of the  $n$ -qubit Pauli group  $\mathcal{P}_n$  and are defined as follows:

- Stabilizer group  $\mathcal{S}$ : An abelian subgroup of  $\mathcal{P}_n$  *not* containing  $-I$ . The common  $(+1)$ -eigenspace of the operators in  $\mathcal{S}$  is the codespace  $\mathcal{C}$  of the subsystem code. Thus, elements of  $\mathcal{S}$  act on the codespace  $\mathcal{C} = \mathcal{H}_L \otimes \mathcal{H}_G$  as the identity  $I_{\mathcal{H}_L} \otimes I_{\mathcal{H}_G}$ .
- Gauge group  $\mathcal{G}$ : A (not necessarily abelian) subgroup of  $\mathcal{P}_n$  whose center  $Z(\mathcal{G})$  is equal to  $\langle \mathcal{S}, iI \rangle$ . Elements of  $\mathcal{G}$  act on the codespace  $\mathcal{C} = \mathcal{H}_L \otimes \mathcal{H}_G$  as  $I_{\mathcal{H}_L} \otimes g_{\mathcal{H}_G}$ , where  $g_{\mathcal{H}_G}$  is an



operator on  $\mathcal{H}_G$ . These are the operators we are allowed to measure.

A standard stabilizer code is simply the special case where  $\dim(\mathcal{H}_G) = 1$  (the gauge subsystem is trivial), and consequently  $\mathcal{G} = \langle \mathcal{S}, iI \rangle$ .

### 6.2.3 Logical operators

Logical operators are operations that act nontrivially on subsystem  $\mathcal{H}_L$  while preserving the codespace  $\mathcal{C} = \mathcal{H}_L \otimes \mathcal{H}_G$ . They are elements of the centralizer of the stabilizer group, denoted  $C(\mathcal{S})$ , that are not in  $\mathcal{G}$  (since elements in  $\mathcal{G}$  act trivially on  $\mathcal{H}_L$ ). We distinguish between two types:

- Bare logicals: Operators  $L \in C(\mathcal{G})$  that are not in  $\mathcal{G}$ . These commute with all gauge operators and act as  $L_{\mathcal{H}_L} \otimes I_{\mathcal{H}_G}$ , where  $L_{\mathcal{H}_L} \neq I_{\mathcal{H}_L}$ .
- Dressed logicals: Operators  $L \in C(\mathcal{S})$  that are not in  $\mathcal{G}$ . These commute with the stabilizers but may anticommute with some gauge operators. They act as  $L_{\mathcal{H}_L} \otimes L_{\mathcal{H}_G}$ , where  $L_{\mathcal{H}_L} \neq I_{\mathcal{H}_L}$ . (We adopt the convention that bare logicals are also dressed logicals, but it should be noted that some authors exclude bare logicals from the definition of dressed logicals.)

In practice, dressed logicals are important because a bare logical multiplied by any gauge operator is a dressed logical that performs the same logical operation on the logical subsystem  $\mathcal{H}_L$ .

### 6.2.4 Description in terms of virtual Pauli operators

A convenient way to understand subsystem codes is to introduce the notion of virtual Pauli operators [97]. Given an  $[[n, k, g, d]]$  subsystem code, one can construct  $n$  pairs of “virtual” Pauli  $X$  and  $Z$  operators  $\{X'_1, Z'_1, X'_2, Z'_2, \dots, X'_n, Z'_n\}$  from  $\mathcal{P}_n$  satisfying the usual commutation and anticommutation relations:

1.  $[X'_j, X'_\ell] = [Z'_j, Z'_\ell] = [X'_j, Z'_\ell] = 0$  for all distinct  $j, \ell \in \{1, \dots, n\}$ ,
2.  $X'_j Z'_j = -Z'_j X'_j$  for all  $j \in \{1, \dots, n\}$ , and
3.  $(X'_j)^2 = (Z'_j)^2 = I$  for all  $j \in \{1, \dots, n\}$ ,

such that the stabilizer group is given by  $\mathcal{S} = \langle Z'_1, \dots, Z'_s \rangle$  for  $s = n - k - g$ , and the gauge group is given by  $\mathcal{G} = \langle iI, \mathcal{S}, X'_{s+1}, Z'_{s+1}, \dots, X'_{s+g}, Z'_{s+g} \rangle$ . A set of check measurements for the code is thus  $\{Z'_1, \dots, Z'_s, X'_{s+1}, Z'_{s+1}, \dots, X'_{s+g}, Z'_{s+g}\}$ . Representatives for the logical group are given by  $\{X'_{s+g+1}, Z'_{s+g+1}, \dots, X'_n, Z'_n\}$ .

## 6.3 The Bacon–Shor code

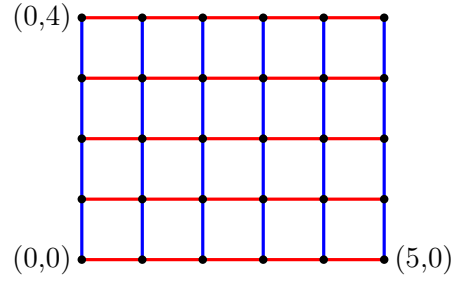
The [Bacon–Shor code](#) [\[109\]](#) is the canonical example of a subsystem code, defined on an  $M \times N$  rectangular lattice of qubits.

### 6.3.1 Construction

Let qubits be placed at coordinates  $(i, j)$  for  $0 \leq i \leq M - 1$  and  $0 \leq j \leq N - 1$  (with the qubit at the lower left corner labeled  $(0, 0)$ ). The code is defined by the gauge group  $\mathcal{G}$  generated by weight-2 Pauli operators acting on adjacent qubits, namely:

- Horizontal checks:  $X_{i,j} X_{i+1,j}$  for  $0 \leq i \leq M - 2$  and  $0 \leq j \leq N - 1$ .
- Vertical checks:  $Z_{i,j} Z_{i,j+1}$  for  $0 \leq i \leq M - 1$  and  $0 \leq j \leq N - 2$ .

See [Figure 12](#) for the Bacon–Shor code lattice when  $M = 6, N = 5$ .



**Figure 12:** The lattice for a  $6 \times 5$  Bacon–Shor code. Red links represent  $XX$  gauge generators, and blue links represent  $ZZ$  gauge generators. Qubits are located at vertices.

### 6.3.2 Stabilizers and logicals

The stabilizers of the Bacon–Shor code are formed by products of gauge operators that commute with all operators in  $\mathcal{G}$ .

- An  $X$ -stabilizer is formed by taking the product of horizontal  $XX$  checks across two adjacent columns:  $\prod_{j=0}^{N-1} X_{i,j} X_{i+1,j}$  for  $0 \leq i \leq M-2$ . Notice this commutes with all the vertical  $ZZ$  checks.
- A  $Z$ -stabilizer is formed by taking the product of vertical  $ZZ$  gauges across two adjacent rows:  $\prod_{i=0}^{M-1} Z_{i,j} Z_{i,j+1}$  for  $0 \leq j \leq N-2$ . This commutes with all the horizontal  $XX$  checks.

These  $M + N - 2$  stabilizers form a minimal generating set for the stabilizer group  $\mathcal{S}$  of the Bacon–Shor code. The  $(M-1)N + M(N-1) = 2MN - M - N$  gauge check operators listed in [Section 6.3.1](#) form a minimal generating set for the gauge group up to phase. Subtracting the  $M + N - 2$  stabilizers gives  $2MN - 2M - 2N + 2$ , so the number of gauge qubits is  $\frac{1}{2}(2MN - 2M - 2N + 2) = MN - M - N + 1 = (M-1)(N-1)$ . In fact, it is possible to explicitly define the  $(M-1)(N-1)$  pairs of virtual gauge operators in such a way that there is a clear one-to-one correspondence between the gauge qubits and the square plaquettes of the lattice; see [110, Sec. II] for a careful exposition.

There is exactly one logical qubit for the Bacon–Shor code. The logical operators are  $\bar{X} = \prod_{j=0}^{N-1} X_{0,j}$  and  $\bar{Z} = \prod_{i=0}^{M-1} Z_{i,0}$ , which respectively correspond to a chain of  $X$  operators spanning the lattice vertically and a chain of  $Z$  operators spanning the lattice horizontally.

#### Exercise 6.1: Distance of Bacon–Shor code

Show that the distance of the  $M \times N$  Bacon–Shor code is  $\min(M, N)$ .

[Hint: recall that the distance of the code is the minimum weight of a dressed logical of the code.]

The parameters of the Bacon–Shor code are thus  $\llbracket MN, 1, (M-1)(N-1), \min(M, N) \rrbracket$ .

### 6.4 Knill–Laflamme conditions for subsystem codes

How do we determine if a subsystem code can correct a specific set of errors  $\mathcal{E}$ ? For this, we have to generalize the standard Knill–Laflamme conditions.

**Theorem 6.1:** Knill–Laflamme conditions for subsystem codes

Let  $\mathcal{H} = (\mathcal{H}_L \otimes \mathcal{H}_G) \oplus \mathcal{C}^\perp$  be a subsystem code with projector  $P$  onto  $\mathcal{C} = \mathcal{H}_L \otimes \mathcal{H}_G$ . The code  $\mathcal{C}$  can correct a set of errors  $\mathcal{E}$  if and only if for any pair of errors  $E_j, E_\ell \in \mathcal{E}$ , there exists an operator  $g_{j\ell}$  acting on the gauge subsystem  $\mathcal{H}_G$  such that

$$PE_j^\dagger E_\ell P = I_L \otimes g_{j\ell}. \quad (6.3)$$

In the standard stabilizer case ( $\dim \mathcal{H}_G = 1$ ),  $g_{j\ell}$  is simply a scalar  $\alpha_{j\ell}$ , recovering the Knill–Laflamme condition  $PE_j^\dagger E_\ell P = \alpha_{j\ell} P$  for subspace codes. In the subsystem case, the error product  $E_j^\dagger E_\ell$  is allowed to disturb the gauge subsystem  $\mathcal{H}_G$  arbitrarily, provided it acts as the identity (or a scalar) on the logical subsystem  $\mathcal{H}_L$ .

**Exercise 6.2:** Knill–Laflamme condition for Pauli errors

If the errors are Pauli operators, prove that the Knill–Laflamme condition simplifies to the following statement: A set of Pauli errors  $\mathcal{E}$  is correctable if and only if for all  $E_j, E_\ell \in \mathcal{E}$ , the product  $E_j E_\ell$  is not a dressed logical operator, that is,  $E_j E_\ell \notin \mathcal{C}(\mathcal{S}) \setminus \mathcal{G}$ .

## 6.5 Topological subsystem codes

Subsystem codes naturally extend to topological settings, where we desire spatially local check operators of low weight. In this section, we introduce subsystem versions of the toric and surface codes introduced in [Section 5.3](#).

### 6.5.1 Subsystem toric code

Defined on a lattice of size<sup>8</sup>  $M \times M$  with periodic boundary conditions, the subsystem toric code [111] uses a layout involving triangular checks of weight 3. See [Figure 13](#) for an example where  $M = 3$ . Qubits are placed on both the vertices and the centers of the edges of the lattice, for a total of  $n = 3M^2$  physical qubits ( $M^2$  vertices and  $2M^2$  edges). For every triangle in the lattice, we have a 3-qubit check operator (which could be either  $X$  or  $Z$ ) involving the vertex qubit and the adjacent edge qubits. These are sometimes called  $X$  or  $Z$  “triangle operators”, and form a generating set for the gauge group (up to phase). There are in total  $2M^2$   $X$  and  $Z$  triangle operators each. For the purposes of calculating the number of gauge qubits, it should be noted that these  $4M^2$  triangle operators do not form a *minimal* generating set for the gauge group (up to phase); any  $X$  (resp.,  $Z$ ) triangle operator is the product of all the other  $2M^2 - 1$   $X$  (resp.,  $Z$ ) triangle operators. To get a minimal generating set, one can remove any one  $X$  and any one  $Z$  triangle operator from the set of  $4M^2$  check operators.

**Exercise 6.3:** Stabilizers of the subsystem toric code

- (a) Each plaquette of the subsystem toric code consists of a pair of  $X$  triangle operators and a pair of  $Z$  triangle operators. Show that the weight-6 operator defined by the product of the two weight-3  $X$  (or  $Z$ ) triangle operators on a single plaquette commutes with all of the triangle operators, and thus are stabilizers of the code.

<sup>8</sup>This refers to the number of *distinct* qubits in each row and column of the lattice. This means that lattice points which are identified (e.g., those on the top and bottom boundaries) are only counted once.

- (b) Show also that the set of such operators generates the stabilizer group of the subsystem toric code, that is, any stabilizer is a product of such weight-6 operators.

[Hint: Each stabilizer is a product of triangle operators. Prove that if a particular triangle operator appears in the product, then the other triangle operator of the same type ( $X$  or  $Z$ ) in the same plaquette must also appear in order for the stabilizer to commute with either triangle operator of the opposite type in the same plaquette.]

- (c) There are  $M^2$  such weight-6  $X$  stabilizers and  $M^2$  such weight-6  $Z$  stabilizers. Show that any  $M^2 - 1$  of the  $X$  stabilizers and  $M^2 - 1$  of the  $Z$  stabilizers form a minimal generating set of the stabilizer group, so  $s = 2(M^2 - 1)$ .

[Hint: The product of all  $M^2$   $X$  or  $Z$  stabilizers is the identity.]

Since the minimal generating set for the gauge group (up to phase) has size  $2(2M^2 - 1)$  and that for the stabilizer group has size  $2(M^2 - 1)$ , we have

$$g = \frac{1}{2} (2(2M^2 - 1) - 2(M^2 - 1)) = M^2. \quad (6.4)$$

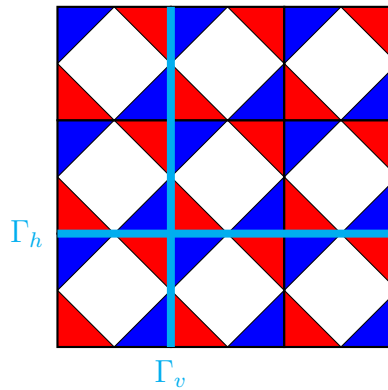
Therefore, this code encodes

$$k = n - g - s = 3M^2 - M^2 - 2(M^2 - 1) = 2 \quad (6.5)$$

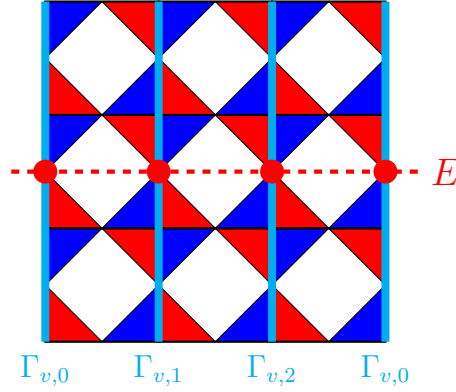
logical qubits.

Let  $\Gamma_h$  (resp.,  $\Gamma_v$ ) denote the set of  $2M$  qubits that lie along a fixed horizontal (resp., vertical) line of the lattice. This represents a noncontractible loop of the torus. The two pairs of (bare) logical operators of the code can be represented by:

- $\bar{X}_1 = \prod_{j \in \Gamma_h} X_j$  and  $\bar{Z}_1 = \prod_{j \in \Gamma_v} Z_j$ ,
- $\bar{X}_2 = \prod_{j \in \Gamma_v} X_j$  and  $\bar{Z}_2 = \prod_{j \in \Gamma_h} Z_j$ .



**Figure 13:** The lattice for the subsystem toric code with  $M = 3$ , with two noncontractible loops  $\Gamma_h$  and  $\Gamma_v$  labeled. Note that the left and right boundaries of the lattice are identified with each other, and so are the top and bottom boundaries. Qubits are placed at each vertex and the center of each edge of the lattice. Red triangles represent  $XXX$  check operators, with an  $X$  check at each vertex of the triangle. Blue triangles represent  $ZZZ$  check operators, with a  $Z$  check at each vertex of the triangle. Each bold cyan line represents a noncontractible loop of the torus made up of  $2M$  qubits and is used in the definition of the logical operators of the code.



**Figure 14:** Distance calculation for the subsystem toric code with  $M = 3$ , showing a weight-3  $X$ -error  $E$  along the vertical edges. Note that the left and right boundaries of the lattice are identified with each other.

To determine the distance of the code, we need to calculate the minimum distance of any nontrivial logical operator. Observe that such a Pauli operator  $E$  must commute with all the stabilizers but must anticommute with at least one other logical operator among  $\bar{X}_1, \bar{X}_2, \bar{Z}_1, \bar{Z}_2$ . Let us assume without loss of generality that  $E$  anticommutes with  $\bar{Z}_1 = \prod_{j \in \Gamma_{v,0}} Z_j$ , where  $\Gamma_{v,0}$  is a vertical line of qubits as labeled in Figure 14. (The vertical lines of qubits are labeled  $\Gamma_{v,0}, \dots, \Gamma_{v,M-1}$ .)

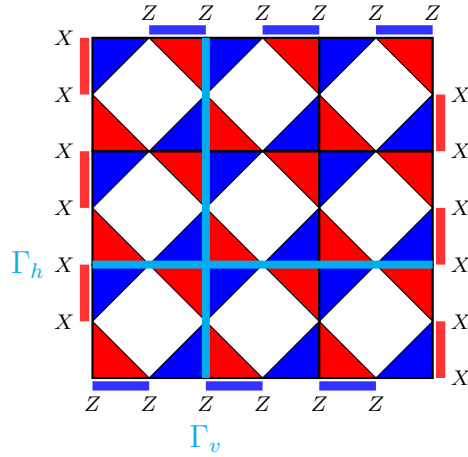
Note then that  $E$  also anticommutes with  $\prod_{j \in \Gamma_{v,i}} Z_j$  for any  $i \in \{1, \dots, M-1\}$ . To see this, observe that it suffices to show  $E$  commutes with the product  $\left(\prod_{j \in \Gamma_{v,0}} Z_j\right) \left(\prod_{j \in \Gamma_{v,i}} Z_j\right)$ , and this latter statement is true because that product is simply the product of all the weight-6  $Z$  plaquette stabilizers lying between the lines  $\Gamma_{v,0}$  and  $\Gamma_{v,i}$ .

Since  $E$  anticommutes with each  $\Gamma_{v,i}$ , it must act nontrivially on at least one qubit of each  $\Gamma_{v,i}$  (specifically, via an  $X$  or  $Y$ ). Thus,  $E$  must act nontrivially on at least  $M$  qubits, showing that the distance of the subsystem toric code is at least  $M$ . If we take  $E$  to be the operator acting as  $X$  on all  $M$  qubits intersecting a fixed horizontal line at half-integer coordinates of the lattice (see Figure 14 for an example), then it can be checked that  $E$  is indeed a nontrivial logical operator (as it commutes with all the stabilizers and anticommutes with  $\bar{Z}_1$ ). In fact, it is not hard to directly check that  $E$  and  $\bar{X}_1$  differ by a product of gauge operators, so  $E$  is a dressed logical that performs the same logical operation as  $\bar{X}_1$ . In light of the discussion above, we see that the subsystem toric code on a lattice of size  $M \times M$  has parameters  $\llbracket 3M^2, 2, M^2, M \rrbracket$ .

### 6.5.2 Subsystem surface code

The subsystem surface code [111] on an  $M \times M$  lattice<sup>9</sup> is defined similarly to the subsystem toric code, but with open boundary conditions. See Figure 15 for an example where  $M = 4$ . As before, qubits are placed on both the vertices and the centers of the edges of the lattice, but since the boundaries of the lattice are not identified, there are now a total of  $n = 3M^2 - 2M$  physical qubits ( $M^2$  vertices and  $2M(M-1)$  edges). Besides the  $4(M-1)^2$  weight-3  $X$  and  $Z$  triangle operators, there are additional  $4(M-1)$  weight-2 operators on the boundaries of the lattice, with  $M-1$  such operators of type  $X$  on each of the left and right boundaries and  $M-1$  such operators of type  $Z$  on each of the top and bottom boundaries. These form all the check operators for the subsystem surface code, and in fact form a minimal generating set (of size  $4(M-1)^2 + 4(M-1) = 4M^2 - 4M$ )

<sup>9</sup>Specifically, the  $M \times M$  lattice for the subsystem surface code comprises  $M$  vertices and  $M-1$  plaquettes along each row and column, as the lattice boundaries are not identified.



**Figure 15:** The lattice for the subsystem surface code with  $M = 4$ . Like the subsystem toric code, qubits are placed at each vertex and the center of each edge of the lattice. Red triangles represent  $XXX$  check operators, with an  $X$  check at each vertex of the triangle. Blue triangles represent  $ZZZ$  check operators, with a  $Z$  check at each vertex of the triangle. Boundary operators of weight-2 are indicated by thick bars with  $X$  or  $Z$  checks at each end. As before, each bolded cyan line is made up of  $2M - 1$  qubits and is used in the definition of the logical operators of the code.

for the gauge group (up to a phase).

#### Exercise 6.4: Stabilizers of the subsystem surface code

- (a) Check that the  $(M - 1)^2$  plaquette operators of type  $X$  and the  $(M - 1)^2$  plaquette operators of type  $Z$  are stabilizers for the subsystem surface code. Verify that this also holds for the  $4(M - 1)$  boundary operators of weight 2.
- (b) Show that these form a minimal generating set for the stabilizer group, and thus  $s = 2M^2 - 2$ .

By Exercise 6.4, we have

$$g = \frac{1}{2} ((4M^2 - 4M) - (2M^2 - 2)) = M^2 - 2M + 1 = (M - 1)^2, \quad (6.6)$$

and so the subsystem surface code encodes exactly

$$k = n - g - s = (3M^2 - 2M) - (M^2 - 2M + 1) - (2M^2 - 2) = 1 \quad (6.7)$$

logical qubit.

Define  $\Gamma_h$  and  $\Gamma_v$  as before. Then, a pair of (bare) logical operators can be defined by  $\overline{X} = \prod_{j \in \Gamma_h} X_j$  and  $\overline{Z} = \prod_{j \in \Gamma_v} Z_j$ . It is straightforward to verify that these do indeed commute with all check operators but not with each other. By a similar argument as in the subsystem toric code, the distance of the subsystem surface code is  $M$ . As such, the subsystem surface code on an  $M \times M$  lattice has parameters  $[[3M^2 - 2M, 1, (M - 1)^2, M]]$ .

## 6.6 Why subsystem codes?

We have seen that subsystem codes mathematically generalize standard stabilizer codes by partitioning the codespace into a logical subsystem and a gauge subsystem. But why introduce this

extra complexity? In this section, we outline the primary practical and theoretical advantages of subsystem codes.

**Reduced weight of check operators.** Historically, the first major motivation for subsystem codes was to simplify error syndrome measurements. As demonstrated in our transition from the 9-qubit Shor code to the Bacon–Shor code, introducing gauge degrees of freedom allows us to replace high-weight stabilizers (e.g., weight-6  $XX$ -checks) with low-weight gauge operators (weight-2  $XX$  and  $ZZ$  checks). Because measuring high-weight operators requires deeper, more error-prone quantum circuits, subsystem codes often yield more robust fault-tolerant threshold behavior in near-term hardware.

**Gauge fixing.** The defining feature of a subsystem code is the presence of the gauge group  $\mathcal{G}$ . Because the encoded logical information is invariant under the action of  $\mathcal{G}$ , we are free to measure any commuting subset of gauge operators. When we do this, we project the system into an eigenstate of those operators, a process called *gauge fixing*. Effectively, the measured gauge operators are temporarily promoted to the stabilizer group  $\mathcal{S}$ . This ability to dynamically alter the code’s stabilizer group on the fly offers several additional advantages, which we detail below.

**Flexibility in handling biased noise.** Gauge fixing allows a subsystem code to adapt to the dominant noise in the physical hardware. Consider the rectangular Bacon–Shor code described earlier in this chapter. If we choose to measure only the vertical  $XX$ -checks, the code collapses into a state stabilized by those  $XX$ -checks and the logical  $Z$ -stabilizers. Conversely, measuring only the horizontal  $ZZ$ -checks stabilizes the system against  $X$  errors.

If a quantum processor suffers from highly biased noise, for example, if  $Z$ -errors occur much more frequently than  $X$ -errors, we can adjust our measurement schedule to perform the  $XX$ -checks much more frequently. This flexibility is impossible in standard stabilizer codes where the check operators are strictly fixed.

**Dynamical logical qubits.** By carefully scheduling which gauge operators are measured at which times, we can create *dynamical logical qubits*. As demonstrated by Alam and Rieffel [110], one can apply a specific period-4 measurement schedule to the Bacon–Shor lattice. Instead of fixing all gauges at once, the schedule alternates between measuring different sets of  $XX$  and  $ZZ$  gauges.

At each round, only  $(M - 1)(N - 1) - 1$  gauge degrees of freedom are fixed (as opposed to  $(M - 1)(N - 1)$ , for the traditional Bacon–Shor construction). This leaves an extra degree of freedom that can store an additional logical qubit. Because the set of measured gauge operators changes at each time step, the logical operators of this extra qubit are “dynamical”: they change at each step of the measurement round. This concept is the foundational mechanism behind the recent explosion of interest in *Floquet codes*. See [Section 7](#) for more details.

**Enhanced decoding via schedule-induced gauge fixing.** In the standard decoding of subsystem codes, the error syndrome is extracted by multiplying the outcomes of several gauge operator measurements to reconstruct a composite stabilizer. For instance, in the subsystem toric code, a weight-6  $X$ -stabilizer is formed by taking the parity of two adjacent weight-3  $X$ -triangle operators. By only passing this final product to the decoder, we discard the individual measurement outcomes of the triangles, which contain finer-grained information about an error’s exact spatial and temporal location. Higgott and Breuckmann demonstrated [112] that this lost information can be recovered using *schedule-induced gauge fixing*. By adjusting the measurement schedule to repeat a specific gauge measurement before any anti-commuting operators are introduced, the gauge measurement outcome becomes deterministic relative to its previous value.

Geometrically, this changes the structure of the decoding matching graph. In a minimum weight perfect matching (MWPM) decoder, the subsystem toric code without gauge information produces a dense triangular matching graph, where each weight-6 stabilizer is a vertex of degree 6. When the weight-3 triangle gauge operators are “fixed” by the schedule, each weight-6 vertex effectively “splits” into two distinct vertices of degree 3. This transforms the matching graph into a sparse hexagonal lattice. With a lower connectivity, there are fewer intersecting paths for the decoder to confuse, which significantly boosts the decoding accuracy and increases the circuit-level depolarizing threshold.

**Code switching for fault-tolerant logical gates.** To achieve universal FTQC, we need a universal set of logical gates (e.g., Clifford +  $T$ ). However, the Eastin–Knill theorem dictates that no single quantum error-correcting code can have a universal set of transversal gates. Certain subsystem codes (e.g., gauge color codes) offer a workaround via *code-switching*. For example, a 3D color code defined on a tetrahedral 3-complex [113] allows for a transversal  $T$  gate but lacks a transversal Hadamard ( $H$ ) gate. Conversely, a 2D color code [85] allows for a transversal  $H$  gate but lacks a transversal  $T$  gate. As studied by Butt *et al.* [114], by treating the 2D color code as a gauge-fixed version of a 3D subsystem color code, one can fault-tolerantly switch the encoded information between the 3D and 2D geometries. This allows the computer to execute a transversal  $T$  gate in the 3D geometry, project down to the 2D geometry via gauge fixing to execute a transversal  $H$  gate, and switch back. Similar code-switching principles apply to other families, such as the doubled color codes [115] and quadratic residue codes [116, 117]. Code-switching thus offers a pathway to universality that significantly reduces the reliance on incredibly costly magic state distillation protocols. We will return to code switching briefly in [Section 14.6](#).

In all, subsystem codes represent a powerful paradigm shift in quantum error correction. By relaxing the strict requirements of the stabilizer formalism and allowing certain degrees of freedom to remain unprotected (the gauge subsystem), we gain a substantial amount of flexibility. This flexibility translates into lower-weight check operators, dynamic responses to biased noise, enhanced decoding graphs, and novel pathways to fault-tolerant universal gate sets via code-switching. As hardware continues to mature, leveraging these gauge degrees of freedom will be a central theme in lowering the overhead of FTQC.



## 7 Dynamical codes

As we saw in [Section 6](#), we do not necessarily need commuting observables to define a codespace and to extract error syndromes. In a subsystem code, its codespace  $Q$  is stabilized by the phaseless Pauli operators in the center of the gauge group  $\mathcal{G}$  generated by the set of non-commuting check measurements. However, in extracting error syndromes we do actually measure these non-commuting checks which effectively evolve the codestate  $|\bar{\psi}\rangle$  that we are measuring, and therefore the effective codespace at a given time.

**Dynamical codes** [Eq. 260](#) need not have a fixed codespace, as long as logical information remains preserved throughout the syndrome extraction process. For a codespace  $Q(\mathcal{S}_t)$  stabilized by a stabilizer group  $\mathcal{S}_t$  at time  $t$ , measuring an observable  $s'$  that anticommutes with some stabilizers in  $\mathcal{S}_t$  on codestate  $|\bar{\psi}_t\rangle \in Q(\mathcal{S}_t)$  evolves it to a state  $|\bar{\psi}_{t+1}\rangle$  in codespace  $Q(\mathcal{S}_{t+1})$ , which is distinct from  $Q(\mathcal{S}_t)$  but retains the encoded information. For an illustration, see [Figure 16](#). As we will see, dynamical codes are *distinct* from subsystem codes. For a sequence of non-commuting check measurements  $C_1, C_2, \dots$  that defines a dynamical code that encodes a  $K$ -dimensional codespace, a subsystem code with gauge group generated by the same check measurements, that is,  $\mathcal{G} = \langle C_1 \cup C_2 \cup \dots \rangle$ , may not allow any logical information to be encoded in it.

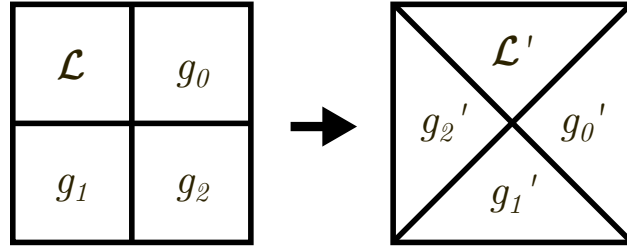
### 7.1 “Hidden” dynamical logical qubit

As discussed in [Section 6](#), an  $[[n, k, g, d]]$  stabilizer subsystem code has  $n$  physical  $q$ -dimensional systems, a  $q^k$ -dimensional logical subspace, a  $q^g$ -dimensional gauge subspace, and a  $q^r$ -dimensional error syndrome subspace. For some choice of gauge group  $\mathcal{G}$ , we could end up with gauges and stabilizers that take up “too much” space such that  $g + r = n$ , leaving no room to encode any logical information. This is the case with Kitaev’s honeycomb code [\[118\]](#), which is a qubit subsystem stabilizer code defined on a hexagonal lattice on the surface of a torus where qubits are placed on the vertices (see [Figure 17](#)). The lattice edges and faces are colored with green, red, or blue such that any three edges sharing a vertex must have different colors and two hexagonal faces at the end of each edge have the same color as that edge.

This code has a gauge group  $\mathcal{G}$  generated by two-qubit Pauli measurements on the edges of the lattice where the edge color determines its corresponding Pauli observable, while the plaquette colors determine the type of 6-qubit stabilizer on that plaquette<sup>10</sup>. The stabilizer group  $\mathcal{S}$  is generated by 6-qubit Pauli operators on the hexagonal faces and two Pauli operators along two loops around the horizontal and vertical directions on the torus (black and yellow lines in [Figure 17](#)).

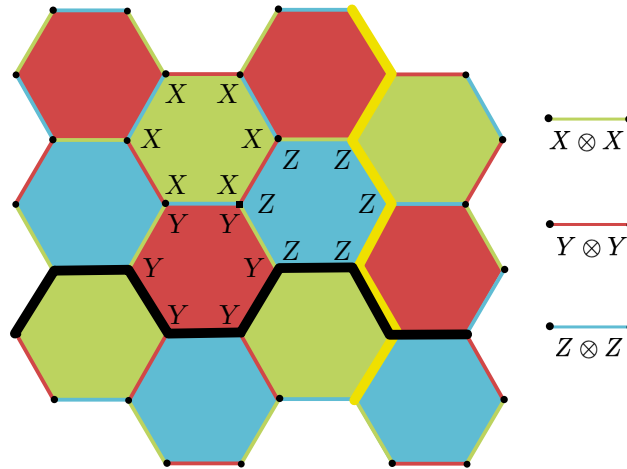
If there are  $n_p$  hexagonal faces on the lattice, then there are  $3n_p$  edges and  $n = 2n_p$  qubits (vertices). Taking the product of all two-qubit Pauli operators  $P_i P_j$  that generate the gauge group  $\mathcal{G}$  of Kitaev’s honeycomb code, gives us the identity, that is,  $\prod_{i,j} P_i P_j = I$ . Thus, the stabilizer code has  $3n_p - 1$  independent generators. On the other hand, the product of the 6-qubit Pauli operators on the faces is also equal to the identity, and so the independent generators of  $\mathcal{S}$  consist of  $n_p - 1$  6-qubit Pauli operators on the faces and two loops wrapping around the torus in perpendicular directions. This gives us  $n_p + 1$  independent generators of  $\mathcal{S}$ . This in turn gives us a space of  $2^r = 2^{n_p+1}$  dimensions that are allocated for the syndrome stabilizers, whereas a space of  $2^g = 2^{n_p-1}$  dimensions are allocated for the gauge qubits (as  $\mathcal{S} \subseteq \mathcal{G}$ ,  $g = (3n_p - 1 - n_p - 1)/2$ ). Since there are  $n = 2n_p$  qubits on the lattice, the number of qubits taken up by the gauge qubits and the stabilizers is  $n_p - 1 + n_p + 1 = 2n_p = n$ , which shows that there is no encoded logical information.

<sup>10</sup>The original Kitaev code’s check measurements are determined by the orientation of the edges such that each plaquette stabilizer is of the same type. Here we adopt the three-colored honeycomb code convention used in Ref. [\[119\]](#), which does not change the point we are trying to make here.

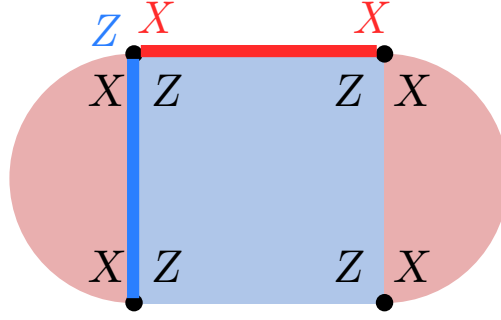


**Figure 16:** A change of logical subspace and syndrome subspaces. A subspace stabilizer code  $Q$  (left-hand side) consists of logical subspace  $\mathcal{L}$  which is the joint  $+1$  eigenspace of three stabilizer generators  $g_0, g_1, g_2$ , and subspaces corresponding to  $-1$  for each stabilizer generator  $g_0, g_1, g_2$ . Any operation performed on a code state  $|\psi\rangle$  of  $Q$ , such as measurements or gates, may change the code  $Q$  to a new subspace stabilizer code  $Q'$  (right-hand side) with different subspace  $\mathcal{L}'$  and three stabilizer generators  $g'_0, g'_1, g'_2$ .

Later, in [Section 7.3](#), we will see how by shifting our perspective from a *fixed* codespace to a *dynamical* codespace, one can *dynamically* encode two logical qubits in Kitaev’s code by looking at how the stabilizer group of the code evolves as a sequence of non-commuting check measurements are performed. This idea is basically what gave rise to the first dynamical code: the [Hastings-Haah honeycomb Floquet code](#) [\[120\]](#). To gain a better understanding of how a codespace changes as we perform non-commuting check measurements while preserving logical information, we will first discuss in [Section 7.2](#) a simpler example of a dynamical code consisting only of four data qubits. In this example, we will also see the phenomenon where one dynamical logical qubit is present in this 4-qubit code when viewed as a dynamical code, although there are no logical qubits present when the check measurements are taken to be the checks for a subsystem code.



**Figure 17:** A 36-qubit Kitaev’s honeycomb code on a hexagonal “honeycomb” lattice with degree 3 on the surface of a torus. The lattice has a periodic boundary condition: The bottom and top boundaries represent the same set of edges, similarly for its left and right boundaries. Colors of hexagonal plaquettes and edges determine 6-qubit stabilizers and 2-qubit check measurements, respectively. In addition to the 6-qubit plaquette stabilizers, there are two stabilizers that loop around the torus along the vertical (yellow line) and horizontal (black line) directions.



**Figure 18:** Four-qubit rotated surface code. We label the top-left qubit as qubit 1, top-right qubit as qubit 2, bottom-left qubit 3, and bottom-right qubit 4. The red colored half-circles represent the  $X_1X_3$  (left) and  $X_2X_4$  (right) stabilizer generator of the code, whereas the blue square represents the  $Z_1Z_2Z_3Z_4$  stabilizer generator. The bright red line and bright blue line represent the  $X$ -type logical operator  $\bar{X} = X_1X_2$  and the  $Z$ -type logical operator  $\bar{Z} = Z_1Z_3$ , respectively.

## 7.2 4-qubit dynamical code

In this section, we discuss a simple dynamical code that uses only four data qubits. We start by considering the 4-qubit rotated surface code, which is a stabilizer error-detecting code with parameters  $[[4, 1, 2]]$ . Its stabilizer group  $\mathcal{S}_0$  is generated by three stabilizer generators:

$$X_1X_3, \quad X_2X_4, \quad Z_1Z_2Z_3Z_4, \quad (7.1)$$

and logical operator representatives  $\bar{X} = X_1X_2$  and  $\bar{Z} = Z_1Z_3$ , where we denote  $P_j$  as a 4-qubit Pauli operator with Pauli operator  $P$  on the  $j^{\text{th}}$  qubit and identity everywhere else. The 4-qubit rotated surface code can therefore be illustrated as in Figure 18.

To better visualize the Pauli operators over the code, matching the illustration in Figure 18, we denote a four-qubit Pauli operator  $P = P_1P_2P_3P_4$  over these four data qubits as

$$P = \begin{pmatrix} P_1P_2 \\ P_3P_4 \end{pmatrix}, \quad (7.2)$$

so that the stabilizer generators and logical operators of  $\mathcal{S}_0$  can be written as

$$g_0 = \begin{pmatrix} ZZ \\ ZZ \end{pmatrix}, \quad g_1 = \begin{pmatrix} XI \\ XI \end{pmatrix}, \quad g_2 = \begin{pmatrix} IX \\ IX \end{pmatrix}, \quad \bar{X} = \begin{pmatrix} XX \\ II \end{pmatrix}, \quad \bar{Z} = \begin{pmatrix} ZI \\ ZI \end{pmatrix}. \quad (7.3)$$

We will adopt this notation in analyzing how the codespace and stabilizer group changes after we measure an observable  $s'$  that anticommutes with some of the stabilizers of the four-qubit code.

### 7.2.1 Updating stabilizer group

We now denote  $Q(\mathcal{S}_0)$  as the codespace of the 4-qubit rotated surface code. Then, consider the following measurements and how a codestate in  $Q(\mathcal{S}_0)$  evolves upon these measurements:

1. Measure  $s'_0 = \begin{pmatrix} IZ \\ ZI \end{pmatrix}$  and  $s'_1 = \begin{pmatrix} ZI \\ IZ \end{pmatrix}$  on arbitrary codestate  $|\bar{\psi}\rangle \in Q(\mathcal{S}_0)$ .
2.  $s'_0$  anticommutes with generators  $g_1 = \begin{pmatrix} IX \\ IX \end{pmatrix}$ ,  $g_2 = \begin{pmatrix} XI \\ XI \end{pmatrix}$ . So, outcome  $o_0 = \pm 1$  of  $s'_0$  is random and we obtain an updated state  $|\bar{\varphi}\rangle = \Pi_o^{s'_0} |\bar{\psi}\rangle$ , for  $\Pi_o^{s'_0} = \frac{I + os'_0}{2}$  being the projector onto the  $o$ -eigenspace of Pauli observable  $s'_0$ . Note that the updated state  $|\bar{\varphi}\rangle$  is stabilized by  $os'_0$ .

3. Since  $g_1, g_2$  anticommute with  $s'_0$ , then  $g_1 \Pi_o^{s'_0} = \Pi_{-o}^{s'_0} g_1$ , the updated state  $|\bar{\varphi}\rangle$  is not stabilized by  $g_1, g_2$ .

4.  $s'_0$  commutes with stabilizer  $s_3 = g_1 g_2 = \frac{XX}{XX}$ . So by using  $\Pi_o^{s'_0} s_3 = s_3 \Pi_o^{s'_0}$  we have

$$s_3 |\bar{\varphi}\rangle = s_3 \Pi_o^{s'_0} |\bar{\psi}\rangle = s_3 \Pi_o^{s'_0} s_3 |\bar{\psi}\rangle = s_3 s_3 (\Pi_o^{s'_0} |\bar{\psi}\rangle) = |\bar{\varphi}\rangle, \quad (7.4)$$

which shows that  $|\bar{\varphi}\rangle$  is stabilized by  $s_3$ .

5. Since  $g_0 = \frac{ZZ}{ZZ}$  commutes with  $s'_0$ , then  $|\bar{\varphi}\rangle$  is stabilized by  $g_0$ .

To summarize, the updated codestate  $|\bar{\varphi}\rangle$  is now stabilized by Paulis

$$os'_0 = o \frac{IZ}{ZI}, \quad s_3 = g_1 g_2 = \frac{XX}{XX}, \quad g_0 = \frac{ZZ}{ZZ}. \quad (7.5)$$

Note that it is also stabilized by

$$os'_0 g_0 = o \frac{ZI}{IZ} = os'_1, \quad (7.6)$$

which means that the outcomes of measurements  $s'_0$  and  $s'_1$  on initial codestate  $|\bar{\psi}\rangle$  must have a parity of 0, that is, their measurement outcomes are equal. More generally in the absence of errors, the product of measurements  $s'_0$  and  $s'_1$  must be equal to the eigenvalue of the eigenspace of  $g_0$  that the initial codestate  $|\bar{\psi}\rangle$  is in.

Thus after we measure  $s'_0, s'_1$  on a codestate  $|\bar{\psi}\rangle$  in  $Q(\mathcal{S}_0)$ , we remove all elements of stabilizer group  $\mathcal{S}_0$  that anticommute with either  $s'_0$  or  $s'_1$  and take the rest to define a new stabilizer group  $\mathcal{S}_1$  consisting of: (1)  $\pm s'_0$  and  $\pm s'_1$  (where the signs depend on the measurement outcomes), and (2) all elements of  $\mathcal{S}_0$  that commute with both  $s'_0$  and  $s'_1$ . The new stabilizer group  $\mathcal{S}_1$  stabilizes the new codestate  $|\bar{\varphi}\rangle$  obtained after measuring observables  $s'_0, s'_1$  on initial codestate  $|\bar{\psi}\rangle$ . We denote this stabilizer group update as

$$\mathcal{S}_0 = \left\langle \frac{ZZ}{ZZ}, \frac{IX}{IX}, \frac{XI}{XI} \right\rangle \xrightarrow[\frac{IZ}{IZ}]{\frac{ZI}{ZI}} \mathcal{S}_1 = \left\langle \frac{XX}{XX}, \frac{ZI}{IZ}, \frac{IZ}{ZI} \right\rangle. \quad (7.7)$$

This stabilizer group update is illustrated in [Figure 19](#).

We now consider the logical operator representatives of the initial code  $Q(\mathcal{S}_0)$ :

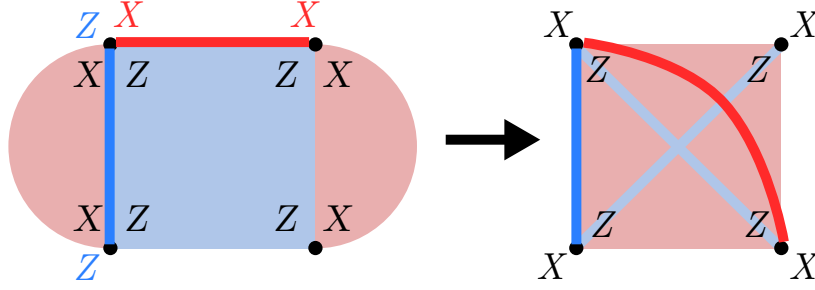
$$\bar{X} = \frac{XX}{II} \quad \text{and} \quad \bar{Z} = \frac{ZI}{ZI}. \quad (7.8)$$

The logical eigenstates  $|\bar{0}\rangle, |\bar{1}\rangle$  of  $\bar{Z}$  after measuring  $s'_0$  with outcome  $o$  become

$$|\bar{i}'\rangle = \Pi_o^{s'_0} |\bar{i}\rangle, \quad (7.9)$$

for  $i \in \{0, 1\}$ . Since  $\bar{Z}$  commutes with  $s'_0$ , we have

$$\bar{Z} |\bar{i}'\rangle = \bar{Z} \Pi_o^{s'_0} |\bar{i}\rangle = \Pi_o^{s'_0} \bar{Z} |\bar{i}\rangle = (-1)^i |\bar{i}'\rangle. \quad (7.10)$$



**Figure 19:** Stabilizer group and logical operator update from the 4-qubit rotated surface code (left) to another 4-qubit code (right). Vertices represent qubits, while red (resp. blue) faces represent  $X$  (resp.  $Z$ ) stabilizer generators.

Thus, we can see that the updated eigenstates of  $\bar{Z}$  are still eigenstates of  $\bar{Z}$ . On the other hand, the eigenstates of logical  $\bar{X}$ , given by

$$|\bar{+}\rangle = \frac{|\bar{0}\rangle + |\bar{1}\rangle}{\sqrt{2}} \quad \text{and} \quad |\bar{-}\rangle = \frac{|\bar{0}\rangle - |\bar{1}\rangle}{\sqrt{2}}, \quad (7.11)$$

are updated to  $|\bar{\pm}'\rangle = \Pi_o^{s'_0} |\bar{\pm}\rangle$ . Note that  $\bar{X}$  anticommutes with  $s'_0$ , so it cannot be a logical operator of the new stabilizer code  $Q(\mathcal{S}_1)$ . However, a different logical operator representative

$$\bar{X}' = g_1 \bar{X} = \begin{matrix} XI \\ IX \end{matrix} \quad (7.12)$$

does commute with  $s'_0$  (and  $s'_1$ ). The action on this logical operator representative on the updated codestate is

$$\bar{X}' |\bar{\pm}'\rangle = \bar{X}' \Pi_o^{s'_0} |\bar{\pm}\rangle = \Pi_o^{s'_0} \bar{X}' |\bar{\pm}\rangle = \pm |\bar{\pm}'\rangle, \quad (7.13)$$

which indicates that the updated eigenstates are the eigenstates of logical operator representative  $\bar{X}'$ . Note that logical operator  $\bar{X}'$  remains a logical operator after measuring  $s'_0$  because we define it as a product between the logical operator representative  $\bar{X}$  that we first pick and a stabilizer  $g_1$  of the initial stabilizer group  $\mathcal{S}_0$ . Both  $\bar{X}$  and  $g_1$  individually anticommute with  $s'_0$  so that their product  $\bar{X}' = \bar{X} g_1$  commutes with  $s'_0$ . The logical operator updates are illustrated in Figure 19.

**General stabilizer group and logical operator update rules.** As we have seen in the stabilizer group update and logical operator update in the 4-qubit code example above, measuring Pauli observables  $s'_0, s'_1$  that anticommute with some of the original stabilizers in  $\mathcal{S}_0$  evolves a codestate  $|\bar{\psi}\rangle \in Q(\mathcal{S}_0)$  to a new codestate  $|\bar{\varphi}\rangle$  stabilized by an updated stabilizer group  $\mathcal{S}_1$  and a new set of logical operators. In fact, one can update any initial stabilizer code  $\mathcal{S}$  and its logical operators after a Pauli measurement  $s'$  is performed, according to whether  $s$  commutes with the elements of  $\mathcal{S}$  and the logical operators. We note that update rules for stabilizer group and logical operators have been known and used for simulating stabilizer circuits (see e.g. [121, Section 7]).

**Lemma 7.1:** Stabilizer group update rules

Update rules for an initial stabilizer group  $\mathcal{S} = \langle G \rangle$  with generators  $G = \{g_1, \dots, g_r\}$  to stabilizer group  $\mathcal{S}' = \langle G' \rangle$  after Pauli measurement  $P$  with outcome  $o = \pm 1$ :

1. If  $\pm P \in \mathcal{S}$ , then  $\mathcal{S}' = \mathcal{S}$ , that is, the stabilizer stays the same. In this case, outcome  $o$

is deterministic.

2. If  $\pm P$  is not in  $\mathcal{S}$  but  $P$  commutes with all  $s \in \mathcal{S}$  (so  $P \in \mathcal{N}(\mathcal{S}) \setminus \mathcal{S}$  is a logical operator), then  $\mathcal{S}' = \langle oP, g_1, \dots, g_r \rangle$ . That is, add  $oP$  to  $G$  to get  $G' = \{oP\} \cup G$ .
3. If  $P$  does not commute with elements in a subset  $W = \{g_1, \dots, g_l\} \subseteq G$ :
  - (a) Let  $W' = \{g_1g_2, g_1g_3, \dots, g_1g_l\}$  ( $|W'| = l - 1$ ). All  $g' \in W'$  commute with  $P$ . (Note that  $\mathcal{S} = \langle g_1, W', G \setminus W \rangle$ , so now  $P$  anticommutes only with  $g_1$ .)
  - (b) Update  $G$  to  $G' = \{oP\} \cup W' \cup (G \setminus W)$  and get  $\mathcal{S}' = \langle G' \rangle$ .

Here we replace generator  $g^*$  with  $oP$ , replace  $W \setminus g_1$  with  $W'$ , and keep the generators that commute with  $P$ .

A proof of this lemma can be found in Ref. [122] for general qudit stabilizer group update (which thus includes qubit stabilizer update as a special case). We give some remarks on each of the rules:

1. Rule 1 is essentially applied when we measure one of the stabilizers in the initial stabilizer group  $\mathcal{S}$ .
2. Rule 2 is applied when we measure a logical operator and thus changes the logical state (which one should not do when extracting error syndromes). Note that if we start with an  $\llbracket n, k \rrbracket$  stabilizer code, then we increase the syndrome dimension to  $n - k + 1$  since we are adding a stabilizer, so that we are left with  $k - 1$  logical qubits. The updated stabilizer code has parameters  $\llbracket n, k - 1 \rrbracket$ .
3. Rule 3 is used when we measure a  $s'$  that anticommutes with some elements of  $\mathcal{S}$ . We used this rule in the 4-qubit code example above, where we removed stabilizer generator  $g_1$  after measuring  $s'_0$  because they anticommute, updated  $g_1$  (which also anticommute with  $s'_0$ ) to  $g_0g_1$ , and kept the generator  $g_3$ .

Similarly, we can use a set of rules to update logical operator representatives of initial stabilizer group  $\mathcal{S}$  after measuring some Pauli observable  $s'$ .

#### Lemma 7.2: Logical operator update rules

Update rules for logical operator (representative)  $L$  of an initial stabilizer group  $\mathcal{S} = \langle G \rangle$  with generators  $G = \{g_1, \dots, g_r\}$  to logical  $L'$  of stabilizer group  $\mathcal{S}' = \langle G' \rangle$  after Pauli measurement  $P$  with outcome  $o = \pm 1$ :

1. If  $P = L$  then we are measuring logical operator  $L$ .  $L$  is now a stabilizer in  $\mathcal{S}'$ .
2. If  $PL = LP$ , then  $L$  remains a logical operator of  $\mathcal{S}'$ .
3. If  $P$  does not commute with  $L$  but commutes with all  $s \in \mathcal{S}$ , then  $L$  is updated to  $L' = oP$ , which is now a stabilizer in  $\mathcal{S}'$ .
4. If  $P$  does not commute with  $L$  and some stabilizers in  $\mathcal{S}$ , then update  $L$  to  $L' = g_1L$ , where  $g_1$  is the stabilizer removed in stabilizer update rule 4.

We give some remarks on each of the logical operator update rules above:

1. Rule 1: we measure logical operator  $L$ . This is basically rule 2 of the stabilizer update rules above where we measure a logical operator so that it is in the updated stabilizer group.
2. Rule 2: measure  $P$  that commutes with  $L$ . If  $|\bar{j}\rangle$  is the +1 eigenstate of  $L$  updated to  $|\bar{j}'\rangle = \Pi_o^P |\bar{j}\rangle$ , then

$$L|\bar{j}'\rangle = L\Pi_o^P |\bar{j}\rangle = \Pi_o^P L|\bar{j}\rangle = \Pi_o^P L|\bar{j}\rangle = |\bar{j}'\rangle. \quad (7.14)$$

3. Rule 3:  $L$  is updated to  $L' = oP$ . When  $P$  does not commute with  $L$  but commutes with all  $s \in \mathcal{S}$ , then  $P \in \mathcal{N}(\mathcal{S}) \setminus \mathcal{S}$  (since all  $s \in \mathcal{S}$  commute with  $L$ ). So  $P$  is a representative of a logical operator of  $\mathcal{S}$  anticommuting with  $L$ , thus measuring  $s'$  on +1 eigenstate  $|\bar{j}\rangle$  of  $L$  gives

$$\Pi_+^P |\bar{j}\rangle \quad \text{or} \quad \Pi_-^P |\bar{j}\rangle \quad (7.15)$$

at random, stabilized by  $+P$  or  $-P$ , respectively (measuring  $P$  means doing a logical measurement that anticommutes with  $L$ , we are adding 1 stabilizer  $oP$ .)

4. Rule 4:  $L$  is updated to  $L' = g_1 L$ .  $P$  does not commute with  $L$ , but  $g_1 L$  is another representative of the same logical operator. Since both  $g_1$  and  $L$  anticommute with  $oP$ , if  $|\bar{j}\rangle$  is the +1 eigenstate of  $L$  updated to  $|\bar{j}'\rangle = \Pi_o^P |\bar{j}\rangle$ , then

$$L'|\bar{j}'\rangle = g_1 L \Pi_o^P |\bar{j}\rangle = g_1 \Pi_{-o}^P L|\bar{j}\rangle = \Pi_o^P g_1 L|\bar{j}\rangle = \Pi_o^P |\bar{j}\rangle = |\bar{j}'\rangle. \quad (7.16)$$

That is,  $|\bar{j}'\rangle$  is the +1 eigenstate of  $L'$  (we used  $L|\bar{j}\rangle = |\bar{j}\rangle$  and  $g_1 |\bar{j}\rangle = |\bar{j}\rangle$ ).

## 7.2.2 Four-qubit Floquet code

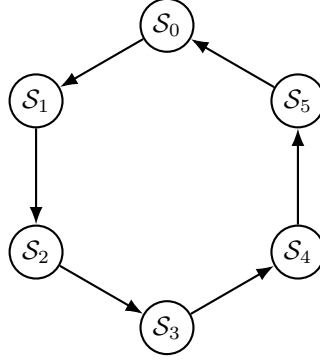
By rule 3 of the stabilizer update rules in [Lemma 7.1](#), the size of the updated stabilizer group  $\mathcal{S}'$  is preserved whenever we measure a  $s'$  that anticommutes with some elements of initial stabilizer group  $\mathcal{S}$ . Thus, logical information is always preserved by measuring Pauli observable  $s'$  such that  $s's = -ss'$  for some stabilizer  $s \in \mathcal{S}$ . By doing this periodically, we can construct a period-6 measurement schedule on 4-qubits so that we have stabilizer groups evolving as

$$\mathcal{S}_0 \mapsto \dots \mapsto \mathcal{S}_5 \mapsto \mathcal{S}_0 \mapsto \dots \quad (7.17)$$

Each of these six stabilizer groups are called an *instantaneous stabilizer group* (ISG). Since the original 4-qubit rotated surface code encodes one logical qubit, all ISGs also encode one logical qubit. The measurement sequence of this 4-qubit dynamical code can be illustrated as follows:

$$\begin{array}{cccccc} IZ & XX & ZI & XI & ZZ & XI \\ ZI & II & ZI & IX & II & XI \\ ZI & II & IZ & IX & II & IX \\ IZ & XX & IZ & XI & ZZ & IX \end{array} \quad \mathcal{S}_0 \xrightarrow{\quad} \mathcal{S}_1 \xrightarrow{\quad} \mathcal{S}_2 \xrightarrow{\quad} \mathcal{S}_3 \xrightarrow{\quad} \mathcal{S}_4 \xrightarrow{\quad} \mathcal{S}_5 \xrightarrow{\quad} \mathcal{S}_0 \rightarrow \dots \quad (7.18)$$

This period-6 measurement schedule gives us a period-6 evolution of ISGs:



The explicit form can be obtained by applying the update rules in [Lemma 7.1](#):

$$\begin{aligned}
 S_0 &= \left\langle \begin{array}{ccc} XI & IX & ZZ \\ XI' & IX' & ZZ \end{array} \right\rangle \xrightarrow[\begin{array}{c} IZ \\ ZI \\ IZ \end{array}]{\begin{array}{c} IZ \\ ZI \\ ZI \end{array}} S_1 = \left\langle \begin{array}{ccc} XX & ZI & IZ \\ XX' & IZ' & ZI \end{array} \right\rangle \\
 \xrightarrow[\begin{array}{c} XX \\ II \\ II \end{array}]{\begin{array}{c} XX \\ II \\ II \end{array}} S_2 &= \left\langle \begin{array}{ccc} XX & II & ZZ \\ II & XX' & ZZ \end{array} \right\rangle \xrightarrow[\begin{array}{c} IZ \\ ZI \\ IZ \end{array}]{\begin{array}{c} ZI \\ ZI \\ IZ \end{array}} S_3 = \left\langle \begin{array}{ccc} XX & ZI & IZ \\ XX' & ZI' & IZ \end{array} \right\rangle \\
 \xrightarrow[\begin{array}{c} XI \\ IX \\ IX \\ XI \end{array}]{\begin{array}{c} XI \\ IX \\ IX \\ XI \end{array}} S_4 &= \left\langle \begin{array}{ccc} XI & IX & ZZ \\ IX' & XI' & ZZ \end{array} \right\rangle \xrightarrow[\begin{array}{c} II \\ II \\ ZZ \end{array}]{\begin{array}{c} ZZ \\ II \\ II \end{array}} S_5 = \left\langle \begin{array}{ccc} XX & ZZ & II \\ XX' & II' & ZZ \end{array} \right\rangle \xrightarrow[\begin{array}{c} IX \\ IX \end{array}]{\begin{array}{c} XI \\ XI \end{array}} S_0.
 \end{aligned} \tag{7.19}$$

So as to not clutter the notation, we assume that all check measurements have an outcome of +1. In practice, these checks have random values, as we have seen in the update from ISG  $S_0$  to  $S_1$ , so that the two-qubit generators in each ISG would have a  $\pm 1$  sign, although their parity must be +1 (since the 4-qubit  $X$  and  $Z$  stabilizers must be in all of the ISGs). The stabilizers and logical operators of each ISG in this dynamical code are illustrated in [Figure 20](#).

By measuring two disjoint weight-2  $Z$ -type Paulis in evolving from ISG  $S_0$  to  $S_1$ , we are measuring the syndrome of

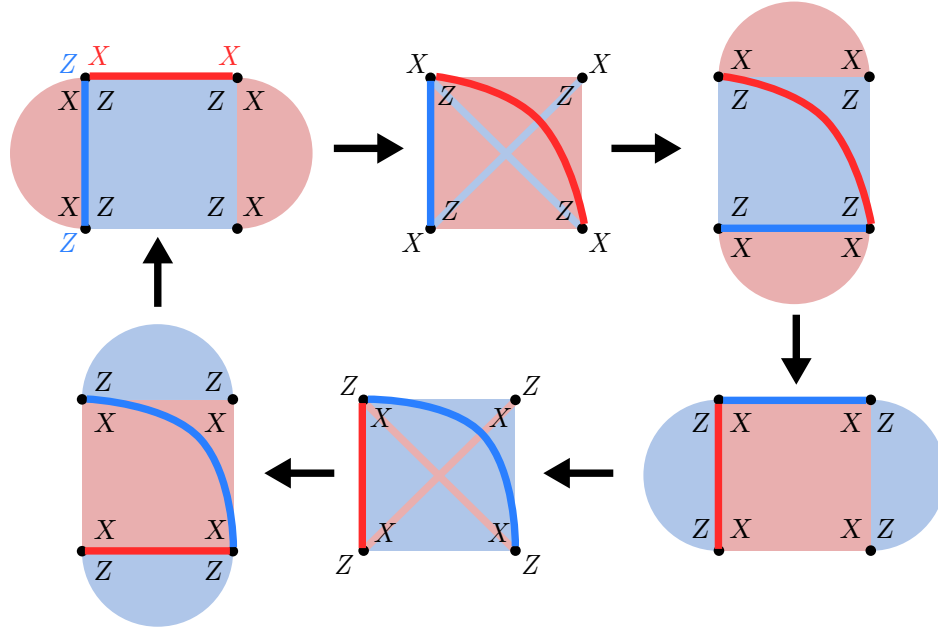
$$\begin{array}{c} ZZ \\ ZZ \end{array}, \tag{7.20}$$

which is a stabilizer in all ISGs. Similarly, by measuring two disjoint weight-2  $X$ -type Paulis in evolving from ISG  $S_1$  to  $S_2$ , we are measuring the syndrome of

$$\begin{array}{c} XX \\ XX \end{array}, \tag{7.21}$$

which is a stabilizer in all ISGs. So, we alternately measure these two weight-4 stabilizers over all six rounds. Thus, every two rounds we obtain the syndrome of the weight-4  $X$  stabilizer (resp., weight-4  $Z$  stabilizer), which allows us to detect a single qubit  $Z$  error (resp., single-qubit  $X$  error) between these two rounds.





**Figure 20:** Four-qubit dynamical code with period-6 measurement schedule given in Eq. (7.19). Red (resp. blue) faces represent  $Z$ -type (resp.  $X$ -type) stabilizers for each ISG. Bright red (resp. bright blue) lines represent  $X$ -type (resp.  $Z$ -type) logical operator representative for each ISG.

**Four-qubit dynamical code as a subsystem code.** As we have mentioned, this 4-qubit dynamical code does not encode any logical qubit when viewed as a subsystem code. To see this, we first construct a gauge group  $\mathcal{G}$  generated by all check measurements:

$$\mathcal{G} = \left\langle \begin{matrix} XX & ZZ & IZ & XX & ZI & XI & II & XI \\ XX & ZZ & ZI & II & ZI & IX & ZZ & XI \end{matrix} \right\rangle, \quad (7.22)$$

where we pick just one out of two check measurements in each round and add the 4-qubit  $X$  and  $Z$  Paulis, since the product of two check measurements in each rounds is either a 4-qubit  $X$  or 4-qubit  $Z$  Pauli. Note that some of these gauge group generators are still redundant. The four weight-2  $Z$  checks can be generated by only three of them, and similarly with the four weight-2  $X$  checks. So we can generate the gauge group only with 6 generators as:

$$\mathcal{G} = \left\langle \begin{matrix} XX & ZZ & XX & IZ & XI & II \\ XX & ZZ & II & ZI & IX & ZZ \end{matrix} \right\rangle. \quad (7.23)$$

Note that each of the first two generators

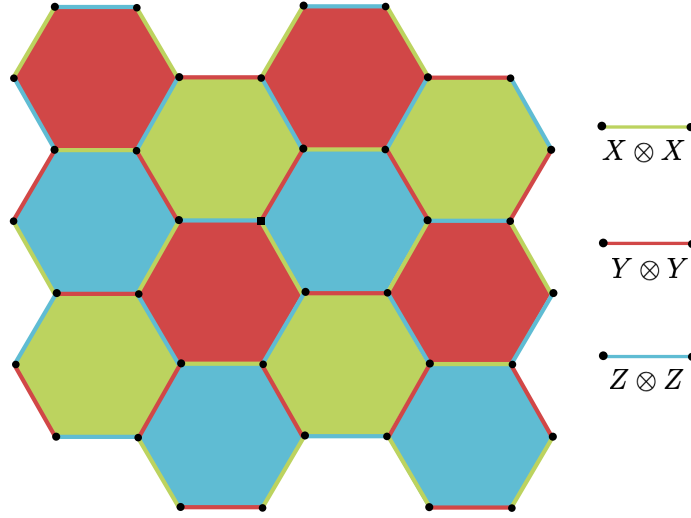
$$g'_1 = \begin{matrix} XX \\ XX \end{matrix}, \quad g'_2 = \begin{matrix} ZZ \\ ZZ \end{matrix} \quad (7.24)$$

commute with all the other generators, so that they are in the center of  $\mathcal{G}$ , that is,  $Z(\mathcal{G}) = \langle \pm i, g'_1, g'_2 \rangle$ . Thus, they generate the stabilizer group of this subsystem code:  $\mathcal{S} = \langle g'_0, g'_1 \rangle$ . On the other hand,

$$g'_3 = \begin{matrix} XX \\ II \end{matrix}, \quad g'_4 = \begin{matrix} IZ \\ ZI \end{matrix} \quad (7.25)$$

anticommute with each other but commute with all the other generators. Therefore, they are an  $X, Z$  operator pair of a gauge qubit. Similarly,

$$g'_5 = \begin{matrix} XI \\ IX \end{matrix}, \quad g'_6 = \begin{matrix} II \\ ZZ \end{matrix} \quad (7.26)$$



**Figure 21:** Honeycomb lattice for a 36 qubit honeycomb code. Vertices in the lattice represent qubits, edges represent two-qubit check measurements, and hexagonal plaquettes represent stabilizers of the code. Edge color indicates its check measurement basis, where 2-qubit  $X$  measurements are performed on green edges, 2-qubit  $Y$  measurements on red edges, and 2-qubit  $Z$  measurements on blue edges. Similarly, green plaquettes are 6-qubit  $X$  stabilizers, red plaquettes are 6-qubit  $Y$  stabilizers, and blue plaquettes are 6-qubit  $Z$  stabilizers.

is also an  $X, Z$  operator pair of a gauge qubit. Therefore, we have  $r = 2$  stabilizer generators and  $g = 2$  gauge qubits:

$$\mathcal{G} = \left\langle \underbrace{\begin{pmatrix} XX & ZZ \\ XX & ZZ \end{pmatrix}}_{2 \text{ stabilizers}}, \underbrace{\begin{pmatrix} XX & IZ \\ II & ZI \end{pmatrix}}_{\text{gauge-q 1}}, \underbrace{\begin{pmatrix} XI & II \\ IX & ZZ \end{pmatrix}}_{\text{gauge-q 2}} \right\rangle, \quad (7.27)$$

which means that there is  $k = n - r - g = 4 - 2 - 2 = 0$  logical qubits, as claimed.

### 7.3 Hastings-Haah honeycomb Floquet code

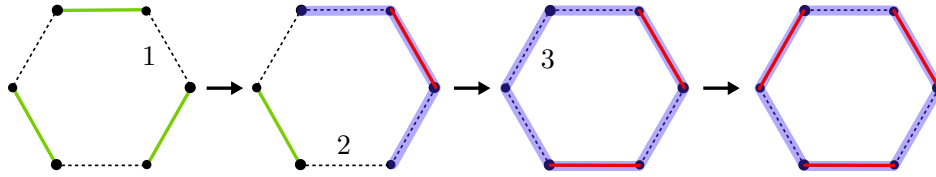
Now let us return to the Kitaev's honeycomb code [118], discussed in Section 7.1 which is a subsystem code with zero logical qubits. We will show that by picking an appropriate measurement schedule, each consisting of a set of 2-qubit measurements, we can actually encode two logical qubits *dynamically*. This was discovered by Hastings and Haah in their seminal work [120] in 2021, 15 years after Kitaev introduced the subsystem honeycomb code.

Recall the construction of Kitaev's code by tessellating the surface of a torus with hexagons, resulting in a regular hexagonal lattice, and then coloring the edges and faces with three colors (see Figure 21 and caption therein). Since all checks of the same color commute (in fact, they act on disjoint qubits), we can measure all of them in the same round, simultaneously. So, let us consider a period-3 measurement schedule cycling between the green, red, and blue checks:

$$\text{green} \rightarrow \text{red} \rightarrow \text{blue} \rightarrow \text{green} \rightarrow \text{red} \rightarrow \dots \quad (7.28)$$

Let us denote the set of green, red, and blue checks as  $C_g, C_r, C_b$ , respectively. Also denote the set of green, red, and blue plaquette stabilizers as  $A_g, A_r, A_b$ , respectively. Let us first focus on a red check measurement around a hexagonal plaquette and assume that we have just measured all green checks so that we start with a stabilizer group  $\mathcal{S} = \langle C_g \rangle$ . Note that a blue plaquette is surrounded

by three red  $Y$  checks and three green  $X$  checks. Then, we measure the three red checks one by one, starting from the pre-existing three green  $X \otimes X$  checks, as indicated in the following figure:



Here, we label the three edges where the red  $Y \otimes Y$  checks are measured as 1, 2, and 3. Let us describe what happens to the stabilizer generators on this plaquette as we update them using the stabilizer update rules in [Lemma 7.1](#) as illustrated in the diagram above from left to right:

1. We start with stabilizer generators  $g_1, g_2, g_3$  (three green  $X$  checks) on this plaquette.
2. Measure red check 1  $r_1$ :  
(Rule 3) Remove one noncommuting green check  $g_1$ , update the other weight-2 green check  $g_2$  to  $g_1g_2$  (resulting in weight-4  $X$  stabilizer highlighted in blue), put red check 1  $r_1$  in the stabilizer group. The updated stabilizers on this plaquette are generated by:  $r_1, g_1g_2, g_3$ .
3. Measure red check 2  $r_2$ :  
(Rule 3) Remove the one remaining green check  $g_3$  since it anticommutes with  $r_2$ . Update the weight-4  $X$  check  $g_1g_2$  to  $g_1g_2g_3$ . Put red check 2  $r_2$  in the stabilizer group. The updated stabilizers on this plaquette is generated by:  $r_1, r_2, g_1g_2g_3$ .
4. Measure red check 3  $r_3$ :  
(Rule 2)  $r_3$  commutes with  $r_1, r_2, g_3g_1g_2$ , so we put red check 3  $r_3$  in the stabilizer group. The updated stabilizers on this plaquette is generated by:  $r_1, r_2, r_3, g_1g_2g_3$ .

Note that by the end of the red check round, there is one weight-6  $X$  stabilizer generator and three weight-2  $Y$  stabilizer generators. Also, note that the product of all of these four stabilizers is a 6-qubit Pauli  $Z$  operator, which is also a stabilizer, that is,  $g_3g_1g_2r_1r_2r_3 = ZZZZZZ$ . This is a blue  $Z$  plaquette stabilizer in  $A_b$  whose syndrome we have just obtained by measuring the three green checks  $g_1, g_2, g_3$  (in the assumed previous round) and the three red checks  $r_1, r_2, r_3$  around it.

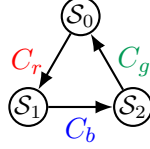
In the next round, we measure all blue  $Z$  checks and perform the same update steps on each green plaquettes. Each green plaquette is surrounded by three red  $Y$  checks  $r_1, r_2, r_3$ , which will be updated to their product  $r_1r_2r_3$  with three blue  $Z$  checks  $b_1, b_2, b_3$  surrounding this plaquette. Then, by taking their product, we obtain a 6-qubit green  $X$  plaquette stabilizer  $r_1r_2r_3b_1b_2b_3 = XXXXXX$  in  $A_g$ . Then, the remaining stabilizers on this plaquette are the three blue  $Z$  checks  $b_1, b_2, b_3$  and the 6-qubit stabilizer  $r_1r_2r_3b_1b_2b_3 = XXXXXX$  in  $A_g$ . A similar analysis on each red plaquette in the next round where we measure green  $X$  checks will give us three green  $X$  checks  $g_1, g_2, g_3$  and a 6-qubit stabilizer  $g_1g_2g_3b_1b_2b_3 = YYYYYY \in A_r$  as the stabilizer generators on each red plaquette.

In summary, the above procedure initializes the honeycomb Floquet code as follows:

1. At round  $r = -4$  we start with ISG  $\mathcal{S}_{-4}$  consisting only of the identity  $I^{\otimes n}$ .
2. At round  $r = -3$  the ISG is  $\mathcal{S}_{-3} = \langle C_g \rangle$ .
3. At round  $r = -2$  the ISG is  $\mathcal{S}_{-2} = \langle A_b, C_r \rangle$ .
4. At round  $r = -1$  the ISG is  $\mathcal{S}_{-1} = \langle A_g, A_b, C_b \rangle$ .

5. At round  $r = 0$  the ISG is  $\mathcal{S}_0 = \langle A_r, A_g, A_b, C_g \rangle$ .
6. At round  $r > 0$  the ISG is  $\mathcal{S}_r = \langle A_r, A_g, A_b, C_l \rangle$  where  $l = g$  if  $r = 0 \pmod 3$  and  $l = r$  if  $r = 1 \pmod 3$  and  $l = b$  if  $r = 2 \pmod 3$ .

So, we have obtained a period-3 measurement sequence that evolves the ISGs in period-3:



The checks in each round, the three ISGs and the plaquette stabilizer syndromes obtained in each round can be summarized as follows:

| round $i \pmod 3$   | 0                                                    | 1                                                    | 2                                                    |
|---------------------|------------------------------------------------------|------------------------------------------------------|------------------------------------------------------|
| checks              | green $X$                                            | red $Y$                                              | blue $Z$                                             |
| ISG $\mathcal{S}_i$ | $\mathcal{S}_0 = \langle A_r, A_g, A_b, C_g \rangle$ | $\mathcal{S}_1 = \langle A_r, A_g, A_b, C_r \rangle$ | $\mathcal{S}_2 = \langle A_r, A_g, A_b, C_b \rangle$ |
| syndromes           | $A_r(Y)$                                             | $A_b(Z)$                                             | $A_g(X)$                                             |

**Honeycomb Floquet code as a subsystem code.** When we take the check measurements of the honeycomb Floquet code to generate a gauge group  $\mathcal{G}$ , we will recover Kitaev's subsystem honeycomb code with this gauge group which does not allow any logical information to be encoded. The honeycomb Floquet code however does encode two logical qubits, dynamically. We can see this by counting the number of independent stabilizer generators for each ISG  $\mathcal{S}_i$ .

First, note that the honeycomb lattice has  $n$  qubits,  $n/2$  plaquettes, and  $3n/2$  edges. Let us first focus on ISG  $\mathcal{S}_0 = \langle A_r, A_g, A_b, C_g \rangle$  and consider the following facts:

1. There are  $\frac{n}{2}$  plaquette stabilizers  $A = A_r \cup A_g \cup A_b$ .
2. Green checks  $X \otimes X$  are  $\frac{1}{3}$  of all edges:  $\frac{1}{3} \frac{3n}{2} = \frac{n}{2}$  edge stabilizers (all green checks).
3. Plaquette stabilizers are not independent (product of all plaquettes is  $I$ ). So, there are  $\frac{n}{2} - 1$  independent plaquette stabilizers. (e.g., take all plaquettes, except for a green plaquette stabilizer to form an independent generator.)
4. Green checks  $C_g$  and the remaining  $\frac{n}{2} - 1$  plaquette stabilizers in  $A$  are not independent since their product is  $I$ . (i.e., take the product between all green  $X$  checks and all green  $X$  plaquette stabilizers.) So there are  $\frac{n}{2} - 1$  independent green checks.

From point 3 and 4 above,  $\mathcal{S}_0$  has  $\frac{n}{2} - 1$  2-qubit green  $X$  checks and  $\frac{n}{2} - 1$  6-qubit plaquette stabilizers as a set of independent generators. Thus, there are  $r = \frac{n}{2} - 1 + \frac{n}{2} - 1 = n - 2$  independent stabilizer generators in total and

$$k = n - r = n - (n - 2) = 2 \quad (7.29)$$

logical qubits. By doing a similar analysis to  $\mathcal{S}_1$  and  $\mathcal{S}_2$ , we will reach to the same conclusion.

We remark that the difference between ISG  $\mathcal{S}_i$  and the stabilizer group of Kitaev's honeycomb code is that the non-trivial loops are not in any of the ISGs  $\mathcal{S}_i$  since it is never measured in the honeycomb Floquet code's measurement schedule. Namely, in Kitaev's code, the value of non-trivial loop operators  $\ell_1, \ell_2$  are fixed to  $+1$ , that is, measuring them will give  $+1$  outcome. On the other hand in ISG  $\mathcal{S}_i$  of the honeycomb Floquet code, they are logical operators for the two logical qubits, that is, there exists another pair of non-trivial loop operators  $\ell'_1, \ell'_2$  such that  $\ell_1 \ell'_1 = -\ell'_1 \ell_1$  and  $\ell_2 \ell'_2 = -\ell'_2 \ell_2$  but  $\ell'_1$  commutes with both  $\ell_2, \ell'_2$  and  $\ell'_2$  commutes with both  $\ell_1, \ell'_1$ .

## 7.4 Conservation of logical information in dynamical codes

As measurements in general change the state of a quantum system, it is essential to ensure that check measurements do not destroy any encoded logical information while also extracting error syndromes. This property, which we call the *conservation of logical information*, is necessary for fault-tolerant quantum memory implemented through a dynamical code. In particular, one must ensure that the check measurements in round  $r + 1$  of the measurement schedule induces a mapping from any code state  $|\psi^{(r)}\rangle$  of the previous round  $r$  to a code state  $|\psi^{(r+1)}\rangle$  of round  $r + 1$  such that the code state amplitudes are conserved. More formally, round  $r + 1$  check measurements  $\mathcal{M}_{r+1}$  induce a mapping of the form

$$|\psi^{(r)}\rangle = \sum_j \alpha_j |j^{(r)}\rangle \mapsto |\psi^{(r+1)}\rangle = \sum_j \alpha_j |j^{(r+1)}\rangle \quad (7.30)$$

for any amplitude  $\{\alpha_j\}_j$ , where  $\{|j^{(r+1)}\}_j$  is an arbitrary basis of the logical codespace in round  $r$ .

The conservation of logical information has been noted since the conception of dynamical codes, where Hastings and Haah explicated the electric-magnetic automorphism of the honeycomb Floquet code's dynamical logical operator in consecutive rounds [120, Section 1.4] (such a logical operator is referred to as the *outer* logical operator by Hastings and Haah). Namely, for a representative of a dynamical logical operator  $\bar{L}^{(r)}$  of ISG  $\mathcal{S}^{(r)}$  in round  $r$  and logical operator  $\bar{L}^{(r+1)}$  of ISG  $\mathcal{S}^{(r+1)}$  in round  $r + 1$ , their product  $\bar{L}^{(r+1)}\bar{L}^{(r)}$  is equal to a round- $r$  stabilizer  $s_r \in \mathcal{S}^{(r)}$ . Since the product between  $\bar{L}^{(r+1)}$  and a stabilizer  $s^{(r+1)} \in \mathcal{S}^{(r+1)}$  gives a different representative of the same logical operation, this is equivalent to

$$s^{(r)}\bar{L}^{(r)} = s^{(r+1)}\bar{L}^{(r+1)}. \quad (7.31)$$

Alam and Rieffel show [110] that this equality is a sufficient condition for the conservation of logical information in Eq. (7.30) to hold (see [110, Eq. 2 and Appendix A]).

## 8 Bosonic codes

In this chapter, we take a brief detour into continuous-variable (CV) encodings, which differ in a fundamental way from the finite-dimensional setting emphasized so far. Previous chapters focused on codes built from many finite-dimensional subsystems whose tensor-product structure enables entangled codewords with protection against environmental noise. In the CV setting, the physical carrier is intrinsically infinite dimensional, which enables [monolithic encodings](#) [\[Ec 200\]](#): codespaces embedded in the Hilbert space of a single physical system rather than distributed across multiple subsystems. Monolithic encodings can also be realized in single high-dimensional qudits [\[123, 124\]](#), but continuous-variable platforms make such single-system embeddings especially natural. Prominent instances include bosonic encodings in oscillator-like degrees of freedom [\[125\]](#), as well as related constructions using atomic and molecular structure [\[126, 127\]](#). With this perspective in place, we now begin our study of bosonic codes.

### 8.1 Bases of a simple harmonic oscillator

Bosonic quantum error correction refers to encoding quantum information in physical systems that can be modeled as simple harmonic oscillators (SHOs). The oscillator modes could be electromagnetic (e.g., microwave cavities, optical cavities, optical fibers, or free space) or mechanical (e.g., vibrations of atomic or molecular nuclei, acoustic resonators, nanomechanical resonators, or trapped-ion crystals). The appeal of bosonic encodings is that a single oscillator mode already has a large, well-structured Hilbert space. Rather than assembling protection solely from many two-level systems, one can try to use this internal structure directly. This leads to code families whose protection mechanisms are naturally expressed in Fock space, phase space, or the coherent-state picture, which we will properly introduce below. These code families provide a vast playground to explore for quantum error correction.

SHOs allow monolithic encodings because their Hilbert spaces are infinite dimensional and admit continuous-variable representations. In particular, an oscillator state can be described in the position or momentum basis, whose generalized basis states  $|x\rangle$  and  $|p\rangle$  are labeled by real numbers  $x, p \in \mathbb{R}$ .<sup>11</sup> These generalized basis states are not normalizable physical states, but they make the continuous-variable structure of an oscillator explicit. The canonical position and momentum bases are summarized in [Box 8.1](#).

#### Box 8.1: Position and momentum bases

|                | Position basis                                        | Momentum basis                                        |
|----------------|-------------------------------------------------------|-------------------------------------------------------|
| States         | $\{ x\rangle \mid x \in \mathbb{R}\}$                 | $\{ p\rangle \mid p \in \mathbb{R}\}$                 |
| Eigenoperators | $\hat{x} x\rangle = x x\rangle$                       | $\hat{p} p\rangle = p p\rangle$                       |
| Orthonormality | $\langle x y\rangle = \delta(x - y)$                  | $\langle p q\rangle = \delta(p - q)$                  |
| Completeness   | $\int_{\mathbb{R}} dx  x\rangle \langle x  = \hat{I}$ | $\int_{\mathbb{R}} dp  p\rangle \langle p  = \hat{I}$ |
| Conversion     | $\langle p x\rangle = \frac{1}{\sqrt{2\pi}} e^{-ipx}$ | $\langle x p\rangle = \frac{1}{\sqrt{2\pi}} e^{ipx}$  |

<sup>11</sup>Since eigenvalues and operators are often denoted by the same symbols in continuous-variable systems, we adopt the  $\hat{\phantom{x}}$  notation for operators in this chapter. Ket notation is also somewhat overloaded: the same symbol  $|\cdot\rangle$  will be used for position ( $x$ ), momentum ( $p$ ), Fock ( $n$ ), and coherent states ( $\alpha$ ). Usually, the intended basis is clear from context; when needed, we add subscripts such as  $|\cdot\rangle_x$  and  $|\cdot\rangle_p$ . As a rule of thumb, integer, complex-number, and real-number labels typically denote Fock states, coherent states, and position or momentum states, respectively.

Throughout this chapter, we use dimensionless quadratures and set  $\hbar = 1$ , so that  $[\hat{x}, \hat{p}] = i$ . The energy spectrum of an SHO is governed by the Hamiltonian

$$\hat{H} = \frac{\omega}{2} (\hat{p}^2 + \hat{x}^2) , \quad (8.1)$$

where  $\omega$  depends on the particular system at hand. A natural basis of states for any quantum system is the set of energy eigenvectors. We find them by solving the Schrödinger equation

$$\hat{H} |n\rangle = E_n |n\rangle , \quad (8.2)$$

uncovering the energy eigenstates

$$|n\rangle = \int_{\mathbb{R}} dx \frac{1}{\sqrt{2^n n!}} \frac{1}{\pi^{1/4}} e^{-\frac{x^2}{2}} H_n(x) |x\rangle \quad (8.3)$$

with energies

$$E_n = \omega \left( n + \frac{1}{2} \right) . \quad (8.4)$$

Here,  $H_n$  denotes the  $n^{\text{th}}$  Hermite polynomial [128]. These states look cumbersome and are practically impossible to work with. A simple change of basis eliminates this complexity and makes the SHO one of the simplest and most widely studied quantum systems. For this, we introduce the *annihilation* ( $\hat{a}$ ), *creation* ( $\hat{a}^\dagger$ ) and *number* ( $\hat{n}$ ) operators:

$$\hat{a} = \frac{1}{\sqrt{2}} (\hat{x} + i\hat{p}) , \quad (8.5)$$

$$\hat{a}^\dagger = \frac{1}{\sqrt{2}} (\hat{x} - i\hat{p}) , \quad (8.6)$$

$$\hat{n} = \hat{a}^\dagger \hat{a} . \quad (8.7)$$

Using these definitions, we can rewrite the Hamiltonian as

$$\hat{H} = \omega \left( \hat{n} + \frac{1}{2} \right) . \quad (8.8)$$

This is already reminiscent of Eq. (8.4) because every energy eigenstate  $|n\rangle$  is exactly the eigenstate of  $\hat{n}$  with eigenvalue  $n$ . Thus, we introduce the *Fock basis* for the SHO system below.

### Box 8.2: Fock basis

|                |                                                                                                 |
|----------------|-------------------------------------------------------------------------------------------------|
| States         | $\{  n\rangle \mid n \in \mathbb{Z}_{\geq 0} \}$                                                |
| Eigenoperator  | $\hat{n}  n\rangle = n  n\rangle$                                                               |
| Orthonormality | $\langle n   m \rangle = \delta_{nm}$                                                           |
| Completeness   | $\sum_{n=0}^{\infty}  n\rangle \langle n  = \hat{I}$                                            |
| Conversion     | $\langle x   n \rangle = \frac{1}{\sqrt{2^n n!}} \frac{1}{\pi^{1/4}} e^{-\frac{x^2}{2}} H_n(x)$ |

The action of the annihilation and creation operators on the Fock states can be simply described as

$$\hat{a} |n\rangle = \sqrt{n} |n-1\rangle , \quad (8.9)$$

$$\hat{a}^\dagger |n\rangle = \sqrt{n+1} |n+1\rangle . \quad (8.10)$$

One can imagine the annihilation (creation) operator “annihilating” (“creating”) one quantum of energy in the system and therefore reducing (increasing) the total energy by one unit.

Finally, we introduce *coherent states* for the SHO below in [Box 8.3](#).

**Box 8.3:** Coherent-state basis

|                  |                                                                                                |
|------------------|------------------------------------------------------------------------------------------------|
| States           | $\{ \alpha\rangle \mid \alpha \in \mathbb{C}\}$                                                |
| Eigenoperator    | $\hat{a} \alpha\rangle = \alpha \alpha\rangle$                                                 |
| Overlap          | $\langle\alpha \beta\rangle = e^{-\frac{ \alpha ^2}{2} - \frac{ \beta ^2}{2} + \alpha^*\beta}$ |
| Overcompleteness | $\int \frac{d^2\alpha}{\pi}  \alpha\rangle\langle\alpha  = \hat{I}$                            |
| Conversion       | $\langle n \alpha\rangle = e^{-\frac{ \alpha ^2}{2}} \frac{\alpha^n}{\sqrt{n!}}$               |

Coherent states are not exactly orthogonal, but the magnitude of their overlap  $|\langle\alpha|\beta\rangle| = e^{-|\alpha-\beta|^2/2}$  becomes negligible for large separations  $|\alpha - \beta|$ .

## 8.2 Error channels

Possessing a differently structured Hilbert space, bosonic systems are naturally prone to different kinds of noise channels than qubits. We describe some of the most prominent bosonic error channels in this subsection.

**Definition 8.1:** Lindblad description of quantum channels

A *quantum channel* is a map that takes an input density matrix  $\rho(0)$  to an output density matrix  $\rho(t)$ . One common way to describe such a channel is through a set of *Kraus operators*  $\{\hat{E}_k\}$ :

$$\rho(t) = \sum_k \hat{E}_k \rho(0) \hat{E}_k^\dagger, \quad (8.11)$$

with  $\sum_k \hat{E}_k^\dagger \hat{E}_k = \hat{I}$ . Each Kraus operator can be viewed as one possible error process contributing to the overall noisy evolution.

When the noise acts continuously in time and memory effects can be neglected, it is often convenient to describe the dynamics instead by a *Lindblad equation*

$$\frac{d\rho}{dt} = -i[\hat{H}, \rho] + \sum_j \kappa_j \left( \hat{L}_j \rho \hat{L}_j^\dagger - \frac{1}{2} \{ \hat{L}_j^\dagger \hat{L}_j, \rho \} \right), \quad (8.12)$$

where the Hamiltonian  $\hat{H}$  describes coherent evolution and the operators  $\hat{L}_j$  describe dissipative processes such as photon loss or dephasing. Solving this differential equation for a finite time interval produces a quantum channel, which can then be written in Kraus form. Thus, the Lindblad equation describes the infinitesimal time evolution, while the Kraus operators describe the resulting finite-time channel.



### 8.2.1 Photon loss

*Photon loss* [129] (a.k.a. the pure-loss or bosonic amplitude-damping channel) is the most common source of incoherent errors in optical and microwave cavities, which are among the most developed platforms for realizing bosonic quantum computing. In the continuous-time description of [Definition 8.1](#), evolution under this channel is described by the Lindblad equation

$$\frac{d\rho}{dt} = \kappa \left( \hat{a}\rho\hat{a}^\dagger - \frac{1}{2}\{\hat{n}, \rho\} \right), \quad (8.13)$$

where  $\rho$  is the density matrix of the state and  $\kappa$  is the excitation-loss rate determined by the physical setup. This channel is equivalently described in the Kraus representation as

$$\rho(t) = \sum_{\ell=0}^{\infty} \hat{E}_\ell \rho(0) \hat{E}_\ell^\dagger, \quad (8.14)$$

$$\hat{E}_\ell = \left( \frac{\gamma}{1-\gamma} \right)^{\ell/2} \frac{\hat{a}^\ell}{\sqrt{\ell!}} (1-\gamma)^{\hat{n}/2} \quad \forall \ell \geq 0, \quad (8.15)$$

where  $\gamma = 1 - e^{-\kappa t}$ . The  $\ell^{\text{th}}$  Kraus operator induces the loss of  $\ell$  photons. The relative weight of this process decreases exponentially with  $\ell$  at a fixed time. Thus, multi-photon losses are exponentially suppressed. This is analogous to high-weight Pauli errors being exponentially suppressed in qubit systems.

At an intuitive level, photon loss causes the oscillator to drift toward lower-energy states by randomly removing some number of excitations. This makes codes with well-separated Fock-state support especially natural for detecting and correcting such errors. Each photon loss is also accompanied by a *back-action* term  $(1-\gamma)^{\hat{n}/2}$ . Notably, even for the zero-loss operator  $\hat{E}_0$ , the system experiences some redistribution of probabilities due to this back-action. This process favors lower-energy states by damping higher Fock states. Consequently, only the vacuum state  $|0\rangle$  survives in the infinite-time limit since

$$\lim_{t \rightarrow \infty} (1-\gamma)^{n/2} = \delta_{n0}. \quad (8.16)$$

### 8.2.2 Dephasing

The second most common incoherent error in electromagnetic cavities is *dephasing*, caused by fluctuations in the cavity frequency. A general description of the dephasing noise channel  $\mathcal{N}_{\text{dep}}$  [130] acting on a density matrix  $\rho$  is

$$\mathcal{N}_{\text{dep}}(\rho) = \int_{-\infty}^{\infty} d\phi p(\phi) e^{-i\hat{n}\phi} \rho e^{i\hat{n}\phi} \quad (8.17)$$

$$= \sum_{m,n \in \mathbb{Z}_{\geq 0}} \rho_{mn} \left[ \int_{-\infty}^{\infty} d\phi p(\phi) e^{-i(m-n)\phi} \right] |m\rangle \langle n| \quad (8.18)$$

where

$$p(\phi) = \frac{1}{\sqrt{2\pi\gamma_{\text{dep}}}} e^{-\phi^2/2\gamma_{\text{dep}}} \quad (8.19)$$

is a probability distribution and  $\rho_{mn}$  are the matrix elements of  $\rho$  written in the Fock basis. This noise channel induces Fock-number-dependent phase shifts  $\phi$  to the state governed by some probability distribution. The probability distribution  $p(\phi)$  is peaked at  $\phi = 0$  and decays with increasing

$|\phi|$ , indicating that larger dephasing events are less likely. The Lindblad equation for this channel is

$$\frac{d\rho}{dt} = \kappa_{\text{dep}} \left( \hat{n}\rho\hat{n}^\dagger - \frac{1}{2}\{\hat{n}^2, \rho\} \right), \quad (8.20)$$

and the Kraus operators are

$$\hat{E}_k = \sqrt{\frac{\gamma_{\text{dep}}^k}{k!}} e^{-\frac{\gamma_{\text{dep}}}{2}\hat{n}^2} \hat{n}^k \quad \forall k \geq 0 \quad (8.21)$$

where  $\gamma_{\text{dep}} = \kappa_{\text{dep}}t$ . Just as in the photon-loss channel, these Kraus operators contain a back-action term, here  $e^{-\gamma_{\text{dep}}\hat{n}^2/2}$ . For example, the  $k = 0$  Kraus operator, corresponding to the no-jump dephasing branch, still redistributes the population in favor of the lower energy Fock states. However, it is important to keep in mind that the full dephasing channel does not redistribute Fock-state populations; when the measurement record is ignored, only the coherences between different Fock states decay.

For the purposes of code construction, one typically simplifies the Kraus operators to be proportional to  $\hat{a}^\ell$  and  $\hat{n}^k$  as the loss and dephasing error operators. The back-action terms are justifiably ignored due to their relatively negligible contributions at small time scales when  $\hat{a}^\ell$  and  $\hat{n}^k$  dominate. In simple terms, dephasing preserves photon number but scrambles the relative phases between different Fock components. Codes that organize information by symmetry in phase space or by carefully chosen parity structure can therefore be made robust against such noise.

### 8.2.3 Additive white Gaussian noise (AWGN)

The Gaussian displacement noise [131] evolves a state  $\rho$  as

$$\int_{-\infty}^{\infty} d^2\alpha p_\sigma(\alpha) \hat{D}(\alpha) \rho \hat{D}(\alpha)^\dagger, \quad (8.22)$$

where

$$p_\sigma(\alpha) = \frac{1}{\sigma^2 2\pi} e^{-|\alpha|^2/2\sigma^2} \quad (8.23)$$

is the probability of the occurrence of a displacement by  $\alpha$ . The displacement operator

$$\hat{D}(\alpha) = \exp\left(\alpha\hat{a}^\dagger - \alpha^*\hat{a}\right) \quad (8.24)$$

displaces a coherent state  $|\beta\rangle$  by an amount  $\alpha$ , up to a phase:

$$\hat{D}(\alpha) |\beta\rangle = e^{(\alpha\beta^* - \beta\alpha^*)/2} |\beta + \alpha\rangle. \quad (8.25)$$

Its action on the position- and momentum-basis states is

$$\hat{D}(\alpha) |x\rangle = e^{i\sqrt{2}\alpha_I x + i\alpha_R \alpha_I} \left| x + \sqrt{2}\alpha_R \right\rangle, \quad (8.26)$$

$$\hat{D}(\alpha) |p\rangle = e^{-i\sqrt{2}\alpha_R p - i\alpha_R \alpha_I} \left| p + \sqrt{2}\alpha_I \right\rangle, \quad (8.27)$$

where  $\alpha = \alpha_R + i\alpha_I$  for  $\alpha_R, \alpha_I \in \mathbb{R}$ . Thus, it displaces the quadratures of the state, up to a phase. In the special case where the displacement parameter is purely real or purely imaginary, it displaces only one quadrature. These special cases correspond to the operators  $e^{-i\alpha\hat{p}}$  and  $e^{i\alpha\hat{x}}$ , which displace  $x$  and  $p$  by an amount  $\alpha \in \mathbb{R}$ , respectively.

Eq. (8.22) can be viewed as Kraus evolution with a continuum of Kraus operators  $\hat{E}_\alpha = \sqrt{p_\sigma(\alpha)} \hat{D}(\alpha)$ . Thus, the action of the channel is to displace the state's quadratures by random amounts governed by a Gaussian probability distribution peaked at zero.

### Exercise 8.1: Position and momentum displacement operators

Prove that the operators  $e^{-i\alpha\hat{p}}$  and  $e^{i\alpha\hat{x}}$  act as position and momentum displacements, respectively.

While we described some of the most prominent noise channels here, our list is not exhaustive. The actual noise model for a system can vary depending on the implementation and the architecture used. For example, superconducting cavities can be afflicted by higher-order nonlinearities inherited from the Josephson junctions. It is common to use a transmon (two-level-system) qubit for control and measurements of bosonic states, which makes the system prone to leakage and additional errors. For the purposes of this work, we will describe codes only in the context of photon loss, dephasing, and AWGN channels but encourage the reader to look at Refs. [132–139] for more details on other kinds of noise channels.

## 8.3 Code examples

The following sections describe several well-studied codes. These codes are not only among the most popular choices in practice, but also provide a useful baseline for thinking about various aspects of bosonic error correction. Very roughly, the main families below protect information in different geometric ways: dual-rail and binomial codes use separation in Fock space, GKP codes use a lattice in phase space, and cat codes use separated constellations of coherent states. Keeping these three pictures in mind makes it easier to see why different bosonic codes are naturally matched to their respective noise models. We begin with codes based on the Fock-state description of the Hilbert space.

### 8.3.1 Fock-state codes

The [dual-rail code](#) [Ec<sub>200</sub>](#) is arguably the simplest bosonic code, with logical information encoded in joint states of two bosonic modes (or “rails”). These two modes can be realized either as two orthogonal modes of the same resonator or as two separate oscillators, depending on the physical platform. It detects up to one photon-loss error with the code states

$$\begin{aligned} |0\rangle_L &= |0, 1\rangle, \\ |1\rangle_L &= |1, 0\rangle. \end{aligned} \tag{8.28}$$

Here, the two numbers in  $|n_1, n_2\rangle$  represent the Fock-state occupation numbers of the two individual modes. Photon loss can be detected simply by measuring the total photon number, which is 1 for the codespace and 0 for the erroneous state  $|0, 0\rangle$ . Since the code cannot correct any errors, it relies on post-selection: after detection, one either keeps or resets the state depending on the outcome. Despite not correcting any errors, it is a popular choice due to ease of implementation and has enjoyed many experimental demonstrations [140–146].

[Binomial codes](#) [Ec<sub>200</sub>](#) [147] are single-mode, Fock-state-based encodings that can be engineered to protect against any desired number of losses and any desired dephasing order. The canonical binomial code, which corrects up to one photon-loss error, that is, the error set  $\{\mathbb{I}, \hat{a}\}$ , admits the basis

$$\begin{aligned} |0\rangle_L &= \frac{1}{\sqrt{2}} (|0\rangle + |4\rangle), \\ |1\rangle_L &= |2\rangle. \end{aligned} \tag{8.29}$$

The syndrome for this code can be measured by determining the parity of the total photon number, that is, by measuring the operator  $e^{i\pi\hat{n}}$ , with the following outcomes:

- +1 implies no error,
- -1 implies a photon-loss error.

In general, binomial codes are a family of  $q$ -dimensional codes described by parameters  $(N, S)$ , with code states  $\{|i\rangle_L \mid i \in \mathbb{Z}_q\}$  admitting a basis

$$|i\rangle_L = \frac{1}{\sqrt{q^N}} \sum_{\substack{p=0 \\ p \equiv i \pmod{q}}}^{(q-1)(N+1)} \sqrt{\binom{N+1}{p}_q} |p(S+1)\rangle. \quad (8.30)$$

Here,  $\binom{N}{p}_q$  are the extended binomial coefficients, defined as the coefficient of  $x^p$  in  $(1 + x + x^2 + \dots + x^{q-1})^N$ . They can be computed recursively using

$$\begin{aligned} \binom{N}{p}_1 &= 1, \\ \binom{N}{p}_q &= \sum_{k=0}^N \binom{N}{k} \binom{k}{p-k}_{q-1}. \end{aligned} \quad (8.31)$$

An  $(N, S)$  code protects against  $L$  photon losses,  $G$  photon gains, and dephasing up to order  $D$ , where  $S = L + G$  and  $N = \max(L, G, 2D)$ . For intuition, consider a code that protects against up to  $L$  photon losses and no photon gains. This code has  $S = L$ , meaning that each codeword is supported on Fock states spaced by at least  $L + 1$  from the support of every other codeword. Hence, any photon loss  $\hat{a}^\ell$  with  $\ell \leq S$  cannot mix two codewords. This structure ensures that the off-diagonal Knill-Laflamme conditions of the form  $\langle \vec{i} | \hat{a}^{\dagger m_1} \hat{a}^{m_2} | \vec{j} \rangle$  are satisfied for all  $m_1, m_2 \leq L$  and  $i \neq j$ . The remaining on-diagonal conditions are satisfied by the choice of binomial coefficients; see Ref. [147, Appendix C]. The syndrome measurement for a general member of the family is more sophisticated, and we refer the interested reader to Ref. [129, Sec. VI.D] for details.

### Exercise 8.2: Binomial code parameters

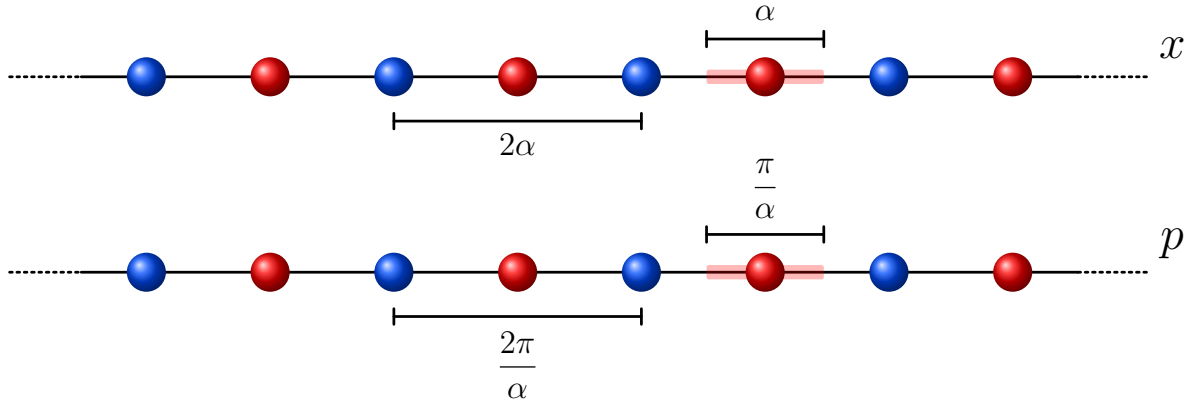
Determine the parameters  $q, N$  and  $S$  for the canonical binomial code in Eq. (8.29).

## 8.3.2 Phase space codes

Phase space codes are most conveniently defined using the position and momentum bases. Arguably the most studied bosonic family, the [Gottesman-Kitaev-Preskill \(GKP\) codes](#) [Eq. 260](#) are examples of phase-space codes. They are designed to protect predominantly against the AWGN (or displacement) noise model, but can still provide exceptional practical performance against photon loss [129]. They first appeared in the year 2000 in the seminal paper [148] by its namesakes: Daniel Gottesman, Alexei Kitaev, and John Preskill. For a canonical GKP code (see [Figure 22](#)), the code states are

$$|i\rangle_L = \sum_{s=-\infty}^{\infty} |\alpha(i + ds)\rangle_x. \quad (8.32)$$

We label the position and momentum bases by the subscripts  $x, p$ , respectively. The central idea is to encode information into a periodic grid in phase space so that sufficiently small shifts can



**Figure 22:** Code states for the  $d = 2$  GKP code are depicted by red and blue spheres. The red region shows the shifts for which the errors are correctable if the initial state was the red state.

be rounded back to the nearest lattice point. In this sense, GKP codes convert small continuous displacement errors into correctable syndromes.

Position displacement errors  $e^{-i\Delta_x \hat{p}}$  for  $|\Delta_x| \leq \alpha/2$  can be corrected by projecting the position-displaced state onto its nearest codeword. An equivalent description is given in terms of the stabilizers

$$\hat{S}_Z = e^{2\pi i \hat{x}/\alpha} \quad (8.33)$$

$$\hat{S}_X = e^{-id\hat{p}\alpha}. \quad (8.34)$$

Since  $\hat{S}_X$  is a displacement operator that shifts position states by an amount  $d\alpha$ , the code states are equal superpositions of position states spaced by  $d\alpha$ . The  $\hat{S}_Z$  stabilizer suggests an equivalent description via equally spaced momentum states, and indeed the code states (up to a change of logical basis) can be written in the momentum basis as

$$|j\rangle_L = \sum_{s=-\infty}^{\infty} \left| \frac{2\pi}{d\alpha} (j + ds) \right\rangle_p. \quad (8.35)$$

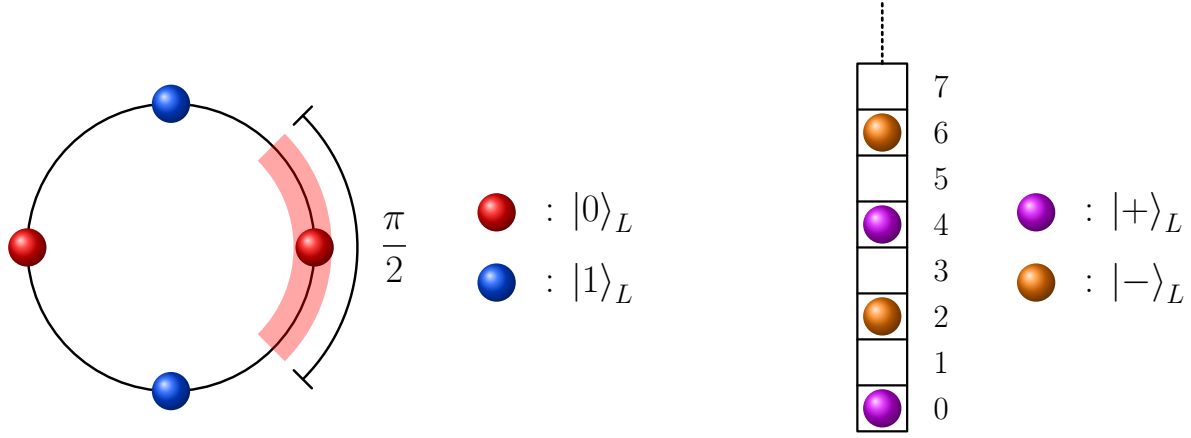
They analogously correct displacement errors  $e^{i\Delta_p \hat{x}}$  for  $|\Delta_p| \leq \frac{\pi}{d\alpha}$ .

Strictly speaking, the states in Eq. (8.32) are idealized and have infinite energy. Physical implementations use finite-energy approximate GKP states, which retain the same geometric intuition while only approximately realizing the perfect lattice picture.

In general, GKP-code stabilizers are isomorphic to symplectic integral lattices [149]. This correspondence can be used to define other kinds of single-mode and multi-mode GKP codes with interesting properties. We refer the interested reader to Jonathan Conrad's work for further exploration in this direction [131, 149–151]. Several broader reviews [152–155] may also be helpful to the reader in their exploration of the rich world of GKP codes.

### 8.3.3 Coherent state codes

Just like Fock- and phase-space-based encodings, one can think of codes defined natively in the coherent-state basis. Like Fock states, coherent states are closely tied to energy, but unlike the Fock basis, there is a continuum of coherent states with the same expected photon number. For fixed



**Figure 23:** (Left) Computational basis states for the cat code are visualized in the coherent state basis. The red region shows the dephasing shifts for which the errors are correctable if the initial state was the red state. (Right) Hadamard basis states for the cat code are visualized in the Fock basis.

$|\alpha\rangle$ , this family of states  $|e^{i\theta}\alpha\rangle$  is parameterized by  $\theta$  and traces a circle in the complex plane:

$$\langle e^{i\theta}\alpha | \hat{n} | e^{i\theta}\alpha \rangle = |\alpha|^2 \quad \forall 0 \leq \theta < 2\pi. \quad (8.36)$$

Coherent-state-based encodings typically use this manifold of equal-energy states to encode logical information. Although more challenging in practice, recent works have begun to explore other kinds of coherent-state-based encodings as well [156].

Equal-energy coherent states of the form  $|e^{i\theta}\alpha\rangle$  are in one-to-one correspondence with the points on a circle. **Cat code**  $\frac{\text{Ec}}{\text{zo}}$  states are equal superpositions of equiangular points on that circle; see [Figure 23](#). One such cat code has code states that are superpositions of the antipodal pairs  $\{\alpha, -\alpha\}$  and  $\{i\alpha, -i\alpha\}$  in the complex plane:

$$\begin{aligned} |0\rangle_L &= \frac{1}{\mathcal{N}} (|\alpha\rangle + |-\alpha\rangle), \\ |1\rangle_L &= \frac{1}{\mathcal{N}} (|i\alpha\rangle + |-i\alpha\rangle), \end{aligned} \quad (8.37)$$

where  $\mathcal{N}$  is a normalization factor. This code can correct up to one photon loss and protect against dephasing errors of the form  $e^{i\phi\hat{n}}$  for  $|\phi| < \pi/4$ . It is easy to see from [Figure 23](#) that the code states can be unambiguously distinguished if the rotation angle  $\phi$  is less than  $\pi/4$ . To gain intuition for loss correction, it is useful to view the code states in the Fock basis:

$$\begin{aligned} |+\rangle_L &= \frac{1}{\sqrt{2}} (|0\rangle_L + |1\rangle_L) \propto \sum_{n=0 \bmod 4} \frac{\alpha^n}{\sqrt{n!}} |n\rangle, \\ |-\rangle_L &= \frac{1}{\sqrt{2}} (|0\rangle_L - |1\rangle_L) \propto \sum_{n=2 \bmod 4} \frac{\alpha^n}{\sqrt{n!}} |n\rangle. \end{aligned} \quad (8.38)$$

Since the two code states are supported on Fock numbers differing by at least 2, any state resulting from a single photon loss can be mapped unambiguously to its original code state by measuring the photon number modulo 4. This Fock-state separation leads to satisfaction of the off-diagonal Knill–Laflamme conditions, similar to the case of binomial codes. The remaining conditions are satisfied by the choice of antipodal points for each codeword.

More generally, one can define a cat code in which each code state corresponds to an equal superposition of the vertices of a  $p$ -gon inscribed in a circle. By taking symmetrically rotated copies of the polygon, one obtains different codewords and hence a logical space of arbitrary dimension  $d$ . Such a  $p$ -gon code corrects up to  $p - 1$  photon-loss errors and dephasing errors for angles of magnitude up to  $\pi/(pd)$ . The code states can be written in the computational basis as

$$|j\rangle_L = \sum_{s=0}^{p-1} \left| \exp \left[ \frac{2\pi i}{p} \left( \frac{j}{d} + s \right) \right] \alpha \right\rangle. \quad (8.39)$$

Cat states can be generalized to the multi-mode setting in various nontrivial ways. [Quantum spherical codes](#) [\[157\]](#) and [tiger codes](#) [\[158\]](#) take superpositions of discrete and continuous constellations in higher-dimensional complex spheres, while quantum cubature codes [\[156\]](#) consider constellations spread across multiple spherical shells of different radii, that is, different energies. Many of them boast superior performance compared to trivial multi-mode generalizations based on independent copies of single-mode codes or simple concatenation schemes, and have opened up fascinating new directions in bosonic coding theory.

#### Exercise 8.3: Coherent state normalization

Why is the normalization factor  $\mathcal{N}$  in [Eq. \(8.37\)](#) different from  $\frac{1}{\sqrt{2}}$ ? Calculate  $\mathcal{N}$  as a function of  $\alpha$ . [Hint: consider the overlap between different coherent states.]

Taken together, these examples illustrate the main themes of bosonic coding theory. Dual-rail codes emphasize simplicity and error detection, binomial codes use carefully engineered Fock-space structure, GKP codes are especially well adapted to small displacement noise, and cat codes exploit geometric separation among coherent states. Which family is most attractive in practice often depends on the dominant noise, the available control operations, and the experimental platform. Overall, which code is the ultimate winner is still an open question.

## 8.4 Further reading

This chapter should serve as a starting point for anyone interested in bosonic quantum error correction. While we have tried to make it reasonably self-contained, many interesting aspects of bosonic codes lie beyond the present discussion. As with Clifford gates in qubit systems, bosonic systems admit an “easy” set of gates known as *Gaussian gates*. There is a theorem analogous to the Eastin-Knill no-go result [\[159\]](#), which says that this set alone cannot suffice for universal computation. Hence, computation with bosonic codes typically requires more sophisticated protocols to achieve universality [\[160, 161\]](#). Another interesting direction is concatenation with qubit-based encodings [\[162–165\]](#). Such codes use an outer qubit-code structure in which each individual qubit is itself a protected bosonic system. These proposals can reduce the effective physical noise afflicting the constituent qubits and hence lead to better overall performance. Control of bosonic qubits is another important topic and differs substantially from conventional qubit systems. In fact, ordinary qubits are often used as ancillas to control and measure bosonic states. These interactions can introduce additional noise into the bosonic system of interest. Engineering fault-tolerant control protocols for bosonic states remains an important area of research [\[153, 154, 166–169\]](#).

## 9 Decoders

To protect quantum information from noise, we encode logical qubits into a quantum error-correcting code. Noise then perturbs the encoded state. Although the physical state of the code has changed, the logical information may still be preserved. To continue the computation, one must apply a recovery operation that returns the system to the codespace without changing the encoded quantum information. Because direct measurement of the error would generally destroy the logical state, quantum error correction proceeds indirectly. One measures *syndromes*, that is, observables that reveal information about the error while commuting with the logical degrees of freedom. A *decoder* is the classical algorithm that takes this syndrome data and outputs a suitable recovery operation [170]. Decoders therefore form the interface between noisy quantum hardware and fault-tolerant logical computation.

A simple back-of-the-envelope estimate already reveals the scale of the challenge: the number of possible syndrome outcomes, and hence the number of candidate error patterns, typically grows exponentially with the number of physical qubits. At the same time, the decoder must run concurrently with the quantum processor and provide recovery operations on the fly, often on microsecond to millisecond time scales depending on the hardware platform. Thus, a useful decoder must be not only accurate, but also fast and scalable. In practice, decoder design is therefore governed by a three-way trade-off between accuracy, latency, and resource overhead [170, 171].

### 9.1 The decoding problem

Let us turn towards an explicit formulation of decoding, starting with a short recap of concepts introduced in earlier sections. Suppose a QEC code is subject to a set of  $n_e$  possible errors  $\{E_i\}_{i=1}^{n_e}$ , where error  $E_i$  occurs with probability  $p_i$ . The goal of decoding is to infer from syndrome information which error has occurred and the corresponding recovery operation that should be applied.

Because measuring the errors directly would typically destroy the encoded state, we instead measure a set of  $n_s$  syndrome observables  $\{S_j\}_{j=1}^{n_s}$  that commute with the logical operators. For qubit stabilizer codes, these syndrome observables are the stabilizers of the code, namely multi-qubit Pauli operators of the form

$$S_j = \bigotimes_{i=1}^n P_i, \quad (9.1)$$

with single-qubit Pauli operators  $P_i \in \{I, X, Y, Z\}$ . Measuring  $S_j$  yields  $s_j \in \{0, 1\}$ , where we map Pauli eigenvalue  $+1 \rightarrow s_j = 0$ , and eigenvalue  $-1 \rightarrow s_j = 1$ . In the absence of errors, we have  $s_j = 0, \forall j$ , while  $s_j = 1$  indicates that the  $j$ th stabilizer has flipped due to errors.

The relation between possible errors and syndrome outcomes is encoded by the parity-check matrix  $H \in \{0, 1\}^{n_s \times n_e}$ . Its rows correspond to the  $n_s$  syndrome measurements, and its columns correspond to the  $n_e$  possible errors. We have that  $H_{ij} = 1$  when the  $j$ th error flips the  $i$ th syndrome measurement; otherwise it is zero. Thus, given an error configuration  $\mathbf{e} \in \{0, 1\}^{n_e}$ , the syndrome is given by  $\mathbf{s} = H\mathbf{e}$ , where matrix-vector multiplication is performed modulo 2. Thus, one can think of the syndrome measurements as providing circumstantial information about the error.

The decoder must then identify a recovery operation that restores the code to the codespace while acting trivially on the logical information. Importantly, successful recovery does not require reconstructing the exact microscopic error. Rather, it is enough to find a correction whose combined action with the physical error is trivial on the logical subspace. For stabilizer codes under Pauli noise, this often reduces to applying an appropriate Pauli correction. We illustrate this process in detail for the Steane code in Examples 9.1 to 9.3.



**Example 9.1:** How to decode Steane code (part 1)

We now present a hands-on example of decoding, showing how to decode the [Steane code](#)  [62], which we introduced in [Example 4.5](#). As a short recap, the Steane code is a CSS code with parameters  $[[7, 1, 3]]$ . Here, 7 physical qubits encode 1 logical qubit and the code distance is 3, so it can correct any *single-qubit* Pauli error.

After the code is subject to an error  $E$  and we have measured the stabilizer syndromes, we infer a Pauli recovery operator  $F$  (typically the one with the lowest Pauli weight) such that  $FE$  is a stabilizer and therefore acts trivially on the logical state. The Steane code is a CSS code, so we can decode  $X$ -type errors and  $Z$ -type errors *independently*, using the same machinery. It is built from the classical  $[7, 4, 3]$  Hamming code. One convenient choice of parity-check matrix is

$$H = \begin{pmatrix} 1 & 1 & 1 & 0 & 1 & 0 & 0 \\ 1 & 1 & 0 & 1 & 0 & 1 & 0 \\ 1 & 0 & 1 & 1 & 0 & 0 & 1 \end{pmatrix}. \quad (9.2)$$

A CSS code is specified by two binary parity-check matrices  $H_X$  for  $X$ -type checks and  $H_Z$  for  $Z$ -type checks. For the Steane code, we take

$$H_X = H_Z = H. \quad (9.3)$$

We now label the physical qubits  $1, 2, \dots, 7$ . Each row of  $H$  defines:

- $X$ -type stabilizer generator: put an  $X$  on each qubit where the row has a 1;
- $Z$ -type stabilizer generator: put a  $Z$  on each qubit where the row has a 1.

Concretely, if the first row is  $(1, 1, 1, 0, 1, 0, 0)$ , then the corresponding generators are

$$S_X^{(1)} = X_1 X_2 X_3 X_5, \quad S_Z^{(1)} = Z_1 Z_2 Z_3 Z_5, \quad (9.4)$$

and similarly for rows 2 and 3. Altogether, there are 3  $X$ -type and 3  $Z$ -type stabilizers, that is, 6 independent generators, leaving  $7 - 6 = 1$  logical qubit.

**Example 9.2:** How to decode Steane code (part 2)

Notably, for CSS codes such as the Steane code, we can decode  $X$  and  $Z$  errors separately. Suppose the physical error is a Pauli operator

$$E = X^{\mathbf{e}_X} Z^{\mathbf{e}_Z}, \quad (9.5)$$

where  $\mathbf{e}_X, \mathbf{e}_Z \in \{0, 1\}^7$  indicate on which qubits an  $X$  or  $Z$  component occurs (and  $Y$  corresponds to both bits being 1 on the same qubit).

- Measuring  $Z$ -type stabilizers detects  $X$ -type errors:  $X$  anticommutes with  $Z$ .
- Measuring  $X$ -type stabilizers detects  $Z$ -type errors:  $Z$  anticommutes with  $X$ .

Thus, we get two syndrome vectors:

$$\mathbf{s}_X \in \{0, 1\}^3 \quad (\text{from } Z\text{-type checks, diagnosing } X \text{ errors}), \quad (9.6)$$

$$\mathbf{s}_Z \in \{0,1\}^3 \quad (\text{from } X\text{-type checks, diagnosing } Z \text{ errors}). \quad (9.7)$$

For the Steane code with  $H_X = H_Z = H$ , the syndrome equations are simply

$$\mathbf{s}_X = H \mathbf{e}_X \pmod{2}, \quad \mathbf{s}_Z = H \mathbf{e}_Z \pmod{2}. \quad (9.8)$$

Because the Steane code has distance 3, it corrects any single-qubit Pauli error. The most likely error is the one with the highest probability, which corresponds to the error of lowest weight. If we assume that at most one  $X$  error occurred, then  $\mathbf{e}_X$  is either the all-zero vector or a vector with a single 1.

## 9.2 Optimization formulation of decoding

We now sharpen the previous discussion by explicitly showing how to decode. While the syndromes give us information about what errors may have occurred, the syndromes alone do not point to a unique error. The main hurdle is that there are many possible errors that fit the syndrome constraint. In fact, the kernel of the parity check matrix  $H$  is the set of undetectable error configurations; modulo stabilizers, nontrivial elements are logical operators. Thus, a natural approach to decoding is to find the error  $\mathbf{e}^*$  with the highest probability that is consistent with the syndrome. This is called *most likely error* (MLE) decoding.

Formally, we assume that our code can be subject to a set of  $n_e$  errors  $\{E_i\}_i$ , with the  $i^{\text{th}}$  error occurring with probability  $p_i$ . The total probability of observing a particular error configuration  $\mathbf{e} \in \{0,1\}^{n_e}$  is given by

$$P(\mathbf{e}) = \prod_i (1 - p_i)^{1 - e_i} p_i^{e_i} = \prod_i (1 - p_i) \prod_i \left( \frac{p_i}{1 - p_i} \right)^{e_i}. \quad (9.9)$$

Then, error configuration  $\mathbf{e}$  is achieved by the following maximization problem:

$$\max_{\mathbf{e}} \ln(P(\mathbf{e})) = C - \sum_i w_i e_i \quad (9.10)$$

$$\text{s.t. } \mathbf{s} = H\mathbf{e}, \quad (9.11)$$

where we have applied logarithms to Eq. (9.9) for convenience, defined weights  $w_i = \ln((1 - p_i)/p_i)$  and an irrelevant constant  $C = \sum_i \ln(1 - p_i)$ . The weights  $w_i$  represent the Bayesian prior encoding our knowledge of the error probabilities. Assuming all error probabilities are the same, that is,  $p_i = p$ , then the optimal solution is the error with the lowest weight.

We note that performing MLE decoding requires accurate prior information about both the possible errors that can occur, as well as the probabilities with which each error occurs. Thus, a necessary precondition for QEC is that the noise of the physical platform is well-characterized.

### Example 9.3: How to decode Steane code: Part 3

We conclude the Steane code example with the simplest concrete decoder: a lookup rule. Because the Steane code is small, one can decode single-qubit errors by matching the observed syndrome to a precomputed table of likely corrections. Assuming that each qubit  $j$  is subject to the same error probability  $p_j = p$ , we now find the error of lowest weight that matches the syndrome. For a  $d = 3$  code, this corresponds to at most a single  $X$  error and a single  $Z$  error.

Now, for decoding, let  $H_{(:,j)}$  denote the  $j^{\text{th}}$  column of  $H$  as a 3-bit vector. Then,

*Assuming the most likely error is a single  $X$  error on qubit  $j$ , we have*

$$\mathbf{s}_X = H_{(:,j)}. \quad (9.12)$$

To decode, we compare  $\mathbf{s}_X$  to the columns of  $H$ . If it matches column  $j$ , infer an  $X$  error on qubit  $j$ . The same holds for  $Z$  errors using  $\mathbf{s}_Z$ .

However, what about  $Y$  errors? A single  $Y$  error on qubit  $j$  can be decomposed as  $Y_j = iX_jZ_j$ , so it flips *both* syndrome types. Therefore, independent decoding of  $X$  and  $Z$  will produce corrections  $X_j$  and  $Z_j$ , whose product is  $Y_j$  (up to phase), correcting the error.

Now, we give the decoding algorithm step-by-step. As input, we are given

- Parity check matrix  $H_X$  and  $H_Z$ ,
- measured syndrome bits for the  $Z$ -type stabilizers,  $\mathbf{s}_X \in \{0,1\}^3$ , and for the  $X$ -type stabilizers,  $\mathbf{s}_Z \in \{0,1\}^3$ .

We assume that we measured

$$\mathbf{s}_X = (1, 0, 1), \quad \mathbf{s}_Z = (0, 0, 0). \quad (9.13)$$

- $\mathbf{s}_Z = 0$  suggests no  $Z$  error.
- $\mathbf{s}_X = (1, 0, 1)$ : find the column of  $H$  equal to  $(1, 0, 1)$ , which corresponds to column 3 and thus to an  $X_3$  error; then apply the correction  $E = X_3$ .

The above lookup decoding is perfect for *at most one error*. If two physical errors occur, the syndrome is the XOR (mod 2) of two columns, which may coincide with a single column. Then, the minimum-weight decoder might choose the wrong hypothesis (a weight-1 correction) even though the true error had weight 2. This is unavoidable with codes of distance  $d = 3$ : the code is only guaranteed to correct errors of weight  $\lfloor (d-1)/2 \rfloor \leq 1$ . For more general codes with more qubits, lookup decoders are not practical as the number of possible correction operators usually scales exponentially in code size.

### 9.3 Degeneracy

The optimization problem above identifies the most likely physical error, but in QEC this is not always the quantity of operational interest. The reason is that quantum codes possess an intrinsically quantum feature that has no direct classical analogue: *degeneracy* [172].

In a classical error-correcting code, for a fixed transmitted codeword  $c$ , distinct error patterns  $e \neq e'$  produce distinct received words,  $c+e \neq c+e'$ . Thus, the received word uniquely determines the error pattern relative to  $c$ , and identifying the most likely error pattern is equivalent to identifying the corresponding most likely corrupted word.

Quantum codes behave differently. They may be *degenerate*, meaning that distinct physical errors can have the same action on the codespace, up to an irrelevant global phase [173]. More precisely, two errors  $E$  and  $E'$  are degenerate if, for every code state  $|\psi\rangle$ ,

$$E|\psi\rangle = e^{i\theta} E'|\psi\rangle \quad (9.14)$$

for some global phase  $\theta$ . In this case, the two errors are indistinguishable from the perspective of the encoded logical information [174].

For stabilizer codes, degeneracy arises from the stabilizer group. A stabilizer operator  $S$  acts trivially on every code state,

$$S|\psi\rangle = |\psi\rangle. \quad (9.15)$$

Consequently, if an error  $E$  occurs, then any error of the form

$$E' = ES \quad (9.16)$$

has the same action on the codespace, since

$$E'|\psi\rangle = ES|\psi\rangle = E|\psi\rangle. \quad (9.17)$$

Thus,  $E$  and  $ES$  represent the same physical effect on the encoded state. They also produce the same syndrome, because multiplication by a stabilizer does not change the commutation relations with the stabilizer generators.

Therefore, in a degenerate quantum code, decoding should not necessarily be viewed as selecting the single most likely physical error. Rather, one should identify the most likely equivalence class of errors, where errors differing by stabilizers are identified. Mathematically, these equivalence classes are cosets of the stabilizer group. For a fixed syndrome, the decoder must determine which stabilizer coset, or equivalently which induced logical operation, is most likely.

Formally, let  $\mathcal{P}_n$  be the  $n$ -qubit Pauli group,  $\mathcal{S} \subset \mathcal{P}_n$  the stabilizer group, and  $\mathcal{N}(\mathcal{S})$  its normalizer. The logical Pauli operators are given by the quotient group

$$\mathcal{L} \cong \mathcal{N}(\mathcal{S})/\mathcal{S}. \quad (9.18)$$

Any error  $E \in \mathcal{P}_n$  can be decomposed (up to an overall phase) as

$$E = T(s)SL, \quad (9.19)$$

where  $T(s) \in \mathcal{P}_n$  is a *pure error* (or *syndrome representative*) that produces the syndrome  $s$ ,  $S \in \mathcal{S}$  is a stabilizer, and  $L \in \mathcal{L}$  is a logical Pauli operator. The roles of these components are as follows. The operator  $T(s)$  determines the measured syndrome, while the stabilizer  $S$  does not affect either the syndrome or the logical state, that is,  $S|\psi\rangle = |\psi\rangle$ , while the logical operator  $L$  fully captures the effect of  $E$  on the encoded information.

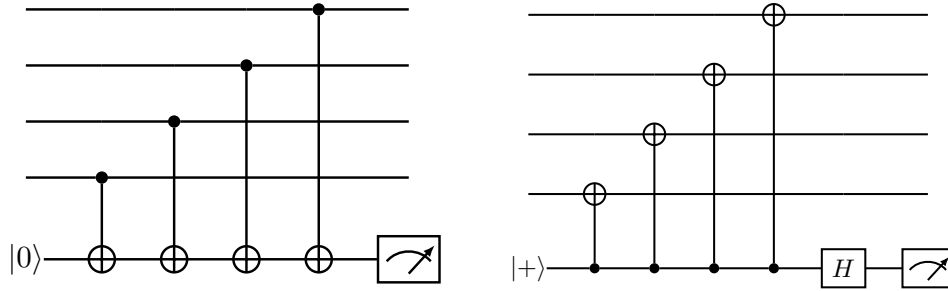
This decomposition shows that the Pauli group can be partitioned as

$$\mathcal{P}_n = \bigsqcup_s T(s)\mathcal{S}\mathcal{L}, \quad (9.20)$$

that is, errors are grouped first by syndrome  $s$ , and within each syndrome class further by their logical action. From this perspective, decoding consists of the following steps

1. Using the syndrome to identify the coset  $T(s)\mathcal{S}$ ,
2. Inferring the most likely logical operator  $L$  within that coset.

A physical error  $E$ , after projection back onto the codespace via  $T(s)$ , induces one of these logical operations  $L$  on the encoded state. Errors that share the same syndrome and induce the same logical operator are said to belong to the same *logical class*. Intuitively, the syndrome determines the stabilizer part of the error, while the logical class determines how the logical information is affected.



**Figure 24:** Measurement protocol for  $Z$ -type (left) and  $X$ -type (right) stabilizers, here shown for weight 4 stabilizers.

The decoder’s goal is therefore not necessarily to identify the exact physical error, but rather to determine the most probable logical class  $[L]$  consistent with the observed syndrome  $s$ . Thus, it is sufficient for the decoder to infer the most likely logical class of the error and then apply the corresponding logical correction operator  $C$ . If after correction a nontrivial logical operator remains, then a *logical error* has occurred: the syndrome has been corrected, but the encoded information has been altered.

The optimal decoding strategy is therefore *degenerate maximum-likelihood decoding* (DMLD). Instead of identifying the single most probable error, DMLD sums the probabilities of all errors belonging to the same logical class and chooses the logical class with the highest total probability given the syndrome. Unfortunately, computing this quantity is usually extremely difficult, making optimal decoding a computationally challenging problem [174].

A common relaxation is MLE, which we introduced above, which simply searches for the single most likely physical error while ignoring degeneracy.<sup>12</sup> Because it neglects the probability contributions of other degenerate errors, MLE may in general return a suboptimal correction. However, its computational complexity is lower: MLE is only an NP-hard problem. While this implies that MLE takes in the worst case exponential time to solve, it is still relatively easier to compute than the QMLD which is a more difficult #P-complete problem [174, 175].<sup>13</sup> Further, MLE is amenable to straightforward relaxations, which has led to many efficient heuristic decoders that can solve the decoding problem quite well in practice.

## 9.4 How to measure syndromes

Let us turn now towards the more practical aspects of decoding. For example, given a QEC code, how do we measure syndromes? For stabilizer codes, this corresponds to measuring the Pauli operators of the stabilizers. In particular, for CSS codes, whose stabilizer checks are either purely  $Z$ -type or purely  $X$ -type, one can measure syndromes using the circuit shown in Figure 24.

Here, one initializes an ancillary qubit in the  $|0\rangle$  state for  $Z$ -type stabilizers, while  $|+\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$  for  $X$ -type stabilizers. Then, CNOT gates are implemented for each data qubit that the stabilizer has support on. For example, for a code with weight 4-stabilizers such as the surface

<sup>12</sup>MLE is also called quantum maximum likelihood decoding, following the notation of Ref. [174]. Note that both DMLD and MLE are maximum-likelihood problems, just over different probability distributions.

<sup>13</sup>To be more formal, NP is the class of decision problems for which a proposed solution can be checked efficiently, but finding the solution may be difficult. An NP-hard problem is at least as hard as every problem in NP, so solving such problems is expected to take exponential time in the worst case. A #P-complete problem asks not just whether solutions exist, but also how many, which is substantially more difficult.

or Steane code, this measurement involves 4 data qubits. The syndrome outcome is recorded by measuring the auxiliary qubit, where for an  $X$ -type stabilizer we have to measure in the eigenbasis of  $|+\rangle$ , which requires an additional Hadamard gate  $H$ .

## 9.5 Noise models for QEC simulation

To model the decoding process, we need an accurate description of the errors that can occur and their corresponding probabilities. On the one hand, this is essential when decoding actual experiments, where our noise model should reflect the experimental reality. On the other hand, this is also essential for classical simulation of the codes and to understand their error correction capabilities. This raises the question of how to model errors in the first place. In fact, the choice of noise model represents a trade-off between physical realism and computational tractability. This has led to a hierarchy of standard simulation models in the literature:

1. **Code-capacity model:** This is the most idealized model. It assumes that the syndrome extraction process (including all ancilla operations and measurements) is perfectly error-free. Noise, typically i.i.d. depolarizing noise, is applied only to the data qubits once per QEC cycle. This model is not realistic but is invaluable for theoretical studies, as it allows for the calculation of a code’s maximum possible error tolerance (its “code capacity”) and provides an upper bound on its performance.
2. **Phenomenological model** [26, 83]: This model takes a step towards realism by acknowledging that syndrome measurements are also faulty. It introduces a probability of measurement error, which effectively flips the classical syndrome bits after they are extracted. This also encapsulates a simplified simulation of errors associated with qubit initialization (state preparation) and reset. However, it still abstracts away the details of the gate-level errors that occur during the syndrome extraction circuit.
3. **Circuit-level model:** This is the most comprehensive and realistic noise model by explicitly simulating the QEC circuit. It accounts for all potential sources of error within the quantum circuit used for a QEC cycle. A noise channel is applied after every physical operation, such as single-qubit gates, two-qubit gates, qubit idling (decoherence), state preparation, and measurement readout.

Luckily, these noise models can be simulated efficiently for one of the most important cases: stabilizer codes under Clifford noise models, which includes Pauli noise models. Here, all operations are Clifford and can be efficiently simulated using a tableau formalism in  $O(n^3)$  time [121]. For more general noise models, such as coherent noise, simulation methods for specific settings have also been found [176].

## 9.6 Detector model

So far, we described decoding in terms of syndrome vectors and parity-check matrices. That language is natural for introducing decoding algebraically, especially for code-capacity and phenomenological models. However, for repeated rounds of syndrome extraction and for circuit-level noise, it is often more convenient to work with the *detector model*. In this representation one tracks changes in measurement outcomes over time, which leads to a particularly clean graphical picture used by many modern circuit-level decoders.

In practical QEC, especially for large codes, it is often inconvenient to work directly with Pauli operators and stabilizer generators. Instead, one can reformulate the problem in terms of *detectors*

and *detection events*, which leads to a very clean graphical decoding picture [177, 178]. The key idea is to track *changes* in stabilizer measurement outcomes rather than the outcomes themselves.

Consider repeated rounds of stabilizer measurements. Each stabilizer measurement produces a binary outcome (e.g.,  $\pm 1$ ). A *detector* is defined as a function of one or more measurement outcomes such that, in the absence of errors, it always evaluates to  $+1$  (or equivalently, 0 in binary form). A *detection event* occurs when a detector flips from its expected value. Equivalently, a detector outputs  $-1$ . For example, in a circuit with repeated measurements, a common detector is the *parity* of a stabilizer measurement between two consecutive rounds:

$$d_i(t) = m_i(t) \cdot m_i(t-1), \quad (9.21)$$

where  $m_i(t)$  is the measurement outcome of stabilizer  $i$  at time  $t$ . If no error occurs, the measurement is stable and  $d_i(t) = +1$ . A flip indicates that an error has occurred.

Each physical error (e.g., a Pauli error on a qubit or a measurement error) produces a characteristic pattern of detection events. We can represent this as a mapping

$$E \mapsto \mathbf{d}(E) \in \mathbb{F}_2^N, \quad (9.22)$$

where  $\mathbf{d}(E)$  is a binary vector indicating which detectors click.

A crucial property is that most physical errors trigger a *small number* of detectors. As such, the pattern of detection events depends only on the error up to stabilizers. Thus, the detector model naturally incorporates degeneracy as different physical errors may produce the same detection pattern. This framework is widely used because it separates the *physics* (error processes) from the *inference problem* (decoding), enabling efficient and scalable decoders.

## 9.7 Decoder landscape

In this section, we give a brief overview of the landscape of decoders for QEC. No single decoder is uniformly best: the preferred choice depends on the code family, the noise model, and the available latency budget. Broadly speaking, different decoder families are favored in different regimes. For topological codes such as the surface code, minimum-weight perfect matching (MWPM) and Union-Find are among the most important practical baselines, with MWPM emphasizing accuracy and Union-Find emphasizing speed. For qLDPC codes, belief-propagation-based decoders and their post-processed variants play a particularly central role, while tensor-network decoders are among the strongest high-accuracy approaches for structured local codes. Finally, for more general detector-based circuit-level decoding beyond graph-like settings, newer search-based approaches such as decision-tree and Tesseract decoders provide a flexible alternative. For quick reference, we summarize the main trade-offs between the most common decoder families discussed in this chapter in Table 3.

## 9.8 Mixed-integer programming decoders and relaxations

In general, the MLE problem in Eq. (9.10) can be formulated as a mixed-integer program (MIP), or for specific choices of error probabilities  $p_i$  as an integer program [96]. In general, these constitute NP-hard problems, so such MIP decoders likely require exponential time to find the optimal solution.

However, if one is satisfied with potentially non-optimal solutions, one can relax the MLE problem so that decoding can be achieved in polynomial time. An example of such a relaxation is given by linear program (LP) decoders [179, 180]. LP decoders relax the original binary constraint  $e_i \in \{0, 1\}$  into a continuous optimization problem with  $e'_i \in [0, 1]$ . Then, the LP can be solved in



| Decoder                                                       | Best for                                                | Speed          | Main advantage                                         | Main drawback                                  |
|---------------------------------------------------------------|---------------------------------------------------------|----------------|--------------------------------------------------------|------------------------------------------------|
| Lookup table<br>( <a href="#">Example 9.3</a> )               | small toy codes;<br>pedagogy                            | very fast      | easiest to teach and<br>implement                      | does not scale                                 |
| MIP / exact MLE<br>( <a href="#">Section 9.8</a> )            | small generic stabilizer<br>codes; benchmarking         | slow           | clean near-optimal<br>baseline                         | exponential cost in<br>practice                |
| LP / SOS relaxations<br>( <a href="#">Section 9.8</a> )       | approximate<br>optimization-based<br>decoding           | moderate–slow  | systematic relaxations<br>of MLE                       | can yield weak or costly<br>relaxations        |
| MWPM ( <a href="#">Section 9.9</a> )                          | surface code; graph-like<br>detector models             | moderate–fast  | strong practical<br>baseline for topological<br>codes  | not naturally suited to<br>arbitrary codes     |
| Union-Find<br>( <a href="#">Section 9.10</a> )                | low-latency<br>surface-code decoding                    | very fast      | near-linear and<br>hardware-friendly                   | usually less accurate<br>than MWPM             |
| Belief propagation<br>(BP) ( <a href="#">Section 9.11</a> )   | qLDPC and<br>sparse-check codes                         | fast           | scales well; strong<br>hybrid building block           | short loops and<br>degeneracy hurt plain<br>BP |
| Tensor networks<br>( <a href="#">Section 9.12</a> )           | high-accuracy decoding<br>for structured local<br>codes | slow           | excellent accuracy;<br>captures degeneracy             | computationally<br>expensive                   |
| Machine learning<br>( <a href="#">Section 9.13</a> )          | correlated noise; soft<br>information                   | fast inference | flexible once trained                                  | training is costly                             |
| Decision-tree /<br>Tesseract ( <a href="#">Section 9.14</a> ) | general detector-model<br>decoding                      | moderate–slow  | very flexible, including<br>circuit-level settings     | runtime can grow<br>quickly                    |
| Max-SAT / QUBO<br>( <a href="#">Section 9.15</a> )            | optimization-based<br>exact or near-exact<br>decoding   | slow           | leverages mature<br>optimizers and special<br>hardware | usually too slow for<br>real-time use          |
| Renormalization group<br>( <a href="#">Section 9.16</a> )     | fast multiscale<br>topological decoding                 | fast           | parallelizable and<br>low-latency                      | may neglect global<br>error correlations       |

Table 3: Comparison of commonly used quantum decoders. The speed labels are qualitative and meant only as a rough guide.

polynomial time using standard solvers. After optimization, the real-valued  $e'_i$  is rounded to the nearest binary number to give a valid error configuration. However, LP decoders often struggle with quantum codes, yielding nonphysical fractional solutions that result in logical errors. This issue can be improved by adding post-processing steps [181].

A recent work proposed a systematic relaxation of MLE using sum-of-squares (SOS) or Lasserre hierarchies [182]. Here, one approximates the constraint that  $e_i$  is binary by using polynomials of sum of squares. The resulting relaxed problem can be solved as a semidefinite program (SDP). The maximal allowed degree  $\ell$  of the sum of squares polynomials allows one to systematically trade-off between accuracy and runtime. While for small  $\ell$ , we see an improvement in runtime, this also results in lower accuracy, whereas a higher value for  $\ell$  yields better decoding solutions but requires more time. Indeed, one finds that with increasing  $\ell$ , the problem converges to the optimal solution. In fact, there is a certificate of optimality via the rank-loop condition: above a certain  $\ell$ , the SOS decoding problem is guaranteed to be optimal. Numerically, one finds that the lowest level  $\ell = 1$  has limited accuracy, while even  $\ell \geq 2$  shows good performance close to the optimal MLE.

## 9.9 Minimum weight perfect matching

For topological codes, the most important accuracy-oriented baseline is minimum weight perfect matching (MWPM). The MWPM algorithm is the canonical and most widely studied decoder for the surface code. It maps the decoding problem onto a graph problem, which can then be solved



using the blossom algorithm in polynomial time [183]. In the worst case, time  $O(n^3)$  is required, though in practice much better decoding times have been observed. However, MWPM in its standard formulation requires that the decoding problem has a graph structure, that is, each error can flip at most two syndromes [184]. This condition is satisfied for surface codes, while for more general codes one has to consider relaxations of the decoding problem. The core reason that MWPM works so well for the surface code is that a single local data-qubit error typically creates either two nearby detection events or one event together with a boundary. Decoding can therefore be reformulated as the problem of pairing up defects in the most likely way. The decoding process of vanilla MWPM proceeds as follows:

1. **Graph construction:** A graph is constructed where the vertices represent the locations of non-trivial syndrome bits (often called “defects” or “anyons”). An additional set of “boundary” vertices is included to handle error chains that terminate at the edges of the code.
2. **Edge weighting:** An edge is drawn between every pair of vertices. The weight of an edge is calculated as the negative logarithm of the probability of the most likely physical error chain that could create that pair of defects. For simple i.i.d. noise, this weight is often approximated by the Manhattan distance between the vertices on the qubit lattice.
3. **Matching:** The algorithm then finds a *perfect matching* on this graph (a set of edges that pairs up all vertices with no vertex left over) such that the sum of the weights of the edges in the matching is minimized. This is a standard problem in computer science that can be solved efficiently with algorithms like Edmonds’ blossom algorithm [183].
4. **Correction:** The minimum-weight matching corresponds to the most probable set of error chains that explains the observed syndrome. A correction operator is then constructed by applying Pauli operators along the paths indicated by the matching.

MWPM is highly accurate for surface codes under independent, depolarizing noise, achieving thresholds near the optimal limit. As a simple example, if two nearby detection events appear in the lattice, MWPM will tend to connect them by the shortest and therefore most probable error chain; if a defect lies close to a boundary, it may instead be cheaper to match it to that boundary. In this way, the matching problem directly encodes the competition between different plausible recovery paths.

Note that there have been many extensions of MWPM to address its restriction to graph-like decoding problems. For example, the color code, an important code class, does not naïvely support MWPM. However, modifications of MWPM exist that allow extensions to color codes [185, 186]. Further, recent work proposed decoding via the Minimum-Weight Parity Factor (MWPF) problem, which generalizes MWPM to hypergraphs [187]. MWPF decoding works on general decoding problems, thus making it far more versatile for generic codes. We note that this is an ongoing and popular research topic, with many other extensions existing.

## 9.10 Union-Find

If one is willing to trade some accuracy for substantially lower latency, a natural alternative is the Union-Find (UF) decoder. The UF decoder is a faster, nearly linear-time alternative to MWPM, making it a compelling candidate for real-time decoding on large-scale systems [188]. The UF algorithm operates by growing clusters of defects:

1. Each defect initially forms its own cluster.
2. The algorithm iterates through all possible single-qubit errors in increasing order of size.

3. For each potential error, it checks if it connects two existing clusters or a cluster to a boundary. If so, the clusters are merged (the “union” operation).
4. This process continues until only clusters with even numbers of defects remain, which are then corrected internally.

The UF decoder’s primary advantage is its speed, with a complexity close to  $O(n\alpha(n))$ , where  $\alpha(n)$  is the extremely slow-growing inverse Ackermann function. This makes it significantly faster than MWPM for large codes. This speed comes at the cost of slightly lower accuracy; the threshold achieved by UF decoders is typically a bit lower than that of MWPM under standard noise models. However, its hardware-friendly structure has inspired highly optimized implementations like Riverlane’s Collision Clustering decoder [189], designed for deployment on FPGAs.

### 9.11 Belief propagation

For sparse qLDPC codes, the graphical structure of the decoding problem can be quite different from that of topological matching-based decoders. In this regime, belief propagation (BP) has been established as a central algorithmic idea. BP is an iterative message-passing algorithm [190, 191]. It is widely used in wireless communication [192], and has also found widespread application in QEC decoding [193]. To decode, BP operates on the code’s Tanner graph. This is a bipartite graph consisting of variable nodes that represent errors, and check nodes that represent stabilizer constraints. An edge connects a variable node to a check node if the corresponding qubit participates in that stabilizer.

Given a measured syndrome  $s$ , the goal of BP is to estimate for each qubit  $i$  the *marginal probability*

$$P(e_i = 1 \mid s), \quad (9.23)$$

that is, the probability that an error occurred on that qubit conditioned on the observed syndrome. Instead of solving this global inference problem directly (which is computationally hard), BP approximates it by passing *local messages* along the edges of the Tanner graph. Each edge in the graph carries two types of messages:

- $m_{i \rightarrow a}$ : from variable node  $i$  to check node  $a$ ,
- $m_{a \rightarrow i}$ : from check node  $a$  to variable node  $i$ .

Intuitively, a variable node tells a check, “I believe I am in error with probability  $p$ ,” while a check node tells a variable, “Given the syndrome constraint and what other qubits are doing, you should (or should not) have an error.” Messages are initialized using the physical error model. For example, for independent bit-flip noise with error rate  $p$ , one initializes

$$m_{i \rightarrow a}^{(0)} = \log \frac{1-p}{p}, \quad (9.24)$$

often expressed in terms of *log-likelihood ratios* (LLRs) for numerical stability.

Each check node enforces a parity constraint, that is, the sum modulo 2 of the neighboring error variables must match the syndrome. The message from check node  $a$  to variable node  $i$  is computed from all incoming messages except  $i$ :

$$m_{a \rightarrow i} = 2 \tanh^{-1} \left( \prod_{j \in N(a) \setminus i} \tanh \left( \frac{m_{j \rightarrow a}}{2} \right) \right), \quad (9.25)$$

where  $N(a)$  denotes the neighbors of check  $a$ .

Each variable node combines its prior belief (from the noise model), and incoming messages from all neighboring checks except  $a$ . The update rule is:

$$m_{i \rightarrow a} = \ell_i + \sum_{b \in N(i) \setminus a} m_{b \rightarrow i}, \quad (9.26)$$

where  $\ell_i$  is the initial log-likelihood ratio. These updates are repeated iteratively. After a fixed number of iterations (or upon convergence), one computes the *belief* at each variable node as

$$\Lambda_i = \ell_i + \sum_{a \in N(i)} m_{a \rightarrow i}. \quad (9.27)$$

At the end, a hard decision is then made to decode the error, for example, via

$$e_i = \begin{cases} 1 & \text{if } \Lambda_i < 0, \\ 0 & \text{otherwise.} \end{cases} \quad (9.28)$$

BP works very well on tree graphs (graphs without loops), where it computes the true marginal probabilities. These updates are very efficient, with each iteration requiring only local message updates along edges of the Tanner graph, leading to a computational complexity that scales linearly with the number of qubits per iteration. However, Tanner graphs of quantum codes typically contain many *short loops* as stabilizers often overlap heavily. These loops cause correlations that BP does not properly account for, leading to overconfident or inconsistent beliefs, or convergence to incorrect solutions. Further, degeneracy means that multiple errors correspond to the same syndrome. As a result, standard BP often performs poorly for qLDPC codes. However, BP can be integrated within other methods, making it one of the most widely used building blocks for modern quantum decoders.

As such, several methods have been proposed to improve BP decoding in the quantum setting, of which we mention a few examples here. For example, using automorphisms to randomly permute the Tanner graph can help mitigate trapping in loops [194]. Further, one can run BP multiple times with different initializations [195] or iterate over the least reliable bits [196]. Also, one can enhance BP with classical decoding techniques such as ordered statistics decoding (OSD) [197] or localized statistics decoding (LSD) [198]. These approaches significantly improve performance. This has led BP-based decoders being one of the most popular choices for qLDPC codes, driving their recent popularity and applications [178].

## 9.12 Tensor network-based decoders

Tensor network (TN) decoders are among the most powerful known decoding methods, achieving very high accuracy by directly approximating DMLD [199–201]. Their strength comes from reformulating decoding as a problem in statistical physics and evaluating it using tensor network techniques.

Recall that DMLD requires computing, for each logical class  $L$ , the total probability

$$P(L \mid s) = \sum_{E \in \mathcal{E}(s, L)} P(E), \quad (9.29)$$

that is, summing the error probabilities  $P(E)$  over all physical errors  $E \in \mathcal{E}(s, L)$  consistent with the syndrome  $s$  and logical operator  $L$ . This sum can be interpreted as a *partition function*:

$$Z_L = \sum_{E \in \mathcal{E}(s, L)} e^{-\beta H(E)}, \quad (9.30)$$

where  $H(E)$  is an effective energy (typically proportional to the weight of  $E$ ) and  $\beta$  is an inverse temperature determined by the noise model. Thus, decoding becomes equivalent to evaluating a classical statistical mechanics model, that is, *computing the partition function over all error configurations consistent with the syndrome*.

The key insight is that this partition function can be written as a *tensor network*. Each component of the code is represented as a tensor: variable tensors encode the probability of error on each qubit, constraint tensors enforce stabilizer (parity) constraints, and boundary tensors enforce the observed syndrome and logical class. The full probability  $Z_L$  is obtained by *contracting* the network.

The decoding algorithm proceeds as follows. First, we construct the tensor network corresponding to the code, noise model, and observed syndrome. Then, for each logical class  $L$ , we modify boundary conditions to enforce the corresponding logical operator, and contract the tensor network to compute the partition function  $Z_L$ . Finally, we select the most likely logical class

$$L^* = \arg \max_L Z_L. \quad (9.31)$$

Crucially, TN decoders *sum over all degenerate errors*, rather than selecting a single configuration. This allows them to approximate true DMLD much more accurately than methods such as MWPM or BP. Exact contraction of a tensor network generally scales exponentially with the tree width of the graph. For 2D codes such as the surface code, this leads to a complexity scaling roughly as

$$O(\exp(c\sqrt{n})), \quad (9.32)$$

which is significantly better than brute-force enumeration but still super-polynomial.

In practice, approximate contraction methods are used, such as matrix product state (MPS) contraction, where one contracts the network row by row with controlled bond dimension. A powerful hybrid approach combines BP with tensor networks [202]. Here, BP is used to approximate marginal distributions, and these marginals guide or simplify the tensor contraction. This significantly reduces computational cost while maintaining high accuracy.

### 9.13 Machine learning-based decoders

Unlike the previous model-based decoders, machine learning (ML) decoders aim to learn the map from syndrome to correction directly from data. ML decoders therefore represent a paradigm shift from the model-based approaches described above. Instead of relying on explicit, predefined noise models, ML decoders learn the mapping from syndrome to correction by training on data drawn from simulations or actual experiments [203].

To implement ML decoders, neural networks have been a popular choice, and they can also handle highly correlated noise, even beyond circuit-level noise descriptions [204, 205]. Recent studies have trained ML decoders on simulated data, which can often be generated efficiently. Further, the decoder's performance can be refined using experimental data [206]. Because ML models learn directly from data, they appear to perform well on correlated errors and can also incorporate soft information such as analogue read-out signals [205]. A potential downside of ML decoders is the need for large quantities of training data and the corresponding computational time needed to train them, which may scale unfavorably with code distance.

### 9.14 Tesseract and decision-tree decoders

For more general detector-model decoding beyond graph-like settings, recent works have proposed the decision-tree decoder [207] and the closely related Tesseract decoder for QEC codes [208]. Both

decoders frame decoding as a graph-search problem in a high-dimensional space and are able to decode circuit-level noise models.

These decoders take as input the detector error model and the resulting syndrome bits. They then construct a search tree whose nodes correspond to partial error configurations. The decoders use A\*-style informed search with a carefully designed heuristic to explore only the most promising branches [209]. The goal is to find the lowest-cost set of physical errors consistent with the observed syndrome. By using the established A\* search with an admissible heuristic, these decoders achieve high accuracy while pruning most of the search space [210]. The Tesseract and decision-tree decoders use different heuristics to improve decoding performance. Further, the decision-tree decoder can be combined with other methods such as BP.

Notably, both algorithms can be applied to any stabilizer code including code-capacity and circuit-level noise models, and are not tied to regular lattice structures or graph-like errors. Also, by tuning the search parameters, the decoders can be made equivalent to the optimal maximum likelihood decoding problem, at the cost of vastly increased runtime.

### 9.15 Combinatorial decoders

A different route is to translate decoding into a standard combinatorial optimization problem and then exploit mature solver technology. Recent works have proposed decoders that map the decoding problem onto combinatorial problems and then solve the resulting combinatorial instance.

In particular, the Max-SAT decoder maps MLE to the Max-SAT problem: Max-SAT is an optimization problem in which one aims to find an assignment of binary variables that satisfies the maximum number of clauses [211, 212]. For example, a Max-SAT instance with two Boolean variables  $e_1$  and  $e_2$  may consist of the three clauses

$$(e_1 \vee e_2) \wedge (e_1 \vee \neg e_2) \wedge (\neg e_1 \vee e_2), \quad (9.33)$$

where  $\neg$  is the negation of the literal,  $\vee$  indicates a logical OR, and  $\wedge$  indicates a logical AND. In the above example, the choice  $e_1 = e_2 = 1$  satisfies all three clauses and is therefore the solution of the problem. One can rewrite the syndrome constraints of the decoding problem into a set of such Max-SAT clauses [212].

In Ref. [211], this has been framed as the so-called lights-out problem, where one has to find a configuration of switches (corresponding to errors) that is able to turn off all lights (corresponding to syndromes). In fact, this lights-out problem can then be seen as a Max-SAT problem. The Max-SAT decoder demonstrates high accuracy, even for the color code, which is usually difficult to decode. Straightforward extensions to the phenomenological noise model and even to circuit-level noise exist. However, the Max-SAT decoder is relatively slow and is therefore more focused on high accuracy.

Recent works have also proposed decoding using the coherent Ising machine. In particular, one first maps MLE to a quadratic unconstrained binary optimization (QUBO) problem [213–215]. This represents MLE as an energy-optimization problem over binary variables. Notably, the syndrome constraints require the implementation of additional constraints in the QUBO energy problem. This can be done via energy penalty terms, which penalize violations of the syndrome constraints by assigning them a high energy in the QUBO. Then, the QUBO is solved using numerical implementations of the coherent Ising machine.

### 9.16 Renormalization group decoders

Returning to fast multiscale methods, renormalization-group (RG) decoders are a family of hierarchical decoding algorithms for topological quantum error-correcting codes, with their most notable

application being the surface code [216]. Their central idea is to solve the decoding problem coarsely at large scales first, and then refine the solution by recursively zooming in on smaller scales, mimicking the coarse-graining procedures of the renormalization group in statistical physics.

As a basic idea, the renormalization-group decoder partitions the code into small blocks [216]. Within each block, it computes either a local decoding decision or an effective logical error that represents the block’s behavior at a coarse scale. This block is then replaced by an effective “super-qubit,” and the syndrome information is passed to the next, coarser level. The procedure then repeats from finer to coarser scales, combining small regions into larger ones while summarizing their error behavior. At the top level, the entire lattice is reduced to a very small effective code, which can be decoded exactly with a brute-force decoder. Then, we need to propagate the correction decision to the original lattice. For this, we reverse the information flow, moving from the coarse to the fine scale, applying the correction operations. This creates a multiscale description of the error, efficiently capturing correlations.

Especially for topological codes, the renormalization-group decoder exploits two key structural properties. First, there is locality of error propagation: errors spread only to nearby stabilizers, so small regions can often be decoded nearly independently. Second, topological codes are usually self-similar, and the decoding problem retains the same structure at larger scales. Thus, instead of dealing with an exponentially large global configuration space in the full decoding problem, the renormalization-group decoder breaks the problem into manageable chunks. We also note that by including soft information from a belief-propagation decoder, the performance of renormalization-group decoding can be markedly enhanced [193]. In particular, performance beyond MWPM has been reported. Renormalization-group decoders offer near-linear decoding time and are directly parallelizable, which is essential in practical applications [216, 217]. As such, they are especially useful when one needs low-latency decoding, for example in real-time error correction for a quantum computer.

In this tutorial, we can only cover a small subset of the vast landscape of decoders. The large number of decoding approaches originates from the complex tradeoff between accuracy, speed and latency. Exact or near-exact formulations best capture the underlying maximum-likelihood problem but are usually too slow for real-time use. Matching-based, Union-Find, and renormalization-group decoders are especially attractive for topological codes and low-latency settings, while BP-based methods are central for qLDPC codes. Tensor-network and search-based decoders offer higher accuracy in structured settings, and machine-learning approaches may become particularly appealing when realistic soft information or strongly correlated noise is available. Overall, practical decoding is less about finding a universally best algorithm, and instead about adapting the decoder to the code, the noise, and the quantum hardware constraints.



## 10 Fundamental bounds in quantum error correction

The development of quantum error correction (QEC) is fundamentally a study of trade-offs. To protect fragile quantum information, one must encode it redundantly, using a larger number of physical systems (e.g., qubits) to represent a smaller amount of logical information. This process is characterized by a set of key parameters, most notably the number of physical qubits  $n$ , the number of encoded logical qubits  $k$ , and the code distance  $d$ . The ratio  $R = k/n$  is called the *encoding rate* and is a measure of efficiency, while the distance  $d$  quantifies the code's error-correcting capability. Before considering the practical constraints imposed by physical hardware, it is essential to understand the absolute theoretical limits that govern these parameters. The bounds discussed in this section apply to any quantum code, regardless of how it might be implemented, and thus represent the ideal performance ceiling.

### 10.1 The quantum Hamming bound (sphere-packing limit)

The *quantum Hamming bound*, also known as the sphere-packing bound, provides a powerful upper bound on the achievable encoding rate of a QEC code by relating the code's parameters to the geometry of the underlying Hilbert space [76, 78]. It is a direct quantum analogue of the classical Hamming bound, which constrains classical error-correcting codes through a similar volume-based argument [51]. The central idea behind the quantum Hamming bound is that for a code to be able to unambiguously correct a certain set of errors, the quantum states resulting from those errors must be distinguishable. Specifically, for a *non-degenerate* code, each correctable error operator, when applied to the code's subspace, must map it to a new subspace that is orthogonal to the subspaces generated by all other correctable errors. Here, non-degeneracy means that distinct correctable errors have distinct effects on the codespace, or equivalently produce distinct error syndromes.

Consider the total  $2^n$ -dimensional Hilbert space of  $n$  physical qubits. A QEC code defines a smaller  $2^k$ -dimensional subspace within the full  $2^n$ -dimensional Hilbert space, known as the *codespace*, where logical information is stored. An error-correcting code with distance  $d$  can correct any error affecting up to  $t = \lfloor (d-1)/2 \rfloor$  qubits. When one of these correctable errors occurs, the state is perturbed and moved out of the original codespace into a new, error-affected subspace. For the error to be correctable, this new subspace must be uniquely identifiable. The quantum Hamming bound formalizes the constraint that the total “volume” of all of these distinct, orthogonal error subspaces cannot exceed the total available volume of the Hilbert space.

For a non-degenerate  $[[n, k, d]]$  quantum code over qubits, which can correct any error of weight up to  $t = \lfloor (d-1)/2 \rfloor$ , the quantum Hamming bound is expressed as:

$$2^k \sum_{j=0}^t \binom{n}{j} 3^j \leq 2^n. \quad (10.1)$$

Here, the term  $\sum_{j=0}^t \binom{n}{j} 3^j$  represents the total number of distinct correctable Pauli errors of weight up to  $t$ . The binomial coefficient  $\binom{n}{j}$  counts the number of ways to choose  $j$  qubits out of  $n$  for an error to occur. The factor of  $3^j$  accounts for the three possible types of non-trivial Pauli errors ( $X$ ,  $Y$ , or  $Z$ ) that can occur at each of those  $j$  locations. The inequality states that the dimension of the codespace,  $2^k$ , multiplied by the number of distinct orthogonal subspaces it can be mapped to by correctable errors, must be less than or equal to the total dimension of the ambient Hilbert space,  $2^n$ .

The proof of the quantum Hamming bound for non-degenerate codes is a direct consequence of the sphere-packing argument. Let  $\mathcal{C}$  be the  $2^k$ -dimensional codespace and let  $\{E_a\}$  be the set of

correctable Pauli error operators with weight at most  $t$ . The error correction conditions for a non-degenerate code require that for any two distinct correctable errors  $E_a$  and  $E_b$ , the resulting error spaces  $E_a\mathcal{C}$  and  $E_b\mathcal{C}$  are mutually orthogonal. Each of these error spaces has the same dimension as the original codespace, namely  $2^k$ . The total number of such correctable errors (including the identity for the no-error case) is  $N_E = \sum_{j=0}^t \binom{n}{j} 3^j$ . Since all  $N_E$  of these  $2^k$ -dimensional subspaces must fit within the total  $2^n$ -dimensional Hilbert space as mutually orthogonal subspaces, the sum of their dimensions cannot exceed the total dimension. This directly yields the inequality  $N_E \cdot 2^k \leq 2^n$ , which is the quantum Hamming bound. If a QEC code saturates this bound, it is said to be a *perfect* quantum code. Such codes are exceptionally rare as they represent the most efficient possible packing of error spheres within the Hilbert space. We saw one example of this in [Section 4.3](#), which we highlight again in the following example:

**Example 10.1:** The  $[[5, 1, 3]]$  code

The  $[[5, 1, 3]]$  code is the canonical example of a perfect quantum code. It encodes  $k = 1$  logical qubit into  $n = 5$  physical qubits and has a distance of  $d = 3$ , meaning it can correct any single-qubit error ( $t = 1$ ). Plugging these parameters into the quantum Hamming bound in [Eq. \(10.1\)](#) gives

$$2^1 \left( \binom{5}{0} 3^0 + \binom{5}{1} 3^1 \right) = 2(1 + 5 \cdot 3) = 2(16) = 32. \quad (10.2)$$

Since the dimension of the ambient Hilbert space is  $2^5 = 32$ , the bound is saturated ( $32 \leq 32$ ), confirming that the  $[[5, 1, 3]]$  code is perfect.

On the other hand, the Steane code is *not* a perfect code:

**Example 10.2:** The  $[[7, 1, 3]]$  Steane code

The  $[[7, 1, 3]]$  code, or Steane code, is another well-known code with  $n = 7$ ,  $k = 1$ , and  $d = 3$  ( $t = 1$ ). For this code, the left-hand side of the inequality is

$$2^1 \left( \binom{7}{0} 3^0 + \binom{7}{1} 3^1 \right) = 2(1 + 7 \cdot 3) = 2(22) = 44. \quad (10.3)$$

The total dimension of the Hilbert space is  $2^7 = 128$ . Since  $44 < 128$ , the Steane code does not saturate the bound and is therefore not a perfect code. This illustrates that there is “wasted space” in the Hilbert space that is not utilized for correcting single-qubit errors.

The standard proof of the quantum Hamming bound relies critically on the assumption of non-degeneracy, where each correctable error corresponds to a unique error syndrome and maps the codespace to a distinct orthogonal subspace. A degenerate code is one where multiple distinct physical errors can produce the same error syndrome. This opens up the possibility of a more efficient packing of information, as different error subspaces are no longer required to be orthogonal. This leads to one of the most significant and long-standing open questions in quantum coding theory:

**Open question 10.1**

Can a degenerate quantum code violate the quantum Hamming bound?



For decades, it was conjectured that all quantum codes must obey this bound. A significant body of work has shown this to be true for many large and important classes of codes. For instance, it has been proven that single and double error-correcting binary stabilizer codes, as well as many types of CSS codes, cannot beat the bound. For example, CSS codes over a prime power alphabet  $q \geq 5$  cannot beat the quantum Hamming bound [218]. However, a fully general proof that covers all possible degenerate codes remains elusive. The discovery of a code that provably violates the bound would be a major theoretical breakthrough, suggesting that our understanding of the ultimate limits of information packing in quantum systems is incomplete. While some numerical studies have suggested the possibility of small violations, no definitive counterexample has been found.

## 10.2 The quantum Singleton bound (information-theoretic limit)

The *quantum Singleton bound* establishes a different kind of limit on the parameters of a quantum error-correcting code [24, 78, 219]. Instead of relying on a geometric sphere-packing argument, its origins are deeply information-theoretic, stemming from fundamental principles of quantum mechanics such as the no-cloning theorem. The bound provides a direct trade-off between the amount of redundancy used ( $n - k$ ) and the error-correcting capability ( $d$ ). The core idea is that if a code can correct the erasure of any  $d - 1$  qubits, it means the full logical information must be recoverable from the remaining  $n - (d - 1)$  qubits.

The quantum Singleton bound arises from considering the implications of this recoverability on the distribution of quantum information across the physical qubits. For any  $[[n, k, d]]$  qubit error-correcting code, the quantum Singleton bound is given by the inequality:

$$n - k \geq 2(d - 1). \quad (10.4)$$

This is stricter than its classical counterpart, which is given by

$$n_{\text{cl}} - k_{\text{cl}} \geq d_{\text{cl}} - 1. \quad (10.5)$$

The factor of 2 in the quantum version reflects the need to protect against both bit-flip ( $X$ ) and phase-flip ( $Z$ ) errors.

A valuable insight into the proof of the quantum Singleton bound comes from a no-cloning argument. Consider a code with distance  $d$ . This implies that if any set of  $d - 1$  qubits are lost (an erasure error), their state can be perfectly reconstructed by performing a recovery operation on the remaining  $n - (d - 1)$  qubits. We partition the  $n$  physical qubits into three sets:  $A$ ,  $B$ , and  $C$ , where  $|A| = d - 1$  and  $|B| = d - 1$ . The remaining qubits are in set  $C$ , so  $|C| = n - 2(d - 1)$ . Because the code has distance  $d$ , the logical information of the  $k$  encoded qubits must be fully recoverable from the qubits in sets  $B$  and  $C$  alone (since the  $d - 1$  qubits in  $A$  could have been erased). Similarly, the logical information must also be fully recoverable from the qubits in sets  $A$  and  $C$  alone (since the  $d - 1$  qubits in  $B$  could have been erased).

If  $k > |C|$ , or equivalently  $k > n - 2(d - 1)$ , the subsystem  $C$  alone is too small to contain the full  $k$ -qubit logical state. Thus, the recovery from  $B \cup C$  must use information contained in  $B$ , while the recovery from  $A \cup C$  must use information contained in  $A$ . This would imply that the  $k$  qubits of logical information are simultaneously present in the system  $B \cup C$  and the system  $A \cup C$ . Since  $A$  and  $B$  are disjoint, this would mean that the information has been effectively cloned onto two different physical locations, which is forbidden by the no-cloning theorem. Therefore, to avoid this contradiction, we must have  $k \leq n - 2(d - 1)$ , which we can rearrange to give the quantum Singleton bound in Eq. (10.4).

A *quantum maximal distance separable (MDS) code* is a quantum error-correcting code that saturates the quantum Singleton bound, that is, one that satisfies the relation  $n - k = 2(d - 1)$ .

These codes are optimal in the sense that they achieve the maximum possible distance for a given number of physical and logical qubits.

**Example 10.3:** The  $[[5, 1, 3]]$  and  $[[4, 2, 2]]$  codes

The  $[[5, 1, 3]]$  code is an MDS code, since  $5 - 1 = 4 = 2(3 - 1)$ . The  $[[4, 2, 2]]$  code is also MDS, as  $4 - 2 = 2 = 2(2 - 1)$ .

The search for and classification of quantum MDS codes is an active area of research in quantum error correction.

### 10.3 The quantum Gilbert-Varshamov bound (existence guarantee)

While the quantum Hamming and Singleton bounds define the upper limits of what is possible in quantum error correction, they do not tell us whether codes approaching these limits actually exist. The quantum Gilbert-Varshamov (GV) bound addresses this by providing a lower bound on the performance of the best possible codes, thereby guaranteeing the existence of “good” quantum codes [219, 220]. The GV bound is fundamentally a non-constructive existence proof. The core idea is to show, through a counting argument, that the space of all possible quantum codes is too large to be composed entirely of “bad” codes (i.e., codes with a distance less than some target value  $d$ ). By demonstrating that the number of potential codes exceeds the number of ways a code can fail to have distance  $d$ , one proves that at least one code with the desired distance must exist.

The quantum GV bound for the existence of an  $[[n, k, d]]$  stabilizer code is often stated as an  $[[n, k, d]]$  stabilizer code existing if the inequality

$$\sum_{j=0}^{d-1} \binom{n}{j} 3^j \leq 2^{n-k} \quad (10.6)$$

is satisfied. The proof typically proceeds by a probabilistic method or a direct counting argument within the space of stabilizer codes. One can consider the set of all possible stabilizer groups that define  $[[n, k]]$  codes. Then, one calculates an upper bound on the number of these codes that have a “bad” logical operator, that is, a non-trivial logical operator with weight less than  $d$ . If this number is smaller than the total number of  $[[n, k]]$  codes, it follows that at least one code must exist that has no such low-weight logical operators, and therefore has a distance of at least  $d$ . This greedy selection of stabilizers is a common way to approach the proof.

This result is profound because it proves the existence of *asymptotically good quantum codes*. This is via the GV bound in the asymptotic limit, where the number of physical qubits  $n$  becomes very large. In this regime, the bound guarantees the existence of code families for which both the rate  $R = k/n$  and the relative distance  $\delta = d/n$  are non-zero constants. The asymptotic form of the quantum GV bound is often expressed as:

$$R \geq 1 - H(\delta) - \delta \log_2 3, \quad (10.7)$$

where  $H(p) = -p \log_2 p - (1 - p) \log_2 (1 - p)$  is the binary entropy function. The asymptotically good quantum codes are the theoretical ideal for scalable, fault-tolerant quantum computation, as they offer protection against errors with a constant resource overhead. The GV bound assures us that such powerful codes are not mathematically impossible.

The Hamming and Singleton bounds act as a “ceiling,” establishing the absolute best-case performance that any QEC code can hope to achieve. They delineate the impossible, setting firm limits

on the trade-off between information density (rate) and robustness (distance). In contrast, the GV bound acts as a “floor.” It does not describe the best possible code but rather guarantees a minimum level of performance that is achievable. It assures us that codes with a certain combination of rate and distance must exist. The space between the GV “floor” and the Hamming/Singleton “ceiling” represents the vast, largely unexplored territory of possible code parameters where optimal codes may lie, and it is where much of the effort of quantum error correction research is directed.

### Open question 10.2

Note that the GV bound’s proof is non-constructive; it is a powerful statement of existence that offers no explicit recipe for building the codes it promises. This gives rise to one of the foremost challenges in quantum coding theory: the search for explicit and efficient constructions of code families that can achieve the performance promised by the GV bound.

The ultimate goal is to find constructive families of asymptotically good quantum codes. This quest has been a primary motivator for decades of research and leads directly to the development of sophisticated code families, such as the qLDPC codes that we will discuss in [Section 11](#), which represent the most promising attempts to turn the GV bound’s abstract promise into a physical reality.

## 10.4 The constraint of geometric locality

The bounds discussed so far describe a theoretical ideal, a world where any qubit can interact with any other qubit to perform the complex multi-body measurements required for error correction. However, the physical reality of most leading quantum computing platforms is one of severe constraints on connectivity. Interactions are often limited to nearest-neighbor or next-nearest-neighbor qubits on a physical lattice. This constraint of geometric locality fundamentally alters the rules of the game, imposing harsh new trade-offs on code parameters and giving rise to a distinct class of bounds that have profoundly shaped the direction of experimental quantum error correction.

### Definition 10.1: Geometrically local code

A quantum code is said to be *geometrically local* in  $D$  spatial dimensions if its constituent physical qubits can be arranged on a  $D$ -dimensional lattice (e.g., a two-dimensional square grid) such that all checks (the operators measured to detect errors) act only on qubits that are physically close to one another. More formally, the qubits involved in any single check measurement must be contained within a ball of a fixed radius, where this radius is a constant that does not scale with the total number of qubits,  $n$ .

This is a very strong constraint that directly reflects the physical limitations of architectures like superconducting circuits or quantum dots, where fabricating long-range connections is technologically challenging.

### 10.4.1 The Bravyi-Poulin-Terhal bound

The Bravyi-Poulin-Terhal (BPT) bound is a landmark result that quantifies the severe limitations imposed by geometric locality in two dimensions [221]. It establishes a trade-off between the number of encoded logical qubits ( $k$ ), the code distance ( $d$ ), and the total number of physical qubits ( $n$ ) for any 2D geometrically local code. For any quantum code defined by geometrically local commuting

projectors on a two-dimensional lattice, the BPT bound states:

$$kd^2 \leq cn, \quad (10.8)$$

where  $c$  is a constant that depends on the specifics of the lattice and the interaction range but is independent of  $n$ ,  $k$ , or  $d$ . The implications of the BPT bound are profound:

1. If one desires a code with a finite rate ( $R = k/n$  is a constant, so  $k = \Theta(n)$ ), then the bound implies  $d^2 = O(1)$ , meaning the distance must be a constant. Such a code cannot offer scalable protection against errors as the system size grows.
2. Conversely, if one desires a scalable distance (e.g.,  $d$  grows as a power of  $n$ , like  $d = \Theta(\sqrt{n})$ ), then the bound implies  $kn = O(n)$ , which means  $k$  must be a constant,  $k = O(1)$ . Such a code has a rate that vanishes asymptotically in  $n$  ( $R \rightarrow 0$ ) and suffers from enormous overhead.

Thus, the BPT bound implies that no two-dimensional geometrically-local code can be asymptotically good. The intuition behind the BPT bound involves ideas from geometry and quantum information theory. One approach to proving it involves a clever partitioning of the two-dimensional lattice. Imagine an  $L \times L$  square lattice of physical qubits, so  $n = L^2$ . The lattice can be divided into  $d$  vertical strips, each of width approximately  $L/d$ . A logical operator must span the lattice to change the topological information, and a distance- $d$  code must have logical operators of weight at least  $d$ . One can construct a “smooth” logical operator that acts on qubits primarily within one of these thin, vertical strips. Using fundamental properties of entanglement entropy, specifically that local quantum operations can only generate an amount of entanglement proportional to the size of the boundary they act across, one can bound the information content ( $k$ ) that can be encoded. The argument essentially shows that the ability to correct errors across a region of size  $d$  (the distance) limits the density of logical information ( $k/n$ ) that can be stored, leading directly to the  $kd^2 \leq O(n)$  trade-off.

**Example 10.4:** The surface code saturates the BPT bound

On an  $L \times L$  grid, a standard planar surface code uses  $n \approx 2L^2$  physical qubits to encode  $k = 1$  logical qubit with a distance of  $d = L$ . Plugging these parameters into the BPT bound:

$$kd^2 = (1) \cdot L^2 = O(n) \quad (10.9)$$

This shows that the surface code perfectly saturates the BPT bound. It is not an underperforming code; rather, it is an *optimal* code given the stringent constraints of two-dimensional geometric locality.

The BPT bound can be generalized for codes that are geometrically local in  $D$  spatial dimensions:

$$kd^{2/(D-1)} \leq O(n). \quad (10.10)$$

This result, derived in subsequent work, shows that increasing the spatial dimension of the hardware can alleviate the trade-off. For any fixed dimension  $D$ , it remains impossible to construct an asymptotically good code with only local interactions. For example, in three dimensions ( $D = 3$ ), the bound becomes  $kd \leq O(n)$ . This allows for a linear distance ( $d = \Theta(n)$ ) with a constant number of logical qubits ( $k = O(1)$ ), or a logarithmic distance with a polylogarithmic number of logical qubits, but not linear scaling for both.

### 10.4.2 The Baspin-Krishna bound

The BPT bound applies to codes with strict geometric locality. However, modern hardware and recent code constructions often explore relaxing this locality, such as by allowing a limited number of long-range connections. Baspin and Krishna extended the study of geometric constraints by analyzing the *connectivity graph* associated with a quantum code [222]. Rather than assuming a rigid embedding in a  $D$ -dimensional Euclidean space, they established trade-offs based on the *graph separator* of the code's connectivity graph, where vertices are qubits and edges denote participation in the same parity check. A separator is a subset of vertices whose removal disconnects the graph into disjoint, roughly equal-sized components. Graphs that can be embedded in low dimensions have small separators, whereas highly connected graphs, like expanders, lack small separators.

The Baspin-Krishna bound demonstrates that the size of the graph separator places a hard upper limit on the achievable dimension  $k$  and distance  $d$ . An interesting conclusion of this work is the necessity of expansion: unless the connectivity graph of a qLDPC code contains an *expander graph*, the code parameters are severely limited. Therefore, an expander graph structure is a necessary condition to construct asymptotically “good” qLDPC codes (where  $k = \Theta(n)$  and  $d = \Theta(n)$ ). Furthermore, they quantified the “cost” of outperforming geometrically local codes [223]. For instance, to build a qLDPC code on a two-dimensional layout that achieves a distance  $d \propto n^{1/2+\epsilon}$  (beating the  $O(\sqrt{n})$  BPT limit), the architecture must include  $\Omega(n^{1/2+\epsilon})$  nonlocal interactions of length at least  $\tilde{\Omega}(n^\epsilon)$ . This reveals that mildly relaxing locality is insufficient to achieve optimal code parameters; significant, highly nonlocal wiring is fundamentally required.

## 10.5 Bounds from optimization theory

While analytical bounds like the quantum Hamming and Singleton bounds provide fundamental limits on code parameters, they often leave a significant gap where it is unknown if codes exist. Tighter bounds, especially for specific, finite block lengths, can be obtained numerically using optimization theory, specifically linear programming (LP) and semidefinite programming (SDP). These bounds treat the search for a quantum code as an optimization problem.

### 10.5.1 Weight enumerators and the MacWilliams identity

The foundation of the LP bound is the *quantum weight enumerator*, which generalizes concepts from classical coding theory [77, 80]. For a quantum error-correcting code with code projector  $\Pi$  encoding  $K = 2^k$  logical states in  $n$  physical qubits, we define the *weight enumerator* coefficients  $A_j$  and the *dual weight enumerator* coefficients  $B_j$  as:

$$A_j = \frac{1}{K^2} \sum_{P \in \hat{\mathcal{P}}_n | \text{wt}(P)=j} |\text{Tr}(P\Pi)|^2, \quad (10.11)$$

$$B_j = \frac{1}{K} \sum_{P \in \hat{\mathcal{P}}_n | \text{wt}(P)=j} \text{Tr}(P\Pi P^\dagger \Pi), \quad (10.12)$$

where  $\hat{\mathcal{P}}_n$  is the  $n$ -qubit Pauli group (ignoring overall phases) and  $\text{wt}(P)$  is the weight of Pauli operator  $P$  (i.e., the number of non-identity terms). For a stabilizer code with stabilizer  $\mathcal{S}$ ,  $A_j$  simply counts the number of elements in  $\mathcal{S}$  of weight  $j$ , and  $B_j$  counts the number of normalizer (logical) operators  $N(\mathcal{S})$  of weight  $j$ .

These coefficients are encapsulated in the polynomials  $A(x) = \sum_{j=0}^n A_j x^j$  and  $B(x) = \sum_{j=0}^n B_j x^j$ . Shor and Laflamme showed [77] that they are intimately related by the *quantum MacWilliams identity*.

tity:

$$B(x) = \frac{K}{2^n} (1 + 3x)^n A \left( \frac{1 - x}{1 + 3x} \right). \quad (10.13)$$

Additionally, Rains introduced the *quantum shadow enumerator* [81]  $Sh(x) = \sum_{j=0}^n Sh_j x^j$ , where the coefficients  $Sh_j$  are defined as:

$$Sh_j = \frac{1}{K} \sum_{P \in \hat{\mathcal{P}}_n | \text{wt}(P)=j} \text{Tr}(P \Pi P^\dagger Y^{\otimes n} \Pi^* Y^{\otimes n}), \quad (10.14)$$

with  $\Pi^*$  being the complex conjugate of the projector  $\Pi$ . The shadow enumerator relates to  $A(x)$  via a similar identity:

$$Sh(x) = \frac{K}{2^n} (1 + 3x)^n A \left( \frac{x - 1}{1 + 3x} \right). \quad (10.15)$$

### 10.5.2 The linear programming bound

The existence of a quantum code imposes strict inequalities on these enumerators, giving the quantum linear programming (LP) bound [80, 224]. Specifically, any valid  $((n, K, d))$  QEC code must satisfy the following:

1.  $A_0 = B_0 = 1$ , that is, there is exactly one Pauli of weight 0: the identity.
2.  $A_j \geq 0$ ,  $B_j \geq 0$ , and  $Sh_j \geq 0$  for all  $j \in \{0, \dots, n\}$ .
3.  $B_j - A_j \geq 0$  for all  $j$ , that is, undetectable errors are elements of the normalizer that are not in the stabilizer.
4.  $A_j = B_j$  for  $1 \leq j < d$ , or the error correction condition: a code of distance  $d$  has no undetectable errors of weight less than  $d$ .

To find an upper bound on  $K$  for a given  $n$  and  $d$ , we treat the coefficients  $A_j$ ,  $B_j$ , and  $Sh_j$  as unknown variables in an LP. We seek to check if there is any feasible assignment of  $A_j$  that satisfies all the linear equality and inequality constraints simultaneously, paired with the MacWilliams and shadow identity mappings. If the LP is infeasible for a target set of parameters  $((n, K, d))$ , then no such quantum code can possibly exist. For instance, LP constraints elegantly prove the non-existence of a  $((3, 2, 2))$  quantum code. Writing down the MacWilliams relations for  $n = 3$  and setting  $A_0 = B_0 = 1$  and  $A_1 = B_1$  inevitably forces  $Sh_0 < 0$ , which strictly violates the  $Sh_j \geq 0$  constraint.

### 10.5.3 Semidefinite programming bounds

While LP bounds effectively rule out many parameter regimes by enforcing linear constraints on weight enumerators, they provide only necessary conditions. Semidefinite programming (SDP) bounds [225] generalize the LP approach by enforcing the global positive semidefiniteness of underlying (moment) matrices, yielding strictly tighter limits. By turning the search for quantum codes into a hierarchy of feasibility SDPs, the SDP framework has successfully provided infeasibility certificates proving that codes with parameters such as  $((8, 9, 3))_2$  and  $((10, 5, 4))_2$  do not exist, cases that traditional LP bounds enhanced with shadow inequalities could not resolve [225].



## 11 Quantum low-density parity-check codes

We now introduce *low-density parity-check* (LDPC) codes, a promising class of quantum error-correcting codes. The development of quantum low-density parity-check (qLDPC) codes is deeply rooted in the principles of classical error correction. In his seminal 1960 doctoral thesis [226], Robert Gallager introduced a class of codes that, while initially overlooked, would eventually revolutionize digital communication. These were the LDPC codes. Gallager showed that randomly generated LDPC codes could, with high probability, achieve the Gilbert–Varshamov bound, indicating excellent error-correction capabilities. Gallager’s code construction did not receive much attention until 1982, when Michael Tanner introduced [227] the use of bipartite graphs (now called *Tanner graphs*) as a useful tool for analyzing LDPC codes. Then, over the next decade or so, LDPC codes underwent further development, led by researchers such as David MacKay and others. During this time period, several LDPC code constructions were introduced, such as the high-performing turbo codes, introduced in 1993 [228]. Today, classical LDPC codes play a vital role in technology all around us, such as Wi-Fi for wireless Internet connections, DVB-T for television broadcasting, and 5G for cellular communication.

### 11.1 The LDPC code construction

We start by defining *classical* LDPC codes, as LDPC codes started out in the classical domain before being extended to quantum codes. A classical LDPC code is a binary<sup>14</sup> linear code  $\mathcal{C} \subseteq \mathbb{F}_2^n$  that admits a sparse *parity-check matrix*, which we denote by  $H \in \mathbb{F}_2^{m \times n}$ . This means that for a family of LDPC codes  $\{\mathcal{C}_n\}$ , the number of 1’s in any given row or column of  $H$  is small and, crucially, bounded by a constant that does not grow with the code’s block length  $n$ . If we denote by  $x$  a codeword of the code, then  $x$  lives in the kernel of  $H$ . That is, the code is given by

$$\mathcal{C} = \{x \in \mathbb{F}_2^n : Hx = 0\}. \quad (11.1)$$

Another way to think of classical LDPC codes is that the number of bits involved in each check and the number of checks acting on a given bit are both bound by a constant for all members of the code family.

With this in mind, we introduce qLDPC codes. Much of the early work on qLDPC codes was done by Daniel Gottesman [229] and later expanded upon in a seminal work [230] by Breuckmann and Eberhardt in 2021. A quantum error-correcting code is a qLDPC code if it satisfies the following two conditions:

1. The weight of each check (i.e., the number of qubits it acts on) is bounded by a constant,  $w$ .
2. The degree of each qubit (i.e., the number of checks it participates in) is bounded by a constant,  $l$ .

Since  $w$  and  $l$  are constants, they do not grow as the total code size  $n$  increases. More formally, we have the following:

---

<sup>14</sup>We assume in this chapter that we are working with bits and qubits, though there are constructions for  $q$ -ary classical and qLDPC codes.

**Definition 11.1:** Quantum low-density parity-check (qLDPC) code

An  $[[n, k]]$  stabilizer code is an  $(r, c)$ -LDPC code if there exists a choice of generators  $\{M_1, \dots, M_{n-k}\}$  for the stabilizer  $Q$  of the code such that  $\text{wt} M_i \leq r$  for all  $i$  and the number of generators that act nontrivially (i.e., as  $X$ ,  $Y$ , or  $Z$ ) on qubit  $j$  is at most  $c$  for all  $j \in [n]$ .

Based on this definition, every  $[[n, k]]$  stabilizer code is automatically an  $(n, n - k)$ -LDPC code, but we are usually interested in small values of  $r$  and  $c$ . Furthermore, we are often interested in the so-called "good LDPC codes," which meet the following criteria:

- **Constant rate:**  $\frac{k}{n} \rightarrow R > 0$  as  $n \rightarrow \infty$ ,
- **Linear distance:**  $\frac{d}{n} \rightarrow \delta > 0$  as  $n \rightarrow \infty$ .

Thus, we want  $k = Rn$  and  $d = \delta n$ , but recall from the Bravyi-Poulin-Terhal bound that we discussed in [Section 10.4.1](#) that, for local commuting projector codes (which includes stabilizer codes and thus qLDPC codes), on a  $D$ -dimensional lattice, we have

$$kd^{2/(D-1)} = O(n) . \quad (11.2)$$

If we consider a  $D = 2$ -dimensional lattice and take  $k = Rn$  and  $d = \delta n$ , then plugging these values in, we find

$$kd^2 = O(n^3) \neq O(n) , \quad (11.3)$$

thus violating the bound. Thus, we cannot take  $k$  and  $d$  scaling linearly with  $n$ . In the classical setting, a random parity-check matrix immediately gives us a good LDPC code, but this is not necessarily the case for qLDPC codes. Thus, much of current QEC research into qLDPC codes focuses on finding code constructions that give good LDPC codes, using tools from various fields of mathematics and computer science.

However, note that the BPT bound only applies when we restrict our system to local interactions; if we allow for nonlocal interactions, we may be able to do better than the BPT bound. The natural question is then to ask how much nonlocality is needed to allow for codes that do better than the BPT bound allows in local settings. Dai and Li answered this question in 2024 [231], where they showed that  $\Omega(\max\{k, d\})$  interactions of length  $\Omega(\max\{d/\sqrt{n}, (kd^2/n)^{1/4}\})$  are needed to beat the BPT bound in a 2D embedding.

For practical implementation on physical hardware, it is often desired to arrange the qubits in a physical lattice, where qubits can only interact with their nearest neighbor on the lattice. However, in qLDPC codes, the checks are often allowed to connect qubits which are far apart, as long as the overall connection graph remains sparse. All geometrically local codes are, by definition, qLDPC codes, but the converse is not true. As such, this is a central theme of modern QEC research: while geometric locality enforces the severe constraints from the BPT bound that we saw above, the weaker constraint of nonlocal sparsity may still allow for the construction of highly performant codes. In what follows, we will discuss qLDPC codes that may potentially violate the BPT bound.

## 11.2 The hypergraph product construction

We first consider the *hypergraph product construction*, which yields *hypergraph product* (HGP) codes, initially introduced by Tillich and Zémor [232]. HGP codes have a fixed nonzero rate and distance that grows with the square root of the number of physical qubits  $n$ . The hypergraph product takes the tensor product of two classical code chain complexes to build a new, valid quantum chain



complex. Given two classical "seed" codes  $C_1 = \ker H_1$  and  $C_2 = \ker H_2$  for parity-check matrices  $H_1$  (of size  $r_1 \times n_1$ ) and  $H_2$  (of size  $r_2 \times n_2$ ), respectively, we obtain a quantum CSS code with the following  $X$ - and  $Z$ -type parity-check matrices:

$$H_X = \begin{pmatrix} H_1 \otimes I_{n_2} & I_{r_1} \otimes H_2^\top \end{pmatrix}, \quad (11.4)$$

$$H_Z = \begin{pmatrix} I_{n_1} \otimes H_2 & H_1^\top \otimes I_{r_2} \end{pmatrix}, \quad (11.5)$$

where  $I_m$  is the  $m \times m$  identity matrix. The construction guarantees that the CSS condition is satisfied:

$$H_X H_Z^\top = (H_1 \otimes I_{n_2})(I_{n_1} \otimes H_2)^\top + (I_{r_1} \otimes H_2^\top)(H_1^\top \otimes I_{r_2})^\top \quad (11.6)$$

$$= H_1 \otimes H_2^\top + H_1 \otimes H_2^\top \quad (11.7)$$

$$= 0 \pmod{2}. \quad (11.8)$$

If the input matrices  $H_1$  and  $H_2$  are sparse, the resulting  $H_X$  and  $H_Z$  are also sparse, thus producing a qLDPC code.

The parameters of the resulting  $[[n, k, d]]$  quantum code are determined by the parameters of the classical seed codes  $C_1$  and  $C_2$  used to generate the HGP code,  $[n_1, k_1, d_1]$  and  $[n_2, k_2, d_2]$ . The number of physical qubits in the HGP code is

$$n = n_1 n_2 + r_1 r_2, \quad (11.9)$$

the number of logical qubits is

$$k = k_1 k_2 + k_1^\top k_2^\top, \quad (11.10)$$

where  $k_i^\top$  is the dimension of the transposed code  $C_i^\top$  of  $C_i$ , and the distance is lower bounded by

$$d \geq \min\{d_1, d_2, d_1^\top, d_2^\top\}, \quad (11.11)$$

where  $d_i^\top$  is the distance of the transposed code  $C_i^\top$  of  $C_i$  (where  $d_i^\top = \infty$  if  $k_i^\top = 0$ ). We now look at the toric code through this construction.

### Example 11.1: Toric code

Let  $C_1 = C_2 = C_{\text{rep}}$  be the  $[L, 1, L]$  classical repetition code. This code has codewords  $\{00 \dots 0, 11 \dots 1\}$  and its parity-check matrix, assuming periodic boundary conditions, is an  $L \times L$  circulant matrix:

$$H_{\text{rep}} = \begin{pmatrix} 1 & 1 & 0 & \dots & 0 \\ 0 & 1 & 1 & \dots & 0 \\ \vdots & & \ddots & & \vdots \\ 0 & \dots & 0 & 1 & 1 \\ 1 & 0 & \dots & 0 & 1 \end{pmatrix}. \quad (11.12)$$

For this code,  $n_1 = n_2 = r_1 = r_2 = L$ ,  $k_1 = k_2 = 1$ , and  $d_1 = d_2 = L$ . The transposed code is also a repetition code, so  $k_1^\top = k_2^\top = 1$  and  $d_1^\top = d_2^\top = L$ . Applying the hypergraph product formulae from above yield a quantum code with parameters:

1. **Physical qubits:**  $n = L \cdot L + L \cdot L = 2L^2$ ,
2. **Logical qubits:**  $k = 1 \cdot 1 + 1 \cdot 1 = 2$ ,
3. **Distance:**  $d = \min\{L, L, L, L\} = L$ .

Thus, as expected, we see that the toric code, when analyzed using the HGP construction, is a  $[[2L^2, 2, L]]$  code.

The HGP construction elegantly solves the central challenge of ensuring the CSS commutativity condition,  $H_X H_Z^\top = 0$ , by leveraging the mathematical structure of homological algebra, which we introduced in [Section 5.1.2](#). A CSS code can be viewed as a chain complex, which is a sequence of vector spaces connected by linear maps (boundary operators,  $\partial$ ) such that the composition of any two consecutive maps is zero:  $\partial_i \circ \partial_{i+1} = 0$ . Take the following chain complex:

$$\cdots \xrightarrow{\partial_{j+1}} \mathcal{A}_j \xrightarrow{\partial_j} \mathcal{A}_{j-1} \xrightarrow{\partial_{j-1}} \cdots \xrightarrow{\partial_1} \mathcal{A}_0 \xrightarrow{\partial_0=0} 0, \quad (11.13)$$

where each  $\mathcal{A}_j$  is a vector space over  $\mathbb{F}_q$  and  $\partial_j : \mathcal{A}_j \rightarrow \mathcal{A}_{j-1}$  is a boundary operator. Recall also the fundamental lemma of homology, which states that  $\partial_j \circ \partial_{j+1} = 0$ , that is, the boundary of a boundary is zero. Then, the homology group is

$$H_j(\mathcal{A}_j) = \ker \partial_j / \text{im } \partial_{j+1}. \quad (11.14)$$

Using this, we can take a tensor product of chain complexes. That is, given chain complexes  $(\mathcal{A}, \partial^{\mathcal{A}})$  and  $(\mathcal{B}, \partial^{\mathcal{B}})$ , if we define the tensor product

$$\mathcal{C} = \mathcal{A} \otimes \mathcal{B}, \quad (11.15)$$

then

$$\mathcal{C}_k = \bigoplus_{i+j=k} \mathcal{A}_i \otimes_{\mathbb{F}_q} \mathcal{B}_j \quad (11.16)$$

and

$$\partial_k^{\mathcal{C}} : \mathcal{C}_k \rightarrow \mathcal{C}_{k-1}. \quad (11.17)$$

We can define the boundary operators as follows. Let  $a \in \mathcal{A}_j$  and  $b \in \mathcal{B}_j$ . Then,

$$\partial_k^{\mathcal{C}}(a \otimes b) = (\partial^{\mathcal{A}} a) \otimes b + (-1)^i a \otimes \partial^{\mathcal{B}} b, \quad (11.18)$$

where we get the negative sign from the *Koszul sign rule*. Thus, we take a classical code to get a length-2 chain complex  $C : C_1 \xrightarrow{\partial_C} C_0$ , where  $\partial_C$  represents the parity-check matrix  $H$  of the code. If we take two such codes over  $\mathbb{F}_q$ ,  $C : C_1 \xrightarrow{\partial_C} C_0$  and  $D_1 \xrightarrow{\partial_D} D_0$ , then we can build the following chain complex:

$$(C \otimes D)_2 = C_1 \otimes D_1, \quad (11.19)$$

$$(C \otimes D)_1 = C_1 \otimes D_0 \oplus C_0 \otimes D_1, \quad (11.20)$$

$$(C \otimes D)_0 = C_0 \otimes D_0, \quad (11.21)$$

where we can use the Koszul sign rule to construct the boundary operators as we did above. We will find two such boundary operators, one of which represents the  $X$ -type parity-check matrix  $H_X$  and one that represents the  $Z$ -type parity-check matrix  $H_Z$ .

### 11.3 LDPC codes from algebraic constructions

While we analyzed the HGP codes from the perspective of homological algebra, we can also find codes using polynomials. In this section, we consider one promising and popular code construction, the *bivariate bicycle codes*, to see how we can construct codes from polynomials. While these codes do not yield parameters that are as good as other constructions asymptotically, they are mathematically easier to work with and give good, practical, and experimentally realizable codes for realistic values of  $n$ .

Bivariate bicycle codes were popularized in a recent paper from IBM [178], though they are a specific instance of a more general class of codes, namely the two-block group algebra codes [233]. To define a bivariate bicycle code, we define two polynomials,

$$\begin{aligned} A &= A_1 + A_2 + A_3, \\ B &= B_1 + B_2 + B_3, \end{aligned} \tag{11.22}$$

where  $A_i$  and  $B_i$  are powers of the commuting matrices  $x = S_\ell \otimes I_m$  and  $y = I_\ell \otimes S_m$  for some integers  $\ell$  and  $m$ .  $I_m$  is, as usual, the  $m \times m$  identity matrix and  $S_\ell$  is the  $\ell \times \ell$  *cyclic shift matrix*, which is such that the  $i^{\text{th}}$  row has one element equal to 1 in column  $(i+1) \pmod{\ell}$ . This is easiest to understand by looking at the first two cyclic shift matrices:

$$S_2 = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \quad \text{and} \quad S_3 = \begin{pmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{pmatrix}, \tag{11.23}$$

and so on. In the polynomials  $A$  and  $B$ , we choose  $A_i$  and  $B_i$  such that each term is unique. That is,  $A_i \neq A_j$  and  $B_i \neq B_j$  for  $i \neq j$  and  $A_i \neq B_j$  for all  $i$  and  $j$ . There are a few useful properties of the objects we have introduced here to define the code. First, note that  $x$  and  $y$  commute:

$$xy = (S_\ell \otimes I_m)(I_\ell \otimes S_m) = S_\ell \otimes S_m, \tag{11.24}$$

$$yx = (I_\ell \otimes S_m)(S_\ell \otimes S_m) = S_\ell \otimes S_m, \tag{11.25}$$

$$\implies xy = yx. \tag{11.26}$$

Also, note that  $x^\ell = y^m = I_{\ell m}$  and, by extension,  $A$  and  $B$  commute.

#### Exercise 11.1: Cyclic matrix properties

- (a) Check by explicit calculation that  $x^\ell = y^m = I_{\ell m}$ .
- (b) Show that  $A$  and  $B$  commute.

With  $A$  and  $B$  in hand, we can define the parity-check matrices of the bivariate bicycle code. As these codes are a type of two-block group algebra code, we see that structure reflected in the parity-check matrices:

$$H_X = [A \mid B] \quad \text{and} \quad H_Z = [B^\top \mid A^\top]. \tag{11.27}$$

We can check that the CSS condition holds, as we would expect:

$$H_X H_Z^\top = AB + BA \tag{11.28}$$

$$= AB + AB \tag{11.29}$$

$$= 0, \tag{11.30}$$

where the arithmetic is done modulo 2. Given all of this, we find the code parameters to be the following:

**Theorem 11.1:** Bivariate bicycle code parameters

The matrices  $A, B \in \mathbb{F}_2^{\ell m \times \ell m}$  defined above give a bivariate bicycle code with parameters  $[[n, k, d]]$ , where

$$n = 2\ell m, \quad (11.31)$$

$$k = 2 \dim(\ker A \cap \ker B), \quad (11.32)$$

$$d = \min\{d_X, d_Z\}, \quad (11.33)$$

where the  $X$  and  $Z$  distances are given as

$$d_X = \min\{|v| : v \in \ker H_Z \setminus \text{rs}(H_X)\}, \quad (11.34)$$

$$d_Z = \min\{|v| : v \in \ker H_X \setminus \text{rs}(H_Z)\}, \quad (11.35)$$

where  $\text{rs}(\cdot)$  is the row space and where  $|v|$  is the Hamming weight of the vector  $v \in \mathbb{F}_2^n$ . In particular,  $d_X = d_Z$ , so  $d = d_X = d_Z$ .

We do not prove the code parameters here, though interested readers can find the proof in Ref. [178]. We highlight that, even though these codes do not beat the square-root barrier, they are easy to work with numerically to search for new codes and well-designed for near-term hardware [234]. As such, bivariate bicycle codes may be one error-correcting code that bring the field closer to large-scale, error corrected quantum devices.

#### 11.4 Breaking the $\sqrt{n}$ barrier: advanced constructions

For many years, qLDPC codes could not admit distances that scaled better than  $O(\sqrt{n})$ . Then, researchers introduced several new code constructions that managed to finally break through this barrier. We discuss two such codes in this section: fiber bundle codes and lifted product codes.

**Box 11.1:** Quantum NLTS conjecture

The question of going beyond the square-root barrier has implications beyond quantum error correction. For example, the quantum *no low energy trivial state* (NLTS) conjecture, now a theorem, was proven using properties of good LDPC codes. The quantum NLTS conjecture is related to the *quantum PCP conjecture*, which is one of the biggest open questions in quantum complexity theory. It states that if there is a family  $\{H_n, \alpha(n), \beta(n)\}$  of Hamiltonians such that  $\beta(n) - \alpha(n) = O(1)$ , where  $H_n$  is a local Hamiltonian with  $\text{poly}(n)$  terms, then deciding if the ground state energy of  $H_n$  is less than or equal to  $\alpha(n)$  or greater than or equal to  $\beta(n)$  is QMA-hard.

This statement implies the NLTS theorem, which says that there exists a fixed constant  $\epsilon > 0$  and an explicit family of  $\ell$ -local Hamiltonians  $\mathbf{H}$  for an infinite set of integer values  $n$  in which  $\mathbf{H}$  acts on  $n$  particles, consists of  $m = \Theta(n)$  local terms, such that for any family of states  $\psi$  satisfying

$$\text{Tr}[\mathbf{H}\psi] \leq \epsilon m + \lambda_{\min}(\mathbf{H}), \quad (11.36)$$

the minimum depth of any quantum circuit generating  $\psi$  grows faster than any constant. Quantum PCP implies NLTS, so NLTS is a necessary condition for the quantum PCP conjecture to be true. In particular, if the NLTS theorem is false and the quantum PCP conjecture

is true, then this implies that  $\text{NP} = \text{QMA}$ , which is believed unlikely to be true.

However, when good qLDPC codes were introduced by Panteleev and Kalachev [235, 236], the quantum NLTS theorem was able to be proven by Anshu, Breuckmann, and Nirkhe [237]. The proof for this theorem is beyond the scope of this tutorial, but very briefly, the existence of good LDPC codes provided families of codes that have enough nonlocal structure to construct an NLTS Hamiltonian. This then allowed Anshu, Breuckmann, and Nirkhe to prove the theorem. Proving the quantum NLTS theorem is a necessary, but not sufficient, condition for the quantum PCP conjecture to be true, so while progress has been made towards proving this, there still remains some work to be done.

### 11.4.1 Fiber bundle codes

The first qLDPC code that was introduced to break through the square root distance barrier was introduced by Hastings, Haah, and O'Donnell [238] and is based on *fiber bundles*. **Fiber bundle codes**  achieve a distance of  $\Omega\left(\frac{n^{3/5}}{\text{polylog}(n)}\right)$  and a code dimension of  $\tilde{\Theta}(n^{3/5})$  on  $n$  physical qubits. We will see below how these code parameters are derived, but we begin by taking a brief but fun mathematical detour to understand fiber bundles. Then, we use what we learn to develop the formalism of fiber bundle codes.

Fiber bundle theory is a branch of topology that is of great interest beyond just error correction. Informally, a fiber bundle is a space that *locally* looks like a product space, but *globally* is not necessarily a product space, that is, it can have twists. The classic comparison is between a cylinder and a Möbius strip, where they both locally look the same (an interval  $\times$  a line segment) but the Möbius strip, of course, has a twist; see [Figure 25](#).

We now define things more carefully. We start by defining fiber bundles themselves:

#### Definition 11.2: Fiber bundle

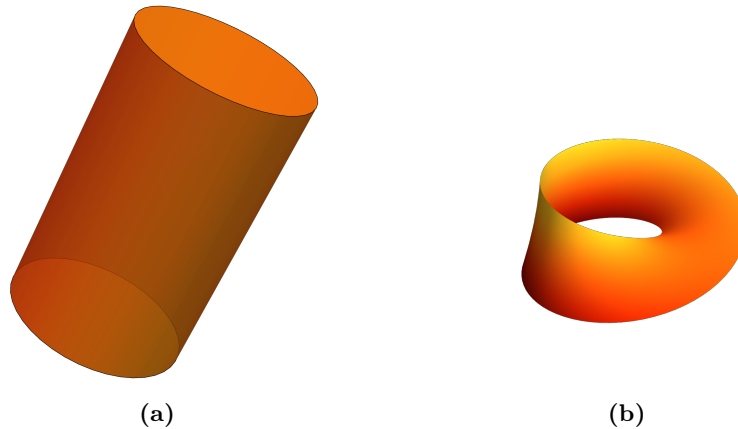
A *fiber bundle* is a set of four objects  $(E, B, \pi, F)$ , usually written as

$$F \hookrightarrow E \xrightarrow{\pi} B, \quad (11.37)$$

where

- $B$  is the *base space*
- $E$  is the *total space*
- $\pi : E \rightarrow B$  is a *projection*
- $F$  is a *fiber*.

This is rather abstract, so we walk through this somewhat intuitively, using a cylinder as a guiding trivial fiber bundle to help us visualize what's going on. Here, the base space  $B$  is the underlying space on which we are moving. For a cylinder, the base space is taken to be a circle, which we denote by  $S^1$  (i.e., a 1D sphere). Then, the fiber  $F$  is something that we attach to each point in the base space. This can be a set, vector space, group, or some other mathematical object. In our cylinder mental model, the fiber is taken to be a line segment or, often, the real line  $\mathbb{R}$ . Think of us dropping a line from above down to a point on the circle; these lines are the fibers (since they



**Figure 25:** (a) A cylinder (a trivial fiber bundle) and (b) a Möbius strip (a simple but nontrivial fiber bundle).

are lines in this case, they really are like fibers!). Then, the total space is the *product* of the base space and the fibers, that is, the combined set of points on the circle and the corresponding fibers. Mathematically, we write this as  $S = B \times F$ , but we took  $B = S^1$  and  $F = \mathbb{R}$ , so a cylinder can, in fiber bundle language, be written as  $S_{\text{cylinder}} = S^1 \times \mathbb{R}$ . Finally, the map  $\pi : E \rightarrow B$  tells us which point in the base space a given fiber sits over. Equivalently, given a base point  $b \in B$ , the map  $\pi^{-1}(b)$  gives us a fiber that looks like  $F$ .

We now add a “twist” to go from a trivial fiber bundle to something nontrivial, like a Möbius strip. First, note that if we zoom into a small region of the base space, call it  $U \subseteq B$ , then the bundle looks like

$$\pi^{-1}(U) \cong U \times F, \quad (11.38)$$

that is, a local product space. Now for the twist. Imagine we cover the base space with many overlapping patches  $U, V, \dots$ . Then, for an overlap region  $U \cap V$ , we may have two different local coordinate systems for the fiber. We relate these via a *gluing map*:

$$f \mapsto g_{UV}(x)f, \quad (11.39)$$

where  $x \in U \cap V$  and  $g_{UV}(x)$  is some symmetry of the fiber. For example, this could be a vector space or a group. For the trivial bundle, the map is chosen to do nothing. For a nontrivial bundle, such as the Möbius strip, though, the map adds a twist, such that if we go around a loop in the base space, we accumulate a nontrivial transformation. Thus, continuing with this Möbius strip example for a moment, the base space is  $B = S^1$  and the fiber  $F$  is a line segment, so that locally everything looks like a cylinder. However, globally, when we go once around the circle, the fiber flips, that is,  $f \mapsto -f$ . This flip is exactly the “twist” we have been talking about. Thus, the Möbius strip is fundamentally a different mathematical object than a cylinder, even though it is described locally in the same way. Other examples of fiber bundles include the tangent bundle and the Hopf fibration, each of which is important in areas of mathematics and physics. We won’t describe those here, but there are abundant resources for interested readers to learn more about these and other fiber bundles.

Now that we have some intuition for what fiber bundles are and how they connect to familiar objects like cylinders and Möbius strips, we dive into some specifics that will help us develop fiber bundle error-correcting codes. We start by connecting the base space, fiber, and bundle to chain complexes, which we introduced back in [Section 5.1.2](#). We mostly follow the presentation in

Ref. [238]. Let  $B$  be a base space and  $F$  be a fiber, as above. Then, we can construct the bundle  $E$  by taking tensor products of the component chain vector spaces. Note that these vector spaces are taken over  $\mathbb{F}_2$  since we will be working with qubits; analogously defined vector spaces over  $\mathbb{F}_q$  for  $q > 2$  exist for qudit fiber bundle codes [239]. An example of these chain vector spaces like the following:

$$\begin{array}{ccccc}
 \cdots & \longleftarrow & \cdots & \xleftarrow{\partial_\mu \otimes I} & B_\mu \otimes F_\nu \\
 \downarrow & & \downarrow & & \downarrow I \otimes \partial_\nu \\
 B_0 \otimes F_1 & \xleftarrow{\partial_1 \otimes I} & B_1 \otimes F_1 & \longleftarrow & \vdots \\
 \downarrow I \otimes \partial_1 & & \downarrow I \otimes \partial_1 & & \downarrow \\
 B_0 \otimes F_0 & \xleftarrow{\partial_1 \otimes I} & B_1 \otimes F_0 & \longleftarrow & \vdots
 \end{array} \tag{11.40}$$

Here, each  $B_\mu$  and  $F_\nu$  is a space and we can connect different products of spaces using the boundary operators  $\partial_\mu \otimes I$  and  $I \otimes \partial_\nu$ . We define the chain space  $E_\rho$  of the bundle as the following direct sum:

$$E_\rho = \bigoplus_{\mu+\nu=\rho} E_{\mu,\nu}, \tag{11.41}$$

where  $E_{\mu,\nu} = B_\mu \otimes F_\nu$  for  $\rho \geq 0$ . Then, we define the boundary map for each  $E_{\mu,\nu}$  as

$$\partial_\rho^E \Big|_{(\mu,\nu)} = I \otimes \partial_\nu^F + \partial_\mu^B \otimes I. \tag{11.42}$$

All of this was for non-twisted bundles, that is, the trivial bundles that we considered above. To extend this to twisted bundles, we first assume that the fiber has an automorphism group  $G$ , which is a collection of permutation actions on a set of  $\nu$ -cells such that the boundary operator commutes with the permutation. That is, for each  $g \in G$  and  $f^\nu \in F_\nu$ , we have

$$g \partial f^\nu = \partial g f^\nu. \tag{11.43}$$

Then, we can define a *twisted boundary map* as follows:

**Definition 11.3:** Twisted boundary map

Given a fiber automorphism  $G$  as in Eq. (11.43), a *connection*  $\varphi$  of a bundle is an arbitrary assignment of an automorphism group element that is called a *twist*, where for each pair of a base cell and one of its boundary cells, we have

$$\{(b, a) : b, a \text{ are cells such that } a \in \partial b\} \xrightarrow{\varphi} G. \tag{11.44}$$

Then, a *twisted boundary map*  $\partial^E$  is defined by  $\varphi$  as

$$\partial_{(0,\nu)}^E(b^0 \otimes f) = b^0 \otimes \partial f, \tag{11.45}$$

$$\partial_{(1,\nu)}^E(b^1 \otimes f) = b^1 \otimes \partial f + \sum_{a^0 \in \partial b^1} a^0 \otimes \varphi(b^1, a^0) f. \tag{11.46}$$

Here, we only consider up to 1-complexes, but this can be generalized to high-dimensional complexes as well, as long as we retain the constraint that  $\partial^E \partial^E = 0$ , that is, the boundary of a boundary is 0. With the complexes defined, we can now consider the (co)homology groups. The

first homology and cohomology of the bundle,  $H_1(E)$  and  $H^1(E)$ , respectively, are isomorphic to the homology and cohomology groups of the base space,  $H_1(B)$  and  $H^1(B)$ . This implies that  $b_1(E) = b_1(B)$ . Then, we can define the *bundle projection*:

**Definition 11.4:** Bundle projection

The *bundle projection*  $\Pi_\rho : E_\rho \rightarrow B_\rho$  is defined as

$$b^\rho \otimes f^0 \mapsto b^\rho, \quad (11.47)$$

$$b^{\rho-j} \otimes f^j \mapsto 0 \text{ for } j > 0 \quad (11.48)$$

for all  $\rho$ -cells  $b^\rho$  and  $(\rho - j)$ -cells  $b^{\rho-j}$  of the base and 0-cells  $f^0$  and  $j$ -cells  $f^j$  of the fiber. The bundle projection also forms a chain map.

With this in mind, we have the following result:

**Theorem 11.2:** Bundle projection properties

The bundle projection induces a vector space isomorphism  $\Pi_* : H_1(E) \rightarrow H_1(B)$  and  $\Pi^* : H^1(B) \rightarrow H^1(E)$  if all of the following conditions are true:

- $B$  is a 1-complex
- The boundary  $\partial f^1$  of any fiber 1-cell  $f^1$  has even weight
- Every fiber 0-chain of even weight is a boundary
- $H_0(B) = 0$ , that is, the zeroth Betti number vanishes
- Every fiber automorphism acts trivially on  $H_1(F)$ .

We refer the reader to [238, Sec. 2.4] for a proof of this result. This forms the last piece of background on fiber bundles that we need to define fiber bundle codes, so without further ado, we present the following definition of fiber bundle codes:

**Definition 11.5:** Fiber bundle code

A *fiber bundle code* is a CSS code whose logical operators are associated with one-dimensional homology and cohomology groups of the twisted bundle complex  $E_2 \rightarrow E_1 \rightarrow E_0$ , constructed from the circle fiber  $F_1 \rightarrow F_0$  and a base  $B_1 \rightarrow B_0$ .

To determine the code parameters, we first define  $n_F = m_F = \ell^2$  for an odd integer  $\ell$ . We then choose a random classical code for the base  $B$  using  $n_B$  bits and  $m_B$  parity-checks, which we can represent using a Tanner graph that has  $n_B$  right-vertices and  $m_B$  left-vertices. In particular, we take  $m_B = \frac{3}{4}n_B$  and we take the vertices to have degree  $\Theta(\log^2 n_B)$ . Then, the minimum distance is  $\Omega(n_B)$  and the parity checks are linearly independent with high probability. That is, the bundle  $E$  has  $n = n_B \cdot m_F + m_B \cdot n_F$  1-cells, which correspond to physical qubits, and the total number of cells is  $(n_B + m_B) \cdot (n_F + m_F)$ . This gives us  $\Theta(n_B)$  logical qubits with  $X$ - and  $Z$ -distances being given by  $d_X = \Omega(m_F / \log^2 n_B)$  and  $d_Z = \Omega(n_B \cdot m_F^{1/2} / \log^2 n_B)$ , respectively, for  $n_B \geq m_F$ . If we take  $n_B \sim m_F$ , then we have  $n = \Theta(n_B^2)$  physical qubits,  $k = \Theta(n^{1/2})$  logical qubits, and  $X$ - and  $Z$ -distances  $d_X = \Omega(n^{1/2} / \log^2 n)$  and  $d_Z = \Omega(n^{3/4} / \log^2 n)$ , respectively. We can perform a



procedure called *distance balancing* to improve these parameters further, yielding fiber bundle codes with parameters  $\llbracket n, \Theta(\sqrt{n}), \Omega(n^{3/5}/\text{polylog}(n)) \rrbracket$ , which beats the square-root distance barrier that plagued qLDPC codes until the introduction of these codes.

### 11.4.2 Lifted product codes

Around the same time that the fiber bundle codes were introduced, the *lifted product codes* were also proposed. This code construction was introduced by Panteleev and Kalachev [236] originally under the name of *generalized hypergraph product codes* [172], but later renamed to lifted product codes. The lifted product code is a Galois-qudit code (see Section 12.3), which means it is defined over the field  $\mathbb{F}_q$  for prime power  $q$ , but we can consider  $q = 2$  for the purposes of this tutorial.

Lifted product codes are CSS LDPC codes that generalize the hypergraph product codes that we explored in Section 11.2. Like the hypergraph product codes, lifted product codes take as their starting point a sparse parity-check matrix of a classical binary code and result in a constant-rate code. However, unlike hypergraph product codes, lifted product codes are able to achieved a *linear distance*, while hypergraph product codes suffer from the square-root distance barrier that we discussed above. As such, lifted product codes are also *good* qLDPC codes.

Recall, hypergraph product codes work with binary matrices over the field  $\mathbb{F}_2$ . Lifted product codes extend this to work over a group algebra  $\mathbb{F}_2[G]$ . For instance, the simplest version of this starts from sparse matrices over the ring

$$R_\ell = \mathbb{F}_2[x]/(x^\ell - 1), \quad (11.49)$$

where each element  $x^s$  from the ring can be represented as an  $\ell \times \ell$  cyclic shift matrix  $P^s$ . Here,  $P$  is the permutation matrix that shifts the standard basis by one position. For example, the  $3 \times 3$  permutation matrix that does this is

$$P = \begin{pmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{pmatrix}. \quad (11.50)$$

A matrix defined over  $R_\ell$  can be expanded into a larger binary matrix by taking  $0 \mapsto 0$  and  $x^s \mapsto P^s$ . Here,  $0$  is the  $\ell \times \ell$  all-zero matrix. This process is called a *lift*, hence the name of the codes. Thus, lifted product codes follow the same hypergraph product code tensor-product structure, but this is done over  $R_\ell$  (or, more generally, over  $\mathbb{F}_2[G]$ ) before expanding into binary blocks.

#### Example 11.2: Small lifted product code

Let  $\ell = 3$  such that

$$P = \begin{pmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{pmatrix} \quad (11.51)$$

and take

$$C = \begin{pmatrix} 1 & x & 0 \\ 0 & 1 & x^2 \end{pmatrix}. \quad (11.52)$$

Using the mapping  $x^s \mapsto P^s$ , we expand  $C$  into blocks as

$$C' = \begin{pmatrix} I_3 & P & 0 \\ 0 & I_3 & P^2 \end{pmatrix}, \quad (11.53)$$

which is a  $6 \times 9$  binary matrix, larger than the original  $2 \times 3$  matrix, but preserving the weight of each column and row.

Lifted product codes follow the same chain complex construction that we used to describe hypergraph product codes, except now the chain complexes are taken over the group algebra. Thus, we have

$$C_2 \xrightarrow{\partial_2} C_1 \xrightarrow{\partial_1} C_0, \quad (11.54)$$

where the fundamental lemma of homology is still satisfied:

$$\partial_1 \partial_2 = 0. \quad (11.55)$$

Taking the parity-check matrices to be

$$H_X = \partial_1, \quad H_Z = \partial_2^\top, \quad (11.56)$$

the CSS condition is automatically satisfied:

$$H_X H_Z^\top = \partial_1 (\partial_2^\top)^\top = \partial_1 \partial_2 = 0. \quad (11.57)$$

Equivalently, given the parity-check matrices  $H_1$  and  $H_2$  for the classical seed codes  $C_1$  and  $C_2$ , respectively, we can construct the  $X$ - and  $Z$ -type parity-check matrices for the lifted product code as

$$H_X = [H_1 \otimes I \mid I \otimes H_2^\top], \quad H_Z = [I \otimes H_2 \mid H_1^\top \otimes I], \quad (11.58)$$

except we replace the 1s with permutation matrices, as discussed above. This lift introduces a controlled global structure that takes the code beyond a local tensor-product construction, which enables lifted product codes to escape the square-root distance barrier.

### Exercise 11.2: HGP codes from LP codes

Show that the lifted product code construction reduces to the hypergraph product code construction when taking  $\ell = 1$  in the ring  $R_\ell = \mathbb{F}_2[x]/(x^\ell - 1)$  and simplifying the resulting  $H_X$  and  $H_Z$ .

There are two ways to introduce the lift: in an abelian way and in a non-abelian way. The abelian lifts are built from  $R_\ell = \mathbb{F}_2[x]/(x^\ell - 1)$  or  $\mathbb{F}_2[G]$  for some abelian group  $G$ . The resulting codes are often quasi-cyclic and easier to describe and implement explicitly. This construction is what was initially proposed by Panteleev and Kalachev [236], the resulting codes use  $n$  physical qubits to encode

$$k = \Theta(\log n) \quad (11.59)$$

logical qubits with a distance scaling as

$$d = \Theta\left(\frac{n}{\log n}\right). \quad (11.60)$$

As the number of logical qubits scales as  $\Theta(\log n)$ , this code is not constant-rate and thus not asymptotically good.

**Exercise 11.3:** Abelian LP codes are not good LDPC codes

Explain why the code dimension scaling as

$$k = \Theta(\log n) \tag{11.61}$$

makes the abelian lifted product codes not asymptotically good.

The non-abelian construction [235], on the other hand, allows for the group algebra to be taken over a non-abelian group. The resulting codes have constant-rate and improved code parameters

$$k = \Theta(n) , \quad d = \Theta(n) , \tag{11.62}$$

enabling asymptotically “good” codes. Thus, lifted product codes are one of the most promising code constructions, yielding great code parameters. However, *decoding* [240–243] remains a challenge for lifted product codes and other types of quasi-cyclic codes, delaying their deployment in practical, real-world instances.

## 12 Qudit codes

Much of quantum information science has been built upon the qubit. However, this two-level system, while powerful, represents only the simplest instance of a unit of quantum information. A more general and potentially more potent framework emerges from the study of qudits, which are the higher-dimensional generalizations of qubits and allow us to work with multi-level systems. In this chapter, we give an overview of qudit error-correcting codes.

### 12.1 Motivation for and basic background on qudits

Before we explore the motivation for qudit codes, we need to understand the motivation for qudits to begin with; after all, shouldn't two-level systems (i.e., qubits) be enough? It turns out, though, that we can gain a lot by considering higher dimensional Hilbert spaces.

A qudit is simply the  $q$ -dimensional generalization of the qubit, for  $q \geq 2$ . This larger state space allows for the storage of more information per computational unit and reduced circuit depth. For example, for qubits, a controlled- $U$  gate only allows one to apply a unitary gate  $U$  if the control qubit is in the state  $|1\rangle$  or do nothing if the control qubit is in the state  $|0\rangle$ . However, if we were working with qutrits, where we have  $q = 3$  basis states  $|0\rangle$ ,  $|1\rangle$ , and  $|2\rangle$ , then we think of more complex control operations: For example, do nothing if the control qutrit is in the state  $|0\rangle$ , apply the gate  $U_1$  if the qutrit is in the state  $|1\rangle$ , and apply the gate  $U_2$  if the qutrit is in the state  $|2\rangle$ . However, if we restrict ourselves to qubit systems, such conditional operations require two control qubits where, for example, we do nothing if the two control qubits are in the state  $|00\rangle$ , apply  $U_1$  if they are in state  $|01\rangle$  or  $|10\rangle$ , and apply  $U_2$  if they are in the state  $|11\rangle$ . Implementing this qubit conditional unitaries may also require more gates (e.g., to check for the parity or to do binary arithmetic on the two control qubits).

More formally, a  $q$ -dimensional qudit system can be represented by a  $q$ -dimensional Hilbert space  $\mathcal{H}_q$ . The Hilbert space  $\mathcal{H}_q$  is spanned by  $q$  orthonormal vectors  $\{|0\rangle, |1\rangle, \dots, |q-1\rangle\}$  such that any single-qudit state can be written as

$$|\psi\rangle = \alpha_0 |0\rangle + \alpha_1 |1\rangle + \dots + \alpha_{q-1} |q-1\rangle, \quad (12.1)$$

where, as usual,  $\alpha_i \in \mathbb{C}$  and

$$|\alpha_0|^2 + |\alpha_1|^2 + \dots + |\alpha_{q-1}|^2 = 1. \quad (12.2)$$

Just as for qubits, if we have  $n$  qudits, then the whole state is a vector in the Hilbert space  $\mathcal{H}_q^{\otimes n}$ , spanned by  $q^n$  orthonormal basis vectors.

To perform quantum computation, we need a set of gates, but these are no longer the qubit gates we have been working with throughout this tutorial. For qudits, though, we still require gates to be unitary operators, where any unitary quantum gate  $U$  can be arbitrarily approximated by a universal gate set  $\{U_k\}_k \subseteq U(q^n)$ , where  $U(q^n)$  is the set of  $q^n \times q^n$  unitary matrices; of course, for qubits, we had  $q = 2$ . Also just like for the case of qubits, there is more than one such universal gate set and the choice of gate set can be informed by factors like hardware constraints.

To take two specific examples of  $q$ -dimensional generalizations of common gates, consider the qudit generalizations of the Pauli- $X$  and  $Z$  gates, which we denote as  $X_q$  and  $Z_q$ , respectively. These gates operate on basis states  $|j\rangle$  as

$$X_q |j\rangle = |j+1 \pmod{q}\rangle, \quad (12.3)$$

$$Z_q |j\rangle = \omega^j |j\rangle, \quad (12.4)$$

where  $\omega$  is the  $q^{\text{th}}$  root of unity:

$$\omega = e^{2\pi i/q}. \quad (12.5)$$

In the basis of the eigenstates of  $Z_q$ , these gates have the form

$$X_q = \begin{pmatrix} 0 & 0 & \cdots & 0 & 1 \\ 1 & 0 & \cdots & 0 & 0 \\ 0 & 1 & \cdots & 0 & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & \cdots & 1 & 0 \end{pmatrix} \quad (12.6)$$

and

$$Z_q = \begin{pmatrix} 1 & 0 & 0 & \cdots & 0 \\ 0 & \omega & 0 & \cdots & 0 \\ 0 & 0 & \omega^2 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & \omega^{q-1} \end{pmatrix}, \quad (12.7)$$

With these in mind, we can then generalize the Pauli group. To do so, we now consider in more detail the specific dimension  $q$ , which, until now, we have been assuming to be any integer. We start by assuming that  $q = p$  for some prime integer  $p$ . For much of the rest of this subsection, we follow the presentation in Daniel Gottesman's lecture notes [89].

**Definition 12.1:** Single-qudit prime Pauli group

Let  $p > 2$  be a prime integer. Then, the *single-qudit Pauli group*  $\mathcal{P}_1(p)$  is defined as the set  $\{\omega^a X_p^b Z_p^c\}$ , where  $a, b, c \in \{0, 1, \dots, p-1\}$  and where  $\omega$  is the  $p^{\text{th}}$  root of unity and  $X_p$  and  $Z_p$  are as defined above for  $q = p$ .

For  $n$  qudits, the Pauli group is, of course, just  $\mathcal{P}_n(p)^{\otimes n}$  and we denote by  $\hat{\mathcal{P}}_n(p) = \mathcal{P}_n(p)/\{\omega^a I\}$  the Pauli group without phases, just as we did for the qubit Pauli group. The commutation relation between the  $X_p$  and  $Z_p$  operators is then

$$Z_p X_p = \omega X_p Z_p. \quad (12.8)$$

One can also represent the qudit Pauli group using the symplectic form, but we refer the reader to [89, Sec. 8.1.1] for more details on this.

If, instead, we consider a *prime power*, such as  $q = p^m$  for prime  $p$  and  $m > 1$ , it is easiest to consider the Pauli group as  $\mathcal{P}_n(q) = \mathcal{P}_m(p)^{\otimes n}$ , that is, as  $m$  copies of the  $p$ -dimensional Pauli group. We now bring in some of the mathematical machinery from the next section on finite field theory, where we label each basis state of a  $q$ -dimensional qudit by an element of the finite field  $\mathbb{F}_q$ . With this in mind, we define the Pauli group for prime power dimensions as follows:

**Definition 12.2:** Prime power Pauli group

Let  $\beta \in \mathbb{F}_q$  and let  $X_q^\beta$  and  $Z_q^\beta$  be the usual generalized Pauli- $X$  and  $Z$  operators, respectively, such that

$$X^\beta |a\rangle = |a + \beta\rangle, \quad (12.9)$$

$$Z^\beta |a\rangle = \omega^{\text{tr}(\beta a)} |a\rangle, \quad (12.10)$$

where  $a \in \mathbb{F}_q$  and  $\text{tr} : \mathbb{F}_q \rightarrow \mathbb{F}_p$  is the trace function that operates as  $\text{tr}(a) = \sum_{j=0}^{m-1} a^{p^j}$ . Then, the Pauli group  $\mathcal{P}_n(q)$  is comprised of elements of the form

$$\eta \omega^b \bigotimes_{j=1}^n X^{\beta_j} Z^{a_j}, \quad (12.11)$$

where  $b \in \mathbb{F}_p$  and  $\beta_j, a_j \in \mathbb{F}_q$ . Here,  $\eta = 1$  if  $q$  is odd, and  $\eta \in \{1, i\}$  if  $q$  is even. We denote  $\hat{\mathcal{P}}_n(q)$  as the set of Pauli representatives without phases, that is,

$$\bigotimes_{j=1}^n X^{\beta_j} Z^{a_j}. \quad (12.12)$$

Now, the commutation relation between the  $X$  and  $Z$  operators is

$$Z^a X^\beta = \omega^{\text{Tr}(a\beta)} X^\beta Z^a. \quad (12.13)$$

We again refer the reader to [89, Sec. 8.1.2.] for further details on this.

Finally, we consider what happens when we have a qudit of dimension  $q$  that is not a prime power. The most straightforward way to deal with this is to factor  $q$  into its prime factors and then treat each prime factor separately with its own Pauli group (e.g.,  $\mathcal{P}_1(2) \otimes \mathcal{P}_1(3)$  for  $q = 6$ ). Alternatively, we could make use of the *Weyl-Heisenberg group*. We do not often consider qudits of such a non-prime-power composite (as opposed to prime) order, so this is all we will say about this for now.

## 12.2 Finite field theory

While the mathematical formalism for qubits is technically based on the finite field  $\mathbb{F}_2$ , we do not often need to worry about the details of fields beyond some basic notions like the addition and multiplication rules (e.g.,  $1 + 1 = 0 \pmod{2}$ ). However, when we expand the local Hilbert space, we need to consider more general fields and mathematical abstractions. In this section, we introduce *finite field theory* (also called *Galois field theory*, after its short-lived founder, Évariste Galois (1811–1832)), as well as some other important notions from algebra that we will find useful.

We assume that the reader is familiar with the group theoretic concepts from [Section 4.1](#); if not, we encourage the reader to review that section now. Before we get to finite fields, we need to introduce *rings*:

### Definition 12.3: Ring

A *ring*  $R$  is a set that is closed under two binary operations: (1) addition:  $x + y \in R$  for any  $x, y \in R$  and (2) multiplication:  $xy \in R$  for any  $x, y \in R$ . Addition and multiplication satisfy the following properties for all  $x, y, z \in R$ : (1) associativity:  $(x + y) + z = x + (y + z)$  and  $(xy)z = x(yz)$ , (2) addition commutativity  $x + y = y + x$ , and (3) distributivity  $x(y + z) = xy + xz$ . Also, there exists a multiplicative identity element 1 and an additive identity element 0. For each element  $x \in R$  there exists a unique additive inverse  $y \in R$ , i.e. a unique  $y \in R$  such that  $x + y = 0$ .

If the multiplication in ring  $R$  is commutative, that is,  $xy = yx$  for all  $x, y \in R$ , then we say that the ring is a *commutative ring*. An important concept from ring theory is that of ring *ideals*:

#### Definition 12.4: Ring ideal

Let  $R$  be a ring. A subset  $I \subseteq R$  is called an *ideal* if  $I$  is an additive subgroup (meaning that if  $x, y \in I$ , then so too is  $x + y$ ) and for any element  $y \in R$  and  $x \in I$ , then  $yx \in I$ .

A *generated ideal* is constructed as follows. Given a subset  $S \subseteq R$  of a ring  $R$ , the ideal generated by  $S$ , which we denote by  $(S)$  (though some references also write this as  $\langle S \rangle$ ), is the set of elements of the form  $\sum_i r_i a_i$ , where the  $a_i \in S$  and the  $r_i \in R$ . The generator elements  $\{a_i\}$  are not unique. For example, in the ring of integers, we have  $(4, 10) = (2)$ . An ideal  $I \subseteq R$  of a ring  $R$  is further said to be *maximal* if the only other ideal to fully contain  $I$  (i.e., contain all of the elements in  $I$ ) is  $R$  itself. A *quotient ring*  $R/I$  of an ideal  $I \subseteq R$  is a ring itself whose elements are the cosets of  $I$  in  $R$  with induced operations (addition and multiplication). Combining the notions of a quotient ring and a maximal ideal, it is a fact that  $R/I$  is a field if and only if  $I$  is a maximal ideal; we will see this below more explicitly.

With this background on ring theory, we can now define a finite field. We start by defining a field in general:

#### Definition 12.5: Field

A *field* is a commutative ring in which every nonzero element has a multiplicative inverse.

Then, more specifically, we have the simple following definition:

#### Definition 12.6: Finite field

A *finite field* is a field with a finite number of elements.

We denote a field with  $q \in \mathbb{N}$  elements by  $\mathbb{F}_q$ , though the notation  $GF(q)$  (for *Galois field* is also common). If  $q$  is prime, then  $\mathbb{F}_q$  is isomorphic to  $\mathbb{Z}/q\mathbb{Z}$ , that is, the integers modulo  $q$ . We write this as  $\mathbb{F}_q \cong \mathbb{Z}/q\mathbb{Z}$ . If  $q = p^m$  for some prime  $p$  and integer  $m \geq 2$  (i.e.,  $q$  is a *prime power*), then  $\mathbb{F}_q$  is *not* isomorphic to  $\mathbb{Z}/q\mathbb{Z}$ . We call  $p$  the *characteristic* of the field. The number of elements in a finite field, which we call the *order* of the field (i.e.,  $q$ ), is always a prime or a prime power; there are no finite fields of order that is a *composite* number, such as 6. We can formalize this in the following theorem:

#### Theorem 12.1: Existence and uniqueness theorem for finite fields

For every  $q = p^m$  for prime  $p$  and integer  $m \geq 1$ , there exists a field with  $q$  elements. Furthermore, any two fields with  $q$  elements are isomorphic. Thus, the field  $\mathbb{F}_q$  is unique.

This is a well-known result and the proof is beyond the scope of this tutorial, but we refer the reader to any good set of lecture notes or textbooks [93] on finite field theory. We can construct a finite field using polynomials, specifically *polynomial quotients*. To see this, we need one additional concept, that of *irreducibility*:

#### Definition 12.7: Polynomial irreducibility

A polynomial  $f(x) \in \mathbb{F}_q[x]$  is *irreducible* if it cannot be factored into two polynomials  $g(x), h(x) \in \mathbb{F}_q[x]$  such that the degrees of both  $g(x)$  and  $h(x)$  are less than the degree

of  $f(x)$ .

With this in mind, we can construct a finite field  $\mathbb{F}_q$  with  $q = p^m$  elements, for prime  $p$  and  $m \geq 1$  as follows. Let  $f(x) \in \mathbb{F}_p[x]$  be an irreducible polynomial, as defined in [Definition 12.7](#). Then,

$$\mathbb{F}_q \cong \mathbb{F}_p[x]/(f(x)). \quad (12.14)$$

Note the parentheses around  $f(x)$ , denoting this as a ring ideal, as defined in [Definition 12.4](#). Thus, we define higher-order finite fields in terms of their characteristic (i.e.,  $p$ ) field. One of the simplest examples is the construction of the field  $\mathbb{F}_4$ :

**Example 12.1:** Finite field with four elements

Let  $p = 2$  and  $f(x) = x^2 + x + 1$ , which is irreducible over  $\mathbb{F}_2$ . We can construct  $\mathbb{F}_4$  as

$$\mathbb{F}_4 = \mathbb{F}_2[x]/(x^2 + x + 1), \quad (12.15)$$

where the four elements of  $\mathbb{F}_4$  are  $\{0, 1, x, x+1\}$ . Arithmetic is then done modulo  $x^2 + x + 1$ .  $\mathbb{F}_4$  is the smallest non-prime field and is thus important for understanding higher-order finite fields.

Notably, every function  $f : \mathbb{F}_q \rightarrow \mathbb{F}_q$  can be represented as a polynomial. We can also consider finite fields from a vector space perspective, as we hinted at in [Section 12.1](#). That is, the field  $\mathbb{F}_{p^m}$  can be worked with as an  $m$ -dimensional vector space over  $\mathbb{F}_p$ . This allows us to use the vast toolbox of linear algebra where we keep in mind that arithmetic operations are done over  $\mathbb{Z}/p\mathbb{Z} \cong \mathbb{F}_p$ .

To conclude, we highlight some interesting properties of finite fields. First, we note that the nonzero elements of a finite field  $\mathbb{F}_q$ , which we denote by  $\mathbb{F}_q^\times$ , form a *cyclic group* under multiplication:

$$\mathbb{F}_q^\times \cong C_{q-1}. \quad (12.16)$$

A cyclic group  $C_{q-1}$  is simply a group that is generated by a single element under multiplication. To connect this concept to finite fields, we note that there is some element  $\alpha \in \mathbb{F}_q^\times$  that we call *primitive* that gives us the whole of  $\mathbb{F}_q^\times$ :

$$\mathbb{F}_q^\times = \{\alpha^k : 0 \leq k < q-1\}. \quad (12.17)$$

We can also denote this as  $\mathbb{F}_q^\times = \langle \alpha \rangle$ . Furthermore, there are exactly  $\varphi(q-1)$  primitive elements in any multiplicative group, where  $\varphi(n)$  is a function called *Euler's totient function*, which counts the number of positive integers up to  $n$  that are coprime to  $n$ . Finally, we note that a finite field  $\mathbb{F}_{p^n}$  has a subfield of size  $\mathbb{F}_{p^m}$  if and only if  $m|n$ , that is,  $m$  divides  $n$ . Each such subfield is unique.

### 12.3 Major families of qudit stabilizer codes

The main class of qudit codes that we focus on in this chapter is that of *qudit stabilizer codes*. There are two axes along which we can define a qudit stabilizer code: the algebra upon which the code is built and the construction style. By the “algebra,” we mean the group or field that all of our subsequent operations are done over, which is, of course, determined by the local dimension of the qudit. Thus, for a qudit code with local qudit dimension  $q$ , we could have codes defined over the integers modulo  $q$ ,  $\mathbb{Z}_q$ , or a finite field  $\mathbb{F}_q$ , depending on what  $q$  is, to name two examples. The “construction” is the other ingredient that gives us a qudit code, and this determines the structure



of the code. Examples include CSS, non-CSS, topological, homological, LDPC, and graph-based codes.

### 12.3.1 Algebra

We start by looking at different qudit stabilizer codes based on their algebra, that is, what group or field the local qudit dimension is defined over.

**Modular-qudit stabilizer codes.** We start by considering [modular-qudit stabilizer codes](#) <sup>Eq. 260</sup>. These codes are the most straightforward generalization of qubit stabilizer codes and were introduced concurrently with the qubit formalism in Daniel Gottesman's PhD thesis [63]. As was the case for qubits, the fundamental group that we work with to build up our stabilizer codes is the Pauli group, except we now consider the *modular-qudit Pauli group* [244], which we define as follows:

#### Definition 12.8: Modular qudit Pauli group

Let  $\mathcal{H}_q$  be the Hilbert space of one modular qudit with computational basis

$$\{|k\rangle : k \in \mathbb{Z}_q\}. \quad (12.18)$$

Define the single-qudit Pauli operators

$$X|k\rangle = |k+1 \pmod{q}\rangle, \quad Z|k\rangle = e^{2\pi i k/q} |k\rangle. \quad (12.19)$$

For  $n$  qudits and vectors  $a, b \in \mathbb{Z}_q^n$ , write

$$X(a) = X^{a_1} \otimes \cdots \otimes X^{a_n}, \quad Z(b) = Z^{b_1} \otimes \cdots \otimes Z^{b_n}. \quad (12.20)$$

Then, the *modular qudit Pauli group* on  $n$  qudits is the subgroup of unitary operators that is generated by all  $X(a), Z(b)$ , and a global phase  $\tau = e^{2\pi i/q}$  that is a  $q^{\text{th}}$  root of unity. In other words,

$$\mathcal{P}_{n,q}^{\text{mod}} = \{\tau^c X(a) Z(b) : c \in \mathbb{Z}_{2q}, a, b \in \mathbb{Z}_q^n\}. \quad (12.21)$$

Now, we represent Pauli strings as vectors in  $\mathbb{Z}_q^{2n}$ , where  $n$  is the number of qudits and  $q$  is the local qudit dimension. Here,  $q$  can be *any* integer; we are not restricted to, for example, primes or prime powers. The object  $\mathbb{Z}_q^{2n}$  is now a *module*, not necessarily just a vector space; see [Box 12.1](#) for a brief introduction to modules. A stabilizer is then a commuting set  $\mathcal{S}$  of these generalized Pauli strings and the code is taken to be the joint  $+1$  eigenspace of  $\mathcal{S}$ . Any single-qudit Pauli string  $X_a Z_b$  for  $a, b \in \mathbb{Z}_q$  can be written using a *modular symplectic representation*  $(a|b) \in \mathbb{Z}_q^2$ , analogous to the binary symplectic representation that we used for qubits. This, of course, extends naturally to the case for  $n$ -qudit systems, where we can write  $(\mathbf{a}|\mathbf{b}) \in \mathbb{Z}_q^{2n}$ .

We denote a modular-qudit stabilizer code encoding into a codespace of dimension  $K = q^k$  into  $n$  physical qudits with distance  $d$  by  $\llbracket n, k, d \rrbracket_{\mathbb{Z}_q}$ , where  $q$  is the local qudit dimension. If  $q$  is prime, then  $K$  is a power of  $q$ , but if  $q$  is composite (e.g.,  $q = 4 = 2^2$  or  $q = 6 = 2 \times 3$ ), then  $K = q^n/|\mathcal{S}|$  (see e.g. [245, Theorem 1]), where  $|\mathcal{S}|$  now does not need to be a power of  $q$  and so  $k$  is not necessarily an integer. For example, consider  $q = 4$  and fourth root of unity  $\omega = e^{2\pi i/4} = i$  so that the generalized Paulis are

$$X = \sum_{j \in \mathbb{Z}_4} |j+1\rangle \langle j| \quad \text{and} \quad Z = |0\rangle\langle 0| + i|1\rangle\langle 1| - |2\rangle\langle 2| - i|3\rangle\langle 3|. \quad (12.22)$$

For a stabilizer group  $\mathcal{S} = \langle Z^2 \rangle = \{I, Z^2\}$ , the corresponding codespace is a two-dimensional subspace spanned by  $\{|0\rangle, |2\rangle\}$  since  $Z^2 = |0\rangle\langle 0| - |1\rangle\langle 1| + |2\rangle\langle 2| - |3\rangle\langle 3|$ . The dimension of the codespace can be clarified by  $K = q/|\mathcal{S}| = 4/2 = 2$ , since the stabilizer group is  $|\mathcal{S}| = 2$ . Thus, the dimension is not any power of 4, that is,  $K = 2 \neq 4^k$  for any positive integer  $k$ .

A  $[[n, k, d]]_{\mathbb{Z}_q}$  modular-qudit code can detect errors on up to  $d - 1$  qudits and can correct erasure errors on up to  $d - 1$  qudits. The code can correct arbitrary errors on at most  $\lfloor \frac{d-1}{2} \rfloor$  qudits. Finally, we note that we can also describe the modular-qudit stabilizer code using its parity-check matrix,  $H = (A|B)$ , where each row  $(a|b) \in \mathbb{Z}_q^{2n}$  of  $H$  represents a stabilizer generator  $X_a Z_b$ .

### Box 12.1: Modules

A *module* can be thought of as a generalization of a vector space, where the field of scalars is replaced by a ring. The ring can be either commutative or non-commutative. Modules have important applications in commutative algebra, homological algebra, algebraic geometry, and algebraic topology. For example, in commutative algebra, ideals and quotient rings (which we defined above in [Section 12.2](#)) are modules. Much of the work with modules has to do with extending the properties of vector spaces to well-behaved rings, but there are several complications that arise when doing this. For example, not all modules have a basis, which is, of course, untrue for vector spaces.

Let  $R$  be a ring (see [Definition 12.3](#)). A *left  $R$ -module* is a set  $M$  with

1. a binary operation  $+$  on  $M$  in which  $M$  is an abelian group and
2. an action of  $R$  on  $M$  (i.e., a map  $R \times M \rightarrow M$ ) denoted by  $rm$  for all  $r \in R$  and  $m \in M$  satisfying:
  - (a)  $(r + s)m = rm + sm$  for all  $r, s \in R, m \in M$ ,
  - (b)  $(rs)m = r(sm)$  for all  $r, s \in R, m \in M$ , and
  - (c)  $r(m + n) = rm + rn$  for all  $r \in R, m, n \in M$ .

If  $R$  has an identity element 1, then we further have a *unital* condition, that is,

- (d)  $1m = m$  for all  $m \in M$ .

In this work, we assume rings are unital.

*Right  $R$ -modules* are defined analogously and if  $R$  is commutative, then we can turn a left  $R$ -module  $M$  into a right  $R$ -module by taking  $rm = mr$  for  $r \in R$  and  $m \in M$ .

A common example of a module is a  $\mathbb{Z}$ -module, where  $\mathbb{Z}$  is, of course, the set of integers. Let  $R = \mathbb{Z}$  and  $A$  be any abelian group with binary operation  $+$ . To make  $A$  into a  $\mathbb{Z}$ -module, we define for any  $n \in \mathbb{Z}$  and  $a \in A$  the quantity  $na$ , where

$$na = \begin{cases} a + a + \dots + a \text{ (} n \text{ times)} & \text{if } n > 0 \\ 0 & \text{if } n = 0 \\ -a - a - \dots - a \text{ (} n \text{ times)} & \text{if } n < 0, \end{cases} \quad (12.23)$$

where 0 is the additive identity in  $A$ . This is the only possible action of  $\mathbb{Z}$  on  $A$  making it a unital  $\mathbb{Z}$ -module. Since  $A$  was taken to be *any* abelian group, we can conclude that *every* abelian group is a  $\mathbb{Z}$ -module.

If  $A$  is an abelian group with element  $x$  of finite order  $n$ , then  $nx = 0$ . That is, for  $\mathbb{Z}$ -modules, there exist nonzero elements  $x \in A$  such that  $nx = 0$  for  $n \neq 0$ , which is not true for vector spaces. If  $A$  has order  $m$ , then  $mx = 0$  for all  $x \in A$  and we say that  $A$  is a module over  $\mathbb{Z}/m\mathbb{Z}$ . Furthermore, if  $p$  is prime and  $A$  is abelian such that  $px = 0$  for all  $x \in A$ , then  $A$  is a  $\mathbb{Z}/p\mathbb{Z}$ -module. However, we know that  $\mathbb{Z}/p\mathbb{Z} \cong \mathbb{F}_p$ , so  $A$  is also an  $\mathbb{F}_p$ -module, which is just a vector space over  $\mathbb{F}_p$ .

**Galois-qudit stabilizer codes.** While for modular-qudit stabilizer codes we allowed  $q$  to be *any* positive integer, if we restrict to qudits that have a local dimension that is a prime power, then we get *Galois-qudit stabilizer codes* [\[Ec26\]](#) [246]. That is, we take  $q = p^m$  for some prime  $p$  and  $m \geq 1$ . Thus, the algebra is now based on the *finite field*  $\mathbb{F}_q$ . The basis states are therefore labeled by elements from  $\mathbb{F}_q$ . Thus, a single-qudit Pauli operator  $X_a Z_b$  for  $a, b \in \mathbb{F}_q$  is written as  $(a|b) \in \mathbb{F}_q^2$ . The notation for  $n$  Galois qudits follows analogously. Furthermore, the Pauli operators are defined such that their commutation relations are determined by the *trace-symplectic inner product*. That is, given two Galois-qudit stabilizers on  $n$  qudits described by  $(\mathbf{a}|\mathbf{b}), (\mathbf{a}'|\mathbf{b}') \in \mathbb{F}_q^{2n}$ , they commute if and only if the following quantity, the trace-symplectic inner product, is zero:

$$\text{Tr}[\mathbf{a} \cdot \mathbf{b}' - \mathbf{a}' \cdot \mathbf{b}] = \sum_{i=1}^n \text{Tr}[a_i b'_i - a'_i b_i] = 0. \quad (12.24)$$

Here, the trace is the *field trace* (see [Definition 12.2](#)). One nice property of Galois-qudit stabilizer codes is that they can be mapped to *self-orthogonal additive codes* over  $\mathbb{F}_{q^2}$ . That is, given an  $n$ -qudit Pauli stabilizer, we can represent it as a length- $n$  vector over the field  $\mathbb{F}_{q^2}$ . Then, given a basis  $(\beta, \beta^q)$  for  $\mathbb{F}_{q^2}$  over  $\mathbb{F}_q$ , a vector  $(a|b) \in \mathbb{F}_q^2$  can be mapped to an element  $a\beta + b\beta^q \in \mathbb{F}_{q^2}$ .

Just as we did for modular-qudit stabilizer codes, we denote a Galois-qudit stabilizer code using  $n$  Galois-qudits to encode  $k$  logical qudits with codespace dimension  $K = q^k$  and distance  $d$  using the notation  $[[n, k, d]]_q$ . Note, though, the subscript  $q$  rather than  $\mathbb{Z}_q$  to denote that these codes are defined over a finite field rather than a ring. Furthermore, just as was the case for modular-qudit stabilizer codes, the logical codespace dimension does not need to be an integer, as we can write  $K = q^{n-r/m}$ , where  $r$  is the number of generators of the stabilizer group, and the quantity  $r/m$  is not necessarily an integer. The distance means the same thing as it did for modular-qudit stabilizer codes as well, namely, the minimum weight of a Galois-qudit Pauli string yielding a nontrivial logical operator. Thus, the code  $[[n, k, d]]_q$  detects and corrects erasure errors on  $d - 1$  qudits and corrects errors on at most  $\lfloor \frac{d-1}{2} \rfloor$  qudits.

**True Galois-qudit stabilizer codes.** Finally, we can consider a subclass of Galois-qudit stabilizer codes whose stabilizers' symplectic representation forms a linear subspace. Such codes are called *true Galois-qudit stabilizer codes* [\[Ec26\]](#). Here, the vectors comprising the stabilizer group form a subspace in which addition and multiplication are closed by elements of  $\mathbb{F}_q$ . Note that this definition adds *multiplication*, where for the Galois-qudit stabilizer codes in the previous section, we only had closure under addition. The number of generators  $r$  is thus an integer multiple of  $m$ , so the quotient  $r/m$  is an integer. Thus, the code has a logical codespace dimension  $K = q^{n-r/m}$ , which is now always an integer, which is nice. Everything else about these codes (e.g., the parity-check matrices, the code parameters, etc.) is the same as for the Galois-qudit stabilizer codes.

### 12.3.2 Construction style

While the algebraic structure told us how to represent the stabilizers and the commutation relations between Pauli operators, the *code construction style* tells us how to build the commuting check operators. We consider four construction styles: CSS, non-CSS, topological/homological, and LDPC.

**CSS-type.** Consider the true Galois-qudit stabilizer codes from above. If we have such a code and we have a set of stabilizer generators that are either  $X$ - or  $Z$ -type Pauli strings, then we have a *Galois-qudit CSS code*. As such, these codes can be defined using chain complexes over  $\mathbb{F}_q$  by extending the techniques that map CSS codes to homology groups for qubits to qudits. The stabilizer generator matrix is now nicely given as

$$H = \begin{pmatrix} 0 & H_X \\ H_Z & 0 \end{pmatrix}, \quad (12.25)$$

where we must have the CSS condition satisfied:

$$H_X H_Z^\top = 0. \quad (12.26)$$

Of course, the nice thing about CSS qudit codes is that they can be derived from  $q$ -ary linear codes, where we take one code to be an  $[n, k_X, d_X]_q$  code  $C_X$  and the other to be an  $[n, k_Z, d_Z]_q$  code  $C_Z$  such that  $C_X^\perp \subseteq C_Z$ . The quantum code parameters are then  $[[n, k, d]]_q$ , where  $k = k_X + k_Z - n$  and  $d \geq \min\{d_X, d_Z\}$ . In particular,

$$d_X = \min\{\text{wt}(c) : c \in C_Z \setminus C_X^\perp\}, \quad d_Z = \min\{\text{wt}(c) : c \in C_X \setminus C_Z^\perp\}. \quad (12.27)$$

Furthermore, we associate  $H_X$  with the parity-check matrix of the classical code  $C_X$  and  $H_Z$  as the parity-check matrix of the classical code  $C_Z$ .

**Non-CSS-type.** For non-CSS-type qudit codes, we do not have the nice separation between  $X$ - and  $Z$ -type stabilizers. These codes generally refer to self-orthogonal codes under a given symplectic form and they relate to the self-orthogonal additive codes over  $\mathbb{F}_{q^2}$  that we mentioned above. This allows for the development of quantum Hamming codes, quantum BCH codes, and other quantum analogues of popular classical  $q$ -ary codes over  $\mathbb{F}_q$ .

**Topological and homological.** As we alluded to above in our discussion of CSS-type qudit codes, we can define qudit stabilizer codes from *chain complexes* and take advantage of the tools from *homological algebra*, which we considered earlier in this tutorial (see, for example, [Section 5.1.2](#)). Basic examples of such codes include qudit versions of the surface code and the toric code. For example, for qudit surface codes, stabilizer generators are of  $X$ - and  $Z$ -type and they act on only a few qudits. We associate these with the stars and plaquettes of a tessellation of a two-dimensional surface. Other examples include the qudit color codes [\[247\]](#), which are just qudit generalizations of the qubit color codes.

**LDPC.** Finally, there exist qudit generalizations of several major classes of LDPC codes, where the stabilizers have the benefit of being sparse. Examples of such qudit LDPC codes [\[239, 248\]](#) include qudit generalizations of the currently-popular bivariate bicycle codes, qudit hypergraph product codes (also very popular), subsystem hypergraph (SHYPS) codes, high-dimensional expander codes, and fiber bundle codes. Qudit LDPC codes inherit many of the favorable qualities of qubit LDPC codes, such as reduced physical qudit overhead, high encoding rates, and sparsity in the check operators, requiring fewer check measurements.

## 12.4 Advances in code bounds and asymptotic performance

Recall the quantum Singleton bound for qubits from [Section 10.2](#), which gives us the relation

$$n - k \geq 2(d - 1) \quad (12.28)$$

for an  $[[n, k, d]]$  code. It turns out that this is the same for qudits [\[74\]](#), except now we have a different size for our logical subspace,  $K = q^k$ , instead of  $K = 2^k$ . Thus, there is no difference in the tradeoff between  $n$ ,  $k$ , and  $d$  when going from qubits to qudits, but we do gain a larger Hilbert space. It is this larger Hilbert space that allows for a richer landscape of MDS codes (those that saturate the quantum Singleton bound), though. Thus, it is easier to find codes over  $\mathbb{F}_q$  for  $q > 2$  that saturate this bound compared to restricting ourselves to  $\mathbb{F}_2$ .

We do, however, get a different quantum Hamming bound, which we discussed in [Section 10.1](#), when we take  $q > 2$ . Recall for qubits we had the following relation, where we define  $t = \lfloor \frac{d-1}{2} \rfloor$ :

$$2^k \sum_{j=0}^t \binom{n}{j} 3^j \leq 2^n. \quad (12.29)$$

For qudits of local dimension  $q$ , we have

$$q^k \sum_{j=0}^t \binom{n}{j} (q^2 - 1)^j \leq q^n, \quad (12.30)$$

which shows that each qudit has  $q^2 - 1$  nontrivial generalized Pauli errors [\[249\]](#). Note that this only holds for nondegenerate codes; finding the Hamming bound for degenerate codes remains an open problem.

Now we discuss some asymptotics. We define the *asymptotic rate* as

$$R = \lim_{n \rightarrow \infty} \frac{\log_q K}{n} = \lim_{n \rightarrow \infty} \frac{k}{n} \quad (12.31)$$

and the *relative distance* as

$$\delta = \lim_{n \rightarrow \infty} \frac{d}{n}. \quad (12.32)$$

The  $q$ -ary Gilbert-Varshamov bound lower bound is given by

$$R \geq 1 - \delta \log_q(q + 1) - H_q(\delta), \quad (12.33)$$

where

$$H_q(x) = x \log_q(q - 1) - x \log_q x - (1 - x) \log_q(1 - x) \quad (12.34)$$

is the  $q$ -ary entropy function [\[220\]](#). As  $q$  grows, the right hand side also grows, giving a larger lower bound and thus a better rate, and therefore a better code. There exist codes such that both  $R$  and  $\delta$  are nonzero for a fixed  $q$  and we call these codes *good codes*.

## 12.5 Experimental realizations of qudit codes

There have been several examples of experimental realizations of qudit-based systems using various platforms, such as superconducting [\[250–255\]](#), photonic [\[256–259\]](#), trapped-ion [\[260–263\]](#), and circuit quantum electrodynamics (QED) [\[264\]](#) devices. However, these are just experimentally implementing qudit systems; there has been less, but still notable, work on implementing qudit-based

*error correction* on real devices. We highlight two examples in this section. The first [265] is an experimental demonstration of a qutrit toric code, which demonstrated the preparation and measurement of a  $\mathbb{Z}_3$  toric code ground state on 24 qutrits. The second [264], which received quite a bit of attention in the community and the press, demonstrated encoding a logical qutrit and a logical ququart using a single-mode square-lattice GKP qudit. While we did not discuss GKP codes in the context of qudits, we refer the reader to Section 8 for more on bosonic codes. Both of these early demonstrations show that implementing error-corrected qudit systems is experimentally viable and pave the way to future, larger-scale experiments.

## 13 Fault tolerance I

A quantum error-correcting code protects *states*. In contrast, a fault-tolerant protocol protects *processes* [24, 64, 69, 89]. The core difficulty is not only detecting and correcting errors, but ensuring that the mechanism of correction does not create more errors than it removes. A *location* is an elementary circuit component: a one- or two-qubit gate, state preparation, measurement, or an idle step. A *fault* is a location at which the physical operation deviates from the ideal one. An ideal circuit contains abstract logical locations: perfect preparations, perfect gates, perfect waits, and perfect measurements. A fault-tolerant implementation replaces each logical location by a *gadget* operating on encoded data. Examples include an encoded CNOT gadget, an encoded measurement gadget, and an error-correction gadget. In this chapter, we make the first steps in understanding *fault-tolerant quantum computation* (FTQC), the pinnacle of the quantum error correction enterprise.

### 13.1 Error propagation

Suppose we are given a circuit with faulty components. If each location of this circuit fails with probability at most  $p$ , then the failure probability associated with a fixed output location scales linearly as  $O(p)$ . For a computation with  $N$  gates, a union bound gives an overall failure probability scaling as  $O(Np)$ . Error correction becomes useful only when encoded gadgets suppress the effective logical error rate below the underlying physical error rate. Merely performing error correction periodically is not sufficient to prevent the buildup of errors, even if it is applied after every encoded gate. Two reasons for this include:

1. **Error propagation through gates:** Encoded gates can cause errors to propagate. For example, let  $U$  be the CNOT gate. The propagation of a Pauli- $X$  error on the control qubit is given by:

$$U(X \otimes I)U^\dagger = X \otimes X \quad (13.1)$$

Similarly, a  $Z$  error on the target qubit propagates backwards:

$$U(I \otimes Z)U^\dagger = Z \otimes Z \quad (13.2)$$

A single fault in a two-qubit gate can thus introduce two errors in the code block, as seen in Figure 26.

2. **Errors introduced by correction:** Error correction itself can introduce errors on the encoded qubits if the ancilla qubits or the gates used to measure the syndrome are faulty.

With this in mind, we define fault tolerance as follows:



$X$  on the control propagates forward to the target.

$Z$  on the target propagates backward to the control.

**Figure 26:** Pauli error propagation through CNOT. The naive use of two-qubit gates can turn one fault into multiple data errors.

**Definition 13.1:** Fault tolerance

For a code correcting up to  $t$  errors per block, a procedure is *fault-tolerant* if any pattern consisting of at most  $s \leq t$  faults during the procedure, together with at most  $r \leq t - s$  pre-existing errors on each input block, produces at most a correctable number of errors on each output block, that is, at most  $t$  errors per output block, and does not alter the encoded logical action.

To satisfy the fault tolerance condition, we design encoded operations such that a failure anywhere during the implementation can only propagate to a small number of qubits on each block. The most robust way to achieve this is via *transversality*.

**Definition 13.2:** Transversal gate

Let a code block consist of  $n$  physical qubits. An encoded gate acting on one or more blocks is *transversal* if it is implemented as a tensor product of gates that couple only corresponding qubits across blocks and never couple two qubits within the same block. For a one-block gate,

$$U_{\text{trans}} = \bigotimes_{i=1}^n U_i. \quad (13.3)$$

For a two-block transversal gate, the  $i^{\text{th}}$  qubit of one block may interact only with the  $i^{\text{th}}$  qubit of the other block.

We demonstrate these ideas with the following example using the Steane code:

**Example 13.1:** The Steane code

The 7-qubit Steane code is a  $[[7, 1, 3]]$  CSS code. A convenient choice of stabilizer generators is

$$g_1^{(X)} = X_1 X_2 X_3 X_4, \quad g_1^{(Z)} = Z_1 Z_2 Z_3 Z_4, \quad (13.4)$$

$$g_2^{(X)} = X_1 X_2 X_5 X_6, \quad g_2^{(Z)} = Z_1 Z_2 Z_5 Z_6, \quad (13.5)$$

$$g_3^{(X)} = X_1 X_3 X_5 X_7, \quad g_3^{(Z)} = Z_1 Z_3 Z_5 Z_7. \quad (13.6)$$

The codespace is the simultaneous  $+1$  eigenspace of these six commuting operators. The logical operators are  $\bar{Z} \equiv Z_1 Z_2 Z_3 Z_4 Z_5 Z_6 Z_7$  and  $\bar{X} \equiv X_1 X_2 X_3 X_4 X_5 X_6 X_7$ .

The Steane code is self-dual as a CSS code, which implies that transversal Hadamard acts logically:

$$\bar{H} = H^{\otimes 7}, \quad H^{\otimes 7} \bar{X} H^{\otimes 7} = \bar{Z}, \quad H^{\otimes 7} \bar{Z} H^{\otimes 7} = \bar{X}. \quad (13.7)$$

Likewise, transversal CNOT between two Steane blocks implements the logical CNOT,

$$\overline{\text{CNOT}} = \text{CNOT}^{\otimes 7}. \quad (13.8)$$

A phase gate is also transversal up to the usual convention for the logical phase. With the logical operators above, one finds

$$S^{\otimes 7} \bar{X} (S^{\otimes 7})^\dagger = -i \bar{X} \bar{Z} \quad \text{and} \quad S^{\otimes 7} \bar{Z} (S^{\otimes 7})^\dagger = \bar{Z}, \quad (13.9)$$

so  $S^{\otimes 7}$  implements  $\bar{S}^\dagger$  and  $(S^\dagger)^{\otimes 7}$  implements  $\bar{S}$ .



## 13.2 Syndrome extraction schemes

Consider an  $n$ -qubit stabilizer code with stabilizer group  $S \subset \mathcal{P}_n$  and generators

$$S = \langle S_1, \dots, S_r \rangle, \quad S_j \in \{I, X, Y, Z\}^{\otimes n}. \quad (13.10)$$

For a CSS code, we can choose generators such that

$$S = \langle S_1^{(X)}, \dots, S_{r_X}^{(X)}, S_1^{(Z)}, \dots, S_{r_Z}^{(Z)} \rangle, \quad (13.11)$$

where each  $S_a^{(X)}$  contains only  $X$  and  $I$ , and each  $S_b^{(Z)}$  contains only  $Z$  and  $I$ . The goal of *syndrome extraction* is to measure the eigenvalues  $s_j \in \{\pm 1\}$  of the generators  $S_j$  fault-tolerantly, that is, such that a single fault does not propagate to multiple data qubits. We will discuss three different fault-tolerant schemes for syndrome extraction, in the sense formalized later in [Definition 13.3](#).

### 13.2.1 Shor scheme

Peter Shor introduced [\[64\]](#) the first method for fault-tolerant syndrome extraction. Shor's method extracts syndromes stabilizer-by-stabilizer, utilizing cat states or GHZ states. For a weight- $w$  stabilizer, Shor's scheme prepares an ancilla in the  $w$ -qubit state:

$$|\text{cat}_w\rangle = \frac{1}{\sqrt{2}} (|0\rangle^{\otimes w} + |1\rangle^{\otimes w}). \quad (13.12)$$

The protocol proceeds in three steps:

1. **Preparation and verification:** The cat state is prepared using a cascade of CNOT gates. However, a single fault during this preparation circuit could easily create a weight-2 error in the cat state. To prevent this from corrupting the data qubits, the cat state must be rigorously verified. Note that a cat state is a repetition code, one uses additional ancilla qubits to measure the parity of adjacent qubits in the cat state (checking that it is indeed in the repetition space). If any verification check fails, the corrupted cat state is instantly discarded, and a new one is prepared.
2. **Transversal CNOT gates:** Once accepted by the verification checks, the cat state interacts with the data block via transversal CNOT gates. Crucially, the  $i^{\text{th}}$  qubit of the cat state interacts exclusively with the  $i^{\text{th}}$  data qubit involved in the stabilizer. Because the interaction is transversal, an error on the cat state can only ever propagate to a single data qubit.
3. **Measurement and majority voting:** After the coupling, the ancilla is measured and the parity of its outcomes yields the stabilizer eigenvalue. Because ancilla preparation and measurement are noisy, a single extraction of the same stabilizer is not usually trusted. Instead, the measurement is repeated an odd number of times and a majority vote is taken. The data are corrected only after a consistent syndrome is obtained or after a decoding rule is applied to the repeated outcomes.

Shor extraction is conceptually clean and applies to general stabilizer codes, not only CSS codes. Its main cost is ancilla overhead and time: a separate cat state is needed for each stabilizer measurement, and repeated measurements are typically required.

### 13.2.2 Steane scheme

Steane extraction is tailored to CSS codes and exploits the fact that the  $X$ -type and  $Z$ -type stabilizers arise from classical linear parity checks [58]. Instead of measuring each stabilizer individually, one uses an entire encoded ancilla block. To extract the syndrome of  $Z$ -type stabilizers (equivalently, to diagnose  $X$  errors on the data), prepare an ancilla in the encoded state  $|\bar{0}\rangle$ . Apply a transversal CNOT from the data block to the ancilla block. Then measure the ancilla transversally in the computational basis and classically decode the outcome using the parity-check matrix of the underlying classical code. The measured bit string contains the same syndrome information that would be produced by measuring the  $Z$ -type stabilizers directly.

Similarly, to extract the syndrome of  $X$ -type stabilizers (equivalently, to diagnose  $Z$  errors), prepare  $|\bar{+}\rangle = \bar{H}|\bar{0}\rangle$ , apply a transversal CNOT in the opposite orientation, measure the ancilla transversally in the  $X$  basis, and classically decode the resulting outcomes. The fault tolerance mechanism is twofold:

1. The data-ancilla interaction is transversal, so one gate fault creates at most one error in each block.
2. The ancilla is itself an encoded state, so the syndrome is extracted in a highly redundant form that can be decoded classically.

The subtle point is ancilla preparation. A fault during the preparation of  $|\bar{0}\rangle$  or  $|\bar{+}\rangle$  may create a correlated, high-weight ancilla error. If the ancilla is used without further checks, that correlated error can propagate through the transversal CNOT gates to several data qubits. Therefore, the ancilla must either be *verified* before use or *decoded* afterward so that dangerous correlated faults can be identified and compensated in the Pauli frame. Both approaches appear in the literature; ancilla decoding is especially important when post-selection is expensive. Compared with Shor extraction, Steane extraction uses larger ancillas but fewer rounds of interaction and measurement. On CSS codes such as the Steane code itself, it is usually far more efficient.

### 13.2.3 Knill scheme

Knill's method performs error correction by *teleporting* the logical state through a verified encoded Bell pair [266]. Prepare two encoded blocks in the logical Bell state

$$|\bar{\Phi}^+\rangle = \frac{1}{\sqrt{2}}(|\bar{00}\rangle + |\bar{11}\rangle). \quad (13.13)$$

One half of this Bell pair is then Bell-measured against the incoming data block, using only transversal gates and transversal measurements. As in ordinary quantum teleportation, the unknown logical state is transferred to the second half of the Bell pair, up to a Pauli correction determined by the Bell-measurement outcomes. The key observation is that the Bell measurement simultaneously reveals the error syndrome. In effect, teleportation and error correction are fused into a single step. Any correction can be tracked in software as an update of the *Pauli frame*, so physical recovery operations need not be applied immediately. The Knill scheme has two important practical features:

1. Ancilla verification can be done aggressively *offline*. Bad Bell pairs are discarded before they ever interact with the data.
2. Because the data block is consumed by a teleportation measurement, many faults are converted into known Pauli-frame updates rather than lingering as unknown physical errors.

This often leads to excellent threshold estimates in architectures that can afford abundant ancilla factories. The tradeoff is higher preparation overhead and more demanding ancilla logistics.

| Scheme | Ancilla resource                                                | Fault-tolerance mechanism                                                        | Typical tradeoff                                                    |
|--------|-----------------------------------------------------------------|----------------------------------------------------------------------------------|---------------------------------------------------------------------|
| Shor   | Verified $w$ -qubit cat state for a weight- $w$ stabilizer      | One-to-one ancilla-data coupling; repeated measurement and majority vote         | Broad applicability, but many ancillas and repeated rounds          |
| Steane | Verified encoded $ \bar{0}\rangle$ and $ \bar{+}\rangle$ blocks | Transversal block-wise syndrome transfer plus classical decoding                 | Very efficient for CSS codes, but ancilla preparation is nontrivial |
| Knill  | Verified encoded Bell pairs                                     | Teleportation through a clean ancilla; corrections absorbed into the Pauli frame | High thresholds and flexible scheduling, but large ancilla overhead |

Table 4: Tradeoffs between various syndrome extraction schemes.

### 13.3 Formalism for fault tolerance

To make the threshold theorem precise, one must state exactly what a fault-tolerant gadget is required to do [70, 89]. Let  $\mathcal{C}$  be an  $[[n, k, 2t + 1]]$  code, so that up to  $t$  physical errors are correctable on each block.

#### 13.3.1 Error sets and filters

For a subset  $R \subseteq \{1, \dots, n\}$  of the  $n$  qubit indices, let  $\mathcal{E}(R)$  denote the linear span of Pauli operators supported on  $R$ . The subspace of states with at most  $r$  errors relative to the codespace is

$$\mathcal{V}_r = \sum_{|R| \leq r} \mathcal{E}(R)\mathcal{C}. \quad (13.14)$$

An ideal  $r$ -filter is the projector onto  $\mathcal{V}_r$ . Informally, an  $n$ -qubit block state passes an  $r$ -filter exactly when it differs from some codeword by errors on at most  $r$  qubits. This language is convenient because fault tolerance is not merely a statement about the final decoded state, it is also a statement about the intermediate encoded state remaining inside the correctable sector.

#### Definition 13.3: Fault-tolerant error-correction gadget

An error-correction gadget for an  $[[n, k, 2t + 1]]$  code is  $t$ -fault-tolerant if for every  $r, s \geq 0$  with  $r + s \leq t$  the following hold:

1. **Recovery property.** If the gadget contains at most  $s$  faults, then its output passes an  $s$ -filter.
2. **Correctness property.** If the input passes an  $r$ -filter and the gadget contains at most  $s$  faults, then ideal decoding of the output gives the same state as ideal decoding of the input.

The first property says that faults occurring during error correction leave behind at most a correctable number of residual errors. The second property says that if the total number of pre-existing input errors and faults inside the gadget is at most  $t$ , then the output is correctly decodable to the same logical state. For distance-3 codes ( $t = 1$ ), the definition reduces to two memorable conditions:

1. If one fault occurs during error correction, the output block has at most one error;
2. If the input block has at most one error and the gadget is otherwise fault-free, the output is logically correct.

### 13.3.2 Fault-tolerant gate gadgets

A gate gadget must also satisfy two requirements. First, it must control how pre-existing errors and internal faults propagate to the outputs. Second, after decoding, it must implement the intended logical gate.

#### Definition 13.4: Fault-tolerant gate gadget

Consider a logical gate acting on one or more encoded blocks of an  $[[n, k, 2t + 1]]$  code. A gate gadget is  $t$ -*fault-tolerant* if for every  $r, s \geq 0$  with  $r + s \leq t$ :

1. **Error-propagation property.** If each input block passes an  $r$ -filter and the gadget contains at most  $s$  faults, then each output block passes an  $(r + s)$ -filter.
2. **Logical correctness.** If each input block passes an  $r$ -filter and the gadget contains at most  $s$  faults, then ideal decoding of the output is the same as applying the ideal logical gate to the ideally decoded input. Equivalently, if  $D$  denotes ideal decoding and  $\mathcal{G}_s^U$  denotes any implementation of the logical gate  $U$  with at most  $s$  faults, then logical correctness requires

$$D(\mathcal{G}_s^U(\rho)) = UD(\rho)U^\dagger \quad (13.15)$$

for every admissible input state  $\rho$  satisfying the filter condition above.

For transversal gadgets, the propagation property is usually immediate: a single physical fault affects only the corresponding output qubits, so the number of errors per block grows additively rather than catastrophically.

### 13.3.3 Rectangles and extended rectangles

Threshold proofs are organized around the replacement of each ideal logical location by a level-1 gadget. The gadget implementing a logical gate together with the error-correction gadgets that follow it on every output block is called a *rectangle* (Rec). If one includes the leading error-correction gadgets on the input blocks as well, one obtains an *extended rectangle* (exRec).

The exRec is the natural unit of analysis because faults in the leading error correction can combine with faults inside the gate gadget. A distance-3 exRec is called *good* if it contains at most one fault. The usual exRec-correctness lemma states that every good exRec simulates the ideal logical location correctly. This lemma is the bridge from local gadget properties to the global threshold theorem.

### 13.3.4 Strong and weak fault-tolerant error correction

In the literature, one encounters both *strong* and *weak* notions of fault-tolerant error correction [89, 267]:

- *Strong fault-tolerant EC* requires the recovery property unconditionally: if the EC gadget contains at most  $s$  faults, then regardless of the detailed input state the output lies within the  $s$ -error sector of the code. This notion is highly composable and is especially convenient in recursive proofs.
- *Weak fault-tolerant EC* imposes correctness only for the class of inputs that can arise during a fault-tolerant computation, for example inputs already known to pass an  $r$ -filter for suitable  $r$ . In other words, the gadget is guaranteed to work on *admissible* states even if it need not behave well on arbitrary adversarial inputs.

Weak fault tolerance is often sufficient in practice because encoded gadgets are never fed completely arbitrary states; they are fed outputs of previous gadgets that already satisfy filter conditions. Strong fault tolerance is conceptually cleaner and easier to compose, but it can be more expensive to realize. The distinction matters when optimizing ancilla verification, post-selection, and decoder scheduling.

## 13.4 Noise models

Threshold theorems do not apply to arbitrary noise without assumptions. The meaning of the physical error rate depends on the class of allowed correlations in space and time. Three broad models appear repeatedly.

### 13.4.1 Stochastic noise

The simplest model assigns a fault probability to each location. In an *independent stochastic* model, each location fails independently with probability  $p$ . More general and more useful is *local stochastic noise*: for every set  $R$  of locations [70],

$$\Pr[\text{all locations in } R \text{ are faulty}] \leq p^{|R|}. \quad (13.16)$$

This bound allows arbitrary correlations among faults, provided large correlated sets are exponentially suppressed. The local stochastic model is particularly convenient for combinatorial threshold proofs because the probability of a malignant set of  $m$  faulty locations is at most  $p^m$ . A common special case is stochastic Pauli noise, in which each faulty location is followed by a Pauli channel,

$$\mathcal{N}(\rho) = \sum_{P \in \mathcal{P}_n} q_P P \rho P, \quad \sum_P q_P = 1. \quad (13.17)$$

Because stabilizer propagation is easy to analyze under Pauli channels, this model is ubiquitous in simulations and decoder benchmarking.

### 13.4.2 Non-Markovian noise

A Markovian noise model describes each time step by a completely positive trace-preserving map that depends only on the current system state. In contrast, *non-Markovian* noise allows the environment to retain memory. A standard Hamiltonian model writes

$$H(t) = H_S(t) + H_B(t) + H_{SB}(t), \quad (13.18)$$

| Noise model                  | Typical assumption                                            | What threshold proofs track                               |
|------------------------------|---------------------------------------------------------------|-----------------------------------------------------------|
| Independent                  | Fault sets of size $m$ have probability at most $p^m$         | Counting malignant fault sets inside gadgets              |
| Non-Markovian coherent noise | Local system-bath coupling with small operator norm           | Bounding the norm of the sum of bad fault-path amplitudes |
| Adversarial local noise      | Arbitrary faulty maps subject to locality and strength bounds | Worst-case logical deviation, often in diamond norm       |

Table 5: Comparison of various noise models.

where  $H_{SB}(t)$  couples the computer to its environment. Errors need not be stochastic events with well-defined probabilities; they may interfere coherently across time.

Threshold theorems nevertheless survive under suitable locality assumptions [69]. Roughly speaking, if the system-bath coupling is local and sufficiently weak in operator norm, then the sum of all bad fault paths remains bounded and concatenation still suppresses the effective logical noise. The proof is more delicate because amplitudes, not probabilities, are combined, and coherent cancellation or reinforcement must be controlled.

### 13.4.3 Adversarial noise

The strongest model allows the faulty operation at each bad location to be chosen adversarially, possibly depending on the entire fault pattern, subject only to a locality or norm bound. This worst-case viewpoint is often formulated in terms of the diamond norm distance between the implemented and ideal operations.

Adversarial noise is mathematically severe: one no longer assumes that errors are random Pauli flips or even stochastic events. The strength of fault tolerance in this regime comes from geometry and locality rather than from any favorable probabilistic structure. Rigorous thresholds exist for several adversarial or coherent models, but the proven threshold constants are generally smaller than the thresholds observed under phenomenological stochastic noise.

## 13.5 Threshold theorem

The discovery of QEC was a monumental step, but it introduced a seemingly fatal paradox: the very circuits used to perform error correction (encoding, syndrome measurement, and recovery) are themselves composed of noisy physical gates. This raises a critical question: does the process of error correction introduce more errors than it fixes? The answer, which provides the theoretical foundation for all of scalable quantum computing, is given by the *threshold theorem*. Informally, it states:

If the error rate  $p$  of each physical component in a quantum computer (qubit preparation, gates, measurements, memory) is below a certain constant value, the accuracy threshold  $p_{\text{th}}$ , then it is possible to perform an arbitrarily long quantum computation with an arbitrarily low logical error rate  $\epsilon$ .

The overhead required to achieve this fault tolerance, in terms of the number of physical qubits and gates, scales efficiently, typically as a polynomial in the logarithm of the circuit size and  $1/\epsilon$ . The theorem is best understood through a quantitative recursion. Let  $p_L^{(\ell)}$  denote the failure probability of a level- $\ell$  exRec, and let  $A$  be an upper bound on the number of malignant sets of size  $t+1$  inside the largest level-1 exRec. Then for sufficiently small  $p$ ,

$$p_L^{(1)} \leq Ap^{t+1} + O(p^{t+2}). \quad (13.19)$$

At higher concatenation levels the same reasoning applies recursively,

$$p_L^{(\ell+1)} \leq A(p_L^{(\ell)})^{t+1}. \quad (13.20)$$

The logic is simple: a level- $(\ell+1)$  exRec can fail only if enough level- $\ell$  sub-gadgets fail in a malignant pattern.

### 13.5.1 Distance-3 case

For the most common pedagogical case  $t = 1$ , [Eq. \(13.20\)](#) becomes

$$p_L^{(\ell+1)} \leq A(p_L^{(\ell)})^2. \quad (13.21)$$

Defining  $\lambda = Ap$ , one obtains by induction

$$p_L^{(\ell)} \leq \frac{1}{A} \lambda^{2^\ell}. \quad (13.22)$$

Hence, if  $\lambda < 1$ , or equivalently if

$$p < p_{\text{th}} \approx \frac{1}{A}, \quad (13.23)$$

then the logical error rate decreases doubly exponentially with the concatenation level. This is the hallmark of a threshold.

### 13.5.2 General $t$

For a code correcting  $t$  errors, rescale

$$y_\ell = A^{1/t} p_L^{(\ell)}. \quad (13.24)$$

Then, [Eq. \(13.20\)](#) implies  $y_{\ell+1} \leq y_\ell^{t+1}$ , so

$$p_L^{(\ell)} \leq A^{-1/t} (A^{1/t} p)^{(t+1)^\ell}. \quad (13.25)$$

Thus, the threshold estimate is

$$p_{\text{th}} \approx A^{-1/t}, \quad (13.26)$$

up to higher-order corrections and model-dependent constants.

### 13.5.3 Why the overhead is efficient

Suppose an ideal computation contains  $N$  logical locations. If each level- $\ell$  location fails with probability at most  $p_L^{(\ell)}$ , then the total failure probability is at most  $Np_L^{(\ell)}$  by the union bound. Requiring  $Np_L^{(\ell)} \leq \epsilon$  and using Eq. (13.25) shows that it suffices to choose

$$(t+1)^\ell = O\left(\log \frac{N}{\epsilon}\right), \quad \ell = O\left(\log \log \frac{N}{\epsilon}\right). \quad (13.27)$$

If one level of encoding uses  $n$  physical qubits per logical qubit, then  $\ell$  levels use  $n^\ell$  physical qubits, so the space overhead scales as

$$n^\ell = \left(\log \frac{N}{\epsilon}\right)^{\log_{t+1} n}, \quad (13.28)$$

which is polynomial in  $\log(N/\epsilon)$ . The gate overhead obeys a similar polylogarithmic bound. We note that the constant  $p_{\text{th}}$  is not universal. It depends on the code, the gadget construction, the decoder, the connectivity constraints, and the noise model. Rigorous threshold values derived from malignant-set counting are typically more conservative than thresholds estimated numerically for specific architectures. Concatenation is the recursive architecture that turns the local gadget properties of the previous sections into asymptotically reliable computation [24, 70, 89].

## 13.6 Recursive encoding

Let  $\mathcal{C}$  be an  $[[n, k, d]]$  code. In one level of encoding, each logical block is mapped to  $n$  physical qubits. In two levels of concatenation, each of those  $n$  qubits is itself re-encoded using the same code. Continuing recursively, a level- $\ell$  encoded block uses  $n^\ell$  physical qubits. For self-concatenation of an  $[[n, 1, d]]$  code, the effective parameters are  $[[n^\ell, 1, d^\ell]]$ . The multiplicative distance law follows because an error must cause a logical fault at the outer level and, within each affected outer-level qubit, must also cause a logical fault at the next inner level, and so on recursively. As an example, concatenating the Steane  $[[7, 1, 3]]$  code twice yields a  $[[49, 1, 9]]$  code, and concatenating by  $\ell$  levels gives a  $[[7^\ell, 1, 3^\ell]]$  code.

## 13.7 Recursive gadgets

Concatenation is not merely recursive state encoding. Every logical location in the ideal circuit is replaced by a level-1 gadget. Then every physical location inside that gadget is replaced by its own level-1 gadget to create a level-2 gadget, and so on. Thus a level- $\ell$  encoded gate is a gadget built from level- $(\ell-1)$  encoded gates, measurements, preparations, and error-correction steps. This hierarchical construction ensures self-similarity: once one proves that good level-1 exRecs are correct, the same argument applies at every higher level because a level- $\ell$  exRec behaves as a single effective location for the level- $(\ell+1)$  simulation.

## 13.8 Why concatenation works

Suppose a code corrects one error and the level-1 gadgets are designed so that any single fault inside an exRec is harmless. Then a logical failure at level 1 requires at least two faults in a malignant arrangement. At level 2, a logical failure requires at least two level-1 exRec failures, each of which already requires two physical faults, so the dominant contribution is fourth order. Repeating this recursively yields the double-exponential suppression in Eq. (13.22). This recursive suppression is the conceptual heart of concatenated fault tolerance: *the effective error model becomes progressively more benign at higher levels of encoding, provided the physical noise starts below threshold.*



### 13.9 Why transversality is powerful

Suppose a code corrects one error per block. If a single  $X$  error enters a transversal CNOT on the control block, the error may spread to the corresponding qubit in the target block, but each block still contains only one error. The gate is therefore compatible with distance-3 correction. This is exactly the right notion of safe error spread: faults may duplicate across blocks, but they may not proliferate *within* a block. Many CSS codes admit useful transversal gates. For example:

- transversal CNOT is logical CNOT for any CSS code built from nested classical codes;
- self-dual CSS codes often admit transversal Hadamard;
- doubly-even CSS codes can admit a transversal phase gate, sometimes up to adjoint convention, as in the Steane code.

Despite their usefulness, transversal gates alone can never provide a universal gate set on a finite-dimensional quantum code. In fact, the *Eastin-Knill theorem* says that no quantum error-correcting code that detects arbitrary single-subsystem errors can realize a universal set of logical gates using only transversal unitary gates [159]. It does not say that transversal gates are rare or unhelpful; rather, it says that *any* finite code with nontrivial error-detecting power must sacrifice universality if one insists on transversality alone. In practice, this means that a fault-tolerant architecture needs an additional ingredient beyond transversal Clifford operations. Common remedies include magic-state distillation and injection, code switching, gauge fixing, lattice surgery, and pieceable fault tolerance.

Transversality is a special case of a broader geometric concept: *locality-preserving logical gates*. These are logical operations that map local errors to local errors and can therefore be implemented without long-range spread of damage. For topological stabilizer codes, such gates are severely constrained. The Bravyi-König theorem says that for a  $D$ -dimensional topological stabilizer code, every locality-preserving logical gate belongs to the  $D^{\text{th}}$  level of the Clifford hierarchy [268]. The consequence is especially striking in two dimensions. In a two-dimensional topological stabilizer code, every locality-preserving logical gate is Clifford. Therefore, no two-dimensional topological stabilizer code can realize a universal fault-tolerant gate set by transversal or any other locality-preserving mechanism alone. In three dimensions one may obtain logical gates from the third level of the Clifford hierarchy, such as CCZ in suitable codes, but even that is still not universal by itself.

Transversal gates are indispensable because they are simple, local, and naturally fault-tolerant. But the Eastin-Knill and Bravyi-König theorems show that fault-tolerant universality cannot come from transversality alone. Modern architectures therefore combine a “cheap” fault-tolerant gate set, usually Clifford operations, with a more expensive resource-generation procedure that supplies the missing non-Clifford ingredients.

## 14 Fault tolerance II

In this second part on fault tolerance, we will cover three topics: (1) the seminal work on a constant-overhead FTQC scheme that uses a family of qLDPC codes in [Sections 14.1 to 14.4](#), (2) a magic state distillation scheme that distills 10 noisy magic states to two magic states with suppressed error in [Section 14.5](#), (3) code-switching protocols that allows for an implementation of a universal transversal gate set by using two codes.

We will first discuss the proof of the existence of a decoding threshold for exponential error suppression in the code distance for qLDPC codes in the case of ideal syndrome extraction ([Section 14.2](#)) and noisy syndrome extraction ([Section 14.3](#)). Then, in [Section 14.4](#), we will discuss the overall FTQC scheme and derive a threshold for the entire scheme in detail. Next, in [Section 14.5](#) we will discuss a simple magic state distillation scheme based on a 4-qubit error-detecting code and show how it distills 10 copies of noisy magic states to two magic states with quadratic suppression of their noise rate. Lastly, in [Section 14.6](#), we will discuss two code-switching protocols as an alternative protocol to magic state distillation to enable a universal FTQC.

### 14.1 Constant-overhead FTQC scheme using qLDPC codes

Here we discuss the FTQC scheme with a family of qLDPC codes with non-vanishing asymptotic encoding and distance rate proposed in [\[229\]](#). Namely, we consider a family of qLDPC codes  $Q_1, Q_2, \dots$  with parameters  $[[n_i, k_i, d_i]]$  such that each code has a non-vanishing asymptotic encoding rate  $R = \lim_{i \rightarrow \infty} \frac{k_i}{n_i} > 0$  and a non-constant distance  $\lim_{i \rightarrow \infty} d_i = \infty$ . We will first discuss two important results showing that there exists a decoder for such a family of codes such that errors are suppressed exponentially in the code distance  $d_i$  whenever physical error rates are below some threshold values. First, in [Section 14.2](#) we will show a decoding threshold when the syndrome extraction circuit is assumed to be ideal. Then, in [Section 14.3](#) we will show that such decoding threshold still exists even when the syndrome extraction circuit is noisy, particularly by doing multiple cycles of syndrome extraction. Lastly in [Section 14.4](#), we will give a detailed proof on the existence of a threshold for this constant-overhead FTQC scheme.

Here we will consider a *local stochastic noise* model where the probability of faults of a certain size occurring decays exponentially with the fault size.

#### Definition 14.1: Local stochastic noise

For a quantum circuit  $\mathcal{C}$  with locations  $\text{loc}(\mathcal{C})$ , a local stochastic noise model with rate  $p$  implies that the probability of faults on all locations in a subset  $S \subseteq \text{loc}(\mathcal{C})$  is given by:

$$\Pr[\text{All locations in } S \text{ are faulty}] \leq p^{|S|}. \quad (14.1)$$

For an  $n$ -qubit system, a local stochastic noise with rate  $p$  implies that the probability of faults on all qubits in a set  $S \subseteq [n]$  is given by

$$\Pr[\text{All qubits in } S \text{ are faulty}] \leq p^{|S|}. \quad (14.2)$$

### 14.2 Decoding threshold for ideal syndrome extraction

Theorem 3 of Ref. [\[229\]](#) shows that for a family  $\{Q_i\}_i$  of  $(r, c)$ -qLDPC codes with parameters  $[[n_i, k_i, d_i]]$  with  $n_i \rightarrow \infty$ ,  $d_i \rightarrow \infty$ , local stochastic noise with rate  $p$ , there exists a decoding algorithm and a threshold  $p_0 > 0$  such that the logical error rate is  $p_L = O(n_i(p/p_0)^{d_i/2})$  whenever

$p < p_0$ . This result generalizes the result by Kovalev and Pryadko [269] on the existence of a threshold for such qLDPC codes under independent noise model to local stochastic adversarial noise model. We go through the proof of this result.

To obtain this threshold, we first consider a decoding algorithm that receives a syndrome string  $s(F) \in \mathbb{Z}_2^l$  (corresponding to generators  $g_1, \dots, g_l$  of stabilizer group  $\mathcal{S}_i$ ) from Pauli error  $F \in \hat{\mathcal{P}}_{n_i}$  and outputs a correction Pauli operator  $E \in \hat{\mathcal{P}}_{n_i}$  with lowest weight that is consistent with syndrome  $s(F)$  (choosing one at random if there are multiple minimum-weight consistent Paulis). This decoder is successful whenever  $EF \in \mathcal{S}_i$  and fails whenever  $EF \in \mathcal{N}(\mathcal{S}_i) \setminus \mathcal{S}_i$ , since  $EF$  is a logical operation and has a trivial syndrome as  $s(EF) = s(E) + s(F) = 0^l$  because  $s(E) = s(F)$ . When the latter happens, the decoding procedure results in a logical error. Now we introduce a graph we call the *code adjacency graph* that captures how the qubits are connected by the stabilizers.

**Definition 14.2:** Code adjacency graph

For an  $n$ -qubit stabilizer code  $Q$  with stabilizer generators  $g_1, \dots, g_l$ , a simple graph  $G$  is the *code adjacency graph* of  $Q$  if its vertices  $V(G)$  and edges  $E(G)$  are given by

$$\begin{aligned} V(G) &= [1, n] \quad \text{and} \\ E(G) &= \{(u, v) : \exists i \in [1, l], u, v \in \text{supp}(g_i)\}. \end{aligned} \tag{14.3}$$

Namely, the vertices of  $G$  are labels for the  $n$  qubits and two vertices are connected by an edge iff the two corresponding qubits are in the support of a stabilizer generator.

For an  $(r, c)$ -qLDPC code, each qubit is involved in at most  $c$  generators and each generator has support on at most  $r$  qubits. Therefore, the degree of each vertex  $u$  in the adjacency graph  $G$  of an  $(r, c)$ -qLDPC as defined in Definition 14.2 is bounded as

$$\deg(u) \leq (r - 1)c. \tag{14.4}$$

Note also that a generator  $g_i$  corresponds to a clique in  $G$ , since qubits involved in  $g_i$  are connected to each other.

Now, let us call a subset of vertices  $W \subseteq V$  a *cluster* if the induced subgraph  $G(W)$  is connected. For a Pauli  $P \in \hat{\mathcal{P}}_{n_i}$ , we say that a cluster  $W \subseteq \text{supp}(P)$  is a *disjoint cluster* of  $P$  whenever there are no qubits in  $\text{supp}(P)$  that are in the border of  $W$ , that is,  $\forall v \in \text{supp}(P) \setminus W, \forall u \in W, (u, v) \notin E(G)$ . A disjoint cluster  $W$  of  $P$  is *minimal* if there is no  $N \subsetneq W$  such that  $N$  is a disjoint cluster of  $P$ . Let us denote the set of all minimal disjoint cluster of  $P$  as  $\text{MDC}(P)$ . Now we will show the following lemma that connects failure of the decoding scheme above to the structure of errors across the  $n_i$  qubits.

**Lemma 14.1:** Decoder failure

If the decoder fails, then there exists a set of connected qubits  $W \subseteq V(G)$  of size  $|W| \geq d_i$  such that at least  $|W|/2$  of qubits in it are faulty.

*Proof.* Suppose that the decoding fails, that is, it chooses a correction Pauli  $E$  such that  $EF \in \mathcal{N}(\mathcal{S}) \setminus \mathcal{S}$ . Then, consider a cluster  $M \in \text{MDC}(EF)$  and  $EF|_M$ , the restriction of  $EF$  to qubits in cluster  $M$ . Note that  $EF|_M$  has the following properties:

1.  $\text{supp}(EF|_M) = M$ , since if there is a qubit  $q \in M$  that is not in  $\text{supp}(EF|_M)$  then we can always find a subset  $N \subseteq M \setminus \{q\}$  that is a disjoint cluster of  $EF$ , contradicting that  $M$  is minimal.

2.  $s(EF|_M) = 0^l$  (it has a trivial syndrome). Restriction  $EF|_M$  corresponds to  $l' \leq l$  syndromes (say,  $g_1, \dots, g_{l'}$ ) while  $EF|_{V \setminus M}$  corresponds to the remaining  $l - l'$  (say,  $g_{l'+1}, \dots, g_l$ ). This is because  $\text{supp}(EF|_M)$  is disconnected from  $\text{supp}(EF|_{V \setminus M})$ , that is,  $(u, v) \notin E$  for all  $u \in \text{supp}(EF|_M)$  and  $v \in \text{supp}(EF|_{V \setminus M})$  (by the definition of a disjoint cluster). So,  $s(EF|_M)$  is a length- $l'$  sub-string of  $s(EF) = 0^l$ .
3.  $EF|_M \in \mathcal{N}(\mathcal{S}_i)$ . The only generators with support on  $M$  are  $g_1, \dots, g_{l'}$ , each giving syndrome 0. So  $EF|_M$  commutes with each of these  $l'$  generators, thus commute with all generators.
4. There exists  $M \in \text{MDC}(EF)$  where  $EF|_M \notin \mathcal{S}_i$  (it is not a stabilizer). If there is no such  $M$ , then all  $M \in \text{MDC}(EF)$  is a stabilizer, which means that  $EF$  is also a stabilizer, contradicting the assumption that the decoding has failed.

Consider  $W \in \text{MDC}(EF)$  such that  $EF|_W \notin \mathcal{S}_i$  (c.f., property 4 above). By property 3, it also holds that  $EF|_W \in \mathcal{N}(\mathcal{S}_i)$ . Thus, for any such  $W \in \text{MDC}(EF)$ , it holds that  $|W| \geq d_i$  since  $EF|_W \in \mathcal{N}(\mathcal{S}_i) \setminus \mathcal{S}_i$ , that is,  $EF|_W$  is a non-trivial logical operator of  $\mathcal{S}_i$  so that its weight must be at least the code distance  $d_i$  since otherwise it is detectable. Now we show that there are at least  $|W|/2$  faulty qubits in  $W$ , that is,  $\text{wt}(F|_W) \geq |W|/2$ . First, we show that  $\text{wt}(E|_W) \leq \text{wt}(F|_W)$ . The decoder always chooses a minimum weight Pauli  $E$  satisfying  $s(E) = s(F)$  so that

$$\text{wt}(E|_W) + \text{wt}(E|_{V \setminus W}) = \text{wt}(E) \leq \text{wt}(F) = \text{wt}(F|_W) + \text{wt}(F|_{V \setminus W}). \quad (14.5)$$

If  $\text{wt}(E|_W) > \text{wt}(F|_W)$  then we can choose  $E'$  such that  $E'|_W = F|_W$  and  $E'|_{V \setminus W} = E|_{V \setminus W}$  so that  $s(E') = s(F|_W) + s(E|_{V \setminus W}) = s(F|_W) + s(F|_{V \setminus W}) = s(F)$  and  $\text{wt}(E') < \text{wt}(E)$  ( $s(E|_{V \setminus W}) = s(F|_{V \setminus W})$  follows from property 2 above that  $s(EF|_{V \setminus W}) = 0$ ). This contradicts that  $E$  is a minimum weight error with syndrome equal to  $s(F)$ , therefore  $\text{wt}(E|_W) \leq \text{wt}(F|_W)$  must be true. So,

$$|W| = \text{wt}(EF|_W) \leq \text{wt}(E|_W) + \text{wt}(F|_W) \leq 2\text{wt}(F|_W) \quad (14.6)$$

which implies that  $\text{wt}(F|_W) \geq |W|/2$ , completing the proof  $\square$

We have shown that a decoding failure implies that there exists a set of connected qubits  $W \subseteq V(G)$  of size  $|W| \geq d_i$  such that at least  $|W|/2$  of the qubits in it are faulty. Thus, the probability of a logical error resulting from decoding failure for a Pauli error  $F$  and (incorrect) correction Pauli  $E$  can be bounded as

$$\begin{aligned} \Pr[\text{Logical error}] &= \Pr[EF \in \mathcal{N}(\mathcal{S}_i) \setminus \mathcal{S}_i] \\ &\leq \Pr\{\exists W \subseteq V(G) : W \text{ connected}, |W| \geq d_i, \text{wt}(F|_W) \geq \lceil |W|/2 \rceil\} \\ &\leq \sum_{\substack{W \subseteq V(G): \\ |W| \geq d_i, \\ W \text{ connected}}} \Pr[\text{wt}(F|_W) \geq \lceil |W|/2 \rceil], \end{aligned} \quad (14.7)$$

by using a union bound to obtain the second inequality. To obtain an upper bound for the logical error in terms of physical error rate  $p$  and the code parameters, we need to put a bound on the number of the connected sets in the sum of Eq. (14.7) and the probability of more than half erroneous qubits in a set.

For a qubit  $v \in V(G)$ , the number of subsets of connected qubits  $W \subseteq V(G)$  of size  $|W| = q$  containing  $v$  can be upper bounded using Lemma 2 of Ref. [229] as

$$|\{W \subseteq V(G) : |W| = q, W \text{ connected}, v \in W\}| \leq \deg(v)e^{q-1}. \quad (14.8)$$

Since there are  $n_i$  qubits in total and the degree of each qubit is bounded by  $(r-1)c$  (c.f. Eq. (14.4)), therefore the total number of connected subsets of size  $q$  in  $G$  is bounded as

$$|\{W \subseteq V(G) : |W| = q, W \text{ connected}\}| \leq n_i((r-1)ce)^{q-1}. \quad (14.9)$$

On the other hand, for a subset  $W \subseteq V(G)$ , the probability that there are  $\lceil |W|/2 \rceil$  erroneous qubits in  $W$  is bounded by

$$\begin{aligned} \Pr[\lceil q/2 \rceil \text{ errors in } W] &\leq \sum_{T \subseteq W : |T| = \lceil |W|/2 \rceil} \Pr[\text{all qubits in } T \text{ are faulty}] \\ &\leq \binom{q}{\lceil q/2 \rceil} p^{\lceil q/2 \rceil} \leq 2^q p^{\lceil q/2 \rceil} \end{aligned} \quad (14.10)$$

by the union bound and the local stochastic noise assumption (c.f. Definition 14.1).

Thus, we can further bound the inequality in Eq. (14.7) as

$$\begin{aligned} \Pr[\text{Logical error}] &\leq \sum_{\substack{W \subseteq V(G): \\ |W| \geq d_i, \\ W \text{ connected}}} \Pr[\text{wt}(F|_W) \geq \lceil |W|/2 \rceil] \\ &\leq \sum_{q=d_i}^{\infty} |\{W \subseteq V(G) : |W| = q, W \text{ connected}, v \in W\}| \\ &\quad \times \Pr[\lceil q/2 \rceil \text{ errors in } W] \\ &\leq \sum_{q=1}^{\infty} n_i((r-1)ce)^{q-1} 2^q p^{\lceil q/2 \rceil} \\ &\leq \frac{n_i}{(r-1)ce} \sum_{q=1}^{\infty} (2(r-1)ce\sqrt{p})^q. \end{aligned} \quad (14.11)$$

The series  $\sum_{q=1}^{\infty} (2(r-1)ce\sqrt{p})^q$  is a geometric series and therefore converges to  $\frac{(2(r-1)ce\sqrt{p})^{d_i}}{1-(2(r-1)ce\sqrt{p})}$  whenever  $2(r-1)ce\sqrt{p} < 1$ . So we can define the threshold as  $p_0 = (2(r-1)ce)^{-2}$  so that whenever  $p < p_0$  we have  $(2(r-1)ce\sqrt{p}) < 1$  and by denoting  $\alpha = (r-1)ce$  we have

$$\Pr[\text{Logical error}] \leq \frac{n_i}{\alpha} \frac{(2\alpha\sqrt{p})^{d_i}}{1-(2\alpha\sqrt{p})} \leq \frac{n_i}{\alpha(1-(2\alpha\sqrt{p}))} \left(\frac{p}{p_0}\right)^{d_i/2}, \quad (14.12)$$

since  $2\alpha\sqrt{p} = \sqrt{p/p_0}$ . So, the logical error probability  $p_L$  is bounded as  $p_L \leq O(n_i(p/p_0)^{d_i/2})$  as claimed (the  $\sqrt{p}$  in the denominator is absorbed into the  $O$  and  $\alpha = (r-1)ce$  is a constant independent of the code parameter).

### 14.3 Decoding threshold for noisy syndrome extraction

In the last section, we showed that a family of qLDPC codes  $[[n_i, k_i, d_i]]$  with growing physical qubits and distance ( $n_i \rightarrow \infty$  and  $d_i \rightarrow \infty$  as  $i \rightarrow \infty$ ) can be decoded with exponential logical error suppression whenever the physical error rate is below a certain threshold, under local stochastic noise model. However, this assumes that the syndrome extraction process itself is noiseless, that is, the syndrome extraction (SE) circuit implementing stabilizer measurements is ideal. In a realistic scenario, components inside the SE circuit can themselves be faulty. To model a realistic scenario

more faithfully, besides qubit errors on the input state, we also need to consider a circuit-level noise model in the QEC procedure, which consists of both qubit errors and measurement errors in the SE circuit.

Theorem 4 of [229] shows that even if SE circuits are noisy, a qLDPC code family whose parameters satisfy the aforementioned properties can also be decoded with exponential logical error suppression given that initial error rate per qubit  $\tilde{p}$ , error rate per qubit at each SE cycle  $p$ , and bit-flip error rate per syndrome bit  $q$  are below certain thresholds, under the circuit-level local stochastic noise model. Such error suppression is obtained when we repeat the SE circuit  $T = d_i$  many times where syndromes between subsequent SE cycles are compared to infer syndromes reliably. In this section, we discuss the proof of this result in detail.

### 14.3.1 Errors in the syndrome extraction circuit

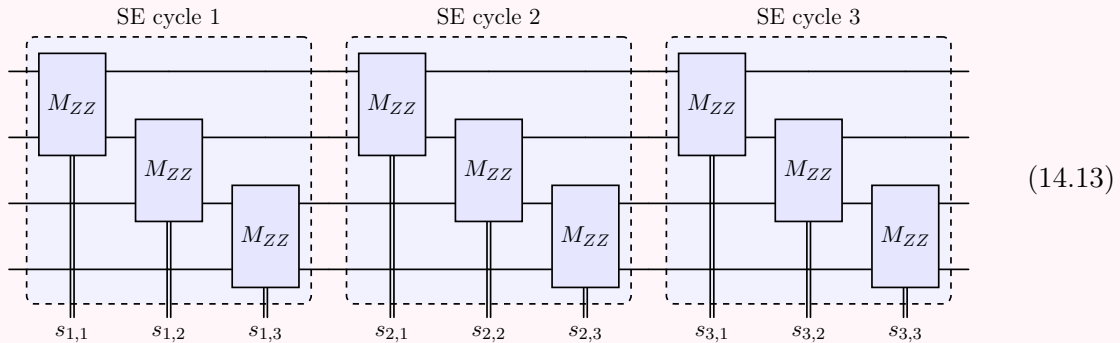
As mentioned above, we first need to consider input errors, syndrome bit errors (resulting from faulty measurements), and gate errors. For this, we consider the following errors at each step  $t \in [1, T]$  in the SE cycle:

1.  $F_t \in \mathcal{P}_n$  the actual  $n$ -qubit physical Pauli errors at timestep  $t$  and  $E_t \in \mathcal{P}_n$  physical Pauli errors deduced by the decoder (i.e., which physical errors the decoder thinks happened at round  $t$ ).
2.  $B_t \in \mathbb{Z}_2^l$  the set of actual syndrome bit errors at timestep  $t$  and  $C_t \in \mathbb{Z}_2^l$  are the sets of syndrome bit errors deduced by the decoder (i.e., which bit-flip errors the decoder thinks happened in round  $t$ ).
3.  $F_1$  and  $E_1$  are actual and deduced input errors, respectively.

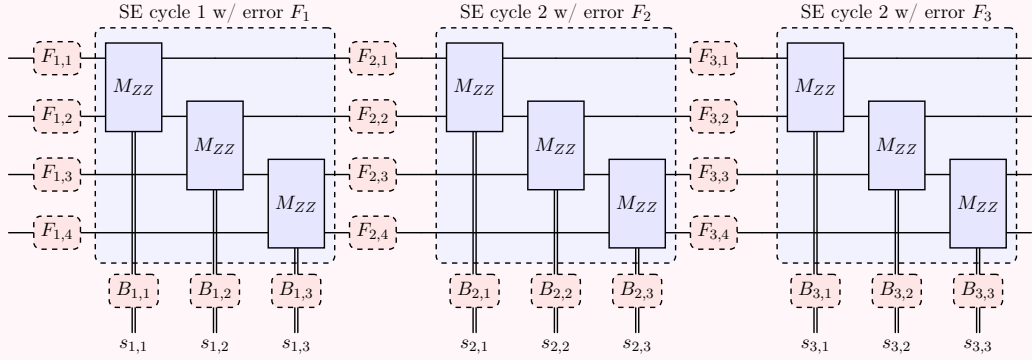
Physical errors in  $F_t$  are those that can trigger a detection in round  $t$  SE. We can express physical error  $F_t$  and syndrome bit error  $B_t$  as  $F_t = F_{t,1} \otimes \cdots \otimes F_{t,n}$  and  $B_t = (B_{t,1}, \dots, B_{t,l})$  where  $F_{t,q}$  are Pauli errors on qubit  $q$  at timestep  $t$  and  $B_{t,j}$  are bit-flip errors on syndrome measurement for stabilizer generator  $g_j$  at round  $t$ . To illustrate this, consider the following example.

#### Example 14.1: Repetition code syndrome extraction circuit

A 3-round SE circuit for the 4-qubit bit-flip repetition code is given by



A noisy implementation of this 3-round SE circuit is therefore given by



Now, let  $\sigma(F_t) \in \mathbb{Z}_2^l$  and  $\sigma(E_t) \in \mathbb{Z}_2^l$  be the syndromes for Pauli errors  $F_t$  and  $E_t$ , respectively, before accounting for syndrome bit errors.  $\sigma$  maps Pauli errors to bit strings so that the syndrome of the product of two errors is the sum of their individual syndromes, that is,  $\sigma(FF') = \sigma(F) + \sigma(F')$ . In other words,  $\sigma$  is a homomorphism from Pauli errors to bit strings. Note that the syndrome  $s_t$  that we observe from timestep  $t$  SE takes into account the cumulative errors  $D_{t-1} = F_{t-1} \dots F_1$  from timesteps 1 to  $t-1$  as well as timestep- $t$  physical errors  $F_t$  and measurement errors  $B_t$ . So the syndrome with perfect measurement from time- $t$  SE is given by  $\sigma(F_t D_{t-1}) = \sigma(F_t) + \sigma(D_{t-1})$  and thus

$$s_t = \sigma(F_t D_{t-1}) + B_t = \sigma(F_t) + \sigma(D_{t-1}) + B_t. \quad (14.15)$$

We call  $s_t$  the *observed* syndrome from SE at timestep  $t$  (note that  $s_t$  depends on  $F_1, \dots, F_t$  and  $B_t$ , but not on  $B_1, \dots, B_{t-1}$  since these are errors on syndrome measurements from timestep 1 to  $t$ , which do not affect the measurement outcomes at timestep  $t$ ). To further clarify this, consider the following example.

**Example 14.2:** Observed syndromes and cumulative errors

Consider a syndrome extraction for a 4-qubit repetition code illustrated in [Example 14.1](#) stabilizer generators  $g_1, g_2, g_3$ . Note that the cumulative errors at the end of timestep 1 is  $D_1 = F_1$ , and suppose that it flips  $g_1$  and  $g_2$  so that  $\sigma(F_1) = (1, 0, 1)$  where there is no measurement errors, that is,  $B_1 = (0, 0, 0)$ . Thus, the syndromes observed at  $t = 1$  SE is

$$s_1 = \sigma(F_1) + B_1 = \sigma(F_1) = (1, 0, 1). \quad (14.16)$$

If there are no errors between timesteps 1 and 2, that is,  $F_2 = I$  and  $\sigma(F_2) = (0, 0, 0)$ , and for measurement errors  $B_2 = (0, 0, 1)$  in the second round of SE, then the observed syndrome at  $t = 2$  is

$$s_2 = \sigma(F_2 D_1) + B_2 = \sigma(F_1) + B_2 = (1, 0, 0). \quad (14.17)$$

Namely, the observed syndrome  $s_2$  from  $t = 2$  SE is the observed syndrome from SE at  $t = 1$  with its third bit flipped due to measurement error  $B_2 = (0, 0, 1)$ .

The observed syndrome from timestep- $t$  SE cycle is given by  $\sigma(F_t) + B_t$ , where syndrome bit errors  $B_t$  are taken into account. Also, we define the change between observed syndromes at timestep



$t - 1$  and  $t$  as

$$\Delta_t = \begin{cases} s_{t-1} + s_t & , \quad t \geq 2 \\ s_1 & , \quad t = 1. \end{cases} \quad (14.18)$$

Since  $D_t = F_t D_{t-1}$  therefore we can write the observed syndrome change  $\Delta_t$  as

$$\Delta_t = \begin{cases} \sigma(F_t) + B_t + B_{t-1} & , \quad t \geq 2 \\ \sigma(F_t) + B_t & , \quad t = 1, \end{cases} \quad (14.19)$$

where the syndrome at timestep  $t = 1$  is compared to the all-zero syndrome, that is, the syndrome corresponding to an ideal initial codestate  $|\bar{\psi}\rangle$ . Syndrome change  $\Delta_t = \sigma(F_t) + B_t + B_{t-1}$  is basically the difference between observed syndromes between  $t - 1$  and  $t$  SE rounds.

**Example 14.3:** Observed syndrome change

Continuing [Example 14.2](#), recall that the syndromes due to physical errors  $F_1$  and  $F_2$  are given by  $\sigma(F_1) = (1, 0, 1)$  and  $\sigma(F_2) = (0, 0, 0)$ , respectively. Note that syndrome change  $\Delta_2$  takes into account *new* errors  $F_2$  between timesteps 1 and 2, as well as measurement errors  $B_1 = (0, 0, 0)$  and  $B_2 = (0, 0, 1)$ . So, it is given by

$$\Delta_2 = s_1 + s_2 = (\sigma(F_1) + B_1) + (\sigma(F_2 D_1) + B_2) = \sigma(F_2) + B_1 + B_2 = (0, 0, 1), \quad (14.20)$$

indicating that the syndrome  $g_3$  has been flipped (in this case due to measurement error  $B_2$ ). Now suppose that at  $t = 3$  SE round, there are physical error  $F_3$  such that  $\sigma(F_3) = (0, 1, 0)$  and measurement error  $B_3 = (0, 1, 0)$  so that we observe syndrome  $s_3 = (0, 0, 0)$ . Recall that  $s_2 = \sigma(F_2 D_1) + B_2 = (1, 0, 0)$ , which means that the first syndrome bit is flipped from  $t = 2$  to  $t = 3$ , that is,  $s_2 + s_3 = (1, 0, 0)$ .

Let us now verify this using the syndrome change formula  $\Delta_3 = \sigma(F_3) + B_3 + B_2$  which takes into account *new* error  $F_3$  between timestep 2 and 3, as well as measurement errors  $B_2$  and  $B_3$ . Since  $D_2 = F_2 D_1$ , we have

$$(1, 0, 0) = s_2 + s_3 = \Delta_3 = \sigma(F_3) + B_2 + B_3 = (1, 0, 0), \quad (14.21)$$

which indicates that we observe that the first syndrome bit is flipped.

Syndrome change  $\Delta_t$  provides information to the decoder about any errors between timestep  $t$  and  $t - 1$ . Up to the point after SE at timestep  $t$  is completed, the decoder have deduced errors  $E_1, E_2, \dots, E_t$  consistent with observed syndromes  $s_1, \dots, s_t$ , namely

$$\begin{aligned} \sigma(F_1) + B_1 &= s_1 = \sigma(E_1) + C_1, \\ \sigma(F_2 F_1) + B_2 + B_1 &= s_2 = \sigma(E_2 E_1) + C_2 + C_1, \\ &\vdots \\ \sigma(F_t \dots F_1) + B_t + B_{t-1} &= s_t = \sigma(E_t \dots E_1) + C_t + C_{t-1}. \end{aligned} \quad (14.22)$$

Since  $\sigma(F_t \dots F_1) = \sigma(F_t) + \sigma(F_{t-1} \dots F_1)$  and  $\sigma(E_t \dots E_1) = \sigma(E_t) + \sigma(E_{t-1} \dots E_1)$ , we obtain a consistency relation between actual errors and deduced errors:

$$\begin{aligned} \sigma(F_t) + B_t + B_{t-1} &= \Delta_t = \sigma(E_t) + C_t + C_{t-1} \quad , \quad \forall t \geq 2 \\ &\text{and} \\ \sigma(F_1) + B_1 &= \Delta_1 = \sigma(E_1) + C_1. \end{aligned} \quad (14.23)$$



Errors  $E_1, \dots, E_T$  deduced by the decoders over the  $T$  SE cycles are the effective correction performed by the decoder. So over the  $T$  SE cycles, the overall effect of both errors and corrections are given by  $\prod_{t=1}^T E_t F_t$ , where  $E_t F_t$  is the discrepancy between what the decoder thinks happened at round  $t$ , that is,  $E_t$ , and the actual error  $F_t$ . Thus  $\prod_{t=1}^T E_t F_t$  is the overall discrepancy in the entire  $T$ -cycle SE circuit. So, if the decoder deduced all errors accurately, that is,  $E_t = F_t$ , then there would be no discrepancy at the end of the SE cycles, that is,  $\prod_{t=1}^T E_t F_t = \prod_{t=1}^T F_t F_t = I$ .

Now note that by consistency condition in Eq. (14.23), we have

$$\begin{aligned} \sigma(F_t E_t) &= \sigma(F_t) + \sigma(E_t) = B_t + B_{t-1} + C_t + C_{t-1} \quad , \quad \forall t \geq 2 \\ &\text{and} \\ \sigma(F_1 E_1) &= \sigma(F_1) + \sigma(E_1) = B_1 + C_1. \end{aligned} \tag{14.24}$$

Suppose that we do a perfect syndrome measurement after cycle  $T$  (no measurement errors and no physical errors after cycle  $T$ ). This gives us syndrome

$$\begin{aligned} \sigma\left(\prod_{t=1}^T E_t F_t\right) &= \sum_{t=1}^T \sigma(E_t) + \sigma(F_t) \\ &= B_1 + C_1 + \sum_{t=2}^T B_t + B_{t-1} + C_t + C_{t-1} \\ &= B_T + C_T \end{aligned} \tag{14.25}$$

since there are pairs of  $B_t$  and  $C_t$  for all  $t$  in the summation which cancels out, except for  $B_T, C_T$ . With this syndrome the decoder will deduce a Pauli error  $G$  which is consistent with syndrome  $\sigma(\prod_{t=1}^T E_t F_t) = B_T + C_T$  and has minimum weight, that is,

$$G \in \arg \min \left\{ \text{wt}(G) : \sigma(G) = \sigma\left(\prod_{t=1}^T E_t F_t\right) \right\}. \tag{14.26}$$

### 14.3.2 Syndrome adjacency graph and error clusters

Now that we have laid the groundwork for syndromes and syndrome errors, before proceeding to the main body of the proof for the error correction threshold of qLDPC codes with syndrome measurement errors, we need to define one more object to help us with relating qubit errors, syndrome errors, and the stabilizer checks. Similar to the proof for a threshold for qLDPC codes with no measurement errors in Section 14.2, we will represent qubit errors and measurement bit-flip errors using a graph that take into consideration both data qubits and syndrome bits over  $T$  SE cycles.

#### Definition 14.3: Syndrome adjacency graph

For an  $n$ -qubit stabilizer code  $Q$  with stabilizer generators  $g_1, \dots, g_l$  and a positive integer  $T$ , a simple graph  $\Gamma$  is the *syndrome adjacency graph* of  $Q$  over  $T$  SE cycles if its vertices  $V(\Gamma) = X(\Gamma) \cup B(\Gamma)$  and edges  $E(\Gamma) = K(\Gamma) \cup L(\Gamma)$  are given by

$$\begin{aligned} X(\Gamma) &= \{(x, t) : x \in [1, n], t \in [1, T+1]\} \quad , \\ B(\Gamma) &= \{(b, t) : b \in [1, l], t \in [1, T]\} \quad , \\ K(\Gamma) &= \{((x, t), (y, t)) \in X(\Gamma) \times X(\Gamma) : \exists i \in [1, l], x, y \in \text{supp}(g_i)\} \quad , \\ L(\Gamma) &= \{((b, t), (x, t')) \in B(\Gamma) \times X(\Gamma) : x \in \text{supp}(g_b), t' \in \{t, t+1\}\} \quad . \end{aligned} \tag{14.27}$$

Namely, qubit vertices  $X(\Gamma)$  are labels for the  $n$  qubits at before and after each  $T$  SE cycles, syndrome vertices  $B(\Gamma)$  are labels of syndrome bits in each of the  $T$  SE cycles, edges  $K(\Gamma)$  connects any two qubits at timestep  $t$  involved in a stabilizer generator, and edges  $L(\Gamma)$  connects syndrome vertices at timestep  $t$  to qubit vertices at timestep  $t$  and  $t + 1$ .

For an  $(r, c)$ -qLDPC code family  $Q_1, Q_2, \dots$  with parameters  $\llbracket n_1, k_i, d_i \rrbracket$ , each qubit vertex  $(x, t)$  is connected to at most other  $r - 1$  qubit vertices at timestep  $t$ , connected to at most  $c$  syndrome vertices at timestep  $t$ , and to at most  $c$  syndrome vertices at timestep  $t + 1$ . So the degree of  $(x, t)$  is at most  $2c + r - 1$ . On the other hand, each syndrome vertex  $(b, t)$  is connected to at most  $r$  qubit vertices at timestep  $t - 1$  and at most  $r$  qubit vertices at timestep  $t$ . Thus, any syndrome vertex  $(b, t)$  has degree of at most  $2r$ . So, a syndrome adjacency graph  $\Gamma$  corresponding to any code  $Q_i$  from an  $(r, c)$ -qLDPC code family  $Q_1, Q_2, \dots$  must have degree of at most  $\max\{2c + r - 1, 2r\}$ , which is a constant in  $n_i$ .

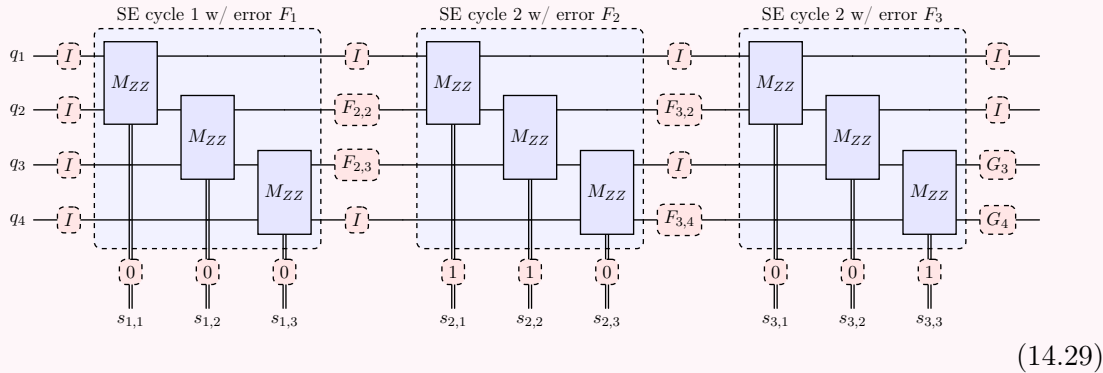
Now consider actual errors  $F_1, \dots, F_T$  and  $B_1, \dots, B_T$ , deduced errors  $E_1, \dots, E_T$  and  $C_1, \dots, C_T$ , and final correction  $G$ . Then consider the following sets of “marked” vertices:

$$\begin{aligned} X_{\text{mrk}} &= \{(x, t) \in X(\Gamma) : t \in [1, T], (x, t) \in \text{supp}(E_t F_t)\}, \\ B_{\text{mrk}} &= \{(b, t) \in B(\Gamma) : (B_t + C_t)_b = 1\}, \\ X_{\text{mrk}}^{T+1} &= \{(x, T+1) \in X(\Gamma) : (x, T+1) \in \text{supp}(G)\}. \end{aligned} \quad (14.28)$$

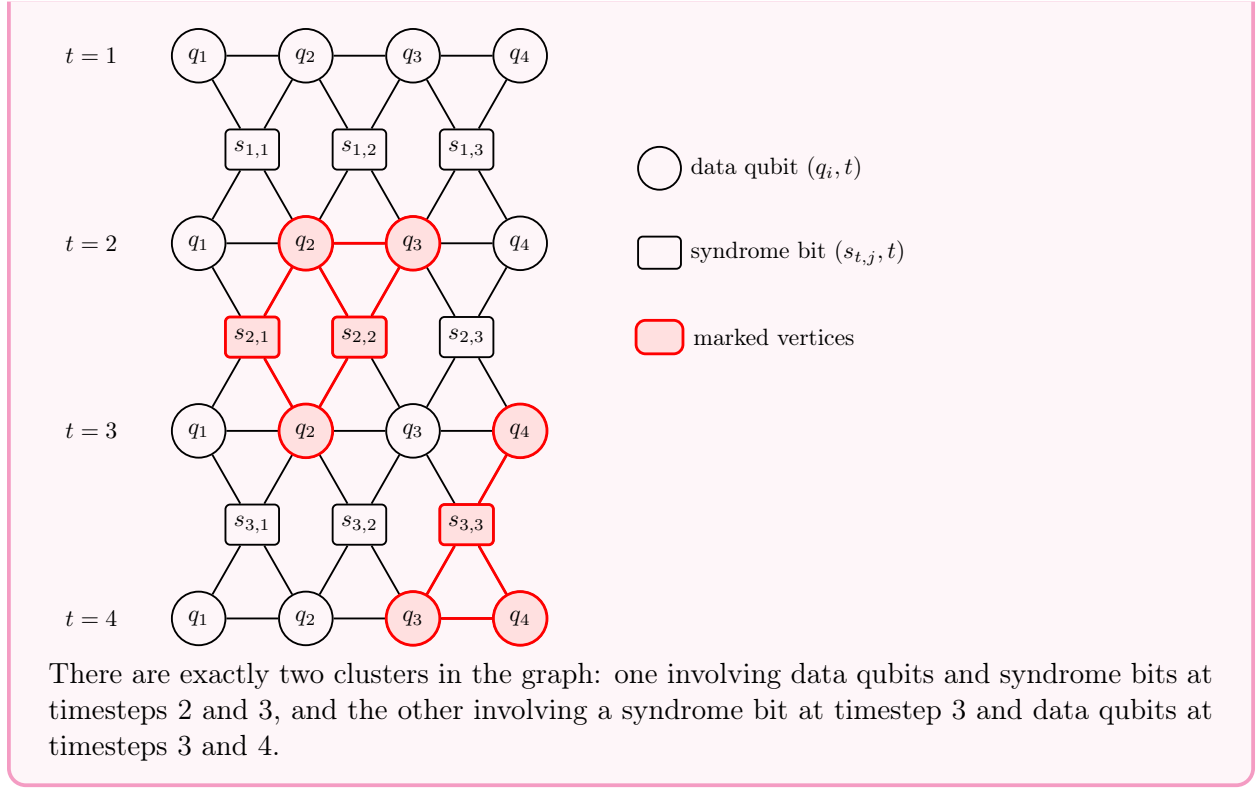
Namely, we mark qubit vertices  $(x, t) \in X(\Gamma)$  if qubit  $x$  at time  $t \in \{1, \dots, T\}$  is in the support of Pauli error  $E_t F_t$ , qubit vertices  $(x, T+1)$  at time  $T+1$  that are in the support of final correction Pauli  $G$ , and syndrome bit vertices  $(b, t)$  at time  $t \in \{1, \dots, T\}$  that are in the support of  $B_t + C_t$ . For the rest of the proof, we will look into largest subsets of marked vertices from  $V_{\text{mrk}} = X_{\text{mrk}} \cup B_{\text{mrk}} \cup X_{\text{mrk}}^{T+1}$  that induces a connected subgraph which we call a *cluster*. More precisely,  $K \subseteq V_{\text{mrk}}$  is a cluster if (1) for all pairs of vertices  $u, v \in K$  there exist a sequence of edges  $(u, w_1), (w_1, w_2), \dots, (w_l, v)$  such that  $w_1, \dots, w_l \in K$  and (2) for all vertices  $v \in K$ , all marked neighbors of  $v$  are  $K$ . As an illustration, consider the following example.

#### Example 14.4: Syndrome adjacency graph and clusters

Consider the 3-round SE circuit for the 4-qubit bit-flip repetition code in Example 14.1 with errors  $F$  and final correction  $G$ :



Its syndrome adjacency graph is given by:



### 14.3.3 Properties of error clusters

As we discussed above, a cluster is a connected induced graph of the syndrome adjacency graph  $\Gamma$  consisting of marked nodes determined by qubit errors  $F = F_1, \dots, F_T$ , syndrome errors  $B = B_1, \dots, B_T$ , and the errors ( $E = E_1, \dots, E_T$ ,  $C = C_1, \dots, C_T$ ,  $G$ ) deduced by the decoder throughout the circuit. Clusters in  $\Gamma$  capture how all errors affect the final residual error  $P = G \prod_{t=1}^T E_t F_t$  at the end of the circuit, which determines whether the decoding is successful. In the following, we will show properties of clusters in a series of lemmas that we use later to characterize logical errors and bound logical error rates to obtain thresholds. First, we show that the residual error  $P_K$  caused by errors in cluster  $K$  must have a trivial syndrome.

#### Lemma 14.2: Connected cluster trivial syndrome

For a connected cluster  $K$ , let  $P_K = G|_K \prod_{t=1}^T E_t|_K F_t|_K$  where  $F$  is the actual errors,  $E$  the deduced errors,  $G$  the final correction, and  $E|_K$  is the restriction of Pauli  $E$  to  $K$ . Then  $\sigma(P_K) = 0$ , that is, it has a trivial syndrome.

*Proof.* Recall that for all bits  $b \in \{1, \dots, l\}$  we have the consistency condition

$$\begin{aligned} \sigma(E_t F_t)_b &= (B_{t-1} + B_t + C_{t-1} + C_t)_b, \quad \forall t \in [2, T] \\ &\text{and} \\ \sigma(E_1 F_1)_b &= (B_1 + C_1)_b, \end{aligned} \tag{14.30}$$

by Eq. (14.24). Let us write  $E_t = E_t|_K E_t|_{K'}$  and  $F_t = F_t|_K F_t|_{K'}$  where  $K'$  is the complement of

cluster  $K$ , so that the consistency conditions become

$$\begin{aligned} \sigma(E_t|_K F_t|_K)_b + \sigma(E_t|_{K'} F_t|_{K'})_b &= (B_{t-1} + C_{t-1})_b + (B_t + C_t)_b, \quad \forall t \in [2, T] \\ &\text{and} \\ \sigma(E_1|_K F_1|_K)_b + \sigma(E_1|_{K'} F_1|_{K'})_b &= (B_1 + C_1)_b. \end{aligned} \quad (14.31)$$

By the definition of a cluster, all marked qubit vertices  $(x, t)$  and  $(x, t+1)$  in  $K'$  for any qubit  $x \in [n_i]$  are not adjacent to syndrome bit vertex  $(b, t) \in K$  since all marked qubit vertices adjacent to  $(b, t)$  must be in  $K$ . Namely, errors  $E_t|_{K'} F_t|_{K'}$  and  $E_{t+1}|_{K'} F_{t+1}|_{K'}$  are not in the support of stabilizer generator  $g_b$  and therefore must have trivial  $b^{\text{th}}$  syndrome:  $\sigma(E_t|_{K'} F_t|_{K'})_b = \sigma(E_{t+1}|_{K'} F_{t+1}|_{K'})_b = 0$ . Similarly, for  $(b, t) \in K'$  errors  $E_t|_K F_t|_K$  and  $E_{t+1}|_K F_{t+1}|_K$  are not in the support of stabilizer generator  $g_b$ , thus its  $b^{\text{th}}$  syndrome is trivial. Thus, for all  $b \in [l]$ ,

$$\begin{aligned} (b, t) \in K &\Rightarrow \sigma(E_t|_{K'} F_t|_{K'})_b = \sigma(E_{t+1}|_{K'} F_{t+1}|_{K'})_b = 0 \\ &\text{and} \\ (b, t) \in K' &\Rightarrow \sigma(E_t|_K F_t|_K)_b = \sigma(E_{t+1}|_K F_{t+1}|_K)_b = 0. \end{aligned} \quad (14.32)$$

Therefore, for  $(b, t) \in K$ , consistency conditions in Eq. (14.31) become

$$\begin{aligned} \sigma(E_t|_K F_t|_K)_b &= \begin{cases} (B_{t-1} + C_{t-1})_b + (B_t + C_t)_b & , \text{ if } (b, t), (b, t-1) \in K \\ 0 & , \text{ otherwise} \end{cases}, \quad \forall t \in [2, T] \\ &\text{and} \\ \sigma(E_1|_K F_1|_K)_b &= \begin{cases} (B_1 + C_1)_b & , \text{ if } (b, 1) \in K \\ 0 & , \text{ otherwise.} \end{cases} \end{aligned} \quad (14.33)$$

Thus, the syndrome of the total physical errors from timestep 1 to  $T$  is

$$\sigma\left(\prod_{t=1}^T E_t|_K F_t|_K\right)_b = \begin{cases} (B_T + C_T)_b & , \text{ if } (b, T) \in K \\ 0 & , \text{ otherwise.} \end{cases} \quad (14.34)$$

Similarly, for timestep  $T+1$ , errors  $G|_{K'}$  are not in the support of stabilizer  $g_b$  for all  $(b, T) \in K$  and errors  $G|_K$  are not in the support of stabilizer  $g_{b'}$  for all  $(b', T) \in K'$  so that

$$(b, T) \in K \Rightarrow \sigma(G|_{K'})_b = 0 \quad \text{and} \quad (b', T) \in K' \Rightarrow \sigma(G|_K)_{b'} = 0, \quad (14.35)$$

which implies that

$$\sigma(G|_K)_b = \begin{cases} (B_T + C_T)_b & , \text{ if } (b, T) \in K \\ 0 & , \text{ otherwise.} \end{cases} \quad (14.36)$$

So, it holds that

$$\sigma(P_K) = \sigma\left(G|_K \prod_{t=1}^T E_t|_K F_t|_K\right) = \sigma(G|_K) + \sigma\left(\prod_{t=1}^T E_t|_K F_t|_K\right) = 0, \quad (14.37)$$

which completes the proof.  $\square$

The lemma above shows that errors  $P_K = G|_K \prod_{t=1}^T E_t|_K F_t|_K$  in a cluster  $K$  do not trigger a syndrome flip. In other words, if we can decompose  $P$  into clusters  $K_1, \dots, K_m$ , then  $\sigma(K_i) = 0$

for all  $i \in [m]$ , and from the proof of [Lemma 14.2](#), this is due to any two nodes  $(\xi, \tau) \in K_i$  and  $(\zeta, \tau') \in K_j$  for  $j \neq i$  must not share an edge and therefore cannot trigger the same syndrome check  $(b, t)$ .

The syndrome triviality of  $P_K$  could mean two things: (1)  $P_K$  is a non-trivial logical error or (2)  $P_K$  is a stabilizer. The following lemma shows that if the residual error  $P = G \prod_{t=1}^T E_t F_t$  at the end of the  $T$  cycles is a non-trivial logical operator (thus resulting in a logical error), then there is a cluster  $K$  which contains errors  $P_K$  which itself is a logical operator and therefore has weight of at least the distance  $d$  of the code.

**Lemma 14.3:** Error weight lower bound

If  $P = G \prod_{t=1}^T E_t F_t$  is a non-trivial logical operator of qLDPC code  $Q$  with stabilizer group  $\mathcal{S}$  and parameter  $\llbracket n, k, d \rrbracket$  (i.e.,  $P \in \mathcal{N}(\mathcal{S}) \setminus \mathcal{S}$ ), then there exists a cluster  $K$  such that  $\text{wt}(P_K) \geq d$  for  $P_K = G|_K \prod_{t=1}^T E_t|_K F_t|_K$ .

*Proof.* Let  $H = \prod_{t=1}^T E_t F_t$  so that  $P = GH$  and assume that it is a non-trivial logical operator, that is,  $GH \in \mathcal{N}(\mathcal{S}) \setminus \mathcal{S}$ . We will then show that  $P_K$  is in  $\mathcal{N}(\mathcal{S}) \setminus \mathcal{S}$  for at least one cluster  $K$  of  $P$ , that is, it is a logical operator.

Consider clusters  $K_1, \dots, K_m$  of  $P$  so that  $P = \prod_{i=1}^m P_{K_i}$ . By [Lemma 14.2](#), for each  $i \in [m]$  we have  $\sigma(P_{K_i}) = 0$  so that  $P_{K_i} \in \mathcal{N}(\mathcal{S})$ . Now, assume that  $P_{K_i} \in \mathcal{S}$  for all  $i \in [m]$ . Then this means that  $P = \prod_{i=1}^m P_{K_i} \in \mathcal{S}$ , which contradicts the assumption that  $GH \in \mathcal{N}(\mathcal{S}) \setminus \mathcal{S}$ . So,  $P_K \in \mathcal{N}(\mathcal{S}) \setminus \mathcal{S}$  must hold. All non-trivial logical operator of  $Q$  must have weight of at least  $d$ . So,  $\text{wt}(P_K) \geq d$ .  $\square$

By how we set our decoder, it chooses minimum weight Paulis that are consistent with the observed syndrome (c.f. consistency conditions in [Eq. \(14.23\)](#)). Now we show that the minimum weight property of deduced errors restricted to any cluster  $K$  also holds.

**Lemma 14.4:** Total error weight

Consider cluster  $K$  and minimum weight deduced errors

$$\{E_1, C_1, \dots, E_T, C_T\} \in \arg \min \text{wt}(E, C), \quad (14.38)$$

where  $\text{wt}(E, C) = \sum_{t=1}^T \text{wt}(E_t) + \text{wt}(C_t)$  and the minimization is over errors  $\{E_t, C_t : t \in [T]\}$  satisfying consistency constraints in [Eq. \(14.23\)](#). Then, it holds that

$$\sum_{t=1}^T \text{wt}(E_t|_K) + \text{wt}(C_t|_K) \leq \sum_{t=1}^T \text{wt}(F_t|_K) + \text{wt}(B_t|_K), \quad (14.39)$$

where  $E_t|_K$  is the errors at timestep  $t$  restricted to cluster  $K$ .

*Proof.* Consider a cluster  $K$  and its complement  $K'$  and (not necessarily minimum weight) deduced errors  $E'_1, C'_1, \dots, E'_T, C'_T$  such that

$$E'_t|_K = F_t|_K, \quad C'_t|_K = B_t|_K, \quad E'_t|_{K'} = E_t|_{K'}, \quad \text{and} \quad C'_t|_{K'} = C_t|_{K'}. \quad (14.40)$$

Namely,  $(E', C')$  are equal to actual errors  $(F, B)$  in cluster  $K$ , but equal to minimum weight deduced errors  $(E, C)$  in  $K'$ . Note that  $E'_1, C'_1, \dots, E'_T, C'_T$  satisfy the constraints in [Eq. \(14.23\)](#)

since for  $t \geq 2$

$$\begin{aligned}
 \sigma(E'_t|_K) + C'_t|_K + C'_{t-1}|_K &= \sigma(F_t|_K) + B_t|_K + B_{t-1}|_K, \\
 \sigma(E'_t|_{K'}) + C'_t|_{K'} + C'_{t-1}|_{K'} &= \sigma(E_t|_{K'}) + C_t|_{K'} + C_{t-1}|_{K'} = \sigma(F_t|_{K'}) + B_t|_{K'} + B_{t-1}|_{K'}, \\
 &\text{and} \\
 \sigma(E'_1|_K) + C'_1|_K &= \sigma(F_1|_K) + B_1|_K, \\
 \sigma(E'_1|_{K'}) + C'_1|_{K'} &= \sigma(E_1|_{K'}) + C_1|_{K'} = \sigma(F_1|_{K'}) + B_1|_{K'}.
 \end{aligned} \tag{14.41}$$

(See the proof of [Lemma 14.2](#).) Since  $\text{wt}(E'_t|_{K'}) + \text{wt}(C'_t|_{K'}) = \text{wt}(E_t|_{K'}) + \text{wt}(C_t|_{K'})$  and  $\text{wt}(E'_t|_{K'}) + \text{wt}(C'_t|_{K'}) = \text{wt}(F_t|_{K'}) + \text{wt}(B_t|_{K'})$ , then we have

$$\text{wt}(E', C') - \text{wt}(E, C) = \left( \sum_{t=1}^T \text{wt}(F_t|_K) + \text{wt}(B_t|_K) \right) - \left( \sum_{t=1}^T \text{wt}(E_t|_K) + \text{wt}(C_t|_K) \right). \tag{14.42}$$

Since  $E, C$  are chosen as consistent errors with minimum weight, then  $\text{wt}(E', C') \geq \text{wt}(E, C)$  must hold. Thus [Eq. \(14.39\)](#) holds.  $\square$

Recall that clusters are defined by both the actual (qubit and syndrome) errors  $F, B$  as well as the errors  $E, C, G$  deduced by the decoders. As such, it is not clear how many of the actual errors occurred in a cluster. In the following lemma, we show that the total number of actual errors  $F, B$  in each cluster  $K$  must be either at least half of the size of  $K$ , or at least a quarter of the size of  $K$ .

**Lemma 14.5:** Cluster faults lower bound

Consider a cluster  $K$  with respect to qubit errors  $E_1F_1, \dots, E_TF_T, G$  and syndrome bit errors  $B_1, C_1, \dots, B_T, C_T$ . If  $K$  contains no vertices of the form  $(x, T+1)$  (i.e.,  $G|_K = I$ ), then the number of faults  $A_K$  in cluster  $K$  satisfies

$$A_K = \sum_{t=1}^T \text{wt}(F_t|_K) + \text{wt}(B_t|_K) \geq \frac{|K|}{2}. \tag{14.43}$$

Also, if  $K$  contains at least one vertex of the form  $(x, T+1)$  (i.e.,  $G|_K \neq I$ ), then

$$A_K = \sum_{t=1}^T \text{wt}(F_t|_K) + \text{wt}(B_t|_K) \geq \frac{|K|}{4}. \tag{14.44}$$

*Proof.* Consider the total weight of all deduced errors  $D_K$ , all qubit errors  $U_K$ , and all syndrome errors  $V_K$  in cluster  $K$ :

$$\begin{aligned}
 D_K &= \sum_{t=1}^T \text{wt}(E_t|_K) + \text{wt}(C_t|_K), \quad U_K = \sum_{t=1}^T \text{wt}(F_t|_K) + \text{wt}(E_t|_K), \\
 \text{and } V_K &= \sum_{t=1}^T \text{wt}(B_t|_K) + \text{wt}(C_t|_K).
 \end{aligned} \tag{14.45}$$

Now recall that each qubit node  $(q, t)$  for  $1 \leq t \leq T$  is marked if qubit  $q$  is in the support of  $E_tF_t$ , each syndrome bit node  $(b, t)$  is marked if the  $b^{\text{th}}$  bit of  $B_t + C_t$  is 1, and each qubit node  $(q, T+1)$  is marked if  $G|_q \neq I$ . Denote the number of marked qubits in  $K$  from timestep 1 to  $T$  by  $s_q$ ,

the number of marked syndrome bits in  $K$  by  $s_c$ , and the number of marked qubits in  $T + 1$  by  $s_g = \text{wt}(G|_K)$ , so that  $|K| = s_c + s_q + s_g$ .

First we will show that  $A_K \geq |K|/2$  assuming that  $G|_K = I$  (so,  $s_g = 0$ ). Note that  $U_K \geq s_q$  since for each qubit node  $(q, t) \in K$  either  $E_t|_q \neq I$  or  $F_t|_q \neq I$ . Similarly, we have  $V_K \geq s_c$  since for each syndrome bit node  $(b, t) \in K$  either  $C_t|_b = 1$  or  $B_t|_b = 1$ . Then, we have

$$A_K + D_K = U_K + V_K \geq s_c + s_q = |K|. \quad (14.46)$$

By Lemma 14.4, it holds that  $D_K \leq A_K$ . So, we obtain

$$|K| \leq A_K + D_K \leq 2A_K. \quad (14.47)$$

which implies that  $A_K \geq |K|/2$  as claimed.

Now we will show that  $A_K \geq |K|/4$  if  $G|_K \neq I$ . We will first show an upper bound to the number of vertices in  $G|_K$  as  $s_g = \text{wt}(G|_K) \leq U_K$ . First note that  $\sigma(G) = B_T + C_T = \sigma(\prod_{t=1}^T E_t F_t)$  by the definition of correction  $G$ . This is true for all the errors and correction restricted to  $K$ , that is,  $\sigma(G|_K) = B_T|_K + C_T|_K = \sigma(\prod_{t=1}^T E_t|_K F_t|_K)$  since  $\sigma(G|_K)_b = (B_T|_K + C_T|_K)_b$  whenever  $(b, T) \in K$  and  $\sigma(G|_K)_b = 0$  otherwise as all marked qubit nodes  $(q, T + 1)$  adjacent to a  $(b, T) \in K$  must be in  $K$ . By a similar argument,  $\sigma(\prod_{t=1}^T E_t|_K F_t|_K)_b = (B_T|_K + C_T|_K)_b$  whenever  $(b, T) \in K$  and  $\sigma(\prod_{t=1}^T E_t|_K F_t|_K)_b = 0$  otherwise. (This is the same line of argument given in the proof of Lemma 14.2). Since  $G$  is chosen to be the smallest weight operator such that  $\sigma(G) = \sigma(\prod_{t=1}^T E_t F_t)$ , we have

$$s_g = \text{wt}(G|_K) \leq \text{wt}\left(\prod_{t=1}^T E_t|_K F_t|_K\right) \leq \sum_{t=1}^T \text{wt}(E_t|_K F_t|_K) \leq U_K. \quad (14.48)$$

By the argument for the  $G|_K = I$  case above, we have  $A_K + D_K \geq s_c + s_q$ . Then, since  $|K| = s_q + s_c + s_g$  and  $U_K \leq A_K + D_K$  we obtain

$$|K| \leq A_K + D_K + U_K \leq 2(A_K + D_K). \quad (14.49)$$

Since  $D_K \leq A_K$ . By Lemma 14.4, we have

$$|K| \leq 4A_K, \quad (14.50)$$

which implies that  $A_K \geq |K|/4$ , as claimed.  $\square$

So far we have not considered the probability of a certain number of actual errors in a cluster. We now show an upper bound to the probability of at least  $m$  actual errors (or, “faults”) in a cluster  $K$  (in fact we show this for any subset of vertices that are not necessarily connected).

**Lemma 14.6:** Probability of faults in cluster

For a subset of vertices  $U \subseteq V(\Gamma)$  and local-stochastic noise model with error parameter  $p_*$ , it holds that

$$\Pr \left[ \text{There are at least } m \text{ faults in } U \right] \leq p_*^m 2^{|U|}. \quad (14.51)$$

*Proof.* Note that by a union bound and by the definition of the local-stochastic noise model, we have

$$\begin{aligned} \Pr \left[ \text{There are at least } m \text{ faults in } U \right] &\leq \sum_{V \subseteq U: |V| \geq m} \Pr[\text{All qubits in } V \text{ are faulty}] \\ &\leq \sum_{V \subseteq U: |V| \geq m} p_*^m. \end{aligned} \quad (14.52)$$

There are  $\binom{|U|}{k}$  subsets of  $U$  with size  $k$ . Thus,

$$\begin{aligned} \Pr \left[ \text{There are at least } m \text{ faults in } U \right] &\leq \sum_{k=m}^{|U|} \binom{|U|}{k} p_*^k \\ &\leq p_*^m \sum_{k=m}^{|U|} \binom{|U|}{k} \leq p_*^m 2^{|U|}, \end{aligned} \quad (14.53)$$

since  $\sum_{k=0}^s \binom{s}{k} = 2^s$ .  $\square$

Now we show the last property of clusters in a graph. This is the same property we used in the proof of decoding threshold without syndrome errors (c.f. Eq. (14.8) of Section 14.2), which uses Lemma 2 of [229].

**Lemma 14.7:** Upper bound on number of clusters [229, Lemma 2]

For any graph  $\Gamma$  with maximum degree  $z'$ , the number of connected clusters  $K$  of size  $|K| = s$  containing vertex  $v \in V(\Gamma)$  is at most  $(z'e)^{s-1}$  and the total number of connected clusters of size  $s$  is at most  $|V(\Gamma)|(z'e)^{s-1}$ .

**Bounding logical error probability** Now we will bound the logical error rate that grows either as  $O(nT(p'/p_i)^d)$ ,  $O(n(p'/p_f)^d)$ ,  $O(n(p''/p_i)^d)$ , or  $O(n(p''/p_f)^d)$ , according to different types of clusters causing the logical error, for some  $p_i, p_f$  and for  $p' = \max\{p, q\}$  and  $p'' = \max\{\tilde{p}, p, q\}$ . First, let us consider errors and corrections  $G, E_1, B_1, \dots, E_T, B_T, F_1, C_1, \dots, F_T, C_T$  so that a decoding failure results in a logical error  $P = G \prod_{t=1}^T E_t F_t \in \mathcal{N}(\mathcal{S}) \setminus \mathcal{S}$ , which restriction to a cluster  $K$  yields  $P_K = G|_K \prod_{t=1}^T (E_t F_t)|_K$ , with actual number of faults  $A_K = \sum_{t=1}^T \text{wt}(F_t|_K) + \text{wt}(B_t|_K)$ . By Lemma 14.3, a logical error implies the existence of cluster  $K$  such that  $\text{wt}(P_K) \geq d$ , so that the size  $|K|$  of such “bad” cluster must satisfy  $|K| \geq \text{wt}(P_K) \geq d$ . So, the probability of a logical error can be bounded as

$$\begin{aligned} \Pr[\text{Logical error}] &\leq \Pr[\text{There is a cluster } K \text{ s.t. } \text{wt}(P_K) \geq d] \\ &\leq \Pr[\text{There is a cluster } K \text{ s.t. } |K| \geq d] \\ &\leq \sum_{s \geq d} \Pr[\text{There is a cluster } K \text{ s.t. } |K| = s]. \end{aligned} \quad (14.54)$$

Now, let us consider four types of logical errors according to the bad cluster  $K$  causing it:

1.  $K$  does not contain both initial errors and final errors (i.e.,  $(E_1 F_1)|_K = I$  and  $G|_K = I$ ).
2.  $K$  does not contain any final errors but contains some initial errors (i.e.,  $(E_1 F_1)|_K = I$  and  $G|_K \neq I$ ).



3.  $K$  does not contain any initial errors but contains some final errors (i.e.,  $(E_1 F_1)|_K \neq I$  and  $G|_K = I$ ).
4.  $K$  contains both initial errors and final errors (i.e.,  $(E_1 F_1)|_K \neq I$  and  $G|_K \neq I$ ).

By Lemma 14.5, the number of faults  $A_K = \sum_{t=1}^T \text{wt}(F_t|_K) + \text{wt}(B_t|_K)$  in cluster  $K$  must satisfy

$$A_K \geq |K|/2 \quad \text{if } G|_K = I \quad \text{and} \quad A_K \geq |K|/4 \quad \text{if } G|_K \neq I, \quad (14.55)$$

Then for a cluster  $K$ , we can use Lemma 14.6 to bound the probability of number of faults in  $K$  for each of these four cases as

$$\begin{aligned} \Pr[\text{At least } |K|/2 \text{ faults in } K] &\leq (p')^{|K|/2} 2^{|K|} \quad , \quad \text{if } (E_1 F_1)|_K = I, G|_K = I, \\ \Pr[\text{At least } |K|/4 \text{ faults in } K] &\leq (p')^{|K|/4} 2^{|K|} \quad , \quad \text{if } (E_1 F_1)|_K = I, G|_K \neq I, \\ \Pr[\text{At least } |K|/2 \text{ faults in } K] &\leq (p'')^{|K|/2} 2^{|K|} \quad , \quad \text{if } (E_1 F_1)|_K \neq I, G|_K = I, \\ \Pr[\text{At least } |K|/4 \text{ faults in } K] &\leq (p'')^{|K|/4} 2^{|K|} \quad , \quad \text{if } (E_1 F_1)|_K \neq I, G|_K \neq I, \end{aligned} \quad (14.56)$$

for  $p' = \max\{p, q\}$  and  $p'' = \max\{\tilde{p}, p, q\}$ .

For case 1, we only need to consider vertices contained from timestep 2 to  $T$  since the cluster  $K$  is contained in this interval. Thus there are  $nT$  vertices that could be contained in such  $K$ , and by Lemma 14.7 there are at most  $(nT)(z'e)^{s-1}$  such clusters of size  $s$ . Hence,

$$\begin{aligned} \Pr[\text{Logical error type 1}] &\leq \sum_{s \geq d} \Pr[\exists \text{ a type 1 cluster } K \text{ s.t. } |K| = s] \\ &\leq \sum_{K: |K|=s \geq d} (nT)(z'e)^{s-1} \Pr[\text{At least } |K|/2 \text{ faults in } K] \\ &\leq \frac{nT}{z'e} \sum_{s \geq d} (2z'e\sqrt{p'})^s. \end{aligned} \quad (14.57)$$

Since  $\sum_{a \geq d} a^a = a^d/(1-a)$  for all  $a < 1$ , if  $2z'e\sqrt{p'} < 1$  we have

$$\Pr[\text{Logical error type 1}] \leq \frac{nT}{z'e} \frac{(2z'e\sqrt{p'})^d}{1 - 2z'e\sqrt{p'}}. \quad (14.58)$$

Defining  $p_i = (2z'e)^{-2}$ , we have

$$\Pr[\text{Logical error type 1}] \leq \frac{nT}{z'e(1 - 2z'e\sqrt{p'})} \left(\frac{p'}{p_i}\right)^{\frac{d}{2}}. \quad (14.59)$$

For case 2, the cluster  $K$  must contain at least one of the  $n$  qubits at timestep  $T+1$ . So, by Lemma 14.7 there are at most  $n(z'e)^{s-1}$  such clusters of size  $s$ . Hence,

$$\begin{aligned} \Pr[\text{Logical error type 2}] &\leq \sum_{s \geq d} \Pr[\exists \text{ a type 2 cluster } K \text{ s.t. } |K| = s] \\ &\leq \sum_{K: |K|=s \geq d} n(z'e)^{s-1} \Pr[\text{At least } |K|/4 \text{ faults in } K] \\ &\leq \frac{n}{z'e} \sum_{s \geq d} (2z'e(p')^{\frac{1}{4}})^s. \end{aligned} \quad (14.60)$$

If  $2z'e(p')^{\frac{1}{4}} < 1$ , then we have

$$\Pr[\text{Logical error type 2}] \leq \frac{n}{z'e} \frac{(2z'e(p')^{\frac{1}{4}})^d}{1 - 2z'e(p')^{\frac{1}{4}}}. \quad (14.61)$$

Defining  $p_f = (2z'e)^{-4}$ , we have

$$\Pr[\text{Logical error type 2}] \leq \frac{n}{z'e(1 - 2z'e(p')^{\frac{1}{4}})} \left(\frac{p'}{p_f}\right)^{\frac{d}{4}}. \quad (14.62)$$

For case 3, the cluster  $K$  must contain at least one of the  $n$  qubits at timestep 1. So, by [Lemma 14.7](#) there are at most  $n(z'e)^{s-1}$  such clusters of size  $s$ . Hence,

$$\begin{aligned} \Pr[\text{Logical error type 3}] &\leq \sum_{s \geq d} \Pr[\exists \text{ a type 3 cluster } K \text{ s.t. } |K| = s] \\ &\leq \sum_{K: |K|=s \geq d} n(z'e)^{s-1} \Pr[\text{At least } |K|/2 \text{ faults in } K] \\ &\leq \frac{n}{z'e} \sum_{s \geq d} (2z'e\sqrt{p''})^s. \end{aligned} \quad (14.63)$$

If  $2z'e\sqrt{p''} < 1$ , then we have

$$\Pr[\text{Logical error type 3}] \leq \frac{n}{z'e} \frac{(2z'e\sqrt{p''})^d}{1 - 2z'e\sqrt{p''}}. \quad (14.64)$$

Thus, we can again use  $p_i = (2z'e)^{-2}$  to obtain

$$\Pr[\text{Logical error type 3}] \leq \frac{n}{z'e(1 - 2z'e\sqrt{p''})} \left(\frac{p''}{p_i}\right)^{\frac{d}{2}}. \quad (14.65)$$

For case 4, cluster  $K$  must contain at least  $2T+1$  vertices since it spans all qubits from timestep 1 to  $T+1$  as well as at least  $T$  syndrome bit vertices in between those  $T+1$  layers of qubit vertices, that is,  $|K| \geq 2T+1$ . So, by [Lemma 14.7](#) there are at most  $(2T+1)(z'e)^{s-1}$  such clusters of size  $s$ . Hence,

$$\begin{aligned} \Pr[\text{Logical error type 4}] &\leq \sum_{s \geq 2T+1} \Pr[\exists \text{ a type 4 cluster } K \text{ s.t. } |K| = s] \\ &\leq \sum_{K: |K|=s \geq 2T+1} n(z'e)^{s-1} \Pr[\text{At least } |K|/4 \text{ faults in } K] \\ &\leq \frac{n}{z'e} \sum_{s \geq 2T+1} (2z'e(p')^{\frac{1}{4}})^s. \end{aligned} \quad (14.66)$$

If  $2z'e(p')^{\frac{1}{4}} < 1$ , then we have

$$\Pr[\text{Logical error type 4}] \leq \frac{n}{z'e} \frac{(2z'e(p')^{\frac{1}{4}})^{2T+1}}{1 - 2z'e(p')^{\frac{1}{4}}}. \quad (14.67)$$

Thus, for  $p_f = (2z'e)^{-4}$ , we have

$$\Pr[\text{Logical error type 4}] \leq \frac{n}{z'e(1 - 2z'e(p')^{\frac{1}{4}})} \left(\frac{p'}{p_f}\right)^{\frac{1}{4}} \left(\frac{p''}{p_f}\right)^{\frac{T}{2}}. \quad (14.68)$$

To summarize, the probability of each type of logical errors is bounded as:

$$\begin{aligned}
 \Pr[\text{Logical error type 1}] &\leq \frac{nT}{z'e(1-2z'e\sqrt{p'})} \left(\frac{p'}{p_i}\right)^{\frac{d}{2}}, \\
 \Pr[\text{Logical error type 2}] &\leq \frac{n}{z'e(1-2z'e(p')^{\frac{1}{4}})} \left(\frac{p'}{p_f}\right)^{\frac{d}{4}}, \\
 \Pr[\text{Logical error type 3}] &\leq \frac{n}{z'e(1-2z'e\sqrt{p''})} \left(\frac{p''}{p_i}\right)^{\frac{d}{2}}, \\
 \Pr[\text{Logical error type 4}] &\leq \frac{n}{z'e(1-2z'e(p'')^{\frac{1}{4}})} \left(\frac{p''}{p_f}\right)^{\frac{d}{4}} \left(\frac{p''}{p_f}\right)^{\frac{d}{2}},
 \end{aligned} \tag{14.69}$$

where we set the total number of timesteps as  $T = d$  for  $p_f = (2z'e)^{-4}$  and  $p_i = (2z'e)^{-2}$ .

#### 14.3.4 Residual error rate

We now show that the remaining residual errors after  $T$  rounds of syndrome extraction are local stochastic when the decoding is successful. Note that a successful decoding means that  $G \prod_{t=1}^T E_t F_t$  is in the stabilizer group  $\mathcal{S}$ , whereas decoding failure means that it is in  $\mathcal{N}(\mathcal{S}) \setminus \mathcal{S}$ . For an initial codestate  $\rho$ , the actual state after  $T$  syndrome extraction rounds with faults  $F = F_T \dots F_1$  and inferred errors  $E = E_T \dots E_1$  is

$$F\rho F^\dagger = \left(E \prod_{t=1}^T E_t F_t\right) \rho \left(E \prod_{t=1}^T E_t F_t\right)^\dagger = \left(EG^\dagger G \prod_{t=1}^T E_t F_t\right) \rho \left(EG^\dagger G \prod_{t=1}^T E_t F_t\right)^\dagger, \tag{14.70}$$

since  $F = E \prod_{t=1}^T E_t F_t$  (up to some phase). If  $G \prod_{t=1}^T E_t F_t \in \mathcal{S}$ , then it acts trivially to codestate  $\rho$  and we obtain

$$F\rho F^\dagger = EG\rho GE. \tag{14.71}$$

Since  $E$  is the errors that the decoder has inferred throughout the  $T$  rounds, then the actual remaining residual error is  $G$ <sup>15</sup>. Since  $G$  is the residual error that will act as an initial error in the next SE cycle, we need to determine its error rate.

If the decoding results in no logical error and there is no cluster spanning from timestep 1 to  $T+1$ , then the correction operator  $G$  is local stochastic (c.f. Definition 14.1) over the output  $n$  qubits with error rate  $p_0/3$  for some  $p_0 \in (0, 1)$  if qubit error rate  $p$  and syndrome bit error rate  $q$  are small enough. Namely, if  $G \prod_{t=1}^T E_t F_t \notin \mathcal{N}(\mathcal{S}) \setminus \mathcal{S}$  and  $G|_K = I$  or  $E|_1 F|_1 = I$ , then the probability of errors on output qubits  $S$  is

$$\Pr[\text{errors } S \subseteq \text{supp}(G)] \leq (p_0/3)^{|S|}, \tag{14.72}$$

whenever  $p_0 > 3(4z'e^2\sqrt{p'})$  and  $e(1-2z'e(p')^{1/4}) \geq 1$ , where  $p' = \max\{p, q\}$ .

Consider a subset  $S$  of the  $n$  qubits at timestep  $T+1$  of size  $|S| = a$ . Then consider all clusters  $K_1, K_2, \dots, K_m$  intersecting  $S$  from  $E_1 F_1, \dots, E_T F_T, G$ , that is, all clusters  $K_j$  such that  $K_j \cap S \neq \emptyset$  and  $(\bigcup_j K_j)|_{T+1} = S \subseteq \text{supp}(G)$ . Now let  $s_{j,q} = \sum_{t=1}^T \text{wt}(E_t|_{K_j} F_t|_{K_j})$  be the number of marked qubit vertices from timestep 1 to  $T$  in cluster  $K_j$  and  $s_{j,c} = \sum_{t=1}^T \text{wt}(B_t|_{K_j} C_t|_{K_j})$  the marked syndrome vertices and  $s_{j,g} = \text{wt}(G|_{K_j})$  the marked qubit vertices at timestep  $T+1$ . So, the size of

<sup>15</sup>Here,  $E$  can be removed because it is the frame that is applied in software (not a gate that is actually performed) to the codestate once it is inferred by the decoder.

each such cluster is  $|K_j| = s_{j,q} + s_{j,c} + s_{j,g}$  and the total size of all clusters  $K_1, \dots, K_m$  intersecting  $S$  is

$$s = \sum_{j=1}^m s_{j,q} + s_{j,c} + s_{j,g}, \quad (14.73)$$

since all clusters are disjoint.

Now note that the total size of output qubits in clusters intersecting  $S$  is  $\sum_j s_{j,g} = a$ . Since  $G|_K$  is a minimum-weight error with syndrome  $\sigma(G|_K) = B_T|_K + C_T|_K = \sigma(\prod_{t=1}^T E_t|_K F_t|_K)$ , we have

$$\text{wt}(G|_K) \leq \text{wt}\left(\prod_{t=1}^T E_t|_K F_t|_K\right) \leq \sum_{t=1}^T \text{wt}(E_t|_K F_t|_K) \quad (14.74)$$

by the same argument as given in the proof of [Lemma 14.4](#). Thus for the clusters intersecting  $S$  we have  $s_{j,g} \leq s_{j,q}$ , and therefore the total size of the union of clusters  $K_1, \dots, K_m$  is lower bounded as

$$s = \sum_{j=1}^m s_{j,q} + s_{j,c} + s_{j,g} \geq \sum_{j=1}^m 2s_{j,g} \geq 2a. \quad (14.75)$$

Now, since all clusters  $K_1, \dots, K_m$  intersecting  $S$  has some vertices in timestep  $T + 1$  in it, by [Lemma 14.5](#) the total number of actual faults  $A_j$  in each cluster  $K_j$  is lower bounded as

$$A_j = \sum_{t=1}^T \text{wt}(F_t|_{K_j}) + \text{wt}(B_t|_{K_j}) \geq \frac{|K_j|}{4}. \quad (14.76)$$

Thus the total number of actual faults in the union of clusters  $K_1, \dots, K_m$  is lower bounded as

$$\sum_{j=1}^m A_j \geq \frac{1}{4} \sum_{j=1}^m |K_j| = \frac{s}{4}. \quad (14.77)$$

So, by [Lemma 14.6](#) the probability of at least  $s/4$  faults in  $K_1, \dots, K_m$  is

$$\Pr[s/4 \text{ faults in } K_1, \dots, K_m] \leq p^{s/4} 2^s. \quad (14.78)$$

By Lemma 2 of [\[229\]](#), the number of sets  $U$  of size  $s$  that are a union of connected clusters in a graph  $\Gamma$  with maximum degree  $z'$  such that  $U$  contains a subset of vertices  $S$  of size  $|S| = a$  is upper-bounded by  $e^{a-1}(z'e)^{s-a}$ . Thus, the probability of clusters  $K_1, \dots, K_m$  such that sets  $(\bigcup_j K_j)|_{T+1} = S$  such that  $|S| = a$ , given that the decoding succeeded, is upper-bounded as

$$\begin{aligned} \Pr\left[\left(\bigcup_j K_j\right)|_{T+1} = S, S \subseteq \text{supp}(G), |S| = a\right] &\leq \sum_{s \geq 2a} e^{a-1}(z'e)^{s-a} (p')^{s/4} 2^s \\ &= \frac{1}{e(z')^a} \sum_{s \geq 2a} (2z'e(p')^{1/4})^s, \end{aligned} \quad (14.79)$$

since the total size  $s = \sum_j |K_j|$  of clusters  $K_1, K_2, \dots$  must be at least  $2a$  by [Eq. \(14.75\)](#) and the total number of actual faults across them must be at least  $s/4$  by [Eq. \(14.77\)](#). Note that we use  $p' = \max\{p, q\}$ , which is the maximum between qubit error rate  $p$  and syndrome error rate  $q$ , since we assume that clusters with support from timestep 1 to  $T + 1$  is considered as a decoding failure. Since we only consider all clusters containing some output qubit vertices, these clusters cannot

contain initial qubits if we assume that the decoding succeeded. Lastly, if  $2z'e(p')^{\frac{1}{4}} < 1$  then we can use the geometric sum identity  $\sum_{s \geq t} c^s = \frac{c^t}{1-c}$  for  $c < 1$  to obtain

$$\begin{aligned} \Pr \left[ \left( \bigcup_j K_j \right) |_{T+1} = S \subseteq \text{supp}(G) \right] &\leq \frac{1}{e(z')^{|S|}} \frac{(2z'e(p')^{1/4})^{2|S|}}{(1 - 2z'e(p')^{1/4})} \\ &= \frac{(4z'e^2\sqrt{p'})^{|S|}}{e(1 - 2z'e(p')^{1/4})}. \end{aligned} \quad (14.80)$$

Thus if  $p_0 > 3(4z'e^2\sqrt{p'})$  and  $e(1 - 2z'e(p')^{1/4}) \geq 1$ , then we have

$$\Pr[S \subseteq \text{supp}(G)] \leq \left( \frac{p_0}{3} \right)^{|S|}, \quad (14.81)$$

which means that output error  $G$  is local stochastic with rate  $p_0/3$ .

### 14.3.5 Bounding logical error rates

Lastly let us pick the thresholds  $p_0, p_1, p_2$  for the initial errors  $\tilde{p}$ , qubit SE errors  $p$ , and syndrome bit errors  $q$ , respectively, so that the local-stochastic output error in Eq. (14.72) is satisfied and the logical error rates in Eq. (14.69) are bounded by functions exponential in code distance  $d$ .

First, recall the logical error probability bounds for the four different types of errors given in Eq. (14.69):

$$\begin{aligned} \Pr[\text{Logical error type 1}] &\leq \frac{nT}{z'e(1 - 2z'e\sqrt{p'})} \left( \frac{p'}{p_f} \right)^{\frac{d}{2}}, \\ \Pr[\text{Logical error type 2}] &\leq \frac{n}{z'e(1 - 2z'e(p')^{\frac{1}{4}})} \left( \frac{p'}{p_f} \right)^{\frac{d}{4}}, \\ \Pr[\text{Logical error type 3}] &\leq \frac{n}{z'e(1 - 2z'e\sqrt{p''})} \left( \frac{p''}{p_f} \right)^{\frac{d}{2}}, \\ \Pr[\text{Logical error type 4}] &\leq \frac{n}{z'e(1 - 2z'e(p'')^{\frac{1}{4}})} \left( \frac{p''}{p_f} \right)^{\frac{1}{4}} \left( \frac{p''}{p_f} \right)^{\frac{d}{2}} \end{aligned} \quad (14.82)$$

for  $p' = \max\{p, q\}$  and  $p'' = \max\{\tilde{p}, p, q\}$  and  $p_f = (2z'e)^{-4}$ <sup>16</sup>. Let us set our first threshold  $p_0$  as  $p_0 = p_f = (2z'e)^{-4}$ .

In the output error rate analysis above, we need  $p' = \max\{p, q\}$  to be small enough such that  $p_0 > 3(4z'e^2\sqrt{p'})$  and  $e(1 - 2z'e(p')^{1/4}) \geq 1$ . So,

$$p' \leq \min \left\{ \left( \frac{p_0}{3(4z'e^2\sqrt{p'})} \right)^2, \left( \frac{e-1}{2z'e} \right)^4 \right\}. \quad (14.83)$$

Thus, we need to set threshold  $p_1$  and  $p_2$  so that  $p \leq p_1$  and  $q \leq p_2$ , such that the upper bound on  $p'$  holds. This can be satisfied by setting

$$p_1 = p_2 = \frac{p_f^2}{144(z')^2 e^4} = (192(z')^5 e^6)^{-2}. \quad (14.84)$$

<sup>16</sup>Note that we use  $p_f$  for all the bounds, since the bounds in Eq. (14.69) with  $p_i = (2z'e)^{-2}$  is stricter compared to ones with  $p_f < p_i$ .

Thus whenever  $\tilde{p} < p_0$ ,  $p < p_1$ , and  $q < p_2$ , the logical error rates are bounded as

$$\begin{aligned}
 \Pr[\text{Logical error type 1}] &\leq O\left(nT\left(\frac{p'}{p_0}\right)^{\frac{d}{2}}\right), \\
 \Pr[\text{Logical error type 2}] &\leq O\left(n\left(\frac{p'}{p_0}\right)^{\frac{d}{4}}\right), \\
 \Pr[\text{Logical error type 3}] &\leq O\left(n\left(\frac{p''}{p_0}\right)^{\frac{d}{2}}\right), \\
 \Pr[\text{Logical error type 4}] &\leq O\left(n\left(\frac{p''}{p_0}\right)^{\frac{d}{2}}\right).
 \end{aligned} \tag{14.85}$$

#### 14.4 Threshold theorem for constant-overhead FTQC scheme

Gottesman's FT scheme [229] simulates logical quantum circuit  $C$  with  $k$  qubits and  $f(k)$  locations using  $O(k)$  physical qubits at any given time and  $O(f(k)\text{poly } k)$  circuit locations. Namely, this scheme has a constant  $\frac{O(k)}{k} = O(1)$  space overhead and polynomial  $\frac{O(f(k)\text{poly } k)}{f(k)} = O(\text{poly } k)$  time overhead. For simplicity, we also assume that the circuit  $C$  consists of Clifford gates  $\{H, \text{CNOT}, S\}$  and a non-Clifford gate  $T$ .

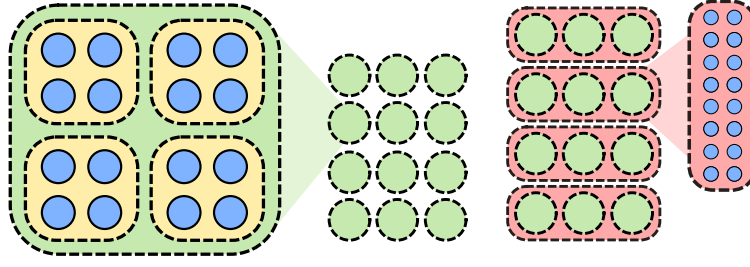
The code family  $\{Q_i\}$  with parameters  $\llbracket n_i, k_i, d_i \rrbracket$  used in the scheme is  $(r, c)$ -LDPC: each stabilizer generator acts on at most  $r$  qubits and each qubit involved in at most  $c$  generators, for constants  $r, c$  independent of  $i$ . Moreover, we also demand some additional properties of the code family:

1. Encoding rate  $R = \lim_{i \rightarrow \infty} k_i/n_i > 0$ .
2. Distance  $d_i \rightarrow \infty$ .
3. Growth of number of physical qubits:  $0 < n_i - n_{i-1} < n_{i-1}^\beta$  for  $\beta > 0$ .
4. If thresholds  $\tilde{p} < p_0$ ,  $p < p_1$ , and  $q < p_2$  are satisfied and  $T(n_i)$  SE rounds are performed, then the code is decodable with logical error rate  $p_L = O(1/g(n_i))$ , for  $g(n) = \Omega(\text{poly } n)$  and  $T(n) = \text{poly } n$ .

Other than the qLDPC code family, below we list key ingredients and ideas that go into this FTQC scheme, each of which we discuss in detail:

1. Block-encode  $k$  logical qubits into  $M = \lceil \frac{k}{k_i} \rceil$  blocks of  $(r, c)$ -qLDPC codes  $Q_i$  satisfying the conditions above (for an illustration comparing this block encoding scheme to the concatenated code encoding in [Section 13](#), see [Figure 27](#)).
2. *Sequential FT circuit*: one logical operation at a time.
3. Cycle error correction (EC) across  $M$  blocks of  $Q_i$ , sharing ancillas.
4. *Shor-style syndrome extraction (SE)* with cat-state ancillas.
5. *Knill-style teleportation* for logical gates and measurements with offline ancilla preparation using concatenated codes.

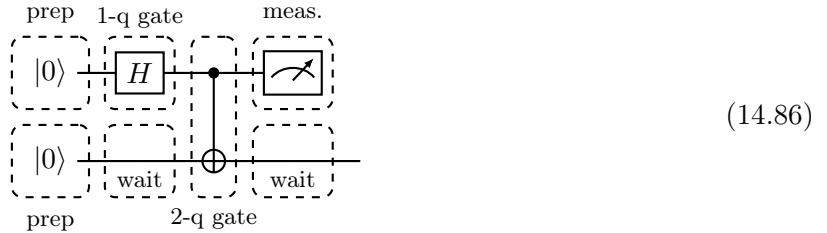
First, though, we discuss the fault tolerance and noise model used for the scheme.



**Figure 27:** Illustration of encoding by concatenation (left) and block encoding (right). In both figures, blue circles represent physical qubits and green circles represent logical qubits. On the left-hand side, each green circle represent an outer  $[[4, 1]]$  code, where each one of its four qubits (yellow, rounded squares) is itself a  $[[4, 1]]$  code (where the four physical qubits are the blue circles inside a yellow rounded square). Thus, the 12 logical qubits use  $12 \times 16 = 192$  physical qubits through two levels of concatenation. On the right-hand side, each red, rounded rectangle is a  $[[16, 3]]$  code, where the 16 physical qubits are represented by blue circles and the 3 logical qubits by green circles. Thus, with 4 copies of the  $[[16, 3]]$  code, we can encode 12 logical qubits using  $4 \times 16 = 64$  physical qubits.

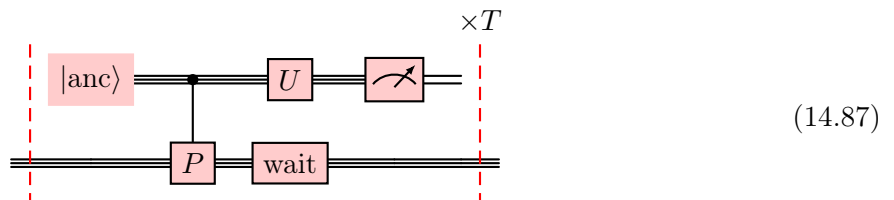
#### 14.4.1 Basic model and simplified model of fault tolerance

Here we adopt the *basic model of fault tolerance*. In this model, we consider locations of a circuit  $C$ , denoted by  $\text{loc}(C)$ , as the set of all potentially faulty components, which includes state preparation, gates, measurements, and wait locations. For example, consider the following circuit with two state preparation locations, one single-qubit gate location, one two-qubit gate location, one measurement location, and two wait locations:



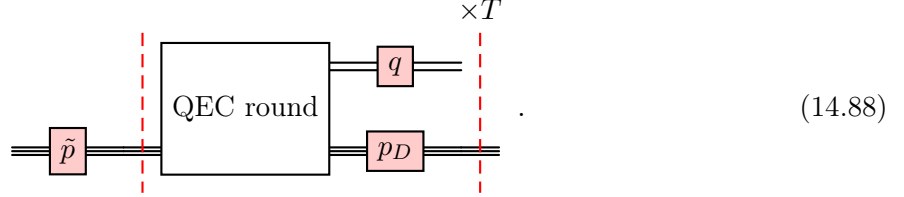
We adopt the local stochastic noise model as defined in [Definition 14.1](#) so that for an error rate  $p$ , the probability of faults in a subset of locations  $S \subseteq \text{loc}(C)$  is at most  $p^{|S|}$ .

Later, for the fault tolerance analysis of the error correction cycles, we will instead use a *simplified model of fault tolerance*. In the simplified model, we instead consider three different types of faults with local-stochastic noise: initial input errors with rate  $\tilde{p}$ , output data qubit errors with rate  $p_D$ , and syndrome bit-flip errors with rate  $q$ . Note that this is the model that we use in the qLDPC code decoding threshold in [Section 14.3](#). To illustrate this more concretely and to compare between the basic model and simplified model of fault tolerance in syndrome extraction circuits, consider the following  $T$  iterations of a generic syndrome extraction circuit in the basic model:



Note that here we have ancilla  $|anc\rangle$  preparation locations, 2-qubit and 1-qubit gate locations, wait locations, and measurement locations where faults may occur with rate  $p$ . In the simplified model,

we can abstract the locations inside the circuit to a “black-box” QEC round circuit with input errors happening before the entire  $T$ -round QEC cycle with rate  $\tilde{p}$  and for each of the  $T$  QEC rounds, syndrome bit-flip errors with rate  $q$  and data qubit errors with rate  $p_D$ :



We summarize the two models in the following table:

| Aspect | Basic model                           | Simplified model                                |
|--------|---------------------------------------|-------------------------------------------------|
| Scope  | Full FT protocol: gates, EC, ancillas | Abstract QEC cycle ( $T$ SE rounds)             |
| Noise  | $k$ locations local stoch. $\leq p^k$ | Local stochastic: rates $\tilde{p}, p, q$       |
| Errors | Location-based (gate/prep/meas/wait)  | Initial $\tilde{p}$ , data $p_D$ , syndrome $q$ |

With these FT models and using the ingredients for the FTQC scheme we listed above, we can show that FTQC with constant space overhead can be achieved whenever error rates are below some thresholds. In particular, this FTQC scheme uses a family of qLDPC codes  $\{Q_i\}_i$  with parameters  $\llbracket n_i, k_i, d_i \rrbracket$  satisfying: (1) a non-zero asymptotic rate  $R = \lim_{n_i \rightarrow \infty} \frac{k_i}{n_i}$ , (2) a condition on the growth of number of physical qubits  $n_i$ , and (3) existence of a decoding with logical error rate and residual error rate guarantees. Using this qLDPC code family, we can show that a  $k$ -qubit logical circuit  $C$  with  $f(k)$  locations can be simulated by a circuit  $\tilde{C}$  using  $O(k)$  qubits at any given time and  $O(f(k)\text{poly}(k))$  total number of physical locations, where the output distribution of  $\tilde{C}$  is  $\epsilon$ -far from  $C$ . This is formalized in the following threshold theorem.

**Theorem 14.1:** Constant-overhead FTQC threshold [229, Theorem 1]

Let  $\{Q_i\}$  be a family of QEC codes with:

1.  $Q_i$  is an  $\llbracket n_i, k_i \rrbracket$   $(r, c)$ -LDPC code, for constants  $r, c$ .
2. As  $n_i \rightarrow \infty$ ,  $\frac{k_i}{n_i} \rightarrow R > 0$ .
3.  $0 < n_i - n_{i-1} < n_{i-1}^\beta$  for some  $\beta > 0$ .
4. Decoding guarantees local-stochastic noise: Under local stochastic noise (with initialization error rate  $\tilde{p}$  per qubit, error rate in QEC cycle  $p_D$  per data qubit per syndrome extraction (SE), syndrome bit flip rate  $q$ ),  $\tilde{p} < p_0$ ,  $p_D < p_1$ , and  $q < p_2$  implies logical error  $O(1/g(n_i))$  after decoding and residual errors  $\leq p_0/3$  given a polynomial  $T(n_i)$  syndrome extractions each with depth  $h(n_i)$  if there is no logical errors. ( $T, h, g$  are non-decreasing functions.)

Choose  $\alpha < 1$  such that  $0 < \alpha\beta < 1$ . Then, for all  $\eta > 1$ ,  $\epsilon > 0$  and for all polynomials  $f(n) = o(g(n^\alpha))$ , there exists  $p_T(\eta)$  and  $k_0(\eta, f, \epsilon)$  such that for all sequential circuit  $C$  with  $k > k_0$  qubits and  $f(k)$  locations, there exists a simulation  $\tilde{C}$  using at most  $\eta k/R$  physical qubits. If local-stochastic error rate  $p$  satisfies  $p < p_T(\eta)$ , statistical distance between  $\tilde{C}$  and  $C$  is at most  $\epsilon > 0$ . Furthermore for  $\alpha' = \max(\alpha, \alpha\beta)$ , simulation  $\tilde{C}$  uses:



1.  $O\left(\frac{f(k)T(2k^{\alpha'})k^{\alpha'}}{\eta-1} + f(k)k^{\alpha'}\text{polylog}(k/\epsilon)\right)$  physical locations.
2. Classical computation time of  $O(h(2k^{\alpha'}) + \log \log(k/\epsilon))$  per logical step.
3. If  $T(n_i)$  polynomial, then  $|\tilde{C}| = \text{poly } k$ ; if decoding can be done in polynomial time, then classical computation time complexity is also polynomial.

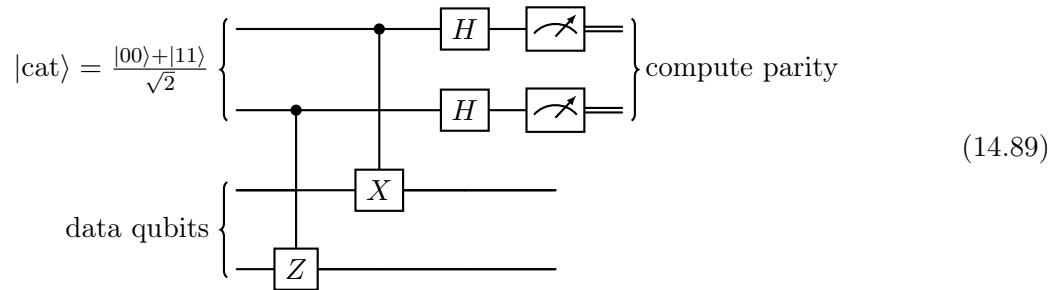
Note that the parameters  $\beta > 0$  and  $\alpha < 1$  determine the number of locations  $\text{loc}(\tilde{C})$  and the number of classical computation steps. We will later see how these parameters are chosen. Also, note that this threshold theorem holds for big enough logical circuits  $C$ , that is, ones that uses  $k > k_0$  qubits, where  $k_0$  also depends on these parameters.

#### 14.4.2 Error correction

We now discuss how error correction is performed in this FTQC protocol, the rate at which errors remain, and how much overhead it incurs. To perform reliable error correction, we measure each syndrome  $T(n_i) = \text{poly}(n_i)$  times, where each syndrome measurement is done using Shor's fault-tolerant error correction (Shor EC) scheme (c.f. [Section 13.2.1](#)). For each stabilizer generator  $g$  with weight  $w \leq r$ , Shor EC for measuring  $g$  consists of three steps:

1. Prepare an ancilla cat state  $\frac{1}{\sqrt{2}}(|0^w\rangle + |1^w\rangle)$  fault-tolerantly (will be discussed below).
2. Apply controlled-Pauli gates from cat qubits to data qubits.
3. Measure each ancilla qubit in  $X$ -basis.

For example, the Shor EC circuit to measure the syndrome of  $X_1Z_2$  is given by



The cat state  $\frac{|0^r\rangle + |1^r\rangle}{\sqrt{2}}$  over  $r$  qubits can be prepared with low fault-probability using a gadget  $\mathcal{G}_{\text{cat}}^r$  that prepares the state and checks for bit-flip errors using pairwise  $Z$  measurement and  $r'$  ancillas. We then discard the output if the error check outputs a  $-1$  outcome and accept it otherwise. For

example, a gadget  $\mathcal{G}_{\text{cat}}^4$  that prepares 4-qubit cat code using  $r' = 1$  check ancilla is given by

$$\mathcal{G}_{\text{cat}}^4 = \left\{ \begin{array}{l} |0\rangle \\ |0\rangle \\ |0\rangle \\ |0\rangle \end{array} \right\} \frac{|0000\rangle + |1111\rangle}{\sqrt{2}} \quad (14.90)$$

where the two-qubit  $Z$  measurement checks the parity of the cat state.

Since each stabilizer generator has weight of at most  $r$ , a Shor EC needs an additional  $r' = O(r)$  ancilla qubits per stabilizer generator (since we also need ancillas to check for errors in the cat state preparation). Since we have  $n_i - k_i$  stabilizers to measure for each of the  $M = \lceil k/k_i \rceil$  code-blocks, a single round of Shor EC will need  $r'M(n_i - k_i)$  ancilla qubits if we measure all stabilizers in all code-blocks simultaneously. To prevent this blowup, we split the  $M$  blocks of  $\llbracket n_i, k_i, d_i \rrbracket$  code to  $s$  sets, each containing  $M/s$  blocks. Then, we measure only the syndromes of  $M/s$  code-blocks at any given time in the QEC cycle while all the other  $(s - 1)M/s$  blocks wait until we measure all syndromes of the  $M/s$  code-blocks before we move on to the next  $M/s$  blocks. This way, we only need additional  $r'M(n_i - k_i)/s$  ancilla qubits for QEC that we reuse to do Shor EC on different blocks in cycle. Since we need to prepare an average of  $\frac{1}{1 - Ap}$  cat states (as we dispose those that fail the error check), the total number of ancilla qubits we need for QEC is

$$\frac{r'M(n_i - k_i)}{s(1 - Ap)}. \quad (14.91)$$

Now let us translate the error rate  $p$  in the basic model of fault tolerance to the physical data qubit error rate  $p_P$  and syndrome bit-flip error rate  $p_B$  to obtain the local-stochastic data qubit error rate  $p_D$  and syndrome bit-flip error rate  $q$  the simplified model for a QEC cycle. Let us consider all fault locations in the SE circuit causing a physical data qubit error:

- Cat state faults: cat state preparation error can propagate via controlled gates. Each ancilla has  $A$  fault locations (dependent on  $r$ ) and each qubit involved in at most  $c$  syndrome measurements. So, the probability of qubit error from bad ancilla up to post-selection (with success probability  $\geq 1 - pA$ ) is  $\frac{cAp}{1 - pA}$
- Gate faults: at most  $c$  controlled-Pauli gates from each syndrome measurements the qubit is involved in, each with probability  $p$ .
- Wait faults: idling errors during waits. Single syndrome measurement takes  $l$  time steps and measured every  $s$  cycles. So each qubit has to wait at most for  $sl$  steps, each contributing to error probability  $p$ .

Thus, by putting all three error sources together, the physical data error rate  $p_D$  is given by:

$$p_P = \underbrace{\frac{cAp}{1 - pA}}_{\text{ancilla propagation after post-selection}} + \underbrace{cp}_{\text{controlled gate error}} + \underbrace{slp}_{\text{wait time error}} \quad (14.92)$$

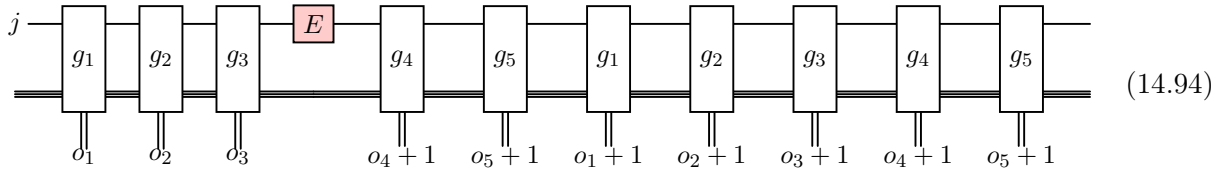
Note that the data qubit error rate satisfy local stochastic condition:  $\Pr[a \text{ data errors}] \leq p_P^a$  because Shor SE is fault-tolerant (see [Section 13.3.1](#)). Now let us consider all fault locations in the SE circuit causing a syndrome bit-flip error:

- Cat state faults: similar to the physical data qubit faults above, faults in the cat state preparation circuit causes bit-flip error with probability  $\frac{Ap}{1-pA}$ .
- Gate and measurement faults: there are  $r$  controlled-Pauli,  $r$  Hadamard gates, and  $r$  measurements in an SE circuit, where each location fails with probability  $p$ . Thus the probability of a bit-flip error from gate and measurement faults is  $3rp$ .

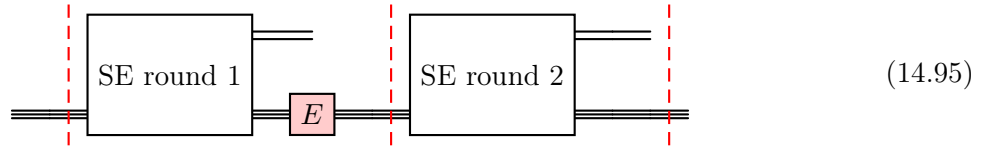
Putting the two error sources together gives a syndrome bit-flip error rate of

$$p_B = \underbrace{\frac{Ap}{1-pA}}_{\text{Cat-state prep}} + \underbrace{rp}_{\text{controlled-Paulis}} + \underbrace{rp}_{\text{Hadamard gates}} + \underbrace{rp}_{\text{measurements}}. \quad (14.93)$$

Note that there are correlations between data qubit errors and syndrome errors, where a physical data error in the basic FT model can cause both syndrome bit-flip errors and data qubit errors in the simplified model. For example, suppose that a qubit  $j$  is involved in stabilizers  $g_1, \dots, g_5$ , measured in this sequence. Then, suppose that a physical error  $E$  at qubit  $j$  after measurement  $g_3$  but before  $g_4$  in the first round of SE, and that this error anticommutes with all five stabilizers. Then, this error will flip syndromes  $g_4, g_5$  in the first SE round and then flip  $g_1, g_2, g_3$  in the second SE round while  $g_4, g_5$  remains the same (assuming there have been no other errors), as illustrated in the following:

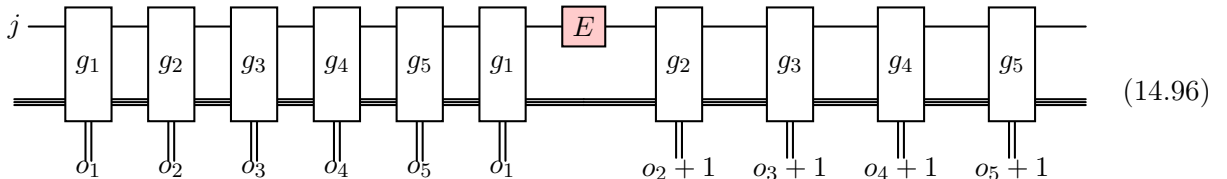


In the simplified model, this data qubit error can be placed between the first and second SE rounds as

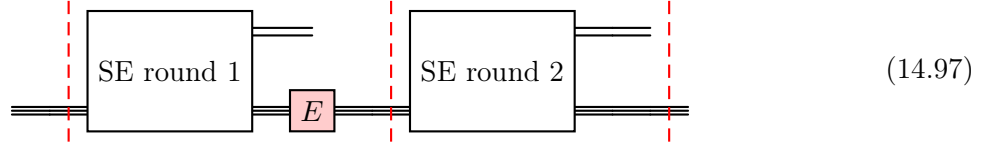


but only  $g_1$  and  $g_2$  syndromes differ between the first and second round. Thus, we translate this fault  $E$  in the basic FT model to a data qubit error  $E$  and syndrome bit errors on  $g_4, g_5$  in the first round of SE.

On the other hand, if error  $E$  at qubit  $j$  happens after measurement  $g_1$  but before  $g_2$  in the second round of SE, it will flip  $g_2, \dots, g_5$ :



In this case, we consider  $E$  to be an error between round 1 and 2 in the simplified model, as



and we translate this fault  $E$  in the basic FT model to a syndrome bit error on  $g_1$  in the second round of SE. Generalizing this to the  $(r, c)$ -LDPC codes where each qubit is involved in at most  $c$  syndrome measurements, a fault in a SE round that flips at most the first  $a \leq \lceil c/2 \rceil$  syndromes is translated to syndrome errors on these first  $a$  syndromes, whereas a fault in a SE round that flips at most the last  $b < \lceil c/2 \rceil$  syndromes is translated to syndrome errors on these last  $b$  syndromes. Thus, there are at most  $\lceil c/2 \rceil$  syndrome bit-flip errors (in the simplified FT model) that can be caused by a physical data qubit error in the basic FT model.

Now we need to bound the physical data error rate  $p_P$  with  $p_D$  and physical bit-flip error rate  $p_B$  with  $q$  such that the data qubit error rate and the syndrome bit error rate are local-stochastic. By using the above bit-flip error accounting, a physical data error causes at most  $\lceil c/2 \rceil$  syndrome errors. Therefore, for set  $S$  with  $b_D$  data errors and  $b_B$  syndrome errors in the simplified FT model, we have

$$b_B = [\text{syndrome bit-flips}] + [\text{from phys. data errors}] \leq a_B + b_D \lceil c/2 \rceil, \quad (14.98)$$

for some  $a_B$  in the interval  $[a_{\min}, b_B]$  for  $a_{\min} = b_B - b_D \lceil c/2 \rceil$ . Since  $b_B = a_{\min} + b_D \lceil c/2 \rceil$  and  $\sum_{a=k}^{\infty} p^a = \frac{p^k}{1-p}$ , the probability of errors  $S$  is bounded as

$$\begin{aligned} \Pr[\text{errors } S] &\leq \sum_{a_B=\max(0, a_{\min})}^{b_B} \binom{b_B}{a_B} p_P^{b_D} p_B^{a_B} \\ &\leq 2^{b_B} p_P^{b_D} \sum_{a_B=\max(0, a_{\min})}^{b_B} p_B^{a_B} \\ &\leq 2^{a_{\min} + b_D \lceil c/2 \rceil} p_P^{b_D} \frac{p_B^{a_{\min}}}{(1-p_B)} \\ &\leq (2^{\lceil c/2 \rceil} p_P)^{b_D} \frac{(2p_B)^{a_{\min}}}{(1-p_B)}. \end{aligned} \quad (14.99)$$

We need to set local-stochastic rates  $p_D$  and  $q$  for  $S$  as

$$\Pr[\text{errors } S] \leq p_D^{b_D} q^{b_B} = p_D^{b_D} q^{a_{\min} + b_D \lceil c/2 \rceil} = (p_D q^{\lceil c/2 \rceil})^{b_D} q^{a_{\min}}. \quad (14.100)$$

This can be satisfied by setting

$$p_D = \frac{\sqrt{p_P}}{1-p_B} \quad \text{and} \quad q = \max \left\{ 2p_P^{1/(c+1)}, \frac{2p_B}{1-p_B} \right\}. \quad (14.101)$$

Then the thresholds for data qubit errors and syndrome errors are given by  $p_D < p_1$  and  $q < p_2$ .

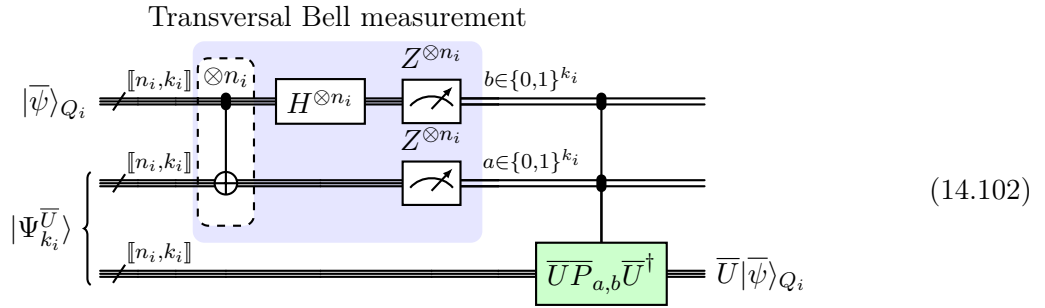
### 14.4.3 Logical gates

In this FTQC scheme, we use Knill's gate teleportation scheme proposed in [24], which allows for an implementation of a universal fault-tolerant gate set [73] by preparing a logical ancilla state

that encodes a gate  $U$  using concatenated codes. Recall that we assume that the logical circuit  $C$  contains gates from a Clifford gate set  $H, S, \text{CNOT}$  and a non-Clifford gate  $T$ . Such a protocol that implements a logical unitary  $\bar{U}$  on a  $[[n_i, k_i]]$ -qLDPC code block  $Q_i$ , uses two ancilla code blocks  $A_1, A_2$  of the  $[[n_i, k_i]]$ -qLDPC codes, and consists of the following three steps:

1. Using concatenated codes (see [270]), prepare  $2n_i$ -qubit state  $|\Psi_{k_i}^{\bar{U}}\rangle = (I \otimes \bar{U})|\overline{\text{Bell}}_{k_i}\rangle$  over the ancilla code blocks  $A_1 \otimes A_2$ . The state  $|\Psi_{k_i}^{\bar{U}}\rangle$  encodes the unitary gate  $\bar{U}$ , which is a logical  $U$  for the  $[[n_i, k_i]]$  code. Here,  $|\overline{\text{Bell}}_{k_i}\rangle = \left(\frac{|00\rangle + |11\rangle}{\sqrt{2}}\right)^{\otimes k_i}$  is the logical Bell state over ancilla blocks  $A_1, A_2$ .
2. Teleport the  $\bar{U}$  gate via transversal Bell measurement over  $Q_i$  and  $A_1$  ( $\bar{X}_{Q_i} \bar{X}_{A_1}, \bar{Z}_{Q_i} \bar{Z}_{A_1}$  basis).
3. Correct with logical Pauli  $\bar{P}_{a,b} = \bar{X}_1^{a_1} \bar{Z}_1^{b_1} \dots \bar{X}_{k_i}^{a_{k_i}} \bar{Z}_{k_i}^{b_{k_i}}$  conjugated by  $\bar{U}$  based on outcomes.

The logical gate teleportation circuit in step 2 above, including correction in step 3 is given by

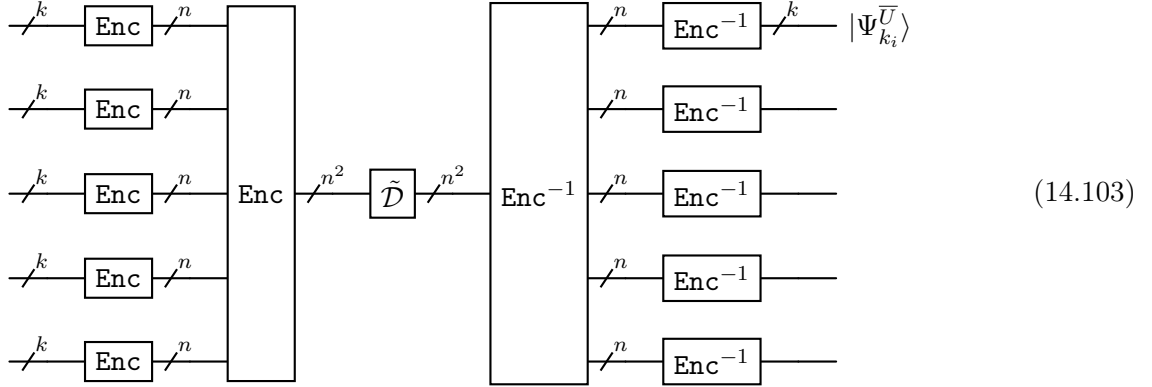


where a transversal CNOT is performed on the first two code-blocks, followed by a transversal Hadamard on the first block, then all qubits in the first two blocks are measured in the  $Z$  basis, whose outcomes determine the logical correction unitary  $\bar{U} \bar{P}_{a,b} \bar{U}^\dagger$  on the third block. Note that when  $U$  is a 2-qubit unitary, such as a CNOT gate, across two code blocks, we prepare a  $4n_i$ -qubit state  $|\Psi_{k_i}^{\bar{U}}\rangle$  over four blocks of ancilla  $A_1^{(1)}, A_1^{(2)}, A_2^{(1)}, A_2^{(2)}$  and do a transversal Bell measurement over two blocks of data qubits  $Q_i^{(1)}, Q_i^{(2)}$  and two blocks of ancilla  $A_1^{(1)}, A_1^{(2)}$ . Then, correction  $\bar{U} \bar{P}_{a,b} \bar{U}^\dagger$  is done over two ancilla blocks  $A_2^{(1)}, A_2^{(2)}$ .

To prepare a  $2n_i$ -qubit state  $|\Psi_{k_i}^{\bar{U}}\rangle$ , we use an  $r$ -level concatenated code  $C^{\otimes r}$  ancilla factory<sup>17</sup>. This ancilla factory encodes the  $2n_i$  qubits in concatenated code  $C^{\otimes r}$ , then prepares the state  $|\Psi_{k_i}^{\bar{U}}\rangle$  using a circuit  $\tilde{\mathcal{D}}$  that encodes an ideal circuit  $\mathcal{D}$  with  $|\mathcal{D}|$  locations that prepares the state  $|\Psi_{k_i}^{\bar{U}}\rangle = (I \otimes U)|\text{Bell}_{k_i}\rangle$ . For an  $[[n, k]]$  code with encoder  $\text{Enc}$  and  $k = 2n_i < n$ , an ancilla factory

<sup>17</sup>A proof that any state can be prepared using a concatenated code can be found in [271, Section 4.2.2] or [272]

preparing  $|\Psi_{k_i}^{\bar{U}}\rangle$  using a two-level concatenation of  $[[n, k]]$  codes can be illustrated as



For logical error rate  $\varepsilon_0$ , the ancilla factory uses

$$\# \text{ qubits in ancilla factory} = O(n_i \text{polylog}(n_i/\varepsilon_0)) \quad (14.104)$$

physical qubits and

$$\text{loc}(\text{ancilla factory}) = O(|\mathcal{D}| \text{polylog}(|\mathcal{D}|/\varepsilon_0)) \quad (14.105)$$

locations for the ancilla factory. By setting the number of code blocks as  $M = \lceil \frac{k}{k_i} \rceil = \text{polylog} k$  and by picking  $k_i = \frac{k}{\text{polylog} k}$ , so that  $n_i = \Theta(k_i) = \Theta(\frac{k}{\text{polylog} k})$ , the number of physical qubits used in an ancilla factory is

$$\# \text{ qubits in ancilla factory} = O(n_i \text{polylog}(n_i/\varepsilon_0)) \approx O(k), \quad (14.106)$$

thus maintaining constant space overhead.

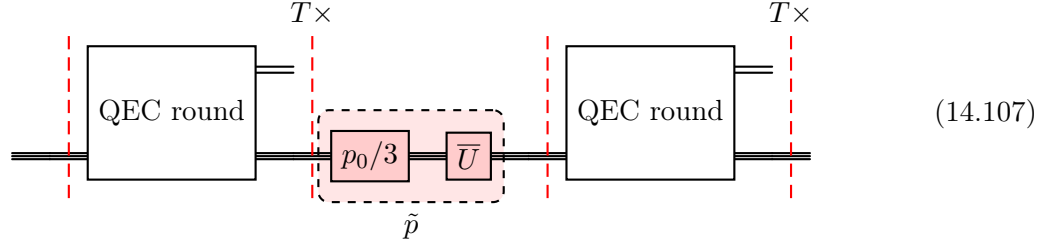
Such an ancilla factory is also used to prepare the logical state  $|\bar{0}\rangle$  at the start of the circuit. On the other hand, it can be used to do logical measurements for general stabilizer code, where the code blocks are encoded into a concatenated code and measured there. In both  $|\bar{0}\rangle$  state preparation and logical measurement, we also use an additional  $O(n_i \text{polylog}(n_i/\varepsilon_0))$  physical qubits.

Lastly, we note that for Clifford gates  $U \in \{H, S, \text{CNOT}\}$ , correction  $\bar{U} \bar{P}_{a,b} \bar{U}^\dagger$  is a logical Pauli which can be applied transversally (or, we can just keep track of it in the Pauli frame). However, for the non-Clifford  $T$  gate,  $\bar{V} = \bar{T} \bar{P}_{a,b} \bar{T}^\dagger$  is a logical Clifford for which transversal implementation may not be possible on the qLDPC code. Therefore, we need to implement  $\bar{V}$  using gate teleportation with another ancilla  $|\Psi_{k_i}^{\bar{V}}\rangle$ . In this case, after the transversal Bell measurement is performed, the data qubits need to wait for this additional ancilla to be prepared and teleported for correction  $\bar{V}$  to be applied.

#### 14.4.4 Initial error rate in the simplified FT model

In this FTQC scheme, a QEC cycle consisting of  $T$  QEC rounds is performed between subsequent logical gates. So, faults from the logical gate implementation right before a QEC round are counted as initial errors for that QEC round with local-stochastic error rate  $\tilde{p}$ . In addition to logical gate faults, the initial errors of a QEC cycle also comes from residual errors from the QEC round right before the logical gate. As an illustration, consider a logical gate  $\bar{U}$  and two QEC cycles before and

after it:



Note that the residual error from the previous QEC cycle is local-stochastic with rate  $p_0/3$  as we showed in [Section 14.3](#), whenever the decoding is successful.

From our previous discussion on how logical gates are implemented, we can now analyze how the logical gate faults cause initial errors at the QEC cycle following every logical gate, along with residual errors from the previous QEC cycle, and determine its local-stochastic error rate  $\tilde{p}$ . A physical data qubit error caused by logical gate faults comes from the following two sources:

1. Ancilla factory: if there are  $B$  locations in an ancilla factory, then faults in it cause a physical data qubit error with probability  $Bp$ .
2. In a teleportation circuit, a data qubit is involved in a Hadamard, a CNOT, a measurement, and a logical Pauli correction. Thus faults on these locations will cause a physical data qubit error with probability  $4p$ .

Note that a logical Clifford  $\bar{U}$  may be a two-qubit gate across two code blocks, such as a CNOT, which will propagate residual errors from the previous QEC cycle across code blocks. So the physical initial error rate  $\tilde{p}_C$  in the simplified error model for QEC cycles after a logical Clifford  $\bar{U}$  is at most

$$\tilde{p}_C = \underbrace{\frac{2p_0}{3}}_{\text{residual if } \bar{U}=\text{CNOT}} + \underbrace{\frac{Bp}{|\Psi_{k_i}^{\bar{U}}\rangle \text{ prep.}}}_{\text{CNOT}} + \underbrace{p}_{\text{H}} + \underbrace{p}_{\text{meas.}} + \underbrace{p}_{\text{Pauli } \bar{U}\bar{P}_{a,b}\bar{U}^\dagger}}. \quad (14.108)$$

On the other hand, for a logical non-Clifford  $\bar{U}$  such as the  $T$  gate, we need an additional ancilla to do Clifford correction  $\bar{V} = \bar{U}\bar{P}_{a,b}\bar{U}^\dagger$  in the gate teleportation circuit. Thus, the physical initial error rate  $\tilde{p}_{NC}$  in the simplified error model for QEC cycles after a logical non-Clifford  $\bar{U}$  is at most

$$\tilde{p}_{NC} = \underbrace{\frac{p_0}{3}}_{\text{residual}} + \underbrace{2}_{\text{two tele.+prep.}} \left( \underbrace{\frac{Bp}{|\Psi_{k_i}^{\bar{U}}\rangle \text{ or } |\Psi_{k_i}^{\bar{V}}\rangle}}_{\text{CNOT}} + \underbrace{p}_{\text{H}} + \underbrace{p}_{\text{meas.}} + \underbrace{p}_{\bar{U}\bar{P}_{a,b}\bar{U}^\dagger \text{ or } \bar{V}\bar{P}_{a,b}\bar{V}^\dagger}} \right). \quad (14.109)$$

We can then choose an initial error rate threshold  $p_0$  that bounds initial input error rate  $\tilde{p}$  which is given by the largest quantity between  $\tilde{p}_C$  in [Eq. \(14.108\)](#) and  $\tilde{p}_{NC}$  in [Eq. \(14.109\)](#).

#### 14.4.5 Threshold and logical error rate

Now let us collect the local-stochastic data qubit error rate  $p_D$  and syndrome error rate  $q$  from [Eq. \(14.101\)](#) and the initial error rate  $\tilde{p} = \max\{\tilde{p}_C, \tilde{p}_{NC}\}$  from [Eq. \(14.108\)](#) and [Eq. \(14.109\)](#) in the simplified FT model. We obtain thresholds  $p_0, p_1, p_2$  for these error rates as follows:

$$\begin{aligned} \tilde{p} = \max\{\tilde{p}_C, \tilde{p}_{NC}\} &= \max\left\{\frac{2p_0}{3} + (B+4)p, \frac{p_0}{3} + 2(B+4)p\right\} < p_0, \\ p_D = \frac{\sqrt{p_P}}{1-p_B} &< p_1, \quad \text{and} \quad q = \max\left\{2p_P^{1/(c+1)}, \frac{2p_B}{1-p_B}\right\} < p_2. \end{aligned} \quad (14.110)$$

Then, we choose a threshold  $p_T$  such that all 4 inequalities above are satisfied whenever  $p < p_T$ . We also need the logical error probability to be at most  $\varepsilon$ . Because circuit  $C$  has  $f(k)$  locations, we need at most  $\varepsilon/f(k)$  logical error rate per logical location:

- From probability  $\varepsilon_0$  of ancilla preparation failure from the ancilla factory, we need  $\varepsilon_0 \leq \varepsilon/(3f(k))$  (at most 2 ancillas per location, for example, for logical  $T$  gates).
- From probability  $D/g(n_i)$  of QEC failure, we choose  $i$  such that  $D/g(n_i) \leq \varepsilon/(3f(k))$  by taking  $n_i > k^\alpha$  for large  $k$  since  $f(k) = o(g(k^\alpha))$  for  $\alpha > 1$ .

Thus, by using  $\varepsilon_0 \leq \varepsilon/(3f(k))$  and  $D/g(n_i) \leq \varepsilon/(3f(k))$ , the logical error rate per logical location can be bounded as

$$\begin{aligned} \Pr[\text{Logical location } \ell \text{ fails}] &\leq \Pr[\text{ancilla prep failure at } \ell] + \Pr[\text{QEC failure at } \ell] \\ &\leq 2\varepsilon_0 + \frac{D}{g(n_i)} \\ &\leq 3\frac{\varepsilon}{3f(k)} = \frac{\varepsilon}{f(k)}. \end{aligned} \tag{14.111}$$

Since  $\Pr[\text{Logical location } \ell \text{ fails}] \leq \frac{\varepsilon}{f(k)}$ , we have

$$\begin{aligned} p_{\text{fail}} &:= \Pr[\text{there is a logical failure}] \\ &\leq \sum_{\ell} \Pr[\text{Logical location } \ell \text{ fails}] \leq f(k) \frac{\varepsilon}{f(k)} = \varepsilon. \end{aligned} \tag{14.112}$$

Let  $\xi$  be an output distribution with logical errors. For ideal output distribution  $p_{\text{ideal}}$  and noisy output distribution  $p_{\text{noisy}} = (1 - p_{\text{fail}})p_{\text{ideal}} + p_{\text{fail}}\xi$ , their total variation distance is therefore bounded as

$$\text{TVD}(p_{\text{ideal}}, p_{\text{noisy}}) \leq p_{\text{fail}} \leq \varepsilon. \tag{14.113}$$

#### 14.4.6 Constant space overhead

We now show that this FTQC scheme incurs a constant space overhead. In particular, for a logical circuit  $C$  using  $k$  qubits, we can construct a circuit  $\tilde{C}$  simulating  $C$  using  $O(1)$  qubits at any given time. Note that the total number of physical qubits used in  $\tilde{C}$  is given by

$$\text{Total physical qubits} = N_{\text{data}} + N_{\text{QEC}} + N_{\text{op}}, \tag{14.114}$$

where  $N_{\text{data}}$  is the number of physical data qubits,  $N_{\text{QEC}}$  is the total number of physical qubits used for ancillas in QEC at any given time, and  $N_{\text{op}}$  is the total number of physical qubits used for ancillas in logical operations (logical gates, measurements, and state preparation) at any given time.

Let us now consider constants  $\omega, \lambda, \mu > 0$  to be determined later and also recall the positive encoding rate assumption in [Theorem 14.1](#), that is,  $R = \lim_{i \rightarrow \infty} k_i/n_i > 0$ . The number of physical data qubits is given by

$$N_{\text{data}} = Mn_i, \tag{14.115}$$

since we are using  $M = \lceil k/k_i \rceil$  blocks of  $[[n_i, k_i, d_i]]$  qLDPC codes. We take a large positive integer  $i$  such that  $k_i/n_i \approx R$ , so that the bound

$$N_{\text{data}} = Mn_i \leq (1 + \omega) \frac{k}{R} \tag{14.116}$$



holds for large  $k$ .<sup>18</sup>

Now, recall that from Eq. (14.91), that the number of physical qubits used for QEC ancillas is given by

$$N_{\text{QEC}} = \frac{M(n_i - k_i)r'}{s(1 - Ap)}, \quad (14.117)$$

since we do QEC only on  $M/s$  blocks at a time, where  $r'$  is the total physical qubits used per cat state and  $1 - Ap$  is the success probability in the cat state post-selection (repeat  $1/(1 - Ap)$  times on average). We can take a large  $s$  such that

$$N_{\text{QEC}} = \frac{M(n_i - k_i)r'}{s(1 - Ap)} \leq \lambda \frac{k}{R}. \quad (14.118)$$

By Eq. (14.104), the number of physical qubits used for each teleportation ancilla  $|\Psi_{k_i}^{\overline{U}}\rangle$  is

$$N_{\text{op}} = O(n_i \text{polylog}(n_i/\varepsilon_0)) = cn_i(\log(n_i/\varepsilon_0))^a, \quad (14.119)$$

where  $\varepsilon_0$  is the logical error rate of the ancilla factory and constants  $c, a > 0$ . Since we only do one logical operation at a time in  $\tilde{C}$ , we need at most  $2cn_i(\log(n_i/\varepsilon_0))^a$  physical qubits (i.e., since we need to prepare two ancillas to implement a logical non-Clifford  $T$ ). We can pick a sufficiently large  $n_i$  such that

$$N_{\text{op}} = cn_i(\log(n_i/\varepsilon_0))^a \leq \mu \frac{k}{R}. \quad (14.120)$$

Now, we can choose  $\eta > 0$  such that

$$N_{\text{tot}} = N_{\text{data}} + N_{\text{QEC}} + N_{\text{op}} \leq (1 + \omega) \frac{k}{R} + \lambda \frac{k}{R} + \mu \frac{k}{R} \leq \eta \frac{k}{R}, \quad (14.121)$$

which gives a constant overhead of  $\eta/R$  since both  $\eta, R$  are constants. Note that we still need to pick  $\omega, \lambda, \mu > 0$  and  $i$  such that:

$$\begin{aligned} N_{\text{data}} &= Mn_i \leq (1 + \omega) \frac{k}{R} \\ N_{\text{QEC}} &= \frac{M(n_i - k_i)r'}{s(1 - Ap)} \leq \lambda \frac{k}{R} \\ N_{\text{op}} &= Cn_i(\log(n_i/\varepsilon_0))^a \leq \mu \frac{k}{R} \\ \frac{D}{g(n_i)} &< \frac{\varepsilon}{3f(k)}. \end{aligned} \quad (14.122)$$

We do this by picking  $\omega > 0$  and  $\lambda > 1$  and  $M \leq \lambda k/k_i$  and large  $s$ , which depend on  $n_i$  chosen below. If  $R/(1 + \omega) < k_i/n_i < R(1 + \omega)$  then we have

$$N_{\text{data}} = Mn_i \leq (1 + \omega) \frac{k}{R} \quad \text{and} \quad N_{\text{QEC}} = \frac{M(n_i - k_i)r'}{s(1 - Ap)} \leq \lambda \frac{k}{R}. \quad (14.123)$$

We want a large  $i$  such that (1) the logical error after decoding and (2) the logical operations qubit overhead are small:

$$\frac{D}{g(n_i)} < \frac{\varepsilon}{3f(k)} \quad \text{and} \quad cn_i(\log(n_i/\varepsilon_0))^a \leq \mu \frac{k}{R}. \quad (14.124)$$

---

<sup>18</sup>Note that this is possible since  $Mn_i \approx \frac{n_i k}{k_i}$  so that  $\lim_{i \rightarrow \infty} Mn_i \approx \lim_{i \rightarrow \infty} \frac{n_i k}{k_i} = \frac{k}{R}$  by the assumption that the asymptotic encoding rate is positive:  $R = \lim_{i \rightarrow \infty} \frac{k_i}{n_i} > 0$ . If instead we have zero asymptotic encoding rate  $R = \lim_{i \rightarrow \infty} \frac{k_i}{n_i} = 0$ , then  $\lim_{i \rightarrow \infty} \frac{n_i}{k_i}$  blows up hence  $\lim_{i \rightarrow \infty} Mn_i$  also blows up and we can no longer upper bound the number of data qubits.

So, we can pick  $i$  such that  $n_i \geq k^\alpha$  (since  $f(k) = o(g(k^\alpha))$  for large  $k$ ) and

$$n_i \leq \frac{k}{R \log(k/(R\varepsilon_0))^{a+1}}. \quad (14.125)$$

Because  $n_i \leq k_i/R \leq k/R$  and  $\varepsilon_0 \leq \varepsilon/(3f(k))$ , for  $t > 0$  it holds that

$$cn_i(\log(n_i/\varepsilon_0))^a \leq \frac{ck}{R \log(k/(R\varepsilon_0))} \left(1 + O\left(\frac{\log \log(k/(R\varepsilon_0))}{\log(k/(R\varepsilon_0))}\right)\right) < \frac{ck}{tR}. \quad (14.126)$$

Lastly, we show why such an  $i$  exists. Recall that we have assumed  $0 < n_i - n_{i-1} < n_i^\beta$  for  $\beta < 1$  for the qLDPC code family. Also,  $\alpha < 1$  such that  $0 < \alpha\beta < 1$  (which determines  $f(n_i) = o(g(n_i^\alpha))$ ). We pick smallest  $i$  such that  $n_i > k^\alpha$  (so  $n_{i-1} < k^\alpha$ ), so that

$$k^\alpha < n_i \leq n_{i-1} + n_{i-1}^\beta \quad (14.127)$$

$$\leq k^\alpha + k^{\alpha\beta} \quad (14.128)$$

$$\leq 2k^{\alpha'} \quad (14.129)$$

$$< \frac{k}{R \log(k/(R\varepsilon_0))^{a+1}} \quad (14.130)$$

$$\leq \frac{k}{R \log(3kf(k)/(R\varepsilon))^{a+1}} \quad (14.131)$$

for large enough  $k$  where  $\alpha' = \min\{\alpha, \alpha\beta\} < 1$ . So,  $n_i = O(k/\text{polylog}(k))$ .

#### 14.4.7 Total number of locations

We now determine the total number of locations in  $\tilde{C}$ . First, let us consider locations in the QEC cycles. Note that there are  $T(n_i)$  SE repetitions each QEC cycle,  $sl$  waits in between, a cat state for each of the  $n_i - k_i$  stabilizer generators, each using  $\frac{A}{1-Ap}$  gates for prep (on average), and  $2r$  gates for measurement. So, the total number of locations per QEC cycle is

$$\# \text{ locations per QEC cycle} \leq T(n_i) \left( sln_i + (n_i - k_i) \left( \frac{A}{1-Ap} + 2r \right) \right). \quad (14.132)$$

At each logical timestep there are  $M/s$  QEC cycles, so the number of locations in QEC cycles at each logical step is

$$\begin{aligned} \# \text{ QEC cycle locations per logical step} &\leq MT(n_i) \left( ln_i + \frac{(n_i - k_i)}{s} \left( \frac{A}{1-Ap} + 2r \right) \right) \\ &= O(n_i T(n_i)). \end{aligned} \quad (14.133)$$

For the number of locations in each logical gate implementation, note that there are  $O(n_i \text{polylog}(n_i/\varepsilon_0)) = O(n_i \text{polylog}(n_i f(k)/\varepsilon))$  locations in concatenated code ancilla factory for  $|\Psi^{\bar{U}}\rangle$  per logical location (since  $\varepsilon_0 \leq \varepsilon/(3f(k))$ ). Therefore, the total number of locations in each logical operation is given by

$$\# \text{ locations per logical operation} \leq O\left(n_i T(n_i) + n_i \text{polylog}(kf(k)/\varepsilon)\right) \quad (14.134)$$

Since  $n_i < 2k^{\alpha'}$  for  $\alpha' = \max\{\alpha, \alpha\beta\}$ , we have  $T(n_i) < T(2k^{\alpha'})$  and  $\text{polylog}(n_i f(k)/\varepsilon) < \text{polylog}(kf(k)/\varepsilon)$ . So, total number of locations in  $\tilde{C}$  is

$$|\tilde{C}| \leq O\left(f(k)k^{\alpha'} \left(T(2k^{\alpha'}) + \text{polylog}(kf(k)/\varepsilon)\right)\right). \quad (14.135)$$

Lastly, we also need to consider how many steps of classical computation are needed per logical step in  $\tilde{C}$ . Note that each QEC decoding runs in  $O(h(n_i)) \leq O(h(2k^{\alpha'}))$  classical steps since  $n_i < 3k^{\alpha'}$  (recall the assumption in [Theorem 14.1](#) that decoding runs for  $h(n_i)$  timesteps for some non-decreasing function  $h$ ). We also need to consider  $O(\log \log \varepsilon_0) = O(\log \log(f(k)/\varepsilon)) = O(\log \log(k/\varepsilon))$  levels of concatenated code ancilla factory where decoding is performed. So, the total number of classical step per logical step is

$$\# \text{ classical steps per logical step} = O(h(2k^{\alpha'}) + \log \log(k/\varepsilon)). \quad (14.136)$$

With this, we conclude the proof of [Theorem 14.1](#).

#### 14.4.8 Further improvements

One shortcoming of Gottesman's constant-overhead FTQC result is that it makes no promise whether decoding can be done in polynomial time (in general it is indeed a hard problem). This was solved first in the work by Fawzi *et al.* [273], where the general Gottesman FTQC scheme is instantiated using [quantum expander code](#) [Ec<sub>260</sub>](#) (which is a [hypergraph product code](#) [Ec<sub>260</sub>](#) [232] with classical expander codes as seed codes). The quantum expander code family has a parameter of  $[\Theta(k_i), k_i, \Theta(\sqrt{k_i})]$  and enables a single-shot decoder with  $O(\log(k_i))$  runtime (even with noisy syndrome measurements).

More recently, a work by Nguyen and Pattison [271] uses [quantum locally-testable codes \(LTC\)](#) [Ec<sub>260</sub>](#) which allows for single-shot decoding that incurs only a constant-time classical computation per logical quantum timestep. Moreover, they also showed that an improvement from  $O(\text{polylog}(k))$  overhead in concatenated ancilla factory to  $(\log k)^{o(1)}$  by using optimized stabilizer state and magic state distillation.

### 14.5 Magic state distillation

Magic state distillation is one of the most, if not the most, widely used subroutines in FTQC schemes to enable a universal fault-tolerant computation based on stabilizer codes. In general, a magic state distillation protocol starts with multiple copies of noisy *magic states*  $\rho^{\otimes m}$  followed by an encoding to logical magic states  $\bar{\rho}^{\otimes k}$  in some quantum error-correcting code (for  $k < m$ ), where errors can be detected or corrected to produce less noisy magic states  $\tilde{\rho}^{\otimes k}$ . Then, one can implement non-Clifford gates using gate teleportation on  $\tilde{\rho}^{\otimes k}$ . Thus, while the Eastin-Knill theorem obstructs a transversal implementation of a universal transversal logical gate set on any code (see [Section 13.9](#)), one could still perform universal quantum computation using a stabilizer code with transversal logical Clifford gates and magic state distillation. Here, we discuss a magic state distillation protocol proposed in [274] using the [\[\[4, 2, 2\]\]](#) [Ec<sub>260</sub>](#) error-detecting stabilizer code.

#### 14.5.1 The [\[\[4, 2, 2\]\]](#) stabilizer code

The [\[\[4, 2, 2\]\]](#) code is defined by its stabilizer group  $\mathcal{S} = \langle s_1, s_2 \rangle$  with logical operator representatives  $\bar{X}_1, \bar{Z}_1, \bar{X}_2, \bar{Z}_2$ , given by

$$s_1 = \begin{smallmatrix} XX \\ XX \end{smallmatrix}, \quad s_2 = \begin{smallmatrix} ZZ \\ ZZ \end{smallmatrix}, \quad \bar{X}_1 = \begin{smallmatrix} XX \\ II \end{smallmatrix}, \quad \bar{Z}_1 = \begin{smallmatrix} ZI \\ ZI \end{smallmatrix}, \quad \bar{X}_2 = \begin{smallmatrix} XI \\ XI \end{smallmatrix}, \quad \bar{Z}_2 = \begin{smallmatrix} ZZ \\ II \end{smallmatrix}, \quad (14.137)$$

where we denote the four qubits in the square arrangement  $\begin{smallmatrix} P_1 P_2 \\ P_4 P_3 \end{smallmatrix}$  where  $P_i$  is the Pauli operator on the  $i^{\text{th}}$  qubit. Also, we note that logical Hadamard gate  $\bar{H}_1 \bar{H}_2$  on both logical qubits of the

$[[4, 2, 2]]$  code can be implemented SWAP-transversally. Namely, it can be implemented by applying  $H^{\otimes 4}$  followed by a logical swap  $\overline{\text{SWAP}}_{1,2} = \text{SWAP}_{2,4}$  (i.e., a physical swap between qubit 2 and 4) since

$$\begin{aligned} H^{\otimes 4} \overline{X}_1 H^{\otimes 4} &= \overline{Z}_2, & H^{\otimes 4} \overline{Z}_1 H^{\otimes 4} &= \overline{X}_2, & H^{\otimes 4} \overline{X}_2 H^{\otimes 4} &= \overline{Z}_1, & H^{\otimes 4} \overline{Z}_2 H^{\otimes 4} &= \overline{X}_1 \\ &\text{and} \\ \overline{\text{SWAP}}_{1,2} \overline{X}_1 \overline{\text{SWAP}}_{1,2} &= \overline{X}_2, & \overline{\text{SWAP}}_{1,2} \overline{Z}_1 \overline{\text{SWAP}}_{1,2} &= \overline{Z}_2, \\ \overline{\text{SWAP}}_{1,2} \overline{X}_2 \overline{\text{SWAP}}_{1,2} &= \overline{X}_1, & \overline{\text{SWAP}}_{1,2} \overline{Z}_2 \overline{\text{SWAP}}_{1,2} &= \overline{Z}_1. \end{aligned} \quad (14.138)$$

### 14.5.2 The $|H\rangle$ magic state

Here, we consider a magic state  $|H\rangle$ , which is the  $+1$  eigenstate of the Hadamard operator  $H$ , given by

$$|H\rangle = R_y(\pi/4)|0\rangle = \cos \frac{\pi}{8}|0\rangle + \sin \frac{\pi}{8}|1\rangle, \quad (14.139)$$

for rotation gate  $R_y(\theta) = e^{-i\theta Y/2}$  about the Pauli- $Y$  axis. First let us consider the Hadamard twirling channel  $\mathcal{T}_H(\cdot) = \frac{1}{2}I \cdot I + \frac{1}{2}H \cdot H$  on a state  $\rho$ , which will give a resource state  $\rho_H$ :

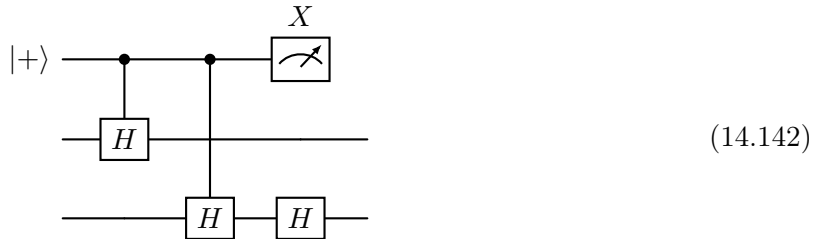
$$\rho_H = \mathcal{T}_H(\rho) = (1-p)|H\rangle\langle H| + p|-H\rangle\langle -H| \quad (14.140)$$

for some  $p \in [0, 1]$  depending on the input  $\rho$  and where  $|-H\rangle$  is the  $-1$  eigenstate of  $H$ . Since  $Y|H\rangle = |-H\rangle$ , we can write the action of  $\mathcal{T}_H$  on state  $\rho$  as

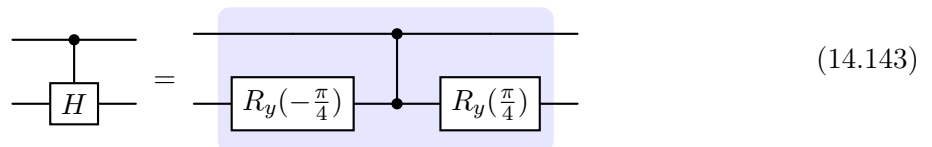
$$\rho_H = (1-p)|H\rangle\langle H| + pY|H\rangle\langle H|Y, \quad (14.141)$$

namely, a Hadamard twirling results in the magic state  $|H\rangle$  with some probability  $1-p$  and  $Y$ -error faulty magic state  $Y|H\rangle$  with probability  $p$ .

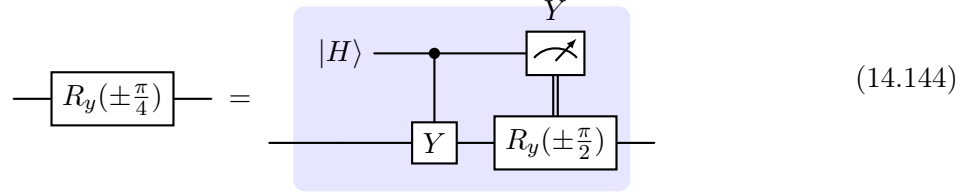
Since  $|H\rangle$  is the  $+1$  eigenstate of  $H$ , one can determine whether the noisy resource state  $\rho_H$  is the magic state  $|H\rangle$  or  $|-H\rangle$  by measuring observable  $H$  (treating  $H$  as an observable, instead of a gate). A  $+1$  outcome then corresponds to a projection onto the  $|H\rangle$  eigenspace and a  $-1$  outcome a projection onto  $|-H\rangle = Y|H\rangle$ . In particular, since the 4-qubit code encodes 2 logical qubits, we need to consider measurement of two-qubit observable  $H_1 H_2$  which projects a two-qubit state onto its  $+1$  subspace  $\text{span}\{|H\rangle|H\rangle, |-H\rangle|-H\rangle\}$  or its  $-1$  subspace  $\text{span}\{|H\rangle|-H\rangle, |-H\rangle|H\rangle\}$ . This measurement can be performed by the circuit



The control- $H$  gate is a non-Clifford gate, which can be decomposed as



whereas the  $Y$ -rotation gate  $R_y(\pm\frac{\pi}{4})$  can be performed using a  $|H\rangle$  state ancilla, a controlled Pauli gate, and measurement feed-forward (conditioning a Clifford gate on a measurement outcome) as


(14.144)

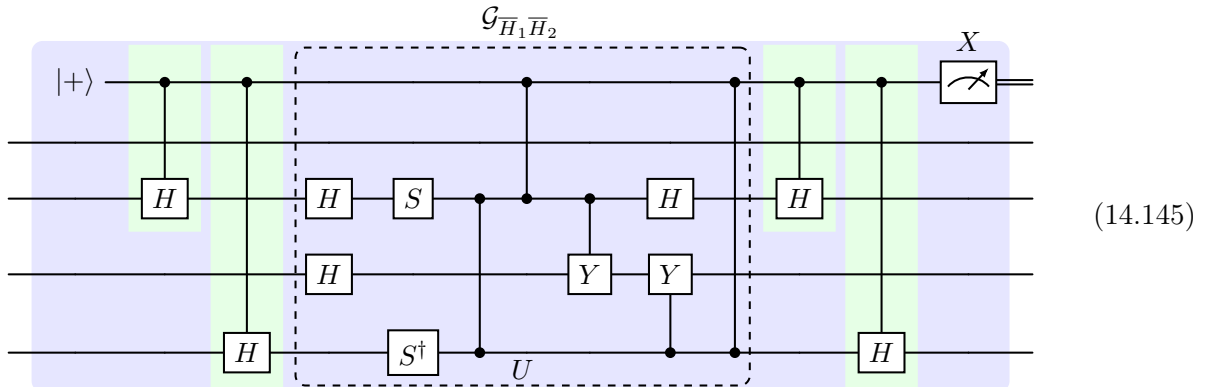
where a  $R_y(\pm\frac{\pi}{2})$  gate is applied when the  $Y$  measurement outcome is  $+1$  and an identity gate is applied otherwise.

### 14.5.3 Distilling two magic states using error-detection

Two logical magic states  $|\overline{H}\rangle$  encoded in the  $[[4, 2, 2]]$  code can be obtained probabilistically using 10 resource states  $\rho_H$  as follows:

1. Encode two resource states  $\rho_H$  to a  $[[4, 2, 2]]$  code block.
2. Perform the logical  $\overline{H}_1\overline{H}_2$  measurement on the code block. Here, we use a  $[[4, 2, 2]]$ -code gadget that implements the circuit in Eq. (14.142) by decomposing the control Hadamard gate as in Eq. (14.143) and the  $\pi/4$   $Y$ -rotation gadget as in Eq. (14.144). This gadget consumes eight resource states  $\rho_H$ , which will be described shortly.
3. A  $-1$  outcome from measuring  $\overline{H} \otimes \overline{H}$  indicates that one copy of the encoded  $\rho_H$  state has a  $\overline{Y}$  error and a  $+1$  outcome indicates that either both encoded resource states are  $|H\rangle$  or  $|-H\rangle$ .
4. Run syndrome extraction of the code by measuring the two stabilizer generators  $s_1, s_2$  in Eq. (14.137) to detect single-qubit errors, then unencode the  $[[4, 2, 2]]$  code block.
5. Accept the two output resource states if: (1)  $\overline{H}_1\overline{H}_2$  measurement outcome is  $+1$  and (2) there is no  $-1$  syndrome. If at least one of the conditions is not satisfied, discard the output state and start over.

It has been shown [274] that the  $\overline{H}_1\overline{H}_2$  measurement can be implemented using the circuit  $\mathcal{G}_{\overline{H}_1\overline{H}_2}$  given by


(14.145)

where the four controlled- $H$  highlighted in green are themselves gadgets described in Eq. (14.143) and Eq. (14.144). Since each controlled- $H$  gadget uses two  $Y$ -rotations and each  $Y$ -rotation is implemented by a gadget that uses a single resource state ancilla  $\rho_H$ , then the entire circuit  $\mathcal{G}_{\overline{H}_1\overline{H}_2}$  uses eight resource states, as mentioned earlier in step 2.

A  $Y$ -rotation gadget in Eq. (14.144) with noisy resource state ancilla  $\rho_H$  instead of  $|H\rangle$  can be described by a  $Y$  error at the ancilla with probability  $p$  which is equivalent to a  $Y$  error at its output

$$\begin{array}{c} \mathcal{G}_{R_y(\pm\pi/4)} \\ |H\rangle \text{---} \text{Y} \text{---} \bullet \text{---} \text{Y} \text{---} \text{R}_y(\pm\pi/2) \end{array} = \begin{array}{c} \mathcal{G}_{R_y(\pm\pi/4)} \\ |H\rangle \text{---} \bullet \text{---} \text{Y} \text{---} \text{R}_y(\pm\pi/2) \text{---} \text{Y} \end{array} \quad (14.146)$$

Thus, for the controlled- $H$  gadget  $\mathcal{G}_{CH}$ , a  $Y$  error on the resource ancilla of the first  $Y$  rotation is equivalent to a  $Z \otimes Y$  error at the output of  $\mathcal{G}_{CH}$

$$\begin{array}{c} \mathcal{G}_{CH} \\ |H\rangle \text{---} \text{Y} \text{---} \bullet \text{---} \text{Y} \text{---} \text{R}_y(\pm\pi/2) \end{array} = \begin{array}{c} \mathcal{G}_{CH} \\ |H\rangle \text{---} \bullet \text{---} \text{Y} \text{---} \text{R}_y(\pm\pi/2) \end{array} \quad (14.147)$$

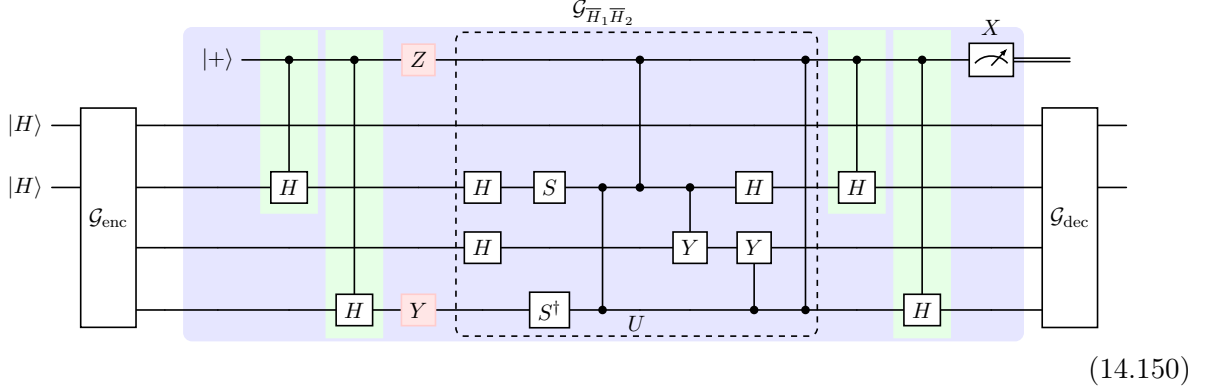
Now, for the two resource states  $\rho_H$  being encoded in the code block, there is a probability  $p$  for each copy that it has a  $Y$  error. So for an encoding circuit  $\mathcal{G}_{\text{enc}}$ , there is a probability  $p$  of a logical error  $\overline{Y}_1$  (resp.,  $\overline{Y}_2$ ) on logical qubit 1 (resp., logical qubit 2):

$$\begin{array}{c} |H\rangle \text{---} \text{Y} \\ |H\rangle \text{---} \text{Y} \end{array} \text{---} \mathcal{G}_{\text{enc}} = \begin{array}{c} |H\rangle \\ |H\rangle \end{array} \text{---} \mathcal{G}_{\text{enc}} \text{---} \overline{Y}_1 \overline{Y}_2 = \overline{Y}_1 \overline{Y}_2 |\overline{H}_1 \overline{H}_2\rangle \quad (14.148)$$

The full distillation circuit then starts with encoding circuit  $\mathcal{G}_{\text{enc}}$  followed by Hadamard measurement gadget  $\mathcal{G}_{\overline{H}_1\overline{H}_2}$  and ends with a decoding gadget  $\mathcal{G}_{\text{dec}}$  that performs syndrome extraction and unencode the code block to two physical qubits:

$$\begin{array}{c} |+\rangle \\ |H\rangle \\ |H\rangle \end{array} \text{---} \mathcal{G}_{\text{enc}} \text{---} \mathcal{G}_{\overline{H}_1\overline{H}_2} \text{---} \mathcal{G}_{\text{dec}} \quad (14.149)$$

Note that a single logical error  $\bar{Y}_j$  triggers a  $-1$  outcome from the Hadamard measurement gadget and becomes a physical  $Y_j$  error on the output state, whereas two logical errors  $\bar{Y}_1\bar{Y}_2$  become physical error  $Y_1Y_2$  at the output. A  $Y$  error on one of the eight resource ancillas for one of the first two  $\mathcal{G}_{\text{CH}}$  gadgets will become either a  $Z_aY_i$  or  $Y_i$  for  $i \in \{2, 4\}$  ( $P_a$  denotes Pauli  $P$  on the top measurement ancilla qubit) error at the end of the gadget. For example, a  $Y$  error on the ancilla resource state for first  $R_y$  gadget in the second  $\mathcal{G}_{\text{CH}}$  gadget gives us errors



It is shown in [274] that a single fault on any of the resource state ancillas in the  $\mathcal{G}_{\text{CH}}$  gadgets will trigger a syndrome flip in the decoding stage, since the  $[[4, 2, 2]]$  can detect any single-qubit error. However, more than one faulty resource state ancillas in the  $\mathcal{G}_{\text{CH}}$  gadgets may not be detected and cause a logical error.

The probability of  $k$  out of ten resource states  $\rho_H = (1-p)|H\rangle\langle H| + p|-H\rangle\langle -H|$  being faulty is given by

$$\Pr[k \text{ faulty resource states}] = \binom{10}{k} p^k (1-p)^{10-k}. \quad (14.151)$$

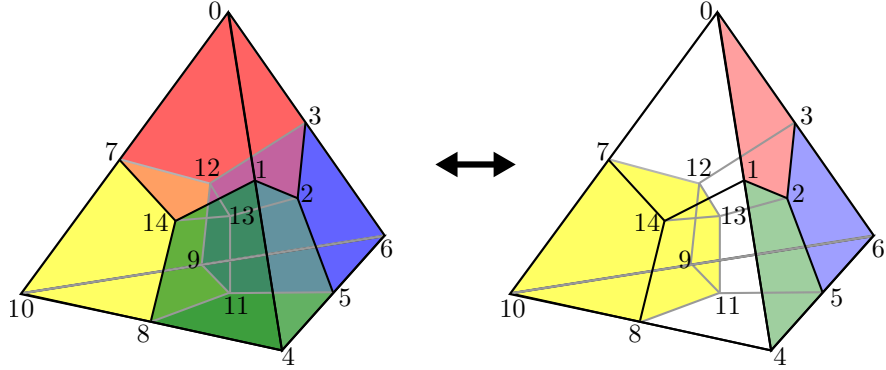
A rejection of the output state resulting from a single fault on the resource state, which is detected by either from the Hadamard check  $\mathcal{G}_{\bar{H}_1\bar{H}_2}$  or the syndrome measurement by  $\mathcal{G}_{\text{dec}}$ . On the other hand, an output state is accepted either when the resources state is non-faulty or if there are multiple faults (evading detection). So, we have an acceptance probability of

$$\begin{aligned} \Pr[\text{output accepted}] &= \Pr[0 \text{ or } k \geq 2 \text{ faulty resource states}] \\ &= (1-p)^{10} + \sum_{k=2}^{10} \binom{10}{k} p^k (1-p)^{10-k} \\ &= 1 - O(p). \end{aligned} \quad (14.152)$$

On the other hand, 2 or more faulty resource states will not trigger any syndrome flips or Hadamard-check flips, but may result in an output error. So, the probability of bad outputs being accepted is

$$\Pr[\text{bad output accepted}] \leq \sum_{k=2}^{10} \binom{10}{k} p^k (1-p)^{10-k} = O(p^2). \quad (14.153)$$

An output error happens when we accept a bad output, that is, where at least one of the output states is not a  $|H\rangle$  state, but no syndrome flips and no Hadamard-check flip. Thus the probability



**Figure 28:** Code switching between Steane’s  $[[7, 1, 3]]$  code (right-hand side face of the tetrahedron on the right-hand side) and  $[[15, 1, 3]]$  tetrahedral 3-D color code (left-hand side). (Figure reproduced from [275] with permission from the authors.)

of output error is

$$\begin{aligned}
 \Pr[\text{output error}] &= \Pr[\text{bad output} | \text{output accepted}] \\
 &\leq \frac{\Pr[\text{bad output accepted}]}{\Pr[\text{output accepted}]} \\
 &= \frac{O(p^2)}{1 - O(p)} = O(p^2).
 \end{aligned} \tag{14.154}$$

Thus, we have shown that using ten input resource states with error rate  $p$  we can use the distillation circuit in Eq. (14.150) to obtain two copies of magic state with quadratically suppressed error rate  $O(p^2)$ .

## 14.6 Code switching

As with magic-state distillation, *code switching* is a method to bypass the Eastin-Knill theorem in which one uses two stabilizer codes  $Q_1$  and  $Q_2$  such that code  $Q_1$  (resp.,  $Q_2$ ) allows for a set of transversal logical gates  $\mathcal{G}_1$  (resp.,  $\mathcal{G}_2$ ), where the two combined logical gate sets form a universal gate set. Then, if we can find two such codes where we can transform one code into the other, preferably in a manner where errors do not propagate much, we can perform a universal FTQC by switching between these two codes to implement any gates from the universal gate set as needed.

One of the most popular pairs of codes to use for code switching is the  $[[7, 1, 3]]$  Steane code [\[276, 277\]](#) and the  $[[15, 1, 3]]$  tetrahedral 3D color code [\[276, 277\]](#) (c.f. Figure 28). The Steane code allows for transversal logical  $H$  and CNOT gates, while the tetrahedral code allows for transversal logical  $T$  gate, thus forming a universal logical gate set.

As illustrated in Figure 28, we need a total of 15 qubits to do this code switching protocol where the Steane code lies only on the 7 qubits (numbered 0 to 6) on one of the triangular faces of the tetrahedron. When we encode the logical information in the Steane code, the remaining 8 qubits (on the vertices of the yellow hexahedron cell, numbered 7 to 14) do not encode any logical information. On the other hand, the tetrahedral code has 14 stabilizers: 4  $X$ -flavored and 4  $Z$ -flavored 4-qubit stabilizers supported on the hexahedra, 3  $Z$ -flavored 4-qubit stabilizers on the triangular face numbered 0 to 6, and 3  $Z$ -flavored 4-qubit stabilizers on the three interior faces of the yellow hexahedron cell (numbered 7,8,9,11,12,13).

Switching from the tetrahedral code to Steane code can be done by measuring three weight-4  $X$  Paulis on the red, green, and blue faces on the triangular face supporting the Steane code. Switching



from the Steane code to the tetrahedral code can be done by measuring three 4-qubit  $Z$  Paulis on the interior faces intersecting the red, blue, and green hexahedra (i.e., faces with vertices numbered 2,5,11,13 and 1,2,13,14 and 2,3,12,13).

A variant of code switching, *transversal code switching*, is a fault-tolerant protocol for universal quantum computation proposed in [278] which uses a self-dual (CSS) code and a (CSS) [tri-orthogonal code](#)  $\text{Ec}_{260}$ . Since a self-dual code admits a transversal logical Hadamard gate, a tri-orthogonal code admits a transversal logical  $T$ -gate, and CSS codes admit a transversal logical  $\overline{\text{CNOT}}$  gate, we can achieve universal quantum computation. Note that, as opposed to more standard code-switching protocols that switch between stabilizer groups within a subsystem code, the transversal code-switching protocol requires two separate stabilizer code blocks. However, fault tolerance of the protocol is guaranteed since all of its operations are transversal.

The protocol involves a transversal implementation of a logical teleportation from one code to another according to which gates one intends to perform. For a self-dual code, we can choose the Steane code, while for a self-orthogonal code, we can choose the tetrahedral code, so that we use a total of  $15 + 7 = 22$  qubits. If we start from the Steane code in logical state  $|\bar{\psi}\rangle_s$ , implementing a transversal logical  $T$  gate followed by a transversal logical  $H$  involves:

1. Preparing a logical  $|\bar{0}\rangle_t$  on the tetrahedral code,
2. Teleporting the Steane code logical state  $|\bar{\psi}\rangle_s$  to the tetrahedral code (a transversal  $\overline{\text{CNOT}}_{s \rightarrow t}$ , logical  $\bar{X}_s$  measurement on the Steane code block, and logical Pauli  $\bar{Z}_t$  on the tetrahedral code block conditioned on  $-1$  outcome of the  $\bar{X}_s$  measurement),
3. Applying transversal  $\bar{T}_t$  on the tetrahedral block (resulting in  $\bar{T}_t|\bar{\psi}\rangle_t$ ),
4. Teleporting the tetrahedral code state to the Steane code (a transversal  $\overline{\text{CNOT}}_{s \rightarrow t}$ , logical  $\bar{Z}_t$  measurement on the tetrahedral code block, and logical Pauli  $\bar{X}_s$  on the Steane code block conditioned on  $-1$  outcome of the  $\bar{Z}_t$  measurement), and lastly
5. Applying transversal  $\bar{H}_s$  on the Steane code block.

## 14.7 Outlook

With *FTQC*, the goal is not only to store quantum information reliably, but also to process it without allowing errors to spread faster than they can be corrected. In practice, this requires not just good code properties, but also a universal logical gate set, control over correlated faults, and an architecture with manageable overhead. A central challenge is the implementation of non-Clifford gates. In many leading architectures, this relies on *magic-state distillation*, a well-developed route to universality, but also one of the most resource-intensive parts of the stack. Much current work therefore aims to reduce its cost through improved protocols, better codes, and hardware-aware optimization. More recently, heuristic alternatives such as *magic-state cultivation* have also been proposed, promising lower-cost preparation of high-fidelity  $T$  states [279].

At the same time, approaches such as *code switching* and *code deformation* seek to combine the strengths of different encodings, for example by using one code for storage and another for specific logical operations. This reflects a broader shift: fault tolerance is increasingly understood not as a property of a single code, but of an entire stack spanning hardware, control, decoding, and compilation.

Looking further ahead, one of the most exciting directions is *constant-overhead fault tolerance* schemes [229]. Surface code-based architectures have set the benchmark for locality and thresholds, but typically require substantial overhead as the target logical performance improves. By contrast,

qLDPC codes and related constructions raise the prospect of protecting many logical qubits with far fewer physical resources, at least asymptotically. Whether this promise can be realized in practice remains open, since finite-size performance, decoder complexity, connectivity constraints, syndrome-extraction schedules, and realistic noise robustness are all crucial. Still, the possibility of going beyond the overheads of current leading schemes makes this a particularly important frontier.

More broadly, quantum error correction is evolving from the study of individual codes to the design of *fully fault-tolerant systems*. This includes faster and more realistic decoders, codes tailored to specific noise biases and hardware platforms, and tighter integration between correction, calibration, and control. It also requires addressing experimentally important effects such as leakage, correlated and non-Markovian noise, and the latency of real-time classical processing. Progress is therefore likely to come from sustained co-design across theory, experiment, and architecture rather than from any single isolated breakthrough.

At the same time, the goals of the field are becoming clearer. In the near term, a key milestone is the demonstration of *logical qubits that outperform physical qubits in a scalable and reproducible way*, including logical memories, logical gates, and small logical processors. Beyond that lies the transition from proof-of-principle error correction to genuinely useful fault-tolerant computation, where protected logical operations support algorithms of scientific or practical value. Achieving this will require not only lower logical error rates, but also credible resource estimates, robust compilation pipelines, and a better understanding of which applications are most likely to benefit first.

In this sense, quantum error correction is no longer only about suppressing decoherence. It has become the central organizing principle of quantum computing as a whole. The key questions now are not simply which code has the highest threshold or best asymptotic scaling, but which combination of codes, gates, decoders, and hardware can deliver useful logical performance under realistic constraints. The coming years will likely be shaped by this transition: from protecting qubits to engineering scalable fault-tolerant quantum computers.

## Acknowledgments

D.J.S. is supported by a graduate research fellowship from the Joint Quantum Institute (JQI) at the University of Maryland, College Park and grant ARL (W911NF-24-2-0107). S.P.J. is supported by the NSF grant OMA-2120757 (QLCI). A.T. is supported by a CQT PhD scholarship, a Google PhD fellowship program, and a CQT Young Researcher Career Development Grant. Z.S. is supported by a CQT PhD scholarship. K.B. is supported by a Hartree fellowship from the Joint Center for Quantum Information and Computer Science (QuICS) at the University of Maryland, College Park.

## References

- [1] B.P. Lanyon et al., “Towards quantum chemistry on a quantum computer”, *Nature Chemistry* **2**, 106–111 (2010).
- [2] Yudong Cao et al., “Quantum chemistry in the age of quantum computing”, *Chemical Reviews* **119**, 10856–10915 (2019).
- [3] Bela Bauer, Sergey Bravyi, Mario Motta, and Garnet Kin-Lic Chan, “Quantum algorithms for quantum chemistry and quantum materials science”, *Chemical Reviews* **120**, 12685–12717 (2020).
- [4] Sam McArdle, Suguru Endo, Alán Aspuru-Guzik, Simon C. Benjamin, and Xiao Yuan, “Quantum computational chemistry”, *Reviews of Modern Physics* **92**, 015003 (2020).
- [5] Mario Motta and Julia E. Rice, “Emerging quantum computing algorithms for quantum chemistry”, *Computational Molecular Science* **12**, e1580 (2021).
- [6] Adam Smith, M.S. Kim, Frank Pollmann, and Johannes Knolle, “Simulating quantum many-body dynamics on a current digital quantum computer”, *npj Quantum Information* **5**, 106 (2019).
- [7] Thomas Ayril, Pauline Besserve, Denis Lacroix, and Edgar Andres Ruiz Guzman, “Quantum computing with and for many-body physics”, *The European Physical Journal A* **59**, 227 (2023).
- [8] Benedikt Fauseweh, “Quantum many-body simulations on digital quantum computers: State-of-the-art and future challenges”, *Nature Communications* **15**, 2123 (2024).
- [9] P.W. Shor, “Algorithms for quantum computation: discrete logarithms and factoring”, *SFCS '94: Proceedings of the 35th Annual Symposium on Foundations of Computer Science*, 124–134 (1994).
- [10] Nicolas Gisin, Grégoire Ribordy, Wolfgang Tittel, and Hugo Zbinden, “Quantum cryptography”, *Reviews of Modern Physics* **74**, 145 (2002).
- [11] Christopher Portmann and Renato Renner, “Security in quantum cryptography”, *Reviews of Modern Physics* **94**, 025008 (2022).
- [12] Feihu Xu, Xiongfeng Ma, Qiang Zhang, Hoi-Kwong Lo, and Jian-Wei Pan, “Secure quantum key distribution with realistic devices”, *Reviews of Modern Physics* **92**, 025002 (2020).
- [13] C.L. Degen, F. Reinhard, and P. Cappellaro, “Quantum sensing”, *Reviews of Modern Physics* **89**, 035002 (2017).
- [14] Benjamin CB Symons, David Galvin, Emre Sahin, Vassil Alexandrov, and Stefano Mensa, “A practitioner’s guide to quantum algorithms for optimisation problems”, *Journal of Physics A: Mathematical and Theoretical* **56**, 453001 (2023).
- [15] Amira Abbas et al., “Challenges and opportunities in quantum optimization”, *Nature Reviews Physics* **6**, 718–735 (2024).
- [16] Richard P. Feynman, “Simulating physics with computers”, *International Journal of Theoretical Physics* **21**, 467–488 (1982).
- [17] Seth Lloyd, “Universal quantum simulators”, *Science* **273**, 1073–1078 (1996).
- [18] Alán Aspuru-Guzik, Anthony D. Dutoi, Peter J. Love, and Martin Head-Gordon, “Simulated quantum computation of molecular energies”, *Science* **309**, 1704–1707 (2005).

- 
- [19] He Ma, Marco Govoni, and Giulia Galli, “Quantum simulations of materials on near-term quantum computers”, *npj Computational Materials* **6**, 85 (2020).
  - [20] Andrew J. Daley, Immanuel Bloch, Christian Kokail, Stuart Flannigan, Natalie Pearson, Matthias Troyer, and Peter Zoller, “Practical quantum advantage in quantum simulation”, *Nature* **607**, 667–676 (2022).
  - [21] William K Wootters and Wojciech H Zurek, “A single quantum cannot be cloned”, *Nature* **299**, 802–803 (1982).
  - [22] Anthony Ransford et al., “Helios: A 98-qubit trapped-ion quantum computer”, *arXiv:2511.05465* (2025).
  - [23] Dorit Aharonov and Michael Ben-Or, “Fault-tolerant quantum computation with constant error rate”, *SIAM Journal on Computing* **38**, 1207–1282 (2008).
  - [24] E. Knill, R. Laflamme, and W. Zurek, “Threshold accuracy for quantum computation”, *arXiv:quant-ph/9610011* (1996).
  - [25] A Yu Kitaev, “Fault-tolerant quantum computation by anyons”, *Annals of Physics* **303**, 2–30 (2003).
  - [26] Austin G. Fowler, Matteo Mariantoni, John M. Martinis, and Andrew N. Cleland, “Surface codes: towards practical large-scale quantum computation”, *Physical Review A* **86**, 032324 (2012).
  - [27] Paul Webster, Lucas Berent, Omprakash Chandra, Evan T. Hockings, Nouédyn Baspin, Felix Thomsen, Samuel C. Smith, and Lawrence Z. Cohen, “The Pinnacle Architecture: Reducing the cost of breaking RSA-2048 to 100 000 physical qubits using quantum LDPC codes”, *arXiv:2602.11457* (2026).
  - [28] Madelyn Cain et al., “Shor’s algorithm is possible with as few as 10,000 reconfigurable atomic qubits”, *arXiv:2603.28627* (2026).
  - [29] Neereja Sundaresan et al., “Demonstrating multi-round subsystem quantum error correction using matching and maximum likelihood decoders”, *Nature Communications* **14**, 2852 (2023).
  - [30] Google Quantum AI, “Suppressing quantum errors by scaling a surface code logical qubit”, *Nature* **614**, 676–681 (2023).
  - [31] Dolev Bluvstein *et al.*, “Logical quantum processor based on reconfigurable atom arrays”, *Nature* **626**, 58–65 (2024).
  - [32] Yifan Hong, Elijah Durso-Sabina, David Hayes, and Andrew Lucas, “Entangling four logical qubits beyond break-even in a nonlocal code”, *Physical Review Letters* **133**, 180601 (2024).
  - [33] Pedro Sales Rodriguez et al., “Experimental demonstration of logical magic state distillation”, *Nature* **645**, 620–625 (2025).
  - [34] A. Paetznick et al., “Demonstration of logical qubits and repeated error correction with better-than-physical error rates”, *arXiv:2404.02280* (2024).
  - [35] N. Lacroix et al., “Scaling and logic in the colour code on a superconducting quantum processor”, *Nature* **645**, 614–619 (2025).
  - [36] Alec Eickbusch et al., “Demonstration of dynamic surface codes”, *Nature Physics* **21**, 1994–2001 (2025).
  - [37] Google Quantum AI and Collaborators, “Quantum error correction below the surface code threshold”, *Nature* **638**, 920–926 (2025).

- [38] Friederike Butt, Ivan Pogorelov, Robert Freund, Alex Steiner, Marcel Meyer, Thomas Monz, and Markus Müller, “Demonstration of measurement-free universal logical quantum computation”, *Nature Communications* **17**, 995 (2026).
- [39] Michael A. Nielsen and Isaac L. Chuang, *Quantum Computation and Quantum Information*, 10th Anniversary edition (Cambridge University Press, 2010).
- [40] A. Yu Kitaev, A.H. Shen, and M.N. Vyali, *Classical and Quantum Computation*, 1st ed., Graduate Studies in Mathematics (American Mathematical Society, Providence, Rhode Island, 2002).
- [41] Scott Aaronson, *Introduction to Quantum Information Science Lecture Notes*, 2018.
- [42] Ronald de Wolf, “Quantum computing: lecture notes”, *arXiv:1907.09415* (2019).
- [43] Richard Jozsa, *Quantum information and computation lecture notes*, 2019.
- [44] John Watrous, “Understanding quantum information and computation”, *arXiv:2507.11536* (2025).
- [45] John Preskill, *Course information for Physics 219/Computer Science 219: Quantum Computation*, 2025.
- [46] Andrew M. Childs, *Lecture notes on quantum algorithms*, 2025.
- [47] Michael Sipser, *Introduction to the Theory of Computation*, 3rd ed. (Cengage Learning, Boston, MA, 2012).
- [48] F.J. MacWilliams and N.J.A. Sloane, *The Theory of Error-Correcting Codes* (North Holland Publishing Co., 1977).
- [49] W. Cary Huffman and Vera Pless, *Fundamentals of Error-Correcting Codes*, 1st ed. (Cambridge University Press, New York, 2003).
- [50] Shu Lin and Juane Li, *Fundamentals of Classical and Modern Error-Correcting Codes* (Cambridge University Press, Cambridge, 2021).
- [51] R.W. Hamming, “Error detecting and error correcting codes”, *The Bell System Technical Journal* **29**, 147–160 (1950).
- [52] C.E. Shannon, “A mathematical theory of communication”, *The Bell System Technical Journal* **27**, 379–423 (1948).
- [53] Gilbert Strang, *Introduction to Linear Algebra, Sixth Edition* (Wellesley-Cambridge Press, Philadelphia, PA, 2022).
- [54] Peter W. Shor, “Scheme for reducing decoherence in quantum computer memory”, *Physical Review A* **52**, R2493(R) (1995).
- [55] Mark M. Wilde, *Quantum Information Theory*, 2nd ed. (Cambridge University Press, Cambridge, 2017).
- [56] Peter W. Shor, “Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer”, *SIAM Journal on Computing* **26**, 1484–1509 (1997).
- [57] Asher Peres, “Reversible logic and quantum computers”, *Physical Review A* **32**, 3266 (1985).
- [58] A.M. Steane, “Error correcting-codes in quantum theory”, *Physical Review Letters* **77**, 793 (1996).
- [59] Raymond Laflamme, Cesar Miquel, Juan Pablo Paz, and Wojciech Hubert Zurek, “Perfect quantum error correcting code”, *Physical Review Letters* **77**, 198 (1996).

- 
- [60] Charles H. Bennett, David P. DiVincenzo, John A. Smolin, and William K. Wootters, “Mixed-state entanglement and quantum error correction”, *Physical Review A* **54**, 3824 (1996).
- [61] A.R. Calderbank and Peter W. Shor, “Good quantum error-correcting codes exist”, *Physical Review A* **54**, 1098 (1996).
- [62] Andrew Steane, “Multiple-particle interference and quantum error correction”, *Proceedings of the Royal Society A: Mathematical, Physical and Engineering Sciences* **452**, 2551–2577 (1996).
- [63] Daniel Gottesman, “Stabilizer codes and quantum error correction”, *arXiv:quant-ph/9705052* (1997).
- [64] P.W. Shor, “Fault-tolerant quantum computation”, in *Proceedings of the 37th Annual Symposium on Foundations of Computer Science*, FOCS ’96 (1996), p. 56.
- [65] A. Yu Kitaev, “Quantum computations: algorithms and error correction”, *Russian Mathematical Surveys* **52**, 1191 (1997).
- [66] J.I. Cirac, T. Pellizzari, and P. Zoller, “Enforcing coherent evolution in dissipative quantum dynamics”, *Science* **273**, 1207–1210 (1996).
- [67] Wojciech Hubert Zurek and Raymond Laflamme, “Quantum logical operations on encoded qubits”, *Physical Review Letters* **77**, 4683 (1996).
- [68] Daniel Gottesman, “Theory of fault-tolerant quantum computation”, *Physical Review A* **57**, 127 (1998).
- [69] Dorit Aharonov and Michael Ben-Or, “Fault tolerant quantum computation with constant error”, *arXiv:quant-ph/9611025* (1996).
- [70] Panos Aliferis, Daniel Gottesman, and John Preskill, “Quantum accuracy threshold for concatenated distance-3 codes”, *Quantum Information and Computation* **6**, 97–165 (2006).
- [71] Eric M. Rains, R.H. Hardin, Peter W. Shor, and N.J.A. Sloane, “A nonadditive quantum code”, *Physical Review Letters* **79**, 953 (1997).
- [72] Debbie W. Leung, M.A. Nielsen, Isaac L. Chuang, and Yoshihisa Yamamoto, “Approximate quantum error correction can lead to better codes”, *Physical Review A* **56**, 2567 (1997).
- [73] Daniel Gottesman, “Fault-tolerant quantum computation with higher-dimensional systems”, *Chaos, Solitons & Fractals* **10**, 1749–1758 (1999).
- [74] Eric M Rains, “Nonbinary quantum codes”, *IEEE Transactions on Information Theory* **45**, 1827–1832 (2002).
- [75] Artur Ekert and Chiara Macchiavello, “Quantum error correction for communication”, *Physical Review Letters* **77**, 2585 (1996).
- [76] Daniel Gottesman, “Class of quantum error-correcting codes saturating the quantum Hamming bound”, *Physical Review A* **54**, 1862 (1996).
- [77] Peter Shor and Raymond Laflamme, “Quantum analog of the MacWilliams identities for classical coding theory”, *Physical Review Letters* **78**, 1600 (1997).
- [78] Alexei Ashikhmin, “Remarks on bounds for quantum codes”, *arXiv:quant-ph/9705037* (1997).
- [79] A. Ashikhmin and S. Litsyu, “Upper bounds on the size of quantum codes”, *IEEE Transactions on Information Theory* **45**, 1206–1215 (1999).
- [80] E.M. Rains, “Quantum weight enumerators”, *IEEE Transactions on Information Theory* **44**, 1388–1394 (1998).



- 
- [81] E.M. Rains, “Quantum shadow enumerators”, *IEEE Transactions on Information Theory* **45**, 2361–2366 (1999).
  - [82] E.M. Rains, “Monotonicity of the quantum linear programming bound”, *IEEE Transactions on Information Theory* **45**, 2489–2492 (1999).
  - [83] Eric Dennis, Alexei Kitaev, Andrew Landahl, and John Preskill, “Topological quantum memory”, *Journal of Mathematical Physics* **43**, 4452–4505 (2002).
  - [84] Sergey Bravyi and Alexei Kitaev, “Universal quantum computation with ideal Clifford gates and noisy ancillas”, *Physical Review A* **71**, 022316 (2005).
  - [85] H. Bombin and M. A. Martin-Delgado, “Topological quantum distillation”, *Physical Review Letters* **97**, 180501 (2006).
  - [86] H. Bombin and M.A. Martin-Delgado, “Statistical mechanical models and topological color codes”, *Physical Review A* **77**, 042322 (2008).
  - [87] Daniel Lidar and Todd Brun, *Quantum Error Correction* (Cambridge University Press, 2013).
  - [88] Frank Gaitan, *Quantum Error Correction and Fault Tolerant Quantum Computing*, 1st ed. (CRC Press, 2013).
  - [89] Daniel Gottesman, *Surviving as a Quantum Computer in a Classical World* (Unpublished, 2024).
  - [90] Michael Kastoryano, *Quantum Error Correction*, 2018.
  - [91] Joschka Roffe, “Quantum error correction: an introductory guide”, *Contemporary Physics* **60**, 226–245 (2019).
  - [92] Victor V. Albert and Philippe Faist, *The Error Correction Zoo*, 2025.
  - [93] David S. Dummit and Richard M. Foote, *Abstract Algebra*, 3rd ed. (John Wiley & Sons, Hoboken, NJ, 2003).
  - [94] C.C. Pinter, *A Book of Abstract Algebra*, International series in pure and applied mathematics (McGraw-Hill, 1982).
  - [95] Zachary P. Bradshaw, Jeffrey J. Dale, and Ethan N. Evans, “Introduction to quantum error correction with stabilizer codes”, *Annals of Physics* **487**, 170353 (2026).
  - [96] Andrew J Landahl, Jonas T Anderson, and Patrick R Rice, “Fault-tolerant quantum computing with color codes”, *arXiv:1108.5738* (2011).
  - [97] David Poulin, “Stabilizer formalism for operator quantum error correction”, *Physical Review Letters* **95**, 230504 (2005).
  - [98] H. Bombin, “Topological subsystem codes”, *Physical Review A* **81**, 032301 (2010).
  - [99] Héctor Bombín, “Gauge color codes: optimal transversal gates and gauge fixing in topological stabilizer codes”, *New Journal of Physics* **17**, 083002 (2015).
  - [100] Nikolas P. Breuckmann and Barbara M. Terhal, “Constructions and Noise Threshold of Hyperbolic Surface Codes”, *IEEE Transactions on Information Theory* **62**, 3731–3744 (2016).
  - [101] Oscar Higgott and Nikolas P. Breuckmann, “Constructions and Performance of Hyperbolic and Semi-Hyperbolic Floquet Codes”, *PRX Quantum* **5**, 040327 (2024).
  - [102] James R. Munkres, *Topology*, 2nd ed. (Prentice Hall, 2000).
  - [103] Seth Lloyd, Silvano Garnerone, and Paolo Zanardi, “Quantum algorithms for topological and geometric analysis of data”, *Nature Communications* **7**, 10138 (2016).

- 
- [104] Allen Hatcher, *Algebraic Topology* (Cambridge University Press, New York, 2001).
  - [105] Dolev Bluvstein et al., “A fault-tolerant neutral-atom architecture for universal quantum computation”, *Nature* **649**, 39–46 (2026).
  - [106] Margarita Davydova, Nathanan Tantivasadakarn, and Shankar Balasubramanian, “Floquet Codes without parent subsystem codes”, *PRX Quantum* **4**, 020341 (2023).
  - [107] David Kribs, Raymond Laflamme, and David Poulin, “Unified and generalized approach to quantum error correction”, *Physical Review Letters* **94**, 180501 (2005).
  - [108] David W Kribs, Raymond Laflamme, David Poulin, and Maia Lesosky, “Operator quantum error correction”, *arXiv:quant-ph/0504189* (2005).
  - [109] Dave Bacon, “Operator quantum error-correcting subsystems for self-correcting quantum memories”, *Physical Review A* **73**, 012340 (2006).
  - [110] M. Sohaib Alam and Eleanor Rieffel, “Dynamical logical qubits in the Bacon-Shor code”, *Physical Review A* **112**, Advance online publication (2025).
  - [111] Sergey Bravyi, Guillaume Duclos-Cianci, David Poulin, and Martin Suchara, “Subsystem surface codes with three-qubit check operators”, *arXiv:1207.1443* (2012).
  - [112] Oscar Higgott and Nikolas P. Breuckmann, “Subsystem codes with high thresholds by gauge fixing and reduced qubit overhead”, *Physical Review X* **11**, 031039 (2021).
  - [113] H. Bombin and M. A. Martin-Delgado, “Exact topological quantum order in  $D = 3$  and beyond: branyons and brane-net condensates”, *Physical Review B* **75**, 075103 (2007).
  - [114] Friederike Butt, Sascha Heußen, Manuel Rispler, and Markus Müller, “Fault-Tolerant Code-Switching Protocols for Near-Term Quantum Processors”, *PRX Quantum* **5**, 020345 (2024).
  - [115] Sergey Bravyi and Andrew Cross, “Doubled Color Codes”, *arXiv:1509.03239* (2015).
  - [116] Matthew Sullivan, “Code Conversion with the Quantum Golay Code for a Universal Transversal Gate Set”, *Physical Review A* **109**, 042416 (2024).
  - [117] Shubham P. Jain and Victor V. Albert, “Transversal Clifford and T-Gate Codes of Short Length and High Distance”, *IEEE Journal on Selected Areas in Information Theory* **6**, 127–137 (2025).
  - [118] Alexei Kitaev, “Anyons in an exactly solved model and beyond”, *Annals of Physics* **321**, 2–111 (2006).
  - [119] Craig Gidney, Michael Newman, Austin Fowler, and Michael Broughton, “A Fault-Tolerant Honeycomb Memory”, *Quantum* **5**, 605 (2021).
  - [120] Matthew B. Hastings and Jeongwan Haah, “Dynamically generated logical qubits”, *Quantum* **5**, 564 (2021).
  - [121] Daniel Gottesman, “The Heisenberg representation of quantum computers”, *arXiv:quant-ph/9807006* (1998).
  - [122] Andrew Tanggara, Mile Gu, and Kishor Bharti, “Simple construction of qudit Floquet codes on a family of lattices”, *arXiv:2410.02022* (2025).
  - [123] Matteo Mezzadri, Alessandro Chiesa, Luca Lepori, and Stefano Carretta, “Fault-tolerant computing with single-qudit encoding in a molecular spin”, *Materials Horizons* **11**, 4961–4969 (2024).
  - [124] Sumin Lim, Junjie Liu, and Arzhang Ardavan, “Fault-tolerant qubit encoding using a spin-7/2 qudit”, *Physical Review A* **108**, 062403 (2023).



- 
- [125] Victor V. Albert, “Bosonic Coding: Introduction and Use Cases”, in *Quantum Fluids of Light and Matter*, Vol. 209 (2025), pp. 79–107.
- [126] Victor V. Albert, Jacob P. Covey, and John Preskill, “Robust encoding of a qubit in a molecule”, *Physical Review X* **10**, 031050 (2020).
- [127] Shubham P. Jain, Eric R. Hudson, Wesley C. Campbell, and Victor V. Albert, “Absorption-Emission Codes for Atomic and Molecular Quantum Information Platforms”, *Physical Review Letters* **133**, 260601 (2024).
- [128] George B. Arfken and Hans J. Weber, *Mathematical Methods for Physicists*, 6th (Elsevier, 2005).
- [129] Victor V. Albert et al., “Performance and structure of single-mode bosonic codes”, *Physical Review A* **97**, 032346 (2018).
- [130] Peter Leviant, Qian Xu, Liang Jiang, and Serge Rosenblum, “Quantum Capacity and Codes for the Bosonic Loss-Dephasing Channel”, *Quantum* **6**, 821 (2022).
- [131] Jonathan Conrad, “The Fabulous World of GKP Codes”, *arXiv:2412.02442* (2024).
- [132] Michele N. Notarnicola, Stefano Olivares, and Matteo G. A. Paris, “Joint Estimation of Noise and Nonlinearity in Kerr Systems”, *APL Quantum* **1**, 036118 (2024).
- [133] Shruti Puri, Samuel Boutin, and Alexandre Blais, “Engineering the Quantum States of Light in a Kerr-nonlinear Resonator by Two-Photon Driving”, *npj Quantum Information* **3**, 18 (2017).
- [134] Fernando Valadares, Ni-Ni Huang, Kyle Timothy Ng Chu, Aleksandr Dorogov, Weipin Chua, Lingda Kong, Pengtao Song, and Yvonne Y. Gao, “On-Demand Transposition across Light-Matter Interaction Regimes in Bosonic cQED”, *Nature Communications* **15**, 5816 (2024).
- [135] Nissim Ofek et al., “Extending the lifetime of a quantum bit with error correction in superconducting circuits”, *Nature* **536**, 441–445 (2016).
- [136] Yaxing Zhang, Jacob C. Curtis, Christopher S. Wang, R. J. Schoelkopf, and S. M. Girvin, “Drive-Induced Nonlinearities of Cavity Modes Coupled to a Transmon Ancilla”, *Physical Review A* **105**, 022423 (2022).
- [137] Iivari Pietikäinen, Ondřej Černotík, Alec Eickbusch, Aniket Maiti, John W.O. Garmon, Radim Filip, and Steven M. Girvin, “Strategies and Trade-Offs for Controllability and Memory Time of Ultra-High-Quality Microwave Cavities in Circuit Quantum Electrodynamics”, *PRX Quantum* **5**, 040307 (2024).
- [138] Maxime Boissonneault, J. M. Gambetta, and Alexandre Blais, “Nonlinear Dispersive Regime of Cavity QED: The Dressed Dephasing Model”, *Physical Review A* **77**, 060305 (2008).
- [139] Takahiro Tsunoda et al., “Error-Detectable Bosonic Entangling Gates with a Noisy Ancilla”, *PRX Quantum* **4**, 020354 (2023).
- [140] Akshay Koottandavida et al., “Erasure Detection of a Dual-Rail Qubit Encoded in a Double-Post Superconducting Cavity”, *Physical Review Letters* **132**, 180601 (2024).
- [141] Yao Lu et al., “High-Fidelity Parametric Beamsplitting with a Parity-Protected Converter”, *Nature Communications* **14**, 5767 (2023).
- [142] Harry Levine et al., “Demonstrating a Long-Coherence Dual-Rail Erasure Qubit Using Tunable Transmons”, *Physical Review X* **14**, 011051 (2024).

- 
- [143] Wenhui Huang et al., “Logical multi-qubit entanglement with dual-rail superconducting qubits”, *Nature Physics*, 1–7 (2026).
  - [144] Kevin S Chou et al., “A superconducting dual-rail cavity qubit with erasure-detected logical measurements”, *Nature Physics* **20**, 1454–1460 (2024).
  - [145] Nitish Mehta et al., “Bias-preserving and error-detectable entangling operations in a superconducting dual-rail system”, *arXiv:2503.10935* (2025).
  - [146] PsiQuantum, “A manufacturable platform for photonic quantum computing”, *Nature* **641**, 876–883 (2025).
  - [147] Marios H. Michael, Matti Silveri, R. T. Brierley, Victor V. Albert, Juha Salmilehto, Liang Jiang, and S. M. Girvin, “New Class of Quantum Error-Correcting Codes for a Bosonic Mode”, *Physical Review X* **6**, 031006 (2016).
  - [148] Daniel Gottesman, Alexei Kitaev, and John Preskill, “Encoding a Qubit in an Oscillator”, *Physical Review A* **64**, 012310 (2001).
  - [149] Jonathan Conrad, Jens Eisert, and Francesco Arzani, “Gottesman-Kitaev-Preskill codes: A lattice perspective”, *Quantum* **6**, 648 (2022).
  - [150] Jonathan Conrad, Ansgar G Burchards, and Steven T Flammia, “Lattices, Gates, and Curves: GKP codes as a Rosetta stone”, *arXiv:2407.03270* (2024).
  - [151] Jonathan Conrad, Joseph T. Iosue, Ansgar G. Burchards, and Victor V. Albert, “Continuous-Variable Designs and Design-Based Shadow Tomography from Random Lattices”, *Physical Review Letters* **135**, 060802 (2025).
  - [152] Barbara M. Terhal, Jonathan Conrad, and Christophe Vuillot, “Towards Scalable Bosonic Quantum Error Correction”, *Quantum Science and Technology* **5**, 043001 (2020).
  - [153] Arne L. Grimsmo and Shruti Puri, “Quantum error correction with the Gottesman-Kitaev-Preskill code”, *PRX Quantum* **2**, 020101 (2021).
  - [154] Anthony J. Brady, Alec Eickbusch, Shraddha Singh, Jing Wu, and Quntao Zhuang, “Advances in bosonic quantum error correction with Gottesman-Kitaev-Preskill codes: Theory, engineering and applications”, *Progress in Quantum Electronics* **93**, 100496 (2024).
  - [155] Richard A Wolf and Pavithran Iyer, “No physics required! A visual-based introduction to GKP qubits for computer scientists”, in *2025 IEEE International Conference on Quantum Computing and Engineering (QCE)*, Vol. 3 (IEEE, 2025), pp. 166–173.
  - [156] Yaoling Yang, Andrew Tanggara, Tobias Haug, and Kishor Bharti, “Quantum Cubature Codes”, *arXiv:2511.23316* (2025).
  - [157] Shubham P Jain, Joseph T Iosue, Alexander Barg, and Victor V Albert, “Quantum spherical codes”, *Nature Physics* **20**, 1300–1305 (2024).
  - [158] Yijia Xu, Yixu Wang, Christophe Vuillot, and Victor V. Albert, “Letting the Tiger out of Its Cage: Bosonic Coding without Concatenation”, *Physical Review X* **15**, 041025 (2025).
  - [159] Bryan Eastin and Emanuel Knill, “Restrictions on transversal encoded quantum gate sets”, *Physical Review Letters* **102**, 110502 (2009).
  - [160] Seth Lloyd and Samuel L. Braunstein, “Quantum Computation over Continuous Variables”, *Physical Review Letters* **82**, 1784–1787 (1999).
  - [161] Christian Weedbrook, Stefano Pirandola, Raul Garcia-Patron, Nicolas J. Cerf, Timothy C. Ralph, Jeffrey H. Shapiro, and Seth Lloyd, “Gaussian Quantum Information”, *Reviews of Modern Physics* **84**, 621–669 (2012).

- 
- [162] Jaehak Lee, Nuri Kang, Seok-Hyung Lee, Hyunseok Jeong, Liang Jiang, and Seung-Woo Lee, “Fault-Tolerant Quantum Computation by Hybrid Qubits with Bosonic Cat-Code and Single Photons”, *PRX Quantum* **5**, 030322 (2024).
  - [163] Kyungjoo Noh and Christopher Chamberland, “Fault-Tolerant Bosonic Quantum Error Correction with the Surface-GKP Code”, *Physical Review A* **101**, 012316 (2020).
  - [164] Christophe Vuillot, Hamed Asasi, Yang Wang, Leonid P. Pryadko, and Barbara M. Terhal, “Quantum Error Correction with the Toric-GKP Code”, *Physical Review A* **99**, 032344 (2019).
  - [165] Jiaxuan Zhang, Yu-Chun Wu, and Guo-Ping Guo, “Concatenation of the Gottesman-Kitaev-Preskill Code with the XZZX Surface Code”, *Physical Review A* **107**, 062408 (2023).
  - [166] Wen-Long Ma, Shruti Puri, Robert J. Schoelkopf, Michel H. Devoret, S. M. Girvin, and Liang Jiang, “Quantum Control of Bosonic Modes with Superconducting Circuits”, *Science Bulletin* **66**, 1789–1805 (2021).
  - [167] Chuanlong Ma et al., “Scalable Optical Links for Controlling Bosonic Quantum Processors”, *arXiv:2512.10706* (2025).
  - [168] Xinyuan You et al., “Crosstalk-Robust Quantum Control in Multimode Bosonic Systems”, *Physical Review Applied* **22**, 044072 (2024).
  - [169] Qian Xu, Pei Zeng, Daohong Xu, and Liang Jiang, “Fault-Tolerant Operation of Bosonic Qubits with Discrete-Variable Ancillae”, *Physical Review X* **14**, 031016 (2024).
  - [170] Barbara M Terhal, “Quantum error correction for quantum memories”, *Reviews of Modern Physics* **87**, 307–346 (2015).
  - [171] Luka Skoric, Dan E Browne, Kenton M Barnes, Neil I Gillespie, and Earl T Campbell, “Parallel window decoding enables scalable fault tolerant quantum computation”, *Nature Communications* **14**, 7040 (2023).
  - [172] Pavel Panteleev and Gleb Kalachev, “Degenerate quantum LDPC codes with good finite length performance”, *Quantum* **5**, 585 (2021).
  - [173] Antonio deMarti iOlius, Patricio Fuentes, Román Orús, Pedro M Crespo, and Josu Etxezarreta Martinez, “Decoding algorithms for surface codes”, *Quantum* **8**, 1498 (2024).
  - [174] Pavithran Iyer and David Poulin, “Hardness of decoding quantum stabilizer codes”, *IEEE Transactions on Information Theory* **61**, 5209–5223 (2015).
  - [175] Min-Hsiu Hsieh and François Le Gall, “NP-hardness of decoding quantum error-correction codes”, *Physical Review A* **83**, 052331 (2011).
  - [176] Sergey Bravyi, Matthias Englbrecht, Robert König, and Nolan Pearld, “Correcting coherent errors with surface codes”, *npj Quantum Information* **4**, 55 (2018).
  - [177] Craig Gidney, “Stim: a fast stabilizer circuit simulator”, *Quantum* **5**, 497 (2021).
  - [178] Sergey Bravyi, Andrew W Cross, Jay M Gambetta, Dmitri Maslov, Patrick Rall, and Theodore J Yoder, “High-threshold and low-overhead fault-tolerant quantum memory”, *Nature* **627**, 778–782 (2024).
  - [179] July X Li and Pascal O Vontobel, “LP decoding of quantum stabilizer codes”, in **2018 IEEE International Symposium on Information Theory (ISIT)** (2018), pp. 1306–1310.
  - [180] Omar Fawzi, Lucien Grouès, and Anthony Leverrier, “Linear programming decoder for hypergraph product quantum codes”, *2020 IEEE Information Theory Workshop (ITW)*, 1–5 (2021).

- 
- [181] Shouzen Gu and Mehdi Soleimanifar, “Power and Limitations of Linear Programming Decoder for Quantum LDPC Codes”, *arXiv:2508.04769* (2025).
  - [182] Nirupam Basak, Ankith Mohan, Andrew Tanggara, Tobias Haug, Goutam Paul, and Kishor Bharti, “Hierarchical quantum decoders”, *arXiv:2601.21715* (2026).
  - [183] Jack Edmonds, “Paths, trees, and flowers”, *Canadian Journal of Mathematics* **17**, 449–467 (1965).
  - [184] Oscar Higgott and Craig Gidney, “Sparse blossom: correcting a million errors per core second with minimum-weight matching”, *Quantum* **9**, 1600 (2025).
  - [185] Kaavya Sahay and Benjamin J Brown, “Decoder for the triangular color code by matching on a Möbius strip”, *PRX Quantum* **3**, 010310 (2022).
  - [186] Craig Gidney and Cody Jones, “New circuits and an open source decoder for the color code”, *arXiv:2312.08813* (2023).
  - [187] Yue Wu, Binghong Li, Kathleen Chang, Shruti Puri, and Lin Zhong, “Minimum-weight parity factor decoder for quantum error correction”, *arXiv:2508.04969* (2025).
  - [188] Nicolas Delfosse and Naomi H Nickerson, “Almost-linear time decoding algorithm for topological codes”, *Quantum* **5**, 595 (2021).
  - [189] Ben Barber et al., “A real-time, scalable, fast and resource-efficient decoder for a quantum computer”, *Nature Electronics* **8**, 84–91 (2025).
  - [190] Frank R Kschischang, Brendan J Frey, and H-A Loeliger, “Factor graphs and the sum-product algorithm”, *IEEE Transactions on Information Theory* **47**, 498–519 (2001).
  - [191] David JC MacKay, *Information Theory, Inference and Learning Algorithms* (Cambridge university press, 2003).
  - [192] Tom Richardson and Ruediger Urbanke, *Modern Coding Theory* (Cambridge University Press, 2008).
  - [193] David Poulin and Yeojin Chung, “On the iterative decoding of sparse quantum codes”, *Quantum Information & Computation* **8**, 987–1000 (2008).
  - [194] Stergios Koutsoumpas, Hasan Sayginel, Mark Webster, and Dan E Browne, “Automorphism ensemble decoding of quantum LDPC codes”, *arXiv:2503.01738* (2025).
  - [195] Tristan Müller, Thomas Alexander, Michael E Beverland, Markus Bühler, Blake R Johnson, Thilo Maurer, and Drew Vandeth, “Improved belief propagation is sufficient for real-time decoding of quantum memory”, *arXiv:2506.01779* (2025).
  - [196] Min Ye, Dave Wecker, and Nicolas Delfosse, “Beam search decoder for quantum LDPC codes”, *arXiv:2512.07057* (2025).
  - [197] Joschka Roffe, David R White, Simon Burton, and Earl Campbell, “Decoding across the quantum low-density parity-check code landscape”, *Physical Review Research* **2**, 043423 (2020).
  - [198] Timo Hillmann, Lucas Berent, Armanda O Quintavalle, Jens Eisert, Robert Wille, and Joschka Roffe, “Localized statistics decoding: A parallel decoding algorithm for quantum low-density parity-check codes”, *Nature Communications* **16**, 8214 (2025).
  - [199] Andrew J Ferris and David Poulin, “Tensor networks and quantum error correction”, *Physical Review Letters* **113**, 030501 (2014).
  - [200] Sergey Bravyi, Martin Suchara, and Alexander Vargo, “Efficient algorithms for maximum likelihood decoding in the surface code”, *Physical Review A* **90**, 032326 (2014).

- 
- [201] Christopher T Chubb, “General tensor network decoding of 2D Pauli codes”, *arXiv:2101.04125* (2021).
- [202] Roy Alkabetz and Itai Arad, “Tensor networks contraction and the belief propagation algorithm”, *Physical Review Research* **3**, 023073 (2021).
- [203] Savvas Varsamopoulos, Ben Criger, and Koen Bertels, “Decoding small surface codes with feedforward neural networks”, *Quantum Science and Technology* **3**, 015004 (2017).
- [204] Giacomo Torlai and Roger G Melko, “Neural decoder for topological codes”, *Physical Review Letters* **119**, 030501 (2017).
- [205] Boris M Varbanov, Marc Serra-Peralta, David Byfield, and Barbara M Terhal, “Neural network decoder for near-term surface-code experiments”, *Physical Review Research* **7**, 013029 (2025).
- [206] Johannes Bausch et al., “Learning high-accuracy error decoding for quantum processors”, *Nature* **635**, 834–840 (2024).
- [207] Kai R Ott, Bence Hetényi, and Michael E Beverland, “Decision-tree decoders for general quantum LDPC codes”, *arXiv:2502.16408* (2025).
- [208] Laleh Aghababaie Beni, Oscar Higgott, and Noah Shutt, “Tesseract: A search-based decoder for quantum error correction”, *arXiv:2503.10988* (2025).
- [209] Peter E Hart, Nils J Nilsson, and Bertram Raphael, “A formal basis for the heuristic determination of minimum cost paths”, *IEEE transactions on Systems Science and Cybernetics* **4**, 100–107 (1968).
- [210] Dragana Grbic, Laleh Aghababaie Beni, and Noah Shutt, “Accelerating the Tesseract Decoder for Quantum Error Correction”, *arXiv:2602.02985* (2026).
- [211] Lucas Berent, Lukas Burgholzer, Peter-Jan HS Derks, Jens Eisert, and Robert Wille, “Decoding quantum color codes with MaxSAT”, *Quantum* **8**, 1506 (2024).
- [212] Mohammadreza Noormandipour and Tobias Haug, “MaxSAT decoders for arbitrary CSS codes”, *arXiv:2410.01673* (2024).
- [213] Jun Fujisaki, Hirotaka Oshima, Shintaro Sato, and Keisuke Fujii, “Practical and scalable decoder for topological quantum error correction with an Ising machine”, *Physical Review Research* **4**, 043086 (2022).
- [214] Jun Fujisaki, Kazunori Maruyama, Hirotaka Oshima, Shintaro Sato, Tatsuya Sakashita, Yusaku Takeuchi, and Keisuke Fujii, “Quantum error correction with an Ising machine under circuit-level noise”, *Physical Review Research* **5**, 043261 (2023).
- [215] Yugo Takada, Yusaku Takeuchi, and Keisuke Fujii, “Ising model formulation for highly accurate topological color codes decoding”, *Physical Review Research* **6**, 013092 (2024).
- [216] Guillaume Duclos-Cianci and David Poulin, “Fast decoders for topological quantum codes”, *Physical Review Letters* **104**, 050504 (2010).
- [217] Guillaume Duclos-Cianci and David Poulin, “Fault-tolerant renormalization group decoder for Abelian topological codes”, *Quantum Information & Computation* **14**, 721–740 (2014).
- [218] Pradeep Sarvepalli and Andreas Klappenecker, “Degenerate quantum codes and the quantum Hamming bound”, *Physical Review A* **81**, 032318 (2010).
- [219] A.R. Calderbank, E.M. Rains, P.W. Shor, and N.J.A. Sloane, “Quantum error correction via codes over  $GF(4)$ ”, *arXiv:quant-ph/9608006* (1997).



- 
- [220] Lingfei Jin and Chaoping Xing, “Quantum Gilbert-Varshamov bound through symplectic self-orthogonal codes”, *IEEE International Symposium on Information Theory Proceedings*, 455–458 (2011).
  - [221] Sergey Bravyi, David Poulin, and Barbara Terhal, “Tradeoffs for reliable quantum information storage in 2D systems”, *Physical Review Letters* **104**, 050503 (2010).
  - [222] Nouédy Baspin and Anirudh Krishna, “Connectivity constrains quantum codes”, *Quantum* **6**, 711 (2022).
  - [223] Nouédy Baspin and Anirudh Krishna, “Quantifying nonlocality: How outperforming local quantum codes is expensive”, *Physical Review Letters* **129**, 050505 (2022).
  - [224] Eric M Rains, “Monotonicity of the quantum linear programming bound”, *IEEE Transactions on Information Theory* **45**, 2489–2492 (2002).
  - [225] Gerard Angles Munné, Andrew Nemec, and Felix Huber, “SDP bounds on quantum codes”, *arXiv:2408.10323* (2024).
  - [226] Robert G. Gallager, “Low density parity check codes”, PhD Thesis (Massachusetts Institute of Technology, Department of Electrical Engineering, 1960).
  - [227] R. Tanner, “A recursive approach to low complexity codes”, *IEEE Transactions on Information Theory* **27**, 533–547 (1981).
  - [228] C. Berrou, A. Glavieux, and P. Thitimajshima, “Near Shannon limit error-correcting coding and decoding: Turbo-codes. 1”, *Proceedings of ICC '93 - IEEE International Conference on Communications* (1993).
  - [229] Daniel Gottesman, “Fault-Tolerant Quantum Computation with Constant Overhead”, *arXiv:1310.2984* (2014).
  - [230] Nikolas P. Breuckmann and Jens Niklas Eberhardt, “Quantum low-density parity-check codes”, *PRX Quantum* **2**, 040101 (2021).
  - [231] Samuel Dai and Ray Li, “Locality vs quantum codes”, *STOC '25: Proceedings of the 57th Annual ACM Symposium on Theory of Computing*, 677–688 (2025).
  - [232] Jean-Pierre Tillich and Gilles Zémor, “Quantum LDPC codes with positive rate and minimum distance proportional to the square root of the blocklength”, *IEEE Transactions on Information Theory* **60**, 1193–1202 (2014).
  - [233] Hsiang-Ku Lin and Leonid P. Pryadko, “Quantum two-block group algebra codes”, *arXiv:2306.16400* (2023).
  - [234] Lukas Voss, Sim Jian Xian, Tobias Haug, and Kishor Bharti, “Multivariate bicycle codes”, *Physical Review A* **111**, L060401 (2025).
  - [235] Pavel Panteleev and Gleb Kalachev, “Asymptotically good quantum and locally testable classical LDPC codes”, *arXiv:2111.03654* (2022).
  - [236] Pavel Panteleev and Gleb Kalachev, “Quantum LDPC codes with almost linear minimum distance”, *IEEE Transactions on Information Theory* **68**, 213–229 (2022).
  - [237] Anurag Anshu, Nikolas P. Breuckmann, and Chinmay Nirkhe, “NLTS Hamiltonians from good quantum codes”, *STOC 2023: Proceedings of the 55th Annual ACM Symposium on Theory of Computing*, 1090–1096 (2023).
  - [238] Matthew B. Hastings, Jeongwan Haah, and Ryan O’Donnell, “Fiber bundle codes: Breaking the  $n^{1/2}\text{polylog}(n)$  barrier for quantum LDPC codes”, *STOC 2021: Proceedings of the 53rd Annual ACM SIGACT Symposium on Theory of Computing*, 1276–1288 (2021).

- 
- [239] Daniel J Spencer, Andrew Tanggara, Tobias Haug, Derek Khu, and Kishor Bharti, “Qudit low-density parity-check codes”, *Quantum* **10**, 2023 (2026).
- [240] Louis Golowich and Venkatesan Guruswami, “Decoding quasi-cyclic quantum LDPC codes”, *arXiv:2411.04464* (2024).
- [241] Mainak Bhattacharyya and Ankur Raina, “Decoding quantum LDPC codes using collaborative check node removal”, *arXiv:2501.08036* (2025).
- [242] Nithin Raveendran, David Declercq, and Bane Vasić, “On the minimum distances of finite-length lifted product quantum LDPC codes”, *arXiv:2503.07567* (2025).
- [243] Asit Kumar Pradhan, Nithin Raveendran, Narayanan Rengaswamy, and Bane Vasić, “Linear time iterative decoders for hypergraph-product and lifted-product codes”, *arXiv:2504.01728* (2025).
- [244] D.M. Appleby, “Symmetric informationally complete-positive operator valued measures and the extended Clifford group”, *Journal of Mathematical Physics* **46**, 052107 (2005).
- [245] Vlad Gheorghiu, “Standard form of qudit stabilizer groups”, *Physics Letters A* **378**, 505–509 (2014).
- [246] Adam Wills, “A review of Galois qudits”, *arXiv:2605.18981* (2026).
- [247] Fern H.E. Watson, Earl T. Campbell, Hussain Anwar, and Dan E. Browne, “Qudit color codes and gauge color codes in all spatial dimensions”, *Physical review A* **92**, 022312 (2015).
- [248] Iryna Andriyanova, Denise Maurice, and Jean-Pierre Tillich, “New constructions of CSS codes obtained by moving to higher alphabets”, *arXiv:1202.3338* (2012).
- [249] Salah A. Aly, “A note on quantum Hamming bound”, *arXiv:0711.4603* (2007).
- [250] Matthew Neeley et al., “Emulation of a quantum spin with a superconducting phase qudit”, *Science* **325**, 722–725 (2009).
- [251] Machiel S Blok et al., “Quantum information scrambling on a superconducting qutrit processor”, *Physical Review X* **11**, 021010 (2021).
- [252] Alexis Morvan et al., “Qutrit randomized benchmarking”, *Physical Review Letters* **126**, 210504 (2021).
- [253] Tanay Roy, Ziqian Li, Eliot Kapit, and David I Schuster, “Two-qutrit quantum algorithms on a programmable superconducting processor”, *Physical Review Applied* **19**, 064024 (2023).
- [254] Noah Goss et al., “High-fidelity qutrit entangling gates for superconducting circuits”, *Nature communications* **13**, 7481 (2022).
- [255] Arkady Fedorov, Lars Steffen, Matthias Baur, Marcus P da Silva, and Andreas Wallraff, “Implementation of a Toffoli gate with superconducting circuits”, *Nature* **481**, 170–172 (2012).
- [256] Hsuan-Hao Lu, Zixuan Hu, Mohammed Saleh Alshaykh, Alexandria Jeanine Moore, Yuchen Wang, Poolad Imany, Andrew Marc Weiner, and Sabre Kais, “Quantum phase estimation with time-frequency qudits in a single photon”, *Advanced Quantum Technologies* **3**, 1900074 (2020).
- [257] Jianwei Wang et al., “Multidimensional quantum entanglement with large-scale integrated optics”, *Science* **360**, 285–291 (2018).
- [258] Yulin Chi et al., “A programmable qudit-based quantum processor”, *Nature communications* **13**, 1166 (2022).

- 
- [259] Christian Reimer et al., “High-dimensional one-way quantum processing implemented on d-level cluster states”, *Nature Physics* **15**, 148–153 (2019).
- [260] Pei Jiang Low, Brendan M White, Andrew A Cox, Matthew L Day, and Crystal Senko, “Practical trapped-ion protocols for universal qudit-based quantum computing”, *Physical Review Research* **2**, 033128 (2020).
- [261] Pavel Hrmo et al., “Native qudit entanglement in a trapped ion quantum processor”, *Nature Communications* **14**, 2242 (2023).
- [262] Florian M Leupold, Maciej Malinowski, Chi Zhang, Vlad Negnevitsky, Adán Cabello, Joseba Alonso, and Jonathan P Home, “Sustained state-independent quantum contextual correlations from a single ion”, *Physical Review Letters* **120**, 180401 (2018).
- [263] Martin Ringbauer, Michael Meth, Lukas Postler, Roman Stricker, Rainer Blatt, Philipp Schindler, and Thomas Monz, “A universal qudit quantum processor with trapped ions”, *Nature Physics* **18**, 1053–1057 (2022).
- [264] Benjamin L Brock, Shraddha Singh, Alec Eickbusch, Volodymyr V Sivak, Andy Z Ding, Luigi Frunzio, Steven M Girvin, and Michel H Devoret, “Quantum error correction of qudits beyond break-even”, *Nature* **641**, 612–618 (2025).
- [265] Mohsin Iqbal et al., “Qutrit toric code and parafermions in trapped ions”, *Nature Communications* **16**, 6301 (2025).
- [266] E. Knill, “Scalable Quantum Computation in the Presence of Large Detected-Error Rates”, *arXiv:quant-ph/0312190* (2004).
- [267] Theerapat Tansuwannont, Balint Pato, and Kenneth R Brown, “Adaptive syndrome measurements for Shor-style error correction”, *Quantum* **7**, 1075 (2023).
- [268] Sergey Bravyi and Robert König, “Classification of topologically protected gates for local stabilizer codes”, *Physical Review Letters* **110**, 170503 (2013).
- [269] Alexey A. Kovalev and Leonid P. Pryadko, “Fault tolerance of quantum low-density parity check codes with sublinear distance scaling”, *Physical Review A* **87**, 020304(R) (2013).
- [270] Dorit Aharonov and Michael Ben-Or, “Fault-tolerant quantum computation with constant error”, in *Proceedings of the twenty-ninth annual ACM symposium on Theory of computing* (1997), pp. 176–188.
- [271] Quynh T Nguyen and Christopher A Pattison, “Quantum fault tolerance with constant-space and logarithmic-time overheads”, in *STOC ’25: Proceedings of the 57th Annual ACM Symposium on Theory of Computing* (2025), pp. 730–737.
- [272] Matthias Christandl, Omar Fawzi, and Ashutosh Goswami, “Fault-tolerant quantum input/output”, *IEEE Transactions on Information Theory* (2026).
- [273] Omar Fawzi, Antoine Grosse, and Anthony Leverrier, “Constant Overhead Quantum Fault-Tolerance with Quantum Expander Codes”, in *2018 IEEE 59th Annual Symposium on Foundations of Computer Science (FOCS)* (2018), pp. 743–754.
- [274] Adam M Meier, Bryan Eastin, and Emanuel Knill, “Magic-state distillation with the four-qubit code”, *arXiv:1204.4221* (2012).
- [275] Derek Khu, Andrew Tanggara, Chao Jin, and Kishor Bharti, “Contextuality of Quantum Error-Correcting Codes”, *PRX Quantum* **7** (2026).
- [276] Adam Paetznick and Ben W. Reichardt, “Universal Fault-Tolerant Quantum Computation with Only Transversal Gates and Error Correction”, *Physical Review Letters* **111** (2013).



- [277] Jonas T. Anderson, Guillaume Duclos-Cianci, and David Poulin, “Fault-Tolerant Conversion between the Steane and Reed-Muller Quantum Codes”, *Physical Review Letters* **113** (2014).
- [278] Sascha Heußen and Janine Hilder, “Efficient fault-tolerant code switching via one-way transversal CNOT gates”, *Quantum* **9**, 1846 (2025).
- [279] Craig Gidney, Noah Shutty, and Cody Jones, “Magic state cultivation: growing  $T$  states as cheap as CNOT gates”, *arXiv:2409.17595* (2024).